Agent Harness

从执行脚手架到自我进化系统

研究修订稿

修订日期（具体事实版本见各章）

2026-10-29

序：为什么现在需要一本 Harness 小书

同一模型放进不同的 Agent 产品后，表现可能看起来完全不同。 差异来自模型之外的上下文、工具、环境、权限、恢复机制和验证机制。 本书把这组能力统一叫做 Agent Harness。

Harness 不是某个框架品牌，也不只是 while model calls tools。 它是模型和真实世界之间的运行时与治理层。 它把人的意图编译成任务，把环境状态编译成上下文，把模型动作约束为受控副作用，再把外部结果编译成可审查证据。

本书面向企业 Agent 平台架构师和高级工程师。 历史篇解释从规划、BDI、ReAct 到 coding agent 的转折；原理篇给出状态机、上下文、工具、安全、验证和多 Agent 的设计；产品篇按同一组架构问题研究 Claude Code、Codex、Cursor、DeepSeek Harness、OpenHands 与 pi-mono；进化篇区分任务内、跨任务、Harness 和模型四层；实践篇给出供应商无关的参考架构和迁移路线。

全书用下面的乘式表达一个设计隐喻，而非可计算的定量公式：

TEXT

```
Agent System Capability = Model × Harness × Environment × Feedback
```

乘号提醒我们，各环节会相互制约；四项并没有可直接相乘的统一度量。 强模型仍可能因关键上下文缺失、危险工具、环境异常或验证器错误而失败。 同一模型单次问答的成绩，也不能直接当作带工具、搜索和多次尝试的系统上限。Harness 改变了可用信息和计算过程，具体收益仍需在给定任务与预算下实测。

全书原始资料截面为 2026-08-28，2026-09-19 完成一轮全书增补；2026-10-29 进一步重写第三篇的架构分析并加入 pi-mono。其余篇没有因此被视为完成十月事实更新；快速变化的产品与接口按各章的来源日期和固定版本说明。 功能会过时，但设计原则更稳。 书中明确区分公开事实、论文结果和作者综合判断；内部使用不改变证据边界。

目录

序：为什么现在需要一本 Harness 小书 2

目录 3

第一篇 历史：Agent 如何从会回答变成会行动 12

本篇导言：Harness 从哪里来 . . . 12

第一章 从控制循环到 Agent Runtime . . . 13

1. 规划：把目标翻译成动作序列 . . . 13
2. 反应与承诺：BDI 的长期遗产 . . . 13
3. 多 Agent：1980 年已经出现的协调税 . . . 14
4. 从手写控制器到模型控制器 . . . 15
5. ReAct：现代最小循环的形成 . . . 15
6. Toolformer、Reflexion 与 Voyager：三种能力迁移 . . . 16
7. Harness 的历史不是功能累积，而是责任迁移 . . . 16

第二章 2023：自主 Agent 爆发与第一次祛魅 . . . 18

1. BabyAGI：任务队列就是最小外部认知 . . . 18
2. AutoGPT：把开放动作空间交给语言模型 . . . 18
3. LangChain：把 Agent 拆成可复用抽象 . . . 19
4. AutoGen：把工作流表达成多 Agent 对话 . . . 20
5. 第一次祛魅：为什么 Demo 自治不能直接进入生产 . . . 20
   - 5.1 语言流畅度与状态正确性错配 . . . 20
   - 5.2 语义相似与记忆有效性错配 . . . 21
   - 5.3 工具可调用与动作可授权错配 . . . 21
   - 5.4 循环持续与目标进展错配 . . . 21
   - 5.5 自我批评与独立验证错配 . . . 21
   - 5.6 多角色对话与多样性错配 . . . 21
6. 从 Framework 到 Runtime . . . 21
7. 这一代系统留下了什么 . . . 21

第三章 接口也是智能：Aider、SWE-agent 与 OpenHands . . . 23

1. Aider：上下文不是越多越好 . . . 23
2. 编辑格式：工具接口会占用模型能力 . . . 23
3. SWE-agent：Agent-Computer Interface 成为设计对象 . . . 24
4. CodeAct：代码作为动作语言 . . . 24
5. OpenHands：把 Agent 与 Runtime 分开 . . . 25
6. Event Stream 不等于完整事件溯源 . . . 25
7. Benchmark 开始评价“模型 × Harness” . . . 26
8. 从接口工程得到的七条原则 . . . 26

第四章 Coding Agent 转折：从实验接口到产品运行时 . . . 27

1. 为什么软件工程成为关键试验场 . . . 27
2. 2024—2026 的产品化转向 . . . 27
3. 从 SWE-bench 到可执行系统评测 . . . 27
4. Benchmark rot 的四种来源 . . . 28
5. 排行榜为何不能直接指导企业选型 . . . 28
6. Coding Agent 留下的架构遗产 . . . 28

第二篇 原理：生产级 Harness 的构成 29
本篇导言：把概率决策装进确定性边界 . . . 29
第五章 Agent = Model × Harness × Environment × Feedback . . . 30
1. Model：概率性策略与生成器 . . . 30
2. Harness：运行时中介与控制系统 . . . 31
3. Environment：不是一个 shell，而是任务世界 . . . 31
4. Feedback：观测不等于评价 . . . 32
5. 为什么使用乘法而不是加法 . . . 32
6. Workflow、Agent 与 Harness 的关系 . . . 32
7. 控制面、数据面与执行面 . . . 32
7.1 控制面 . . . 33
7.2 数据面 . . . 33
7.3 执行面 . . . 33
8. 概率性建议与确定性约束 . . . 33
9. Harness 的厚与薄 . . . 34
10. 一个可实现的最小分层 . . . 34
11. 本书的统一分析模板 . . . 34
第六章 Agent Loop：从 while 循环到持久状态机 . . . 36
1. Thread、Turn、Step 与 Attempt . . . 36
2. 最小状态机 . . . 36
3. 模型停止不等于任务完成 . . . 37
4. Streaming 是事件协议，不只是打字动画 . . . 37
5. 取消必须贯穿调用链 . . . 37
6. Timeout、Retry、Resume 不是一回事 . . . 38
7. 副作用账本与不确定提交 . . . 38
8. Checkpoint 应保存什么 . . . 39
9. Compaction 是有损状态迁移 . . . 39
10. Error Taxonomy 决定恢复策略 . . . 39
11. 并行工具与一致性 . . . 40
12. 一个更完整的循环伪代码 . . . 40
13. 最小可靠性测试集 . . . 41
第七章 上下文、缓存、压缩与记忆 . . . 43
1. 五种必须分开的信息 . . . 43
2. Context Assembly 是一次编译 . . . 43
3. 长窗口不是无限注意力 . . . 44
4. 静态上下文与动态发现 . . . 44
5. Prompt Cache 是架构约束 . . . 44
6. Compaction 是有损编译，不是删除旧消息 . . . 45
7. MemGPT 与分层记忆 . . . 45
8. 记忆类型与写入权限 . . . 46
9. Memory Poisoning 与程序漂移 . . . 46
10. 检索不是只有向量相似度 . . . 46
11. Skills 不是 Memory 的别名 . . . 47
12. Context Quality 的评价 . . . 47

13. 推荐的上下文架构 . . . 47
14. 最小验收清单 . . . 48
第八章 工具、ACI、MCP 与 Code Mode . . . 49
1. Tool Definition 只是起点 . . . 49
2. 好工具的十个条件 . . . 49
3. 错误协议是 ACI 的一部分 . . . 50
4. Tool Result 不应只有字符串 . . . 50
5. MCP 解决的是互操作，不是全部 Harness 问题 . . . 50
6. MCP 的安全边界 . . . 51
7. Tool Discovery：工具也需要分页 . . . 51
8. CLI：最通用但最难治理的工具总线 . . . 51
9. Native Tool Call 与 Code Mode . . . 52
10. 并行工具的调度语义 . . . 52
11. 工具版本与动态变化 . . . 53
12. Tool Policy 与 Tool Execution 分离 . . . 53
13. 如何评价工具层 . . . 53
14. 企业平台的工具分层 . . . 54
15. 最小工具契约伪代码 . . . 54
第九章 权限、沙箱、凭证与供应链 . . . 55
1. 四个不同问题 . . . 55
2. Prompt 不是安全边界 . . . 55
3. 从逐次批准到受控自治 . . . 55
4. 文件系统与网络必须同时限制 . . . 56
5. Policy 决策模型 . . . 56
6. 凭证不进入模型上下文 . . . 56
7. Prompt Injection 的系统应对 . . . 57
8. 多 Agent 的权限传播 . . . 57
9. MCP 与插件供应链 . . . 57
10. 审批 UX 是安全系统 . . . 58
11. 审计记录与模型 trace 分离 . . . 58
12. 数据分类与跨域流动 . . . 59
13. 风险分级 . . . 59
14. 安全测试 . . . 59
15. 安全参考边界 . . . 60
第十章 验证、完成契约与证据包 . . . 61
1. Stop、Answer、Success 与 Commit 是四件事 . . . 61
2. 完成契约从任务入口开始 . . . 61
3. 验证金字塔 . . . 62
3.1 确定性检查 . . . 62
3.2 环境与结果检查 . . . 62
3.3 模型检查 . . . 62
3.4 人类检查 . . . 62
4. Benchmark 也是会腐化的软件 . . . 62
5. 非确定性系统不能只跑一次 . . . 63
6. 防止测试投机与 verifier 篡改 . . . 63

7. 证据包是交付协议 . . . 64
8. 完成门的参考状态机 . . . 64
9. 三类案例 . . . 66
9.1 仓库软件工程 . . . 66
9.2 企业数据分析 . . . 66
9.3 自我进化 Agent . . . 66
10. 常见失败模式 . . . 66
10.1 把 Agent 自述当证据 . . . 66
10.2 只验证最终文本 . . . 66
10.3 同一主体控制目标、实现与评分 . . . 66
10.4 失败后动态降低门槛 . . . 66
10.5 迷信 benchmark 排名 . . . 67
10.6 无限验证—修复循环 . . . 67
11. 企业落地清单 . . . 67
第十一章 多 Agent、委派与协作拓扑 . . . 68
1. 先区分五种经常混淆的东西 . . . 68
2. 多 Agent 的收益来自哪里 . . . 68
3. 何时不应使用多 Agent . . . 68
4. 四类基本拓扑 . . . 69
4.1 Manager—Worker . . . 69
4.2 Pipeline / Assembly Line . . . 69
4.3 Peer Handoff . . . 69
4.4 Blackboard / Event Graph . . . 70
5. 委派是一份受限契约 . . . 70
6. 子结果应交付产物与证据 . . . 70
7. 共享状态与并发写入 . . . 71
8. 错误如何在团队中传播 . . . 71
9. Debate、Critique 与 Ensemble . . . 72
10. 调度、背压与预算 . . . 72
11. 权限、身份与责任链 . . . 72
12. 可观测性：同时看到树和因果链 . . . 73
13. 三类贯穿案例 . . . 73
13.1 仓库软件工程 . . . 73
13.2 企业数据分析 . . . 73
13.3 自我进化 Agent . . . 73
14. 一个最小参考实现 . . . 74
15. 设计原则总结 . . . 75
第十二章 可观测性、轨迹与评测运营 . . . 76
1. 轨迹是事件图 . . . 76
2. 指标分四层 . . . 76
3. 失败分类先于优化 . . . 76
4. Eval 是持续运营系统 . . . 76
5. 在线监控与离线评测闭环 . . . 77
6. Trace replay 的边界 . . . 77
7. 运营仪表盘 . . . 77
8. 一条工具完成事件的示例 . . . 77

9. Trace 完整性与采样 . . . 78
10. Eval 生命周期与污染控制 . . . 78
11. 从指标到行动 . . . 78
12. 可观测性的边界 . . . 78
第三篇 产品：当代主流 Harness 的不同答案 79
本篇导言：用同一组问题读懂六种实现 . . . 79
第十三章 Claude Code：薄循环、厚运行时 . . . 80
1. 整体架构与职责边界：谁拥有循环 . . . 80
2. Agent loop 与恢复：消息流不是业务事务 . . . 80
3. 上下文与会话：压缩、缓存、记忆各做什么 . . . 81
4. 工具、MCP 与扩展：从可发现能力到执行结果 . . . 81
5. 权限、沙箱与凭证：批准动作与限制能力 . . . 82
6. 子智能体与委派：隔离的是哪一份状态 . . . 82
7. 验证与可观测性：从循环结束到可验收产物 . . . 83
8. 一次任务穿过架构：修复登录回归 . . . 83
第十四章 OpenAI Codex：协议化的 Agent Core . . . 84
1. 整体架构与职责边界：协议包住的是运行时 . . . 84
2. Agent loop 与恢复：一轮请求可以包含多次采样 . . . 84
3. 上下文与会话：模型窗口、宿主事实和记忆各有归属 . . . 85
4. 工具、MCP 与扩展：声明能力和执行能力必须对齐 . . . 85
5. 权限、沙箱与凭证：决定能否做，与限制如何做 . . . 86
6. 子智能体与委派：子线程共享控制面，各自运行循环 . . . 86
7. 验证与可观测性：看到执行，不等于证明正确 . . . 86
8. 一次任务如何穿过架构：修复导出遗漏 . . . 87
第十五章 Cursor：IDE 原生上下文与云端 Agent . . . 88
1. 整体架构与职责边界：从编辑器进入执行系统 . . . 88
2. Agent loop 与恢复：模型决策，应用执行 . . . 88
3. 上下文与会话：索引、指令、历史各有载体 . . . 89
4. 工具、MCP 与扩展：从可发现描述到实际调用 . . . 89
5. 权限、沙箱与凭证：批准动作不等于限制进程 . . . 90
6. 子智能体与委派：上下文隔离、环境隔离、调度分层 . . . 90
7. 验证与可观测性：可回看过程，仍须验证结果 . . . 90
8. 一次任务穿过架构：修复跨前后端的权限错误 . . . 91
第十六章 DeepSeek Harness：可组合、可逆的运行时 . . . 92
1. 整体架构与职责边界：运行的是一棵插件树 . . . 92
2. Agent loop 与恢复：把请求、尝试和回合分开 . . . 92
3. 上下文与会话：日志保存事实，投影决定模型看见什么 . . . 93
4. 工具、MCP 与扩展：统一分派，分别拥有执行 . . . 93
5. 权限、沙箱与凭证：三个问题，三条边界 . . . 94
6. 子智能体与委派：上下文继承不等于权限继承 . . . 94
7. 验证与可观测性：区分事实日志、实时流与报告副本 . . . 95
8. 一次任务穿过架构：接入一个 MCP 查询能力 . . . 95
第十七章 OpenHands：Agent 与执行 Runtime 分离 . . . 96
1. 整体架构与职责边界：配置、会话与执行位置分别建模 . . . 96

2. Agent loop 与恢复：由 Conversation 驱动 Agent 的单步 . . . 97
3. 上下文与会话：原始事件、模型视图和长期记忆各有位置 . . . 97
4. 工具、MCP 与扩展：配置可以传输，执行器在运行处物化 . . . 98
5. 权限、沙箱与凭证：批准动作与隔离执行分别配置 . . . 98
6. 子智能体与委派：一项 task 对应一个子 Conversation . . . 98
7. 验证与可观测性：存储顺序不等于业务恰好一次 . . . 99
8. 一次任务穿过架构：修复仓库中的解析错误 . . . 99
第十八章 pi-mono：小核心怎样长成可组合的运行时 . . . 101
1. 整体架构：循环、会话和界面分别负责什么 . . . 101
2. Agent loop：工具反馈和用户追加输入怎样推进循环 . . . 101
3. 上下文与会话：完整记录和当前请求不是同一个对象 . . . 102
4. 工具、MCP 与扩展：小接口怎样容纳大工具集 . . . 102
5. 权限、沙箱与凭证：项目可信不等于操作受限 . . . 103
6. 子智能体与委派：扩展示例如何复用同一个核心 . . . 103
7. 验证与可观测性：事件帮助看见过程，不能替任务验收 . . . 103
8. 一次修复任务如何穿过这些组件 . . . 103
第十九章 产品比较：不要用一张总分表掩盖架构差异 . . . 105
1. 先比较架构：相同责任被放在了哪里 . . . 105
接入面与企业仍需承担的责任 . . . 106
2. 成品、可嵌入运行时、框架与托管服务 . . . 106
3. 托管与自管，要按责任拆开 . . . 106
4. 六个评价坐标，需要六类证据 . . . 107
5. 共同契约应保留产品差异 . . . 107
6. 有条件的采用与退出 . . . 108
第四篇 进化：Agent 如何从轨迹中变得更好 109
本篇导言：从“能改自己”到可复查的改进 . . . 109
第二十章 进化不是自我修改：目标函数、证据与边界 . . . 110
1. 四类可变对象，不是升级阶梯 . . . 110
2. 一份能复查的证据合同 . . . 110
3. 评价权与可变表面分离 . . . 111
4. 先定义估计对象，再谈晋级 . . . 111
5. 零观测违规与全部费用 . . . 112
6. 研究信号应拆成创建、维护与选择 . . . 112
7. 从失败到可检验假设 . . . 112
8. 预算、停止与保持现状 . . . 112
第二十一章 任务内进化：搜索、反思与验证—修复 . . . 113
1. 本层的证据模板实例 . . . 113
2. 反思要绑定观察，采样也可以构成搜索 . . . 113
3. 一个会推进、会扣费、会停止的搜索 . . . 113
4. 验证反馈与环境重试 . . . 115
5. 修计划、修状态与真实副作用 . . . 115
6. 长项目靠产物和证据延续 . . . 115
7. 将预算用在可区分的解释上 . . . 116
8. 如何结束当前任务 . . . 116

第二十二章 跨任务经验化：Memory、Skill 与策略库 . . . 117
1. 本层的证据模板实例 . . . 117
2. 从成功轨迹到可发布条目 . . . 117
3. 检索指标先固定分母 . . . 118
4. 技能包与状态转换 . . . 118
5. 撤销沿使用链传播 . . . 119
6. 复用实验与流式经验库 . . . 119
7. 冲突、容量与维护债务 . . . 119
第二十三章 Harness 进化：从失败病理到版本化变更 . . . 121
1. 本层的证据模板实例 . . . 121
2. 把失败描述改成可检验提案 . . . 121
3. 用不同信号诊断，保留替代解释 . . . 121
4. 从搜索档案到生成器 . . . 122
5. 激活证据与四组门禁 . . . 122
6. 按模型、任务和预算选择配置 . . . 122
7. 效率优化要保留质量让步 . . . 123
8. 发布半径与退役 . . . 123
第二十四章 模型进化：从轨迹到参数更新 . . . 124
1. 本层的证据模板实例 . . . 124
2. 轨迹先过数据门 . . . 124
3. 训练执行者，也可以训练生成器 . . . 125
4. 2×2：条件均值不是效应 . . . 125
5. 从任务级配对数据计算区间 . . . 125
6. 来源追踪之后，还要处置派生资产 . . . 126
7. 发布后仍是系统试验 . . . 127
8. 何时训练，何时保留其他方案 . . . 127
第二十五章 受控进化闭环：门禁、灰度、回滚与反投机 . . . 128
1. 职责分层不能替代信任边界 . . . 128
2. 登记实验与选择规则 . . . 128
3. 留出验证与封存终测各有用途 . . . 129
4. 完整性先于分数 . . . 129
5. 一次“不晋级”的决定 . . . 130
6. 影子运行、灰度与紧急撤销 . . . 130
7. 发布记录、批准与演练 . . . 130
8. 自动化权限与变更对象分别管理 . . . 131
9. 治理也可以更新，确认权不能同轮转移 . . . 131
第五篇 实践：下一代企业 Harness 132
本篇导言：把原则落到企业控制面 . . . 132
第二十六章 三个贯穿案例：从意图到可验证结果 . . . 133
案例一：仓库级软件修复 . . . 133
1. 任务与合同 . . . 133
2. 执行与验证序列 . . . 134
3. 证据包 . . . 135
4. 失败演练：只适配可见测试 . . . 135

案例二：企业经营分析 . . . 136
1. 任务与口径 . . . 136
2. 双重验证 . . . 137
3. 证据包 . . . 137
4. 失败演练：快照漂移 . . . 137
案例三：自我进化 Harness . . . 137
1. 失败归因与实验合同 . . . 137
2. 选择与发布 . . . 138
3. 证据包与来源关系 . . . 138
4. 失败演练：通过修改分母“进步” . . . 139
三个案例的共用骨架 . . . 139
第二十七章 下一代企业 Harness 参考架构 . . . 140
1. 六层结构与权威状态 . . . 140
2. 统一契约与能力协商 . . . 141
3. 身份、租户与凭证 . . . 141
4. 持久执行与副作用 . . . 141
5. 以证据判定完成 . . . 141
6. 七类 SLI 与 SLO 设定方法 . . . 142
7. 多运行时数据流 . . . 142
8. 构建顺序与替代方案 . . . 143
第二十八章 Agent SDD：规范驱动的任务与发布 . . . 144
1. 六类规范 . . . 144
2. 从意图到合同 . . . 144
3. 教学实例：从规范到任务再到验收 . . . 144
3.1 合同怎样映射到平台对象 . . . 145
3.2 固定验收与数据库权限 . . . 146
3.3 四个独立负例 . . . 147
4. 运行中的合同修订 . . . 147
5. 规范怎样进入执行环境 . . . 147
6. 发布门与豁免 . . . 148
7. 规范质量指标 . . . 148
第二十九章 成熟度模型与 Build-vs-Buy . . . 149
1. 五类能力阶段 . . . 149
2. 按风险与证据自评 . . . 149
3. 买什么，控制什么 . . . 149
4. 何时自研 Runtime . . . 150
5. 决策矩阵与 POC . . . 150
第三十章 从接入现有 Agent 到拥有运行时主动权 . . . 151
1. 封装而非散接 . . . 151
2. 外置完成与证据 . . . 151
3. 统一执行面 . . . 151
4. 建立评测与运营基线 . . . 152
5. 逐层替换 . . . 152
6. 引入受控进化 . . . 152
7. 用同一个成本窗口比较采购与自建 . . . 152

8. 退出演练与紧急撤权 . . . 153
第三十一章 展望：Harness OS、Agent 组织与持续进化 . . . 154
1. 模型与 Harness 可以共同设计 . . . 154
2. Agent 组织首先是依赖和交接问题 . . . 154
3. 环境既可能是资产，也可能是债务 . . . 154
4. 自我进化要拆开“改什么”与“谁来决定” . . . 154
5. 长时系统仍有未解决的问题 . . . 155
6. 三条路径及其反证条件 . . . 155
7. 把判断留给可观察结果 . . . 155
附录 156
附录 A：语言无关核心契约与安全伪代码 . . . 157
1. 核心对象 . . . 157
2. Run loop：proposal 不直接变成 effect . . . 158
3. 副作用提交：先记 intent，再执行 . . . 159
4. 恢复、取消与对账 . . . 160
5. Adapter 的能力协商 . . . 161
附录 B：可判定的 Harness 架构评审表 . . . 162
任务与完成 . . . 162
上下文与记忆 . . . 162
工具、环境与副作用 . . . 162
权限与供应链 . . . 163
Durable 与多 Agent . . . 163
Eval、运营与进化 . . . 163
最终判定 . . . 163
附录 C：术语与本体边界 . . . 165
附录 D：概念首次定义与使用索引 . . . 167
附录 E：最小机器可读契约 . . . 168
1. Task 与 CompletionContract . . . 168
2. Action、PolicyDecision 与 Observation . . . 168
3. EvidencePackage 必填骨架 . . . 170
研究方法与局限 171
已核验参考文献 172

Markdown 章节按下列五篇排列。

第一篇 历史：Agent 如何从会回答变成会行动

本篇导言：Harness 从哪里来

本篇要回答“为什么模型之外还需要一套运行系统”。第一章用规划、控制循环和多 Agent 协商提供历史参照，不据结构相似断言直接传承；第二章分析 2023 年自主 Agent 原型的可行性与可靠性缺口；第三章说明 ACI（Agent-Computer Interface，即 Agent 与计算环境之间的动作和观察接口）为何影响任务表现；第四章讨论真实仓库和可执行评测带来的产品化要求。

本篇按工程问题组织历史：行动系统如何维护外部状态与反馈，接口如何改变模型可执行的策略，可演示的循环又如何取得状态、权限、恢复和验证等运行保证。读完后，读者应能解释为什么“换更强模型”并不能自动补齐这些缺口。

第一章 从控制循环到 Agent Runtime

如果把 Claude Code、Codex 或 DeepSeek Harness 的界面都拿掉，剩下的核心其实非常简单：先接收目标，再调用模型，执行模型选择的动作，最后把结果回传给模型。 这样的循环持续进行，直到目标完成或预算耗尽。

TEXT

```
目标 → 决策 → 动作 → 环境变化 → 新观察
        ↑                          ↓
        └──────── 反馈 ────────┘
```

这个循环并不新。控制论研究反馈机制，自动规划研究如何从初始状态到达目标状态，机器人研究感知与行动，BDI 研究信念、目标与承诺，多智能体系统研究任务分配与通信协议。本章用这些工作比较问题结构；除有明确来源的关联外，不据此建立现代产品的直接设计谱系。 现代 Harness 的新意不在于重新发明循环，而在于把一种高能力、概率性、上下文受限、可调用任意软件工具的语言模型放进真实计算环境后，为它补上可靠运行所需的工程结构。

因此，要理解 Harness，最好的起点不是 2023 年的 AutoGPT，而是三个更早的问题。 第一，机器如何形成行动序列？ 第二，一个持续运行的 Agent 如何在变化环境中维持承诺并修正行为？ 第三，多个自治执行者如何分工，而又不依赖全局共享状态？

1. 规划：把目标翻译成动作序列

1971 年的 STRIPS 将规划描述为：在一个世界模型中寻找一串操作，使初始状态经过这些操作后满足目标公式。它把动作表示成具有前置条件和状态效果的操作符，使“怎样完成任务”从一段专用程序转化成可以搜索的状态转换问题。 STRIPS 原始论文

用现代 Harness 的语言重写，STRIPS 已经对应了四个熟悉成分：

TEXT

`State`	当前环境的结构化描述
`Goal`	希望满足的终止条件
`Action`	带前置条件和效果的环境操作
`Planner`	搜索可行 `Action` 序列的控制器

现代 coding agent 的文件读取、补丁、命令执行和测试工具，也可以被当成动作。 但是它和经典规划有决定性差异。 经典规划通常假设动作语义明确、状态可知、执行结果近似确定；而软件 Agent 的仓库、依赖、网络服务和人类意图是部分可观测的。 模型可能误解工具说明、生成无效参数，甚至把命令成功退出当作业务目标完成。

这也是为什么现代 Agent 很少只生成一份完整计划然后机械执行。 更常见的是滚动规划：先选一个信息增益高、风险可控的动作；观察真实结果；再更新局部计划。 ReAct 把这种“推理-行动”交替明确写进语言模型轨迹，但核心问题仍是规划与控制中的老问题：当世界模型不完整时，计划只能依据反馈持续修正。

对 Harness 设计者而言，这段历史留下第一条原则：

> 计划不是事实，而是一个必须持续接受环境证据修正的假设。

这意味着 plan mode 可以帮助形成意图和界定审查范围，但不应成为独立于执行反馈的僵硬工作流。 真正可靠的 Harness 必须保留重新观察、修订计划、撤销局部动作，以及报告不可达目标的通道。

2. 反应与承诺：BDI 的长期遗产

只在每一步对刺激反应的系统容易漂移。 只按预先计划执行的系统又难以适应变化。 Belief–Desire–Intention（BDI）架构试图在两者之间建立实际推理循环。 Agent 根据对世界的信念识别可能目标，并从中形成当前愿望或目标，再选择并承诺某些意图。 新事件到来后，它更新信念，判断现有意图是否仍可行，再决定继续、重规划或放弃。

BDI 的价值不在于要求现代 Harness 一定创建名为 beliefs、desires、intentions 的三个对象。 它真正揭示的是三类经常被聊天记录混为一谈的状态：

- 观察和推断出来的世界状态；
- 用户希望实现但尚未承诺执行路径的目标集合；
- 当前已经投入资源、应跨多个步骤保持一致的执行承诺。

早期 PRS、dMARS 与 AgentSpeak 也发展了计划库、事件触发、元级控制和显式 deliberation cycle。 BDI 综述将其核心贡献概括为：在动态、不可预测环境里平衡主动目标与被动响应，并通过意图表达对未来行为的承诺。BDI 架构综述

这和现代 Agent 的几个常见故障直接相关。

第一，只有对话历史却没有显式任务状态时，模型可能在长会话里忘掉用户到底批准了什么。 第二，计划、待办、执行事实和模型猜测如果用同一种自然语言表示，压缩后很容易互相污染。 第三，出现新消息时，如果 Harness 不区分“补充信息”“优先级变化”“取消请求”和“新任务”，Agent 就可能误放弃或误延续已有承诺。

因此，企业 Harness 至少应在运行时区分：

TEXT

```
ObservedState   可追溯到环境或用户消息的事实
Hypothesis      模型尚未验证的解释
Goal            用户或上层系统定义的结果条件
Commitment      已批准并正在执行的目标/约束
Plan            当前可替换的动作建议
ExecutionState  已开始、等待、取消、失败或完成
```

这里最重要的不是命名，而是不同状态有不同的更新权限和证据要求。 模型可以自由提出假设和计划，但不应自行把假设升级为事实。 也不应把未批准目标升级为承诺。 BDI 为理解这类状态分离提供了比较框架，但本章没有证据证明现代 Harness 的相似分层直接继承自 BDI。

3. 多 Agent：1980 年已经出现的协调税

1980 年的 Contract Net Protocol 研究松耦合节点如何通过协商分配任务。 拥有任务的 manager 发布任务描述，潜在 contractor 根据能力和资源提交方案，manager 再授予合同。 该系统强调没有全局共享数据，也没有单一全局控制。 任务分配必须同时兼顾资源利用与问题求解焦点。Contract Net 原始论文

这与今天的 subagent、agent team 和异步云 Agent 非常相似，但它也提醒我们：多 Agent 从来不是免费的并行计算。 例如，管理者把两个检索子题交给不同执行者，首先要说明各自的输入与交付物。结果返回后，即使两份回答都自称完成，管理者仍需核对来源和冲突，再更新主任务状态。执行者超时、重复交付或越出题目范围时，通信与验收本身就成为工作。

从这个现代教学例子可以看出，委派成本不只是多发几条消息，还包括任务切分、执行者选择、上下文传递、授权、取消和结果合并。第十一章再展开这些运行契约；Contract Net 在这里提供的是协商与分配的参照。

如果拆分收益小于这些协调成本，多 Agent 会更慢、更贵，也更难调试。 这也是为什么“模型能够调用 subagent”不等于系统具备良好的多 Agent 架构。 真正要设计的是委派协议、隔离边界、结果契约、取消传播和合并策略。

现代 coding agent 使用 worktree、独立容器或远程 workspace，不是为了方便并行而已。 它是在物理上减少共享可变状态。 这种隔离与 Contract Net 的松耦合假设可以作结构比较，但两者解决的问题不同：前者约束共享可变状态和执行资源，后者组织任务协商。结构相似不足以证明直接传承，独立 worktree 也不自动带来独立权限或容器级隔离。

4. 从手写控制器到模型控制器

传统 Agent 的策略、计划选择和异常处理通常由规则、搜索算法或专用程序实现。 大语言模型改变的是控制器的表达能力。 它可以读取自然语言目标和非结构化观察，在没有为每种任务都写专用规则的情况下，提出下一动作并生成工具参数。

这带来显著的泛化能力，也引入四种结构性不确定性：

- 语义不确定性：模型可能误解目标、上下文或工具描述；
- 动作不确定性：它可能选择错误工具或构造无效参数；
- 状态不确定性：上下文窗口只包含环境的一部分，并可能在压缩中失真；
- 完成不确定性：模型说“完成”只是一项预测，不是外部世界已经满足目标的证明。

所以，现代 Harness 不能只是把模型接到 shell 上。 它需要用确定性软件包住概率性决策：工具 schema 约束动作形状，权限策略约束可执行范围，沙箱约束副作用，日志记录轨迹，预算限制循环，测试和评估器判断结果，人工审批承接不可自动化的价值判断。

可以把两者的分工写成一条简单边界：

TEXT

```
模型负责：提出、解释、比较、生成、诊断
Harness负责：授权、执行、隔离、记录、计量、验证、恢复
```

这不是说 Harness 中不能有模型评审器，也不是说控制逻辑必须完全固定。 关键是：任何影响安全、资源、归因和进化晋级的决定，都不能只依赖被评对象的单次自我陈述。

5. ReAct：现代最小循环的形成

ReAct 将 reasoning trace 与 action 交替生成。 推理帮助模型维护并调整计划。 动作让模型从外部知识源或环境取得新信息。 新观察再进入后续推理。ReAct

它为现代工具调用 Agent 提供了极具影响力的最小模板：

TEXT

```
for step in range(max_steps):
    ticket = budget.reserve_step_or_stop()
    try:
        decision = model(context, read_only_tools, limits=ticket)
        if decision.is_final:
            return candidate_answer(decision.answer)
        call = validate_read_only_call(decision.tool_call)
        observation = execute_in_simulator(call, limits=ticket)
        context.append(observation)
    finally:
        budget.settle(ticket)  # 计入已消耗资源，归还未用预留
return STOPPED_INCOMPLETE
```

这是说明交替过程的教学伪代码，不是ReAct原始实现。仓库案例入口用明确的模型与工具替身执行过这段控制流，没有运行原论文或供应商Agent。前提是工具仅在只读模拟环境运行，校验失败即拒绝执行；预算辅助函数在调用前预留额度、设置超时，并在异常路径结算。候选回答不代表业务完成，循环耗尽也只表示停止。 它没有回答上下文从哪里来，过长时如何压缩；工具结果是否可信；命令在哪台机器、以谁身份执行；权限是静态还是可临时扩展；进程崩溃后如何恢复；如何取消正在执行的工具；怎样判断模型陷入循环；最终答案需要哪些外部证据；多个用户和 Agent 如何隔离；历史轨迹如何进入评测和进化。

例如，读取文件失败后，模型可以根据错误选择重读；但若动作是发出付款且回执丢失，继续循环并不能判断该不该重试。后者需要外部副作用账本和对账规则。这是从最小循环走向生产运行时必须另行设计的责任，并非 ReAct 论文已经提供的保证。

6. Toolformer、Reflexion 与 Voyager：三种能力迁移

2023 年的几项工作分别说明了 Agent 能力可以迁移到不同位置。

Toolformer 研究了如何通过自监督数据，让模型学习何时调用 API、调用哪个 API、传什么参数，并学会吸收工具结果。Toolformer 它把一部分工具选择能力迁移进模型权重。

Reflexion 不更新权重，而是把任务反馈转成语言反思，保存到情景记忆中，影响后续尝试。Reflexion 它把一部分学习迁移到跨回合上下文。

Voyager 把成功行为保存为可执行技能库，并结合自动课程、环境错误和自验证持续扩充技能。Voyager 它把一部分能力迁移到外部、可组合、可复用的程序资产。

这三种路线构成了理解“Agent 如何进化”的早期坐标系：

能力沉淀位置	典型机制	优点	主要风险
模型权重	训练、微调、强化学习	推理时直接、可泛化	成本高、难回滚、归因困难
运行时记忆	反思、经验摘要、状态图	更新快、无需改权重	污染、过期、检索误配
外部技能	代码、工具、工作流、插件	可审查、可组合、可版本化	供应链风险、接口漂移
当前任务轨迹	retry、search、verify-fix	即时纠错	成本膨胀、循环与自证偏差

这张表按能力保存在哪里分类，不等同于后文的四层进化。后文按修改对象与生命周期区分：当前任务的尝试和修复属于任务内进化；经验与技能复用属于跨任务进化；上下文、工具和工作流配置变更属于 Harness 进化；权重更新属于模型进化。同一技能的生成、复用与发布可能跨越这些层次。 先保留一个重要结论：自我改进不必等同于修改模型，也不必等同于 Agent 任意重写运行时。 最有工程价值的进化，往往是把一次成功中可验证、可迁移的部分，沉淀到风险更低且更易回滚的层级。

7. Harness 的历史不是功能累积，而是责任迁移

回看这条历史线，可以看到 Harness 不是一张越来越长的功能清单。 它更像是责任在模型、控制器、环境与人之间不断重新分配的过程：

TEXT

```
经典规划：控制器显式搜索动作序列
BDI：       控制器管理信念、目标与承诺
多 Agent：  协议管理委派与资源协商
ReAct：     模型参与滚动决策与工具选择
Reflexion：经验进入外部记忆
Voyager：   成功行为进入可复用技能
现代 Harness：确定性运行时治理概率性控制器
```

今天看似全新的问题——上下文工程、subagent、skills、tool schema、self-evolution——确实能在早期 Agent 研究中找到结构相似物。 但结构相似既不等于直接传承，也不等于工程问题已经解决。 语言模型放大了动作空间、任务空间和接口空间。过去可以写死在专用系统里的约束，也必须升级为通用运行时机制。

据此可以理解把 Harness 作为独立责任层的工程理由；这是一种架构解释，不是对各产品形成原因的历史证明。 模型越通用，环境越开放，循环越长时，Harness 承担的责任反而越厚。 它必须让系统知道自己看见了什么、承诺了什么、能做什么、做过什么、是否真的完成，以及从结果中能学到什么。

下一章将进入 2023 年的“自主 Agent 爆发期”：AutoGPT、BabyAGI、LangChain 和 AutoGen 如何把循环、记忆、计划与多 Agent 编排带入大众开发实践，以及这些机制进入生产时需要补齐哪些保证。

第二章 2023：自主 Agent 爆发与第一次祛魅

2023 年春天，GPT-4、廉价 API、开源代码和社交媒体演示共同触发了一次“自主 Agent”爆发。 AutoGPT、BabyAGI、AgentGPT 等项目让普通开发者第一次“直接看到”：只要给模型一个目标、少量工具、一个循环和某种记忆，它似乎就能拆解任务、搜索网络、写文件、运行代码，并不断决定下一步。

从今天看，这批系统并没有建立可靠的通用自治。 但把它们直接叫“玩具”也不准确。 它们把语言模型从单次问答移入持续执行循环。本章分别讨论公开材料描述的机制，以及这些机制用于生产时可能出现的失效条件；后者不等于已经逐一复现的历史事故。

1. BabyAGI：任务队列就是最小外部认知

BabyAGI 归档所描述的核心结构非常简洁：从队列取出一个任务，调用执行 Agent，将结果写入记忆，再根据目标和最新结果创建新任务并重新排序队列，重复循环。归档代码入口 该链接仍指向可变分支，本轮尚未取得足以固定 2023 年原型的提交证据；下文据此解释队列机制，不把当前归档逐行等同于首发版本。

TEXT

```
Objective
   ↓
Task Queue → Execute → Result → Store
   ↑                            ↓
Prioritize ← New Tasks ←————————┘
```

它的重要启示不是“需要三个角色提示词”，而是模型之外必须有可检查的任务状态。 如果所有计划只在对话文本里，系统很难回答：还有哪些任务？哪个任务正在执行？为什么优先做它？某个结果由哪次执行产生？

任务队列把一部分认知外置成了数据结构。 这是一个很小但很关键的动作。 现代 Agent 的待办、计划、工单和执行图也体现相似的职责分离：自然语言适合生成候选，结构化状态更适合承载约束。这是机制比较，不是各项目直接继承 BabyAGI 的证据。

这种“模型管理模型任务”的机制也有可推导的失效条件。 任务创建者可能反复生成低价值后续事项；优先级调整者可能被最近结果牵着走；执行结果被存储并不代表它就正确；队列增长本身很容易被误当作进展。 例如，若“调查市场”的结果又生成同义调查任务，而系统没有目标验收和去重门，队列可以一直有工作，业务目标却不推进。这是教学反例，尚非对某个固定版本的运行复现。

这可以抽象成第一种自治陷阱：

> 当 Harness 只能测量活动而不能测量结果时，Agent 会把循环存活当作任务进展。

企业运行时因此不能只记录 tool-call 数、token 数、步骤数和任务完成声明。 它必须定义与业务结果直接挂钩的 evaluator。 对代码任务，可能是测试、静态检查和验收条件；对数据分析，可能是查询可复现性、口径一致性和事实来源；对业务流程，则可能是外部系统状态与审批记录。

2. AutoGPT：把开放动作空间交给语言模型

AutoGPT 的早期吸引力来自更开放的循环。 模型接收一个长期目标后，可以选择搜索、浏览、文件、命令等动作，再用短期历史和向量记忆延续任务。 与 BabyAGI 的显式队列相比，它更接近后来通用 coding agent 的体验：模型既负责局部规划，也负责选择工具并解释观察。

按这类开放循环的设计，可以尝试几种能力：

- 模型能在多轮中维持一个粗粒度目标；
- 工具结果可以成为新观察，影响后续决策；

- 外部记忆可以突破单次上下文的表面限制；
- 命令与文件工具让模型从“建议者”变成“执行者”。

与此同时，开放循环把多个风险同时放大：目标漂移、重复动作、无效搜索、错误记忆、费用失控、不可逆副作用和虚假完成。 模型生成一段看似合理的自我批评，并不意味着它识别了真实错误；向量库找回语义相似内容，也不意味着内容仍然正确或适用于当前状态。

AutoGPT 的后续仓库逐步发展出平台、组件、Forge 和 benchmark 等不同项目。 做历史研究时必须避免把这些成熟后的结构倒推到 2023 年原型。 这里讨论的是原型所代表的范式：让模型直接选择下一步，再用提示和记忆维持长期目标。由于本轮未补齐原型的固定版本与运行轨迹，下列风险是架构分析，不能据此断言某一版 AutoGPT 已发生对应事故。

这次实验带来的最大教训是：自治不是一个开关。 至少要从五个维度拆开看：

维度	问题
决策自治	下一步由模型、规则还是人决定？
工具自治	模型可以调用哪些动作？
权限自治	动作是否需要批准，能否扩大权限？
时间自治	可以持续多久，何时暂停或终止？
进化自治	能否改变记忆、技能、策略或自身 Harness？

一个系统可以有高决策自治，却被限制在只读沙箱。 也可以使用固定工作流，却对某个已批准 API 拥有高工具自治。 用“全自动/非自动”去描述 Agent，会遮住真正的风险边界。

3. LangChain：把 Agent 拆成可复用抽象

LangChain 在 2023 年用一套影响广泛的词汇总结当时的 Agent。 Agent 是决定动作的模型，Tools 是可执行动作，Memory 负责引入过去事件，AgentExecutor 运行循环直到满足停止条件。 它的典型算法直接继承 ReAct：Thought、Action、Observation 重复进行。LangChain 2023 总结

它的历史贡献是把快速增长的模型供应商、向量库、工具和提示模式装进可复用接口。 开发者不必为每个实验重写消息转换、输出解析和循环控制。 这种“集成框架”大幅降低了入门门槛，也让 Agent、Tool、Memory、Executor 成为广泛流通的工程术语。

从这些接口出发，可以分析四类抽象成本；要判断它们是否发生在具体版本，还需固定实现和故障记录。

第一，模型 API 本身变化很快。 原生 function calling、结构化输出、流式事件和服务端状态不断出现，通用包装层很容易滞后，甚至泄漏底层差异。

第二，Agent 的故障通常发生在跨层边界：提示、工具 schema、消息序列、重试、解析器和供应商响应共同作用。 封装过深时，开发者只会看到“chain failed”，却难以还原模型当时究竟看到了什么。

第三，早期 memory 概念过宽。 对话历史、检索知识、用户偏好、工具结果和执行状态有不同生命周期、可信度和更新规则。把它们都叫 memory，容易制造错误抽象。

第四，生产环境需要的不只是调用组件：还包括持久化、幂等、恢复、租户隔离、审批、流式 UI、观测和评估。 这些能力需要明确的运行语义，单有组件对象之间的连接并不足够。例如输出解析失败时，若只返回统一错误而不保留原始响应和解析器版本，开发者就无法区分模型输出变化与适配器缺陷；这是条件性反例，不是本书实跑过的 LangChain 故障。

这些成本可以解释开发者为何会选择直接调用模型 API，但本章没有社区采用数据来确定这种选择的主要原因。 LangChain 也没有简单消失。 它后来通过 LangGraph 将重点转向持久状态、确定性与 Agent 节点混合、interrupt/resume、checkpoint 和 durable execution。 官方回顾甚至明确提出，最大的竞争者始终是“不使用

框架”。LangGraph 运行时设计

因此更适合把两代设计比较为不同的责任重点，而非一条必然替代路线：

TEXT

```
集成与高层抽象
        ↓ 增加运行语义
显式状态图 + 低层运行时
        ↓
持久化、恢复、观测与部署基础设施
```

2026 年的材料继续呈现框架组合路线：Deep Agents 明确提供子代理的 isolated 与 fork 上下文模式；Microsoft 的 Harness 文档则描述如何组合历史持久化、任务清单、审批、观测和可选的有界循环。Deep Agents 上下文模式，2026-09-08 Microsoft Harness，2026-09-15 更新 这些是官方能力说明，本书未据此实测成本或可靠性；上下文取舍将在第七、十一章讨论。

这组对照对企业自研 Harness 很有提醒意义。 早期最吸引人的往往是统一接口和快速 demo； 长期最难替换的却是状态模型、事件协议和运行语义。 平台应把稳定性投入放在后者。

4. AutoGen：把工作流表达成多 Agent 对话

AutoGen 将多个可定制 Agent 的对话作为应用编排机制。 每个 Agent 可以组合模型、人类输入和工具，交互行为既可由自然语言定义，也可由代码定义。AutoGen 论文

这种设计有很强表达力。 规划者、执行者、评审者和用户代理可以用统一消息隐喻协作； 新角色可通过说明与工具配置快速加入； 人类也能作为对话参与者插入流程。

但“万物皆消息”与“万物皆文件”一样既是统一抽象，也有潜在陷阱。 如果把业务状态、权限决定、执行结果、取消信号和评审结论都退化成自然语言消息，会产生四类问题：

- 难以保证消息被恰好处理一次；
- 难以区分陈述、命令、建议和授权；
- 难以建立严格 schema 与兼容性策略；
- 难以证明终止条件和责任归属。

多 Agent 对话还容易制造“社会性拟真”：不同角色互相赞同、批评或投票，看起来像一个团队， 却可能共享同一个模型偏差、同一错误上下文和同样盲点。 增加说话者数量不自动增加独立证据。

判断对话式编排是否有价值，要看角色是否带来不同的信息、工具、权限或可并行工作，而非只看角色名。例如实现者与审查者若共享同一份错误假设，互相赞同并不增加验证证据；让审查者独立读取需求和补丁才可能提供新的判断。第十一章将用同预算基线比较这类收益与协调成本。

5. 第一次祛魅：为什么 Demo 自治不能直接进入生产

2023 年的演示通常选择开放式目标，比如“研究一个市场并建立网站”。 这类目标给了模型足够空间生成惊喜轨迹，但缺少可重复、可外部判定的完成条件。 生产系统不同：错误成本真实存在，输入分布不断变化，结果必须可审计。

“第一次祛魅”主要来自六个错配。

5.1 语言流畅度与状态正确性错配

模型能生成连贯的“当前进度”，但环境可能根本没发生对应变化。 Harness 必须用工具结果和外部查询维护 canonical state，而不是用模型叙述替代状态。

5.2 语义相似与记忆有效性错配

向量检索擅长找相似文本，但不负责内容真实性、时效性、权限范围或因果相关性。 生产记忆需要来源、时间、作用域、置信度、失效条件和删除机制。

5.3 工具可调用与动作可授权错配

工具出现在 schema 中只说明模型知道如何发起请求，不说明它有权执行。 模型选择、策略判断、用户审批与沙箱执行必须是不同步骤。

5.4 循环持续与目标进展错配

Agent 可以不断产生新的思考、搜索和任务。预算、重复检测、停滞检测和外部里程碑要共同限制循环。

5.5 自我批评与独立验证错配

同一模型阅读自己的输出并说“看起来正确”，只能提供一种弱信号。 强验证必须来自测试、类型系统、约束求解、外部数据、不同信息路径或人类判断。

5.6 多角色对话与多样性错配

共享模型、共享上下文和共享提示风格的多个 Agent 很可能产生相关错误。 有效冗余要测量错误相关性，而不是只看 Agent 数量。

6. 从 Framework 到 Runtime

从前述原型和后续运行时文档看，讨论已从组件调用延伸到运行保证。企业设计需要逐项回答：

- 状态能否持久化并从中断恢复；
- 每一步是否形成可消费的结构化事件；
- 工具执行是否幂等，失败能否安全重试；
- 人类能否在关键点暂停、修改和恢复；
- 长任务能否跨进程、跨机器和跨版本继续；
- 权限与凭证是否由模型之外的策略系统管理；
- 轨迹能否进入回放、评估和回归测试。

这些问题有助于理解 framework 与 runtime 的职责差别。LangGraph 将开发 API 与 PregelLoop runtime 分离；后续产品也各自分离控制、执行或接入协议，具体版本与来源见第三篇。这里不把它们写成从某个早期框架依次演化而来的谱系。

Framework 主要回答“开发者如何表达 Agent”； Runtime 还必须回答“表达出来的 Agent 如何长期、安全、可恢复地运行”。 两者并不互斥，但企业平台如果只拥有前者，就会在每个应用中重复构建后者。

7. 这一代系统留下了什么

AutoGPT 和 BabyAGI 留下开放循环与任务外置；LangChain 留下 Agent/Tool/Memory/Executor 词汇和集成生态；AutoGen 留下对话式多 Agent 编排；LangGraph 则代表从高层魔法回到显式状态和持久运行语义。

它们提示：最小 Agent 可以很小，生产系统所需的运行责任却不能因循环短小而省略。职责可以由已有基础设施承担，并不都要加入 Agent 核心。

TEXT

```
最小 Agent：模型 + 工具 + while loop
```

```
生产 Harness：
  最小 Agent
  + canonical state
  + context lifecycle
  + policy and sandbox
  + durable execution
  + observability
  + verification
  + human control
  + evaluation and governance
```

下一章进入 coding agent 的工程化转折。 Aider 的 repository map、SWE-agent 的 Agent-Computer Interface，以及 OpenHands 的动作—观察事件流，将反复证明一个事实：同一模型的表现，会被它所在接口和环境显著改变。 模型能力并不等于系统能力。

第三章 接口也是智能：Aider、SWE-agent 与 OpenHands

2023 年的通用自主 Agent 证明模型可以循环调用工具。它没有证明模型在大型代码仓库里就能稳定工作。软件工程要求 Agent 定位相关文件、理解跨模块关系、生成可可靠落盘的局部修改、运行测试并解释错误。模型可能知道如何写函数，却可能因为看错文件、破坏补丁格式或忽略仓库约定而失败。

这一阶段最重要的发现可以概括为：

> 裸模型单次问答的成绩不能直接当作带工具系统的上限。模型看见什么、能做什么、动作如何表达、反馈怎样返回，都会影响任务表现；收益大小取决于任务、模型和预算，需要实测。

1. Aider：上下文不是越多越好

大型仓库无法完整塞进上下文窗口。即使窗口足够大，无差别注入也会增加费用、延迟和注意力干扰。Aider 的 repository map 选择另一条路线：提取仓库中的关键符号、类型和签名，以文件依赖关系构图，再在 token 预算内通过图排序选择最相关、最重要的部分。Aider Repository Map

它实际上为模型提供了两级观察：

TEXT

```
Repo Map：低成本、全局但有损的符号地图
Full File：高成本、局部但更完整的源码观察
```

模型先借助地图理解大致结构，再请求具体文件。这比预先猜测所有相关上下文更接近主动感知：Harness 提供可导航地图，模型决定何时放大细节。

这套设计包含三个可迁移原则。

第一，摘要必须服务于后续定位。一个只“概括内容”的摘要可能读起来很好，却无法帮助 Agent 找到下一步需要读取的对象。符号名、路径、签名和依赖边是可操作的索引。

第二，上下文选择需要显式预算。不同信息不是简单的“相关/不相关”，而是在有限 token、延迟和缓存条件下竞争。Harness 应把 context assembly 看成带约束优化问题。

第三，模型应能按需深化观察。一次性静态 RAG 无法预知执行过程中产生的新问题。现代 Cursor 的动态上下文、Claude Code 的搜索工具和 Codex 的文件工具都延续了这一点。

2. 编辑格式：工具接口会占用模型能力

让模型“给出正确代码”和让它“给出 Harness 能可靠应用的修改”是两个不同任务。Aider 为 whole file、search/replace block、unified diff 等多种编辑格式建立端到端 benchmark，测量代码是否正确、格式是否可解析、修改是否成功落盘并通过测试。Aider 编辑 benchmark

这些评测提示，不同模型对 whole、diff、diff-fenced 等格式的适应可能不同。本轮尚未固定早期 function calling 与文本格式比较的完整实验版本，因此不把“更严格的结构反而更差”作为已核验的历史结论。可保留的设计问题是：复杂格式增加了解析和生成约束，是否影响解题表现，应在同任务、同预算下比较。

因此，“结构化接口必然优于文本接口”不是普遍真理。正确问题是：

- 模型是否在训练或后训练中见过这种接口？
- schema 是否贴近任务的自然结构？
- 参数生成需要多少转义、定位和重复内容？
- 接口失败能否返回可修复的局部错误？

- token、延迟、正确率和修改范围之间如何权衡？

这也是模型特定 Harness 的早期证据。同一工具为不同模型提供相同 schema，看似平台中立，实际可能让某些模型承担额外认知税。企业平台需要统一语义，但不必强迫所有模型使用完全相同的表面协议；适配层可以把模型原生动作映射到统一的内部命令。

3. SWE-agent：Agent-Computer Interface 成为设计对象

SWE-agent 把接口明确命名为 Agent-Computer Interface（ACI）。它的论文结论不是只比较模型，而是说明定制 ACI 可以显著改善 Agent 浏览仓库、编辑文件、运行测试和处理程序输出的能力。SWE-agent

ACI 类似人机交互中的 UI，但用户是语言模型。好的 ACI 需要考虑模型的行为偏好和错误模式：

- 命令集合是否小而正交；
- 观察是否包含足够定位信息；
- 大输出是否截断，截断点是否可理解；
- 文件编辑是否容易精确寻址；
- 语法或 lint 错误是否及时返回；
- shell 状态是否跨步骤保持；
- 错误消息是否告诉模型怎样恢复。

传统 API 设计主要面向确定性程序：调用者会严格遵循 schema，并能用代码处理错误。ACI 面向概率性调用者：工具说明本身是模型上下文的一部分，命令命名、示例、错误文本和输出长度都会改变策略。

所以工具质量至少有四层：

TEXT

```
Capability   是否能完成所需动作
Semantics    输入、输出和副作用是否定义清楚
Learnability 模型能否从说明与反馈学会正确使用
Governance   动作能否授权、审计、隔离和撤销
```

很多 MCP 或内部工具只完成第一层：接口“能调用”，却没有为 Agent 的可学习性与企业治理设计。

4. CodeAct：代码作为动作语言

JSON tool call 将动作限制为预定义函数及其参数。CodeAct 提出用可执行 Python 作为统一动作空间，使模型可以组合库调用、中间计算和控制流，并根据执行观察继续修订。CodeAct

代码动作的优势来自组合性。假设 Agent 需要读取数据、过滤、聚合并画图。函数调用模式可能产生多轮 payload 往返；代码模式可以在沙箱内完成多个步骤，只把必要结果送回模型。

但代码动作也扩大了风险面：

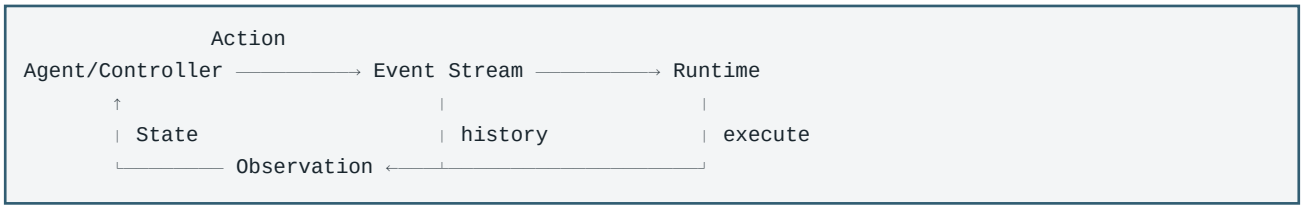
- 动作空间从有限 schema 扩大为通用程序；
- 静态权限判断更困难；
- 中间状态可能留在进程、文件或网络中；
- 可重试性和幂等性更难保证；
- 执行日志可能泄露秘密或产生巨大输出。

因此 CodeAct 不是 function calling 的普遍替代，而是一种适用于高组合性任务的 ACI。一个成熟 Harness 可以同时提供：高风险业务动作使用窄 schema 工具；数据转换和探索在受控代码沙箱中完成；确定性重复流程编译成普通程序或工作流。

5. OpenHands：把 Agent 与 Runtime 分开

本节依据早期论文和历史0.62.0实现说明一种分工：Agent根据状态产生Action，Event Stream记录Action与Observation，Runtime执行动作并返回观察。当前SDK与Agent Server已经采用新的组件边界，见第十七章，不能拿此图代替当前产品接口。OpenHands论文

TEXT



这个分离具有深远的工程意义。

Agent 是决策面：它可以替换模型、提示和策略。Runtime 是执行面：它负责进程、文件、浏览器和网络等真实副作用。Event Stream 则成为二者之间可追溯的协议事实。

如果 Agent 进程崩溃，Runtime 不一定必须销毁；如果 UI 断开，事件仍可持久化；如果要回放问题，可以重建动作—观察历史；如果要并行评估，同一 Agent 可以连接多个隔离 Runtime；如果要支持远程执行，控制面不必进入容器。

历史Docker Runtime在用户镜像中加入action-execution server，通过客户端—服务器接口传递动作和观察。它提供分离执行环境的部署接口，实际隔离取决于容器、挂载、网络和凭证配置；拆成客户端与服务器本身不证明建立了安全域。历史Runtime文档

6. Event Stream 不等于完整事件溯源

按时间记录 Action 与 Observation 很有价值，但不能看到“event stream”就自动假设系统满足严格事件溯源语义。企业实现还要回答：

- event 是否不可变，允许怎样的脱敏和删除？
- 每个 event 是否有稳定 id、因果 id 和幂等键？
- schema 如何版本化？旧 consumer 如何升级？
- 大型输出存正文还是对象存储引用？
- secret 在写日志之前还是之后脱敏？
- 并发工具和 subagent event 如何排序？
- projection 出错后能否从日志重建？
- 用户数据删除与不可变审计如何协调？

因此，动作—观察日志是基础，不是终点。参考架构将在后文区分原始 execution event、面向模型的 observation、面向 UI 的 projection 与面向审计的 security record。它们可能源自同一动作，却有不同数据保留和访问策略。

7. Benchmark 开始评价“模型 × Harness”

Aider 的 benchmark 同时评价模型解题和编辑格式；SWE-agent 比较 ACI；OpenHands 将 Agent 和 Runtime 接入多个软件工程 benchmark。这意味着评价单位开始从裸模型转向系统组合。

可以将任务成功率表示为：

TEXT

```
Success = f(Model, Harness, Environment, TaskDistribution, Budget)
```

如果只报告模型名称而不固定 Harness、环境镜像、工具版本、token 预算和重试策略，结果无法归因。反过来，如果 Harness 更新后分数提高，也不能立即说 Harness 本身更好：可能是评测污染、任务方差、模型后端变化或预算增加。

这为本书后面的 2×2 Model × Harness 评估奠定基础：固定任务集，交叉运行旧/新模型与旧/新 Harness，多次采样，并同时测量成功率、成本、延迟、安全违规和人工介入。只有这样才能区分模型收益、Harness 收益与交互效应。

8. 从接口工程得到的七条原则

这一阶段留下七条稳定原则：

- 全量上下文不是默认答案；应提供可导航的低成本地图和按需深化机制。
- 工具接口是模型行为的一部分，必须用目标模型做端到端评估。
- 表面协议可以模型特定，内部动作语义必须稳定。
- 通用代码动作适合组合计算，但必须进入更强的沙箱与审计域。
- 决策控制面与副作用执行面应具有清晰协议边界。
- Action/Observation 应成为结构化、可关联、可回放的运行时事实。
- 评估对象应是模型、Harness、环境、预算与任务分布的组合。

这些原则为比较 Claude Code、Codex、Cursor 提供了具体问题：它们如何设计上下文、编辑、命令、权限、会话和验证接口。下一章转向真实仓库与产品运行时，再由原理篇系统展开这些责任，建立可用于产品分析和自研设计的统一模型。

第四章 Coding Agent 转折：从实验接口到产品运行时

第三章说明了为什么接口会改变同一模型的能力。本章讨论另一个转折：当 Agent 进入真实仓库后，问题从“能否调用工具”变成“能否在有状态、可执行、多人协作的环境中持续交付”。这推动 Harness 从实验脚本演化为产品运行时，也迫使评测从文本答案转向可重建环境。

1. 为什么软件工程成为关键试验场

软件仓库同时提供了 Agent 研究稀缺的四样东西：持久、可差分的状态；大量可组合工具；编译器和测试形成的外部反馈；Git patch 形成的可审查产物。Agent 可以搜索、修改、执行、失败后再修复，结果还能被另一进程重建。这使“动作—观察—验证”闭环有了具体载体。

TEXT

```
issue / specification
        ↓
repository snapshot → inspect → patch → execute checks
        ↑                          ↓
        └──── diagnostic feedback ┘
                                   ↓
                    reviewable candidate artifact
```

这并不表示软件任务天然简单。依赖、隐藏约束、并发、外部服务和不完整测试让环境仍然部分可观察。区别在于失败通常留下机器可读痕迹，使 Harness 可以把高熵推理放进可重复的反馈回路。

2. 2024—2026 的产品化转向

2024 年的 SWE-agent 工作把 Agent-Computer Interface 作为独立设计轴，并展示仓库导航、编辑与测试接口会显著影响结果。SWE-agent OpenHands 同期把 Agent、EventStream 与执行 Runtime 明确分开，形成可替换模型与沙箱环境的开放平台。OpenHands paper

随后产品重心从单一终端会话扩展到多个表面和更长生命周期：Claude Code 把 hooks、skills、subagents 和 MCP 挂入 loop；Codex 把 core 通过 App Server 提供给 CLI、IDE、桌面与云端；Cursor 将 IDE 状态、动态上下文和云端异步 Agent 结合；DeepSeek Harness 把运行时组织为可替换插件图。第三篇将逐一分析这些系统。这里要强调的是共同变化：运行时开始拥有 session、权限、sandbox、压缩、版本和事件协议，不再只是十几行 ReAct 循环。

产品化还改变了完成语义。实验脚本通常在模型输出 final answer 时结束，而真实产品必须区分 turn 结束、candidate 产生、测试通过、PR 创建、人工合并和生产部署。第十章把这种区别形式化为 CompletionContract 与 EvidencePackage。

3. 从 SWE-bench 到可执行系统评测

SWE-bench 将真实 GitHub issue、仓库 revision 和测试组合成环境，评价系统是否产生可通过检查的 patch。SWE-bench 它的贡献不仅是一张排行榜，而是把评测对象从“模型生成代码片段”推进到“模型 + Harness + 环境”的完整系统。

这也意味着分数不能简单归因于模型。检索、编辑动作、上下文预算、重试、环境构建、测试 patch 和失败处理都会改变结果。两个系统即使使用同一模型，也可能因 Harness 不同而有不同分数；两个模型若使用不同 Harness，比较的是系统，不是纯模型实验。

2024 年推出的 SWE-bench Verified 对人工筛选的任务进行验证，试图减少问题描述、测试和环境质量缺陷。SWE-bench Verified 到 2026 年，OpenAI 又公开说明不再用该集合评价前沿 coding 能力，理由包括污染、测试缺陷和领先系统接近饱和，并建议转向更难、持续维护的评测。退役说明 这段演化说明：可执行测试比文本 judge 更硬，但 benchmark 本身也会老化。

4. Benchmark rot 的四种来源

第一是污染：公开 issue、patch 和讨论进入训练或检索数据。第二是饱和：任务已无法区分前沿系统。第三是基础设施腐烂：依赖、镜像或外部资源不再可重建。第四是规格缺陷：测试只覆盖部分目标，Agent 可以通过 visible check 却偏离真实需求。

一个教学反例是测试只检查函数返回值，却没检查性能或权限边界：候选可能通过硬编码或扩大读取范围得分，实际需求却未满足。它说明评测也需要维护验收规则，具体检查、污染控制与退役流程见第十、十二章。

这段历史的意义在于评价单位发生了变化：不仅要固定候选代码，还要固定测试与环境的版本。分数报告必须注明日期和适用范围，不能把旧榜单直接当作当前产品能力。

5. 排行榜为何不能直接指导企业选型

企业任务的损失函数通常不同于 benchmark。公开集合偏代码修复，组织可能更关心内部框架、合规约束、长时部署、人工复核和错误提交成本。排行榜的预算、权限、模型调用次数和网络条件也未必与生产一致。

选型 POC 至少固定：代表性任务合同、仓库和依赖 snapshot、模型/Harness 版本、最大成本与时长、权限、网络、trial 数和 verifier。报告可信完成率、稳定性、P95 时延、单位成功成本、人工接管、错误完成和安全事件，并按任务族切片。一次“做出来”的演示只证明可达性，不证明稳定运营。

YAML

```yaml
evaluation_claim:
  scope: enterprise-repo-maintenance-v3
  system: model+harness+tools+sandbox
  trials_per_task: 5
  fixed: [task, revision, permissions, verifier, budget]
  reports: [verified_success, reliability, cost, latency, human_load, safety]
```

6. Coding Agent 留下的架构遗产

这一阶段形成了现代 Harness 的五个共同原则：把仓库和环境视为权威状态；按需编译上下文；为模型设计可诊断动作接口；在隔离环境提交副作用；由外部证据判定完成。产品间差异主要在这些原则的边界和工程实现，而不是是否拥有一个循环。

Coding Agent 的历史意义，是把 Agent 从语言产品变成运行系统问题。模型仍负责提出高熵决策，文件、进程、权限、测试、状态和提交则由确定性软件承载。下一篇将把这种分工展开为系统模型、耐久循环、上下文、工具、安全、验证、多 Agent 与评测运营。

第二篇 原理：生产级 Harness 的构成

本篇导言：把概率决策装进确定性边界

本篇先给出全书的设计本体。第五章定义模型、Harness、环境与反馈的职责边界；第六至十二章再依次展开耐久循环、上下文、工具、安全、完成证据、多 Agent 和评测运营。

核心方法是明确分工：模型提出难以预先编码的判断；软件管理身份、状态、预算和外部副作用，并执行定义好的检查。可恢复任务需要保留权威状态，可审计动作需要稳定的动作与观察语义，可信评价需要明确验收规则。完成契约（CompletionContract）把验收、证据、权限、预算、变更和停止规则版本化；简单问答可采用轻量规则，高风险写入则需要独立检查和提交授权。结束一轮、等待用户或取消执行都不等于任务完成，取消也不必等待成功验收。

第五章 Agent = Model × Harness × Environment × Feedback

“Agent = Model + Harness”是一条有用的传播公式，但对企业架构仍然太粗。它容易让人把环境、验证与反馈也塞进 Harness，最终得到“除模型外一切都是 Harness”的不可操作定义。

本书采用一个乘法式系统模型。标题中的 Agent 是系统讨论的简称；严格说，乘式描述的是 Agent System 的整体表现，而非 Agent 这个执行角色：

TEXT

```
Agent System Capability
    = Model × Harness × Environment × Feedback
```

乘号表达各环节相互制约，不是精确数学关系，四项也没有可直接相乘的度量。优秀模型可能因工具缺失或错误权限而失败；Harness 可以借工具、搜索和多次尝试改变可用信息与计算过程，但具体收益仍须实测，不能从这个隐喻推出通用能力上界。不可复现环境会让正确计划执行失败，错误反馈也会把“生成了结果”误判为“结果有效”。

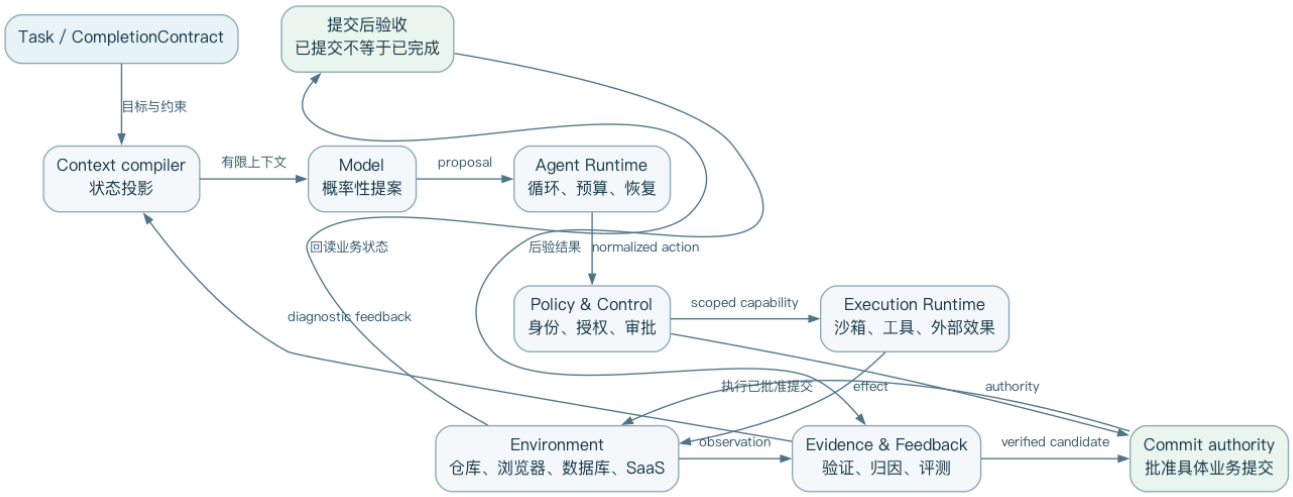


图 5-1 Agent System 的责任边界与反馈方向

1. Model：概率性策略与生成器

模型接收有限上下文，输出文本、结构化动作或代码。它擅长：

- 从非结构化目标中提出解释和候选计划；
- 在不完整信息下选择下一观察或动作；
- 生成代码、查询、文档和工具参数；
- 根据错误反馈诊断并修改方案；
- 在多个候选之间做语义比较。

模型不天然拥有持久状态、真实权限、可靠时钟、事务语义和外部世界真值。即便 API 提供 conversation id 或服务端工具，这些能力仍由模型之外服务提供。

架构上应把模型看成一种概率性决策策略，区别于规定哪些动作获准执行的授权策略：

TEXT

```
proposal ~ Model(context, action_space, sampling_policy)
```

它提出下一步，而不是直接产生已授权副作用。这个区分让平台在模型与环境之间插入验证、策略和审批。

2. Harness：运行时中介与控制系统

Harness 负责把目标、模型和环境组织成可持续执行的任务。综合现代产品与研究，本书把其责任归为十个域：

- 任务契约：目标、范围、约束、交付物、完成证据；
- 循环控制：step、turn、retry、budget、stop、cancel；
- 上下文生命周期：选择、排序、缓存、压缩和重新发现；
- 工具与动作协议：schema、调用、结果、错误、版本和发现；
- 状态与记忆：会话、执行、任务、经验和长期资产；
- 策略与授权：身份、权限、审批、凭证和数据范围；
- 执行协调：Runtime 分配、并发、超时、恢复和隔离；
- 验证与完成：测试、evaluator、证据包和停止门禁；
- 观测与归因：事件、trace、成本、失败分类和人工介入；
- 评估与进化治理：回放、回归、实验、发布、回滚和审计。

“AI Harness Engineering”同样主张能力来自 model-harness-environment system，并列出任务说明、上下文、工具、记忆、任务状态、可观测性、失败归因、验证、权限等责任。AI Harness Engineering

Harness 不必在一个进程或代码库中实现。桌面客户端、App Server、策略服务、沙箱调度器、事件存储和评估平台可以共同构成逻辑 Harness。判断边界的关键不是部署拓扑，而是谁拥有运行语义。

3. Environment：不是一个 shell，而是任务世界

环境包含 Agent 可以观察和改变的外部状态：代码仓库、文件系统、容器、浏览器、数据库、SaaS API、CI、日志、指标以及人类组织。

企业设计常犯的错误，是把“给 Agent 一个终端”当作完成环境建设。Shell 确实是高度组合的通用接口，但环境还需要：

- 可复现的依赖和初始化；
- 与任务对应的身份和网络范围；
- 可观测的应用、日志和指标；
- 快照、fork、清理和资源配额；
- 稳定的服务发现和秘密注入；
- 任务完成后可查询的权威状态。

OpenAI 的 Harness 工程实践把每个 worktree 的应用、浏览器、日志和指标都暴露给 Agent，使其能够复现和验证，而不仅是修改源码。OpenAI Harness Engineering

环境可读性是被低估的能力杠杆。与其反复提示模型“务必检查启动耗时”，不如让它查询启动 trace；与其让模型猜测页面是否正确，不如提供 DOM、截图和浏览器交互；与其把数据库错误复制进 prompt，不如提供只读诊断工具和明确 schema。

4. Feedback：观测不等于评价

工具返回 stdout 是 observation，但不一定是 feedback。Feedback 指能改变系统对“这一步或这次任务有多好”的判断信号。

可以分成四类：

类型	例子	主要用途
执行反馈	exit code、异常、HTTP 状态	即时修复动作
任务反馈	测试、验收规则、业务结果	判断是否完成
人类反馈	批准、修改、拒绝、偏好	处理价值与需求判断
群体反馈	线上指标、回归集、事故、成本	更新 Harness、技能或模型

反馈必须尽可能靠近真实目标。单元测试通过仍可能破坏用户流程；人工点“接受”可能只是没时间审查；模型 judge 的高分可能来自提示泄漏。可信系统需要多信号组合，并记录每个信号的来源和局限。

5. 为什么使用乘法而不是加法

假设两个团队使用同一模型。团队 A 提供快速但不稳定的环境、数十个含糊工具和“模型说完成即完成”；团队 B 提供小而清晰的工具集、可复现 workspace、外部测试和失败恢复。二者不是在相同 Agent 上增加少量功能，而是在构建不同能力的系统。

乘法视角带来三项决策变化。

第一，模型升级不能跳过系统回归。更强模型可能更频繁使用工具、生成更长命令或绕过旧 guardrail，导致安全和成本退化。

第二，Harness 优化必须跨模型验证。某个为模型 A 设计的提示、工具格式或压缩策略可能损害模型 B。

第三，环境与反馈是产品能力，不是“运维配套”。如果 Agent 无法观察真实系统和验证结果，它的自治上限不会因模型参数增加而自然消失。

6. Workflow、Agent 与 Harness 的关系

Anthropic 将 workflow 与 agent 区分：workflow 通过预定义代码路径编排模型和工具；agent 则让模型动态决定过程与工具使用，并建议优先使用简单、可组合模式。Building Effective Agents

Harness 可以同时运行两者：

TEXT

```
Harness Runtime
├── Deterministic workflow node
├── Model routing node
├── Open-ended agent loop
├── Human approval node
└── External evaluator node
```

企业系统不应把“更 agentic”当成天然更先进。稳定、重复、高风险流程应尽量确定化；只有步骤无法事先穷举、需要语义判断或探索时，才扩大模型决策空间。

一个实用判据是：如果下一步能由便宜、稳定、可测试的代码决定，就不应消耗模型不确定性预算。模型应集中处理无法可靠编码的判断。

7. 控制面、数据面与执行面

参考架构将 Harness 拆成三个逻辑平面。

7.1 控制面

管理配置、身份、授权策略、模型目录、工具目录、技能版本、实验、租户和发布。策略管理不必同步参与每次调用，但执行请求仍须经受信任的策略执行点校验；后者可以使用受控缓存或临时授权，不能跳过撤销、期限和动作绑定检查。

7.2 数据面

承载会话、事件、上下文、模型调用、工具请求、状态 projection 和实时交互。它决定“一次任务如何推进”。

7.3 执行面

运行命令、代码、浏览器和外部连接，承载真实副作用。它决定“动作在哪里、以什么权限发生”。

TEXT

```
                Control Plane
          policy / catalog / release
                     |
                     ▼
User ──────→ Harness Data Plane ──────→ Model
                     |
                     ▼
              Execution Plane
          sandbox / browser / APIs
```

这些平面用于区分职责与信任边界，不要求各有独立服务。允许 Agent 修改数据面的临时计划，不代表允许它修改控制面的根权限；允许执行面持有短期凭证，不代表模型上下文可以读取凭证值。后文的证据面进一步归拢数据面中的产物、轨迹和检查记录；进化面则组织候选生成、评测和发布流程，发布权仍受控制面约束。它们是对职责的进一步拆分，不是另一套互斥拓扑。

8. 概率性建议与确定性约束

企业 Harness 应区分“希望模型遵守”与“在明确配置和威胁模型下由软件强制执行”。后者也有覆盖边界，需要检验配置、实现和执行环境。

系统提示中的“不要访问生产数据库”帮助模型理解意图；网络策略和身份权限限制实际访问。提示中的“修改前请询问”可能被误解；审批状态机则记录授权主体、动作与有效期，并在执行时核对这些条件。

这些机制解决不同问题，应组合使用，不能排成一条相互替代的“证明强度”阶梯：

TEXT

```
理解意图：prompt / tool description
组织流程：workflow / mode / model routing
执行授权：policy decision / capability / IAM
限制接触面：network policy / container / VM / separate account
检查验收条件：external verification
核验记录来源与完整性：signature / protected audit
```

签名能帮助核验谁签过哪份记录，不能证明记录中的业务判断正确；验证器也只检查已定义的条件。含主观判断的任务还应指定有权验收的人或服务。越靠近不可逆副作用，越需要把授权、隔离、验收和审计组合起来；提示仍可减少无效尝试，却不能独自承担安全边界。

9. Harness 的厚与薄

现代产品存在明显分歧。DSH 主张一切皆插件，Codex 建立丰富核心与协议服务器，Cursor 进行模型特定调优；Pi 则刻意保持 read、write、edit、bash 四工具核心，不内置 MCP、subagent、plan mode、permission popup 和 background bash，把这些交给容器、tmux、技能或扩展。Pi coding agent README

“厚 Harness”通常提供一致体验、治理和开箱能力，却增加核心复杂度、模型耦合和升级风险。“薄 Harness”更透明、更容易理解和组合，却把安全与运维责任交给部署者。

判断能力是否应进入核心，可以问五个问题：

- 是否影响安全或状态正确性？
- 是否需要跨所有 Agent 保持一致语义？
- 是否位于恢复、取消或审计关键路径？
- 是否必须在模型不可用时仍然工作？
- 是否能由普通环境工具以同等可靠性提供？

前四项多为“是”时倾向核心；第五项为“是”且风险较低时倾向扩展。计划展示可以是插件，权限执行不能只靠插件约定；todo UI 可以替换，事件 idempotency 应进入底层协议。

10. 一个可实现的最小分层

对自研企业平台，建议从六层开始，而不是一次实现所有产品功能：

TEXT

```
L6 Experience      CLI / IDE / Web / API / automation
L5 Agent Patterns  loop / workflow / multi-agent / evaluator
L4 Runtime State   task / session / event / context / memory
L3 Governance      identity / policy / approval / audit
L2 Execution       sandbox / tools / browser / connectors
L1 Model Gateway   provider / routing / cache / quota
```

六层是职责分区，编号不表示严格的调用依赖。执行层在提交时校验治理层签发的受限授权，并通过状态与审计接口记录结果；Agent pattern 调用模型网关，也读取任务状态。关键交互可以补成：

TEXT

```
Agent Patterns → Runtime State / Model Gateway
Agent Patterns → Governance: 请求动作授权
Execution → Governance: 校验授权、期限、约束与资源版本
Execution → Runtime State: 记录动作结果与待对账状态
Runtime State / Execution → audit interface: 关联受保护证据
```

Experience 不直接执行 shell；Agent pattern 不直接读取生产凭证；模型网关不负责业务完成判断；执行器接受规范化动作，不自行解释自然语言意图。

早期平台可以在同一进程或数据库中实现这些职责，但应分清接口、访问权限、状态所有权和保留规则。是否拆服务、拆数据库，应由隔离、规模和故障恢复要求决定。

11. 本书的统一分析模板

后续每个产品案例都使用同一组问题：

- 核心 loop 和停止语义是什么？

- 上下文如何装配、缓存、压缩和恢复？
- 工具如何描述、发现、执行和返回错误？
- 状态和记忆存在哪里，生命周期如何？
- 权限、审批、凭证和沙箱如何分层？
- Agent 如何验证结果并声明完成？
- 多 Agent 如何隔离、委派、取消和合并？
- 客户端与 Harness 通过什么协议交互？
- 运行轨迹如何观测、评估和归因？
- 哪些部分可扩展、可替换或可进化？

这可以避免产品比较退化为功能勾选。两个产品都支持“subagent”，其委派语义可能完全不同；两个产品都支持“sandbox”，一个可能只是默认限制，另一个可能具有企业策略和临时提权；两个产品都支持“memory”，保存的可能分别是聊天摘要、用户偏好或可执行技能。

本章得到的核心结论是：Harness 不是提示词集合，也不只是 while loop。它是模型与环境之间负责运行语义、信任边界和证据闭环的控制系统。下一章将把最核心的 agent loop 展开为状态机，讨论 turn、step、stream、cancel、retry、compaction 和 crash recovery 如何共同决定长任务是否真正可运行。

第六章 Agent Loop：从 while 循环到持久状态机

工具型 Agent 的主循环看起来通常可以用十几行伪代码写清。真正难处理的是执行中的故障，而不是那十几行正常路径。并行工具只完成一半时怎么办？用户在命令执行中取消怎么办？模型给出 final answer 就算结束吗？外部副作用提交后、结果还没落库进程就崩了怎么办？

需要跨进程恢复、审批或提交业务副作用的企业任务，应将这些状态持久化。短暂、无副作用、失败后可从头重算的任务可以使用内存循环，不必为了“企业级”统一增加一套数据库。

1. Thread、Turn、Step 与 Attempt

现代产品对术语并不完全一致。本书的业务执行单位如下；会话Thread是另一种组织方式，不是所有对象的唯一父节点：

TEXT

```
Task          一份完成契约下的业务任务
└── Attempt   使用特定运行时、版本和工作区完成该任务的一次尝试
    └── Turn/Step/Action   会话轮次、可观测阶段和具体动作
        └── ToolTry        同一逻辑动作的传输或工具重试

Thread        协作会话；可承载多个Turn，并映射到不同Task/Attempt
```

Codex 把用户输入到最终 assistant message 的过程称为 turn。其中可以包含多次模型推理和工具调用。Codex Agent Loop Claude Agent SDK 也循环执行“评估—工具—结果”，直到模型输出不含工具调用的响应，并返回包含 token、成本和 session id 的结果消息。Claude Agent Loop

这里的Attempt与附录A一致，表示任务的一次执行尝试。一次API超时后的传输重试记作ToolTry，不另建业务Attempt；它必须沿用同一逻辑动作的幂等键。模型改用了不同方案，则形成新Action，是否产生新业务操作还要由契约判断。分开这些身份后，系统才不会把“重发同一请求”误当成“允许再做一次”。

2. 最小状态机

下面是需要审批和恢复的任务尝试的一种状态模型。Turn结束是会话事件；图中验收与业务终态属于Task/Attempt，不能直接映射成供应商的turn/completed：

TEXT

```
CREATED
  ↓
ASSEMBLING_CONTEXT
  ↓
WAITING_FOR_MODEL
  ↓
DECIDING
  ├──→ WAITING_FOR_APPROVAL
  ├──→ EXECUTING_TOOL
  ├──→ WAITING_FOR_USER
  ├──→ COMPACTING
  └──→ VERIFYING
            ├──→ CANDIDATE_VERIFIED ──→ 按契约决定提交和后验
            ├──→ NEEDS_REPAIR ──→ ASSEMBLING_CONTEXT
            └──→ FAILED

任意活动状态 ──→ CANCELLING ──→ CANCELLED
任意活动状态 ──→ SUSPENDED ──→ 恢复点
```

FAILED、CANCELLED 和 COMPLETED 不能混写。用户取消不代表模型失败；预算耗尽也不代表业务目标不可达成；工具被拒绝可能是策略命中，不一定是技术问题。状态会直接决定能否重试、是否计费、如何告警，以及下次恢复时模型该看到什么。

3. 模型停止不等于任务完成

模型输出无 tool call 的 assistant message，通常结束当前 loop。但它可能只是提问、报告阻塞、拒绝任务，或错误地喊“完成”。

所以至少需要两个判定：

TEXT

```
model_stop       模型本轮不再请求动作
task_completion  外部完成契约已经满足
```

Harness 可按任务类型选择完成策略：

- 对话问答：final response 可能足够；
- 代码修改：要求工作区有预期 diff，并运行指定测试；
- 数据任务：要求产生可复现查询、结果和口径说明；
- 高风险业务动作：要求外部系统确认与审计事件；
- 研究任务：要求来源覆盖、引用验证和未知项披露。

开发检查或获准提供修复反馈的验证集失败时，系统可以回传结构化差异。密封终测失败则停止本轮选择，不把诊断直接喂回候选；若授权降级为开发材料，后续必须重新建立独立终测证据。

4. Streaming 是事件协议，不只是打字动画

长任务需要实时展示模型文本、工具请求、命令输出、审批和状态变化。若 streaming 只是不可恢复的 socket 文本，客户端断线后就无法判断哪些内容被遗漏。

建议每个流事件包含：

TEXT

```
event_id
thread_id / turn_id / step_id / attempt_id
sequence_or_cursor
event_type
timestamp
payload_ref
causation_id / correlation_id
schema_version
visibility_class
```

平台可让客户端通过cursor重连，由服务端重放已持久化事件；大载荷放对象存储，事件保留hash、类型和访问引用。需要另外声明哪些消息只是即时预览，哪些已落库：看到增量文本不等于崩溃后一定能重放。原始事件可以分别供模型、界面和审计使用，三者各自执行脱敏和访问控制。平台的游标保证也不能超过供应商实际导出的事件范围。

5. 取消必须贯穿调用链

用户点击停止后，仅停止下一次模型调用是不够的。正在运行的 shell、浏览器任务、subagent 和远程 job 可能继续带来副作用。

取消应沿所有权树传播：

TEXT

```
Turn cancellation
├── model request cancellation
├── active tool cancellation
|   ├── process group termination
|   └── remote job cancel request
├── child agent cancellation
└── pending approval invalidation
```

取消还有竞态。动作可能在取消信号到达前已提交。状态不能简单写成“cancelled, nothing happened”，而应记录 cancel_requested_at、effect_committed_at 和最终 reconciliation。对不可撤销动作，应返回“取消请求已接收，但动作已提交”。

6. Timeout、Retry、Resume 不是一回事

Timeout是放弃等待一次工具尝试或模型请求；Retry是重新尝试同一个逻辑动作；Resume是从持久状态恢复任务尝试。超时不撤销已经发出的请求，重试不产生新的业务授权，恢复也不是把旧凭证原样放回进程。

自动重试写动作前，需要确认它天然幂等，下游按同一幂等键原子去重，或权威证据确认前次未生效且以后也不可能再生效。本地账本里没有结果、一次查询暂时找不到记录，都不满足最后一项条件。

结果未知时先对账；无法确认时暂停并升级。补偿处理的是已经确认的错误效果，而且补偿也可能失败。重复扣款即便能退款，仍可能占用资金、发出通知或产生费用，因此“有退款接口”不是可以盲目再扣一次的理由。

AWS Durable Execution 文档把幂等定义为重复运行仍产生相同效果，并强调执行确认和 checkpoint 对重试语义的重要性。Idempotency and Retries

模型请求可按策略重试，但每次输出不保证一致，已计费的尝试也不能从成本账中消失。只读调用通常可以重试，仍需考虑快照漂移、限流与读访问审计；发送邮件、付款和合并代码更不能仅因网络超时就重放。

7. 副作用账本与不确定提交

最危险的崩溃窗口是：外部动作已执行，但 Harness 尚未保存结果。

TEXT

```
persist INTENT
authorize exact action
execute external effect
persist OUTCOME
```

如果崩溃发生在第三步到第四步之间，恢复时只知道 intent，不知道 effect 是否发生。它不确定提交状态，不是通过再问模型一次就能修复。

副作用账本至少记录租户、目标、动作、参数hash、幂等键、授权版本、开始时间、下游请求ID和已知结果。相同键配不同参数应拒绝。账本的insert-if-absent只防重复记录，不会阻止两个worker都通过“查无结果”后执行；需要执行所有权与目标系统原子去重共同约束。

异常路径也属于协议。如果executor抛TimeoutError而不是返回一个timeout对象，调用者仍须把已经开始发送的动作记为UNKNOWN_EFFECT。若进程直接死亡而来不及记，恢复器把遗留EXECUTING当成未知效果。对下游不支持去重的系统，只有权威确认旧请求不再可能生效后才可重发；否则交给人工，不承诺跨系统exactly-once。

8. Checkpoint 应保存什么

仅保存聊天历史不足以恢复执行。Checkpoint 至少覆盖：

- 当前 thread/turn/step/attempt 状态；
- canonical task state 与未满足验收条件；
- 已装配上下文的版本或可重建引用；
- 模型、工具、技能、策略和环境版本；
- 已提交与未确定的副作用；
- 活动 child agent 和 remote job handle；
- budget 使用量与剩余额度；
- 当前 workspace snapshot/commit/image 标识。

Claude Code 把消息、工具调用和结果写入 JSONL，支持 resume、rewind 和 fork；这对个人会话很有效。Claude Code Sessions 企业平台还需要数据库一致性、租户隔离、加密、保留策略和跨版本迁移。

9. Compaction 是有损状态迁移

上下文接近上限时，Claude SDK 与 Codex 都会压缩历史。Codex 强调缓存依赖精确前缀匹配，并尽量用追加消息表达中途配置变化；其服务端 compaction 会用较短 items 替代旧 input。Codex Agent Loop

压缩不是普通摘要，而是一次有损状态迁移。至少要保护：

- 用户目标和后续修订；
- 权限与安全约束；
- 已验证事实及来源；
- 未完成承诺和阻塞；
- workspace 关键变化；
- 失败尝试及不应重复的原因；
- 可重新发现原始记录的指针。

可采用两份职责不同的记录：模型上下文允许压缩，权威执行状态和原始证据保持独立。检查不能只发生在压缩前；新投影形成后，还要核对未完成工具调用的配对、契约与权限版本，以及关键信息能否重新检索，检查成功后才切换上下文。模型需要细节时从原记录重读。

10. Error Taxonomy 决定恢复策略

错误不应全部变成一段 tool result。建议至少分类：

类别	例子	默认策略
Model transient	限流、网断	退避重试/切换
Model semantic	无效工具参数	结构化反馈给模型
Policy denied	权限不足	记录拒绝；只有策略允许的升级路径才能再申请
Tool transient	服务超时	先判定有无未知效果，再按幂等条件重试
Tool deterministic	参数非法、文件不存在	修正输入
Environment drift	依赖/分支变化	重新观察或重建环境

类别	例子	默认策略
Verification failure	检查失败	按数据用途决定修复或停止；终测反馈不用于本轮修复
Budget exhausted	token/时间/费用耗尽	暂停并报告
Unknown effect	提交状态不明	reconciliation/人工

错误类型应由确定性层尽量识别。让模型从任意 stderr 猜“要不要重试”，通常会引发 retry storm，并放大重复副作用。

11. 并行工具与一致性

模型可能一次请求多个工具。默认并行只适合互不影响的只读观察。多个写入若基于同一旧状态，容易产生冲突。

Harness 可在执行前计算 action footprint：读取集、写入集、外部目标、凭证域和工作区。若 footprint 重叠，则串行执行、隔离到不同 workspace，或用乐观并发控制并在提交时检查版本。

同理，多 Agent 并行不应共享未经协调的可变目录。Codex 使用 worktree 隔离不同线程，能把“任意文件覆盖”转成“显式合并”问题。

12. 一个更完整的循环伪代码

TEXT

```
function run_turn(turn_id):
    state = restore_or_create_state(turn_id)
    if terminal(state):
        return state.status

    while not terminal(state):
        if cancellation_requested(state):
            return cancel_and_reconcile(state)
        if budget_exhausted(state):
            return suspend_with_checkpoint(state, BUDGET_EXHAUSTED)

        if context_needs_compaction(state):
            projection = compact_model_view(state)
            validate_projection_against_canonical_state(projection, state)
            install_context_projection(projection)

        proposal = call_model_with_budget(build_context(state))
        persist(proposal)

        if proposal.requests_actions:
            for action in plan_execution(proposal.actions):
                state = refresh_state_and_revocations(state)
                if cancellation_requested(state):
                    return cancel_and_reconcile(state)
                if budget_exhausted(state):
                    return suspend_with_checkpoint(state, BUDGET_EXHAUSTED)
                validate_schema(action)
                decision = decode_policy_decision(authorize(action, state))
                if decision.decision == REQUIRE_APPROVAL:
                    persist_bound_approval_request(action, decision)
                    return WAITING_FOR_APPROVAL
                if decision.decision not in {ALLOW, CONSTRAINED_ALLOW}:
                    persist_denied_observation(action, decision)
                    continue
                if not constraints_enforceable(action, decision):
                    persist_denied_observation(action, CONSTRAINT_UNAVAILABLE)
```

```
                    continue
                reservation = reserve_action_budget_or_none(state, action)
                if reservation is None:
                    return suspend_with_checkpoint(state, BUDGET_EXHAUSTED)
                try:
                    outcome = execute_with_effect_ledger(action, decision)
                    persist_and_reduce(outcome)
                finally:
                    settle_actual_cost_and_release_unused(reservation)
                if outcome.status == UNKNOWN_EFFECT:
                    return RECONCILE_REQUIRED
                if outcome.status == PENDING:
                    return WAITING_FOR_EFFECT
            continue

        if proposal.requests_user_input:
            return WAITING_FOR_USER
        if not proposal.has_candidate:
            return TURN_STOPPED
        verification = evaluate_completion_contract(state, proposal)
        if verification.passed:
            return CANDIDATE_VERIFIED
        if not verification.feedback_allowed or not verification.repairable or not repair_bu
dget_remaining(state):
            return BLOCKED_OR_ESCALATED
        append_structured_feedback_and_charge_attempt(verification)
```

这段伪代码的helper有明确责任：模型调用和工具调用都预留并结算预算；拒绝记录包含action_id；执行器在提交前再次检查已撤销授权，并将发送后异常变成未知效果。审批请求绑定参数、资源与契约版本；收到外部答复后恢复原待执行动作，重新授权，不让模型换一个动作套用旧批准。暂停分支直接返回，不能继续执行批次里的后续动作。feedback_allowed还要核对数据用途、实验族反馈余额和接收主体，密封终测固定为否。具体的副作用和恢复契约见附录A，最小可运行参考在仓库examples/。

13. 最小可靠性测试集

声称支持持久恢复的实现，应按自身能力执行以下故障测试。涉及浏览器、远端作业或多Agent的条目在实际启用这些能力时适用：

- 模型返回工具调用后立即崩溃；
- 工具已提交副作用、结果落库前崩溃；
- 并行工具一个成功一个超时；
- 等待审批时服务重启；
- 用户取消与动作提交同时发生；
- compaction 前后继续同一任务；
- 恢复时模型、工具或策略版本已改变；
- child agent 完成但 parent 丢失连接；
- streaming 客户端断线后按 cursor 重连；
- 相同 idempotency key 被并发提交。

只演示正常路径，无法支持崩溃恢复能力的结论。本书的本地参考验证了选定状态与故障模型；它不能代替针对实际进程、网络、供应商服务和存储后端的故障注入。

长任务应有独立于模型上下文的执行身份和可查询副作用记录。需要通过完成契约的是“任务成功”这一声明；提问、拒绝、预算耗尽和用户取消都可以结束本轮，但应保留真实状态。下一章讨论如何在这样的状态记录之上，为模型提供当前需要的信息。

第七章 上下文、缓存、压缩与记忆

模型每轮推理只拿到当前上下文来行动。对 Harness 来说，目标不是“记住一切”。更关键的是，在正确时刻把可信、相关、成本可接受的信息放进模型可见范围，同时把原始事实留好，方便后续复用和核验。

上下文系统常见的问题是把对话历史、任务状态、知识检索、用户偏好、长期经验和可执行技能都塞进一个“memory”里。这些材料的信任等级、生命周期和更新权限不同。下面再按存储职责区分五类信息。

1. 五种必须分开的信息

TEXT

```
Working Context   当前模型请求实际看见的内容
Conversation Log  用户、模型、工具交互的原始历史
Canonical State   任务、约束、执行与副作用的权威状态
Long-term Memory  跨会话复用的事实、偏好和经验
Skills/Artifacts  可执行程序、流程、模板和文档资产
```

Working Context 是临时编译产物，可以压缩和重排。 Conversation Log 用于审计、恢复和重放。 Canonical State 不能依赖模型摘要保持正确。 Long-term Memory 要有来源和失效规则。 Skills 要有版本、权限治理和供应链管理。

如果五类混在一起，压缩可能删掉任务约束。模型写入的猜测可能被误当长期事实。删聊天记录可能顺手删掉审计状态。未经审查的 memory 甚至可能在未来会话里反复注入恶意指令。

2. Context Assembly 是一次编译

每次模型调用前，Harness 都要把上下文从多源编译成可用输入：

TEXT

```
Context = compile(
  system_contract,
  policy_view,
  task_state,
  recent_events,
  retrieved_knowledge,
  active_skills,
  tool_catalog,
  environment_snapshot,
  token_budget
)
```

“编译”意味着过程可重复、可观测、可测试。每段上下文都要记录来源、版本、token 数、选择理由和可见性。出现问题时，工程师才知道模型到底看见了哪版规则、哪些文件、哪些记忆，而不是只留下一段最终 prompt。

Context compiler 还要处理冲突优先级。组织策略、项目规则、用户本轮要求和旧 memory 如果冲突，不能指望它们在 prompt 中的顺序决定结果。确定性层应先判断冲突，再把明确且最小的约束给模型。

子代理的上下文还取决于它与当前工作的关系。Deep Agents 在 2026-09-08 的官方说明中区分 isolated 和 fork：前者从独立任务说明开始，后者继承父状态与历史，将末尾委派调用改写为子任务消息。已完成调查后的修复者可能受益于继承，独立审阅者则更适合只接收需求、补丁和证据，避免先入为主。Deep Agents 上下文模式 这是文档描述的机制；父窗口更短、系统总 token 更少、审阅更可靠仍须分别测量，不能互相代替。

3. 长窗口不是无限注意力

“Lost in the Middle” 研究显示，长上下文模型对信息位置敏感。相关内容位于中间时，模型的表现可能显著下降。Lost in the Middle

这不代表所有现代模型都会同样失效，但它否定了“只要能放下就能等同理解”的假设。长上下文还带来成本上升、延迟增加、缓存失效和矛盾信息增多。

因此上下文要用四类预算管理：

- 容量预算：窗口最多能放多少；
- 注意力预算：模型能否稳定利用；
- 经济预算：输入与 cache read 成本；
- 变化预算：哪些片段变化会破坏前缀缓存。

相关性不是唯一排序依据。任务目标和安全约束即便语义上看似不接近当前动作，也必须保留。最近错误有时比历史成功案例更重要。已失效记忆再相似也要排除。

4. 静态上下文与动态发现

静态上下文每次调用都加载，适合短小稳定、重复高的信息，例如当前目录、关键任务契约、少量项目规则。 动态上下文按需加载，适合大体量、变动快或低频的信息，比如文件、搜索结果、工具查询。

Cursor 描述了从“批量静态注入”向“动态发现”的路径：把长工具输出写入文件，保留文件引用，MCP 描述按需读取，终端输出同步文件系统供 grep。Dynamic Context Discovery

文件在本章里不是万能解法，而是一个清晰、可寻址、可分页、可检索的外部存储。这样可以避免 payload 永久占满 prompt，同时支持模型按需重读原始证据。

常用策略是：

TEXT

```
Always-on: 任务契约、关键策略、当前状态摘要
Index:     文件地图、技能目录、工具目录、记忆索引
On-demand: 原始文件、日志、历史、完整工具 schema
Pinned:    本轮验证所需证据与未解决错误
```

5. Prompt Cache 是架构约束

缓存命中通常依赖可复用前缀。Codex 的公开循环说明讨论了工具顺序、模型、沙箱和工作目录变化对缓存的影响，因此静态内容宜放在前面，兼容的状态更新可追加到后面。Codex Agent Loop 这属于具体实现下的优化，不能推成所有接口参数都不可变化。

所以 context assembly 既要兼顾语义，也要兼顾布局稳定。

TEXT

```
[stable system + stable tools + stable project rules]
[session history with append-only changes]
[latest dynamic observations]
[current user request]
```

工具目录来自动态 MCP 时要稳定排序，并谨慎处理 tools/list_changed。一次不相关工具发现变化，就可能打掉整段长会话缓存收益。

对支持追加更新且语义不冲突的状态，可以保留稳定前缀。旧高优先级规则、工具 schema 或环境说明若已失效，应重建相应上下文并接受缓存失效。权限变更则始终在执行层即时生效，不能等待模型读懂新消息。

接口本身也在变化：OpenAI 2026-09-03 的 Responses API 更新说明，GPT-6 Astra 支持异步工具调用、执行中追加指令，以及在保留缓存前缀的同时调整会话中的推理强度；9 月 8 日又公布适用于 GPT-5.6 及后续受支持模型的缓存诊断功能。API 发布记录，2026-09-19 快照 这些是厂商声明的接口能力，本书未实测命中率。适配器应记录模型、端点和配置版本，分别核对哪些变更影响语义、哪些影响缓存。

6. Compaction 是有损编译，不是删除旧消息

上下文接近阈值时，Harness 可把旧历史压缩成结构化摘要。Claude Code 会发 compaction boundary；Codex 的服务端 compact 用更短输入替代旧 input。Claude Code 之后会重新注入系统 prompt、根级项目规则和 auto memory，但路径细节仍需再次读取相关文件恢复。Claude Context Window

压缩摘要至少包含：

TEXT

```
Goal and acceptance criteria
Confirmed constraints and approvals
Observed facts with source pointers
Workspace/environment changes
Decisions and rationale
Failed attempts and why
Open questions and blockers
Active plan and next action
Pointers to raw history/artifacts
```

压缩前先从权威任务状态提取基线：目标、已批准约束、未完成项和证据引用。压缩后再逐项对照新上下文，并检查引用是否可读；失败就回退或重建，不能只在压缩前检查。高风险任务可在边界处保存检查点，但摘要及其哈希本身不等于完整恢复状态。

以仓库任务 TASK2048 为教学例子：用户只批准改分支，尚未批准合并。若新摘要保留“测试通过”却漏掉合并限制，连续性检查应拒绝该摘要，重新从任务状态生成模型视图；执行层同时继续拒绝未经授权的合并。Anthropic 2026-09-14 发布的按需压缩 beta 接口可返回签名的 compaction block；签名可用于核验来源与完整性，不能代替上述语义检查。API 发布记录

反复压缩会积累失真。应尽量从原始事件和 canonical state 重建新摘要，不要只基于上一次摘要叠加。若成本限制，至少要保留可回源引用并定期重建。

7. MemGPT 与分层记忆

MemGPT 借鉴操作系统分层存储思路，让模型在有限主上下文和外部存储之间移动，并用中断机制控制流转。MemGPT

这个类比有价值，但不能把模型当内核。让模型自己完全决定写入、保留、淘汰，会放大偏差。企业实现通常应让策略层与模型协作：

- 模型提议候选记忆和理由；
- 确定性层校验来源、作用域和敏感等级；
- 高风险或共享记忆进人工审批；
- 检索时同时看相关性、时效、可信度和权限；
- 以反馈更新 memory utility，而不是只按访问频率。

8. 记忆类型与写入权限

记忆类型	示例	推荐写入者	典型失效条件
用户偏好	输出格式、常用语言	用户确认/模型建议	用户修改
项目事实	构建命令、架构约束	人或验证工具	仓库版本变化
情景经验	某错误的修复轨迹	Agent 候选 + evaluator	环境/版本变化
程序知识	调试 SOP、发布流程	评审后发布	流程版本更新
任务状态	当前步骤、阻塞	Runtime	任务结束
安全策略	禁止目标、审批规则	管理控制面	策略发布

安全策略不能当普通 memory 让模型改写；任务状态也不能靠语义检索恢复。 不同类型应有明确的访问权限和生命周期，可以通过同一存储中的逻辑分区实现，不必一律拆库。

Claude Code 明确说明 CLAUDE.md 与 auto memory 只是上下文，不是强制配置。Claude Memory 这点很重要：项目文件里写规则后，真正的访问控制仍应由策略、IAM 和沙箱执行。

9. Memory Poisoning 与程序漂移

长期记忆会持续影响未来行为。攻击者只要让 Agent 写入一次恶意或错误经验，就可能反复触发偏差。常见风险有：

- 把外部文档命令误写成项目规则；
- 将一次偶然修复错误地上升为通用流程；
- 记下过期凭证位置或敏感数据；
- 通过共享 memory 影响其他租户或角色；
- 多次自动总结后形成程序漂移。

每项记忆应保留来源、作用域、有效期、审查状态、作者和支持证据，并提供撤销路径。共享范围越大，晋级门槛应越高。

TEXT

```
episode note → candidate lesson → evaluated skill → reviewed org standard
```

不能从一次轨迹直接升级为组织级规则。MemSecBench 的 v1 用 Write—Execute—Forget 流程考察恶意内容的持久化、后续行为后果和选择性修复，而非只检查入库时是否合格。MemSecBench v1 据此，本书建议撤销时沿来源记录检查已有会话、摘要、检索索引与派生技能：删除原条目后，还要验证后续任务是否继续采用它。派生对象的排查是设计建议，不是本书已复现的论文实验。

10. 检索不是只有向量相似度

推荐采用多阶段检索：

- 按租户、项目、身份、时间和类型先做硬过滤；
- 用关键词、图关系和向量召回候选；
- 按相关性、可信度、时效、成本和风险重排；
- 去重并识别冲突；
- 以带来源标签片段入上下文；

- 记录是否被使用及结果反馈。

检索结果要被标记为“外部证据”，不是系统指令。网页或文档里的 prompt injection 不该因为检索命中而直接升指令优先级。

11. Skills 不是 Memory 的别名

Skill 通常包含可执行或程序化知识：说明、脚本、模板、工具依赖和资产。它比自然语言 memory 具备更高能力，但供应链风险也更高。

Skill 至少应有：

- manifest 与版本；
- 触发条件与能力声明；
- 所需工具、网络和文件权限；
- 安装来源、签名或审核记录；
- 测试与兼容矩阵；
- 执行时最小权限；
- 退役和回滚策略。

技能正文可按需加载，避免永久占据全部上下文。Claude Code 与 Cursor 的相关设计体现了按需发现的方向；常驻哪些字段、何时加载完整内容，需按具体产品版本核对，不能假定实现完全相同。

12. Context Quality 的评价

上下文质量不能只看 token 省钱。至少看这些指标：

指标	含义
Recall of required evidence	必需信息是否被选中
Context precision	独立标注的相关注入单元 / 全部注入单元
Context noise rate	无关注入单元 / 全部注入单元
Constraint retention	压缩后约束是否保持
Provenance coverage	事实是否可回源
Cache hit rate	稳定前缀复用程度
Context latency/cost	装配和推理代价
Memory usefulness	召回后是否提升任务结果
Poisoning rate	不可信内容晋级比例

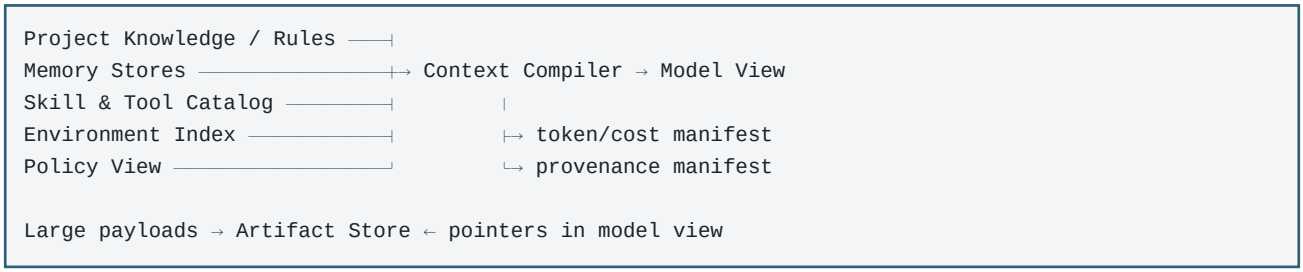
先约定注入单元是片段、token还是证据项。只有同一单位下的相关/无关分类互斥且覆盖全部单元时，精确率与噪声率才互补；不能与按技能激活事件统计的精度混用。

还要做对照评测：移除某段上下文后结果是否变化，交换位置后是否出现位置偏差，注入冲突信息后能否识别。压缩测试应从同一个中间检查点分叉，让未压缩与压缩后的上下文继续执行相同后续步骤；从头重跑整个任务会漏掉中途恢复缺陷。涉及写动作时使用隔离环境或模拟器，避免测试本身重复提交。

13. 推荐的上下文架构

TEXT

```
Raw Event Log ──────────────┐
Canonical Task State ───────┤
```



上下文编译器只读取任务状态，生成给模型看的视图，不负责改写权威事实。模型说“测试通过”时，系统不能据此更新任务数据库；只有测试工具的结果经校验，并绑定到当前补丁版本后，才能记录这项事实。后续摘要可以删去详细日志，但应保留结果和原始日志的可读引用。

14. 最小验收清单

- 能解释每段上下文入选原因；
- 原始历史和模型摘要分离；
- canonical state 不依赖摘要；
- 不同 memory 类型有独立作用域和写入权限；
- 大 payload 可外置并按需读取；
- compaction 前后有连续性测试；
- 动态工具和技能目录稳定排序并可按需发现；
- memory 有 provenance、TTL、撤销和晋级流程；
- 外部内容不能直接提升为系统指令；
- context 策略在不同模型上单独评估。

本章结论是：上下文窗口是模型工作集，不是系统数据库。Memory 是经过治理后的跨时复用机制，不是聊天历史的同义词。下一章将讨论工具层：Action schema、错误协议、MCP、CLI、Code Mode 与能力发现如何共同定义 Agent 的“可行动世界”。

第八章 工具、ACI、MCP 与 Code Mode

工具决定 Agent 能真正执行哪些动作。模型即使理解任务，如果可用工具模糊、冗余或高风险，体验上也会像“能力不足”；设计良好的 ACI 能让模型更容易观察环境、执行动作，并根据错误反馈修正操作。这里的 ACI 沿用第三章的 Agent-Computer Interface。

企业平台不应只看“接入了多少工具”来衡量成熟度，而要看动作语义是否稳定、权限是否清晰、结果是否可验证、失败后是否可恢复。

1. Tool Definition 只是起点

典型工具包含：

TEXT

```
name
description
input_schema
output_schema
side_effect_class
permission_requirements
timeout/retry policy
version
```

多数模型 API 只要求前三项，但企业 Harness 需要后续运行时元数据。否则策略层无法判断工具是否只读，重试器也不知道是否幂等，观测系统也不知道该如何脱敏，兼容层更不知道 schema 是否已变更。

建议把工具拆成两层：

TEXT

```
Model-facing Tool View      为具体模型优化的名字、说明与 schema
Canonical Action Contract 平台内部稳定的动作类型、语义和治理元数据
```

模型表面可以因模型族而变化，内部动作契约则保持稳定。适配层由此既能贴近模型习惯，也能保留统一的审计、权限和评估能力。

2. 好工具的十个条件

- 名称能准确表达动作和对象；
- 描述说明何时可以用，也说明何时不该用；
- 输入 schema 要小而明确，避免多种互斥模式放进一个对象；
- 输出同时包含模型友好摘要和结构化数据；
- 错误能区分可修复输入错误、策略拒绝和系统故障；
- 副作用范围必须可预估；
- 声明取消、超时和重试语义，包括能否幂等及未知结果如何对账；
- 结果包含来源、时间和目标标识；
- 版本变化有兼容策略；
- 能在真实模型和真实任务上端到端评估。

工具说明本身属于上下文。过长说明会占用 token，过短又容易含糊，导致误用。最好的说明不是完整 API 文档，而是支持“选对工具”和“首次调用成功”的最小契约；复杂细节应按需再发现。

3. 错误协议是 ACI 的一部分

模型能否自我修复，很大程度取决于错误是否结构化。推荐返回：

JSON

```
{
  "status": "failed",
  "category": "invalid_argument",
  "retryable": false,
  "message": "line_end must be >= line_start",
  "field_errors": [{"path": "line_end", "code": "range"}],
  "suggested_fix": "Use line_end >= 42",
  "effect_committed": false
}
```

协议错误表示客户端和服务器无法通信；工具执行错误表示调用已被理解，但业务执行失败。MCP 2025-11-25 变更也明确强调，输入校验错误应作为 Tool Execution Error 返回，以便模型自我修正，而不是当作协议错误处理。MCP Changelog

effect_committed 或等价状态非常关键。若状态未知，Harness 不应自动重试写动作。

4. Tool Result 不应只有字符串

纯文本便于模型阅读，却不便于程序、界面和评估器使用；巨大 JSON 便于机器处理，却可能挤占上下文。建议结果分层：

TEXT

```
summary            短模型观察
structured_content 可验证字段
artifact_refs      大内容、文件、图像和日志引用
provenance         来源、目标、时间、版本
execution_meta     时延、attempt、request id、effect 状态
```

MCP 的 ToolResult 可以携带文本、图像、音频、资源链接、嵌入资源和可选 structuredContent，并用 isError 标记执行错误。MCP Schema

模型上下文通常只需要 summary 和少量结构字段；审计与 evaluator 则通过 artifact ref 读取完整结果。

5. MCP 解决的是互操作，不是全部 Harness 问题

MCP采用host-client-server架构：Host管理模型集成、连接权限、用户授权和上下文聚合，Client连接Server，Server提供工具、资源和提示等能力。这里要标明版次：2025-06-18规范描述有状态会话，2026-07-28架构文档已将MCP描述为无状态协议，每个请求携带自己的协议版本和能力。协议没有会话依赖，不等于业务任务不需要持久状态。旧版架构、2026-07-28架构

本章前后的2025-11-25 schema与授权引用保留其版次，用于说明当时的具体接口，不能无条件当作最新规范。接入已有服务时，应按双方实际支持的版本做兼容测试，再决定会话、能力协商和恢复怎样映射到平台。既不能把旧客户端的持久会话假设强加给新接口，也不能只改一个版本标签便宣布迁移完成。

它的重要价值包括：

- 统一能力发现和 JSON-RPC 消息；

- 显式 capability negotiation（能力协商）；
- 本地 stdio 与远程 HTTP server；
- 工具、资源、提示和客户端 sampling/elicitation（采样与澄清）；
- 独立演化的客户端与服务器生态。

但 MCP 不替 Host 决定：是否批准调用、用哪个身份、是否允许访问某数据、结果如何进入上下文、工具是否幂等、任务是否完成。官方架构也把连接权限、安全策略和用户授权放在 Host。

因此企业平台应把 MCP 看成插件与连接协议，而不是安全边界本身。

6. MCP 的安全边界

MCP 的远程 HTTP 授权规范基于 OAuth 2.1，规定受保护资源元数据与令牌受众绑定，并禁止将收到的令牌直接透传给下游服务。这样可以限制令牌被用错目标，以及中间服务替调用者滥用权限的风险；协议合规仍不等于业务动作已获授权。MCP Authorization

即便协议正确实现，平台仍需治理：

- 哪些 Server 可安装；
- Server 发布者与代码供应链是否可信；
- 每个租户和 Agent 可见哪些工具；
- 凭证由谁持有和刷新；
- 工具输出如何分类与脱敏；
- 服务器说明是否含提示注入；
- 工具目录变化是否需要重新审批、重建上下文或使缓存失效；
- 本地 stdio Server 是否能访问宿主机秘密。

“MCP Server 在本地运行”不代表安全。它可能继承用户环境变量和文件权限，供应链风险甚至高于受控远程服务。

7. Tool Discovery：工具也需要分页

数百个工具 schema 会消耗大量上下文并降低选择准确率。Claude Code 默认延迟加载 MCP 工具，只让名称或类别进入初始上下文，由 Tool Search（工具检索）找到相关 schema；官方文档给出的经验是，较大工具集适合搜索，少量工具直接加载更快。Claude Tool Search

Tool discovery 可以类比数据库索引：

TEXT

```
Catalog summary → search(query, policy_scope) → candidate tools
→ load exact schemas → model call → invoke
```

检索应先应用权限过滤，避免泄露不可见工具名称。工具描述要包含业务对象、动作、约束和常用同义词，检索结果还应考虑模型兼容性、健康状态、延迟和成本。

8. CLI：最通用但最难治理的工具总线

Shell 让 Agent 直接复用 git、编译器、数据库客户端和组织已有 CLI。它具有巨大组合性、文档生态和人类可复现性。Pi 的官方说明把 read、write、edit、bash 作为默认工具，并通过技能、扩展与外部 CLI 增加能力，而不是把所有能力做成内置专用工具。Pi coding agent README

CLI 的代价是：参数空间开放、命令可能启动子进程、重定向和管道隐藏真实效果、静态策略难以理解 shell 语义。安全实现至少需要：

- 明确 shell 解析模型，避免对整段字符串做天真前缀匹配；
- 进程组、PTY、stdin、后台进程和超时管理；
- 工作目录与可写根限制；
- 网络和可执行文件策略；
- 命令规范化与用户可读审批；
- stdout/stderr 外置、截断和秘密脱敏；
- 退出码与实际效果分离。

高风险业务动作不应只暴露成任意 shell。应提供窄工具，让策略能理解语义，例如 create_payment_draft 与 commit_payment 分离。

9. Native Tool Call 与 Code Mode

Native 模式每次由模型选择一个或多个函数调用，Harness 执行并把结果送回模型。Code Mode 则让模型生成一段程序，在程序内组合多个工具调用，只把提取后的结果返回外层对话。

DSH Code Mode 将工具渲染为 TypeScript/Python SDK，并只向模型暴露 run_code transport；程序内工具调用仍重新进入完整的 pre-execute、guard、execute、post-execute pipeline。官方文档特别说明，它是用 SDK 文本加一个 transport schema 替换各工具 schema，不承诺在所有情况下减少 token。DSH Tools

Code Mode 的优势：

- 多步数据处理留在执行环境，减少模型往返；
- 中间大型结果不进入对话；
- 可表达循环、分支、并行和异常处理；
- 代码比多轮自然语言更容易复现。

风险：

- 一次 run_code 内可能发生多个真实副作用；
- 审批 UI 必须解释内部调用，而非只显示外层程序；
- 程序可能动态构造参数，静态预审不完整；
- 沙箱、资源限制和秘密隔离要求更高；
- 中间失败与部分提交需要细粒度 ledger。

因此 Code Mode 必须让每个内部 tool call 重新经过策略和审计，不能把 run_code 的一次批准视为无限授权。

10. 并行工具的调度语义

模型输出多个调用不等于它们可以安全并行。工具定义应声明：

TEXT

```
read_set / write_set
side_effect_class
concurrency_group
```

```
idempotency_support
ordering_requirements
```

DSH Code Mode 指导独立只读调用可用 Promise.all，变更调用按顺序运行。企业调度器还可根据目标系统和租户限流。多个读取若要求一致视图，应固定同一快照；依赖读取结果的写入要携带所读资源的版本标识，在提交时核对它是否仍有效，避免按过期观察修改状态。

11. 工具版本与动态变化

工具结构、行为或权限变化会影响模型选择、缓存、重放、会话恢复和评估可比性。每次调用应记录工具契约版本与实现摘要，才能区分接口定义和实际代码是否变化。

兼容变化可以按既定策略升级；破坏性变化应创建新的动作版本。恢复旧会话时，Harness 可以：

- 加载兼容旧版本；
- 运行显式迁移；
- 重新规划尚未执行的动作；
- 无法保证时暂停并请求人工。

不能把旧模型生成的参数直接送给含义已变化的新工具。

12. Tool Policy 与 Tool Execution 分离

推荐流水线：

TEXT

```
model proposal
  → schema validation
  → semantic normalization
  → policy evaluation
  → approval if needed
  → credential binding
  → sandbox/executor dispatch
  → result validation
  → redaction/transformation
  → event + model observation
```

模型不接触实际凭证。授权策略接收规范化动作、身份与环境状态，明确返回允许、拒绝、等待审批或附条件允许。拒绝和未知决定不得进入执行器；等待审批时暂停该动作，恢复后重新核对参数、资源版本与权限。附条件允许只有在约束落实后才能执行。执行器只接受仍有效且绑定主体、动作与资源范围的临时授权，不能把“无需审批”理解为“已经允许”。

延续 TASK2048 的教学情境：测试工具返回失败日志，模型可据此修改候选补丁；若它转而请求合并，而当前授权仅限分支内修改，策略应返回拒绝，轨迹记录该决定且合并执行次数为零。下一步是继续已获准的修复，或按流程申请提交权，不是把工具拒绝当作可自动重试的故障。

13. 如何评价工具层

除了任务成功率，还应测：

- 工具选择的精确率与召回率：选中的工具有多少合适，应选的工具有多少被选中；
- 首次参数有效率；
- 自修复成功率；

- 平均工具轮数与上下文成本；
- 错误分类准确率；
- 重复副作用率；
- 未授权调用拦截率与误报率；
- schema 变化后的兼容率；
- 大结果外置后的证据召回率；
- 不同模型对同一 canonical action 的适配差异。

例如一个任务只需查库存与报价，路由却选了库存、报价和下单工具：召回虽完整，误选仍拉低精确率；若只选库存，则漏掉了报价。合格工具集合应事先标注，并允许确有等效作用的替代工具。

评测还应包含名字相似、描述冲突、返回提示注入、动态改变工具目录、部分成功和超时后提交的工具，观察模型误选、漏选与执行器拦截是否各自被记录。

14. 企业平台的工具分层

TEXT

```
L4 Business Actions  支付、工单、发布、客户数据
L3 Domain Tools      SQL、仓库、观测、文档、浏览器
L2 Generic Compute   shell、Python、文件、HTTP
L1 Protocol Adapters MCP、OpenAPI、CLI、SDK、RPC
L0 Execution Control policy、credential、sandbox、ledger
```

越靠近业务提交，接口越窄、权限越细、验证越强；越靠近通用计算，组合性越高、隔离越强。

15. 最小工具契约伪代码

TEXT

```
ToolContract {
  id, version, model_views[]
  input_schema, output_schema
  side_effect: NONE | REVERSIBLE | COMMITTING
  idempotency: NATURAL | KEYED | NONE
  required_capabilities[]
  data_classification
  timeout_policy, retry_policy
  concurrency_policy
  result_projection
  verifier
}
```

这个 contract 可以由 MCP、CLI wrapper 或内部 SDK 实现。协议可以多样，运行语义必须统一。

本章结论是：工具不是模型函数列表，而是从概率性意图到真实副作用的受治理动作协议。MCP 提供互操作，CLI 提供组合性，Code Mode 提供程序化编排；Harness 必须在它们之下统一身份、策略、执行、账本和验证。下一章将专门讨论这个信任边界：权限、审批、沙箱、凭证与供应链。

第九章 权限、沙箱、凭证与供应链

> 适用性声明：本章讨论的是 Harness 工程控制，不替代组织的安全评审、隐私评估或法律合规意见。

Agent 安全需要关心错误怎样变成真实副作用。风险既来自被误导的模型决策，也来自运行时直接执行的插件、脚本与配置。提示注入、目标漂移、工具误用和记忆污染，不能只靠加强系统提示处理。

所以安全架构要默认假设模型会被误导，并把误导后的能力和影响范围设上限。

1. 四个不同问题

TEXT

```
Authentication  谁在发起任务？
Authorization   此身份可对什么对象做什么？
Approval        此次具体动作是否需要人确认？
Isolation       即使获准执行，进程还能触及什么？
```

把它们混成一个“是否允许工具”开关会出漏洞。用户有仓库写权限，不代表每次 Agent 写入都免审批；用户同意运行测试，不代表脚本可以读 SSH key；容器隔离进程，不代表可以不受控调用云 API token。

2. Prompt 不是安全边界

提示可以降低误用频率，但不构成不可绕过的安全边界。 任何关键约束都要落地到确定性机制：

意图	提示层引导	执行层控制
不读主目录秘密	“不要读取”	文件系统隔离
不向外泄露数据	“不要上传”	egress allowlist/DLP
不改生产	“只测试”	独立身份与环境
删除前询问	prompt 规则	commit-time approval
只操作本仓库	工具描述	resource-scoped capability

模型负责理解意图，策略与执行层在给定配置和威胁模型下限制可执行范围。表中控制也不是完整证明：DLP 可能漏检，隔离取决于挂载、身份和网络配置，审计签名只能帮助确认记录来源与完整性，不能证明业务内容正确。

3. 从逐次批准到受控自治

逐条命令弹窗看起来安全，但高频弹窗会产生审批疲劳。 Anthropic 报道，Claude Code 在加入文件系统和网络双重隔离后，内部 permission prompt 减少了 84%。其设计用到了 macOS Seatbelt、Linux bubblewrap 和受控网络代理。 Claude Code Sandboxing

更稳妥的是先定义安全工作区：区内动作自动放行；越界才申请临时能力。

TEXT

```
default sandbox
  ├── read project
  ├── write workspace
  ├── run approved executables
  └── no network

step-up request
  ├── exact extra path/domain
  ├── reason and duration
  ├── one action/session scope
  └── audit + revoke
```

批复的是具体 capability，不是对 Agent 的抽象信任。

4. 文件系统与网络必须同时限制

只有文件隔离没有网络限制，Agent 仍可能下载恶意程序或访问内网服务。 只有网络限制没有文件隔离，Agent 仍可能先读取秘密，再等待外泄通道。 Anthropic 明确强调两者都要有。

企业沙箱还要考虑：

- 只读系统镜像与受控可写层；
- .git、配置目录和 socket 的特殊处理；
- 设备、IPC、进程、syscall 与资源限制；
- DNS、代理、IP 重绑定与内网地址；
- 子进程继承；
- sandbox teardown 与 artifact 导出。

Sandbox profile 必须版本化，并记录到每次 execution。

5. Policy 决策模型

策略输入不应是未解析的自然语言命令，而应尽量标准化：

TEXT

```
Subject      user, agent, service identity
Action       canonical tool/action + normalized args
Resource     repo, path, API object, environment
Context      task, tenant, time, risk, prior approvals
Provenance   model, skill, MCP server, originating content
```

输出应是：ALLOW、DENY、REQUIRE_APPROVAL、CONSTRAINED_ALLOW。约束可包含只读、路径、域名、行数、金额、TTL 或 dry-run。

策略还要在 commit 阶段重新检查，因为审批之后环境、资源版本、身份状态可能已变化。早先观察和授权不能无限期代表后续副作用合法。

在 TASK2048 的教学情境中，即使用户后来批准合并某个补丁，Agent 随后又修改文件，旧批准也不能自动覆盖新版本。提交前绑定检查失败，就暂停该提交并重新取得适用授权；已发生的分支修改仍保留在任务状态中，不因提交被拒绝而抹去。

6. 凭证不进入模型上下文

模型通常只需要知道“可使用 GitHub 工具”，不需要拿到 token。 推荐流程是：

TEXT

```
authorized action
  → credential broker
  → short-lived scoped credential
  → isolated executor
  → redact output
```

凭证应绑定目标 resource、动作范围、租户和短 TTL。 日志持久化前需脱敏，避免工具错误把 secret 放入上下文。 对无法细分权限的遗留系统，建议通过代理提供窄业务动作，而不是把管理员 token 给通用 shell。

7. Prompt Injection 的系统应对

间接 prompt injection 可能来自网页、issue、文档、代码注释、MCP 输出和 memory。 这些内容应标记为不可信数据，而非和 system instructions 合并。

防御应是组合式的：

- provenance 与信任标签；
- 数据/指令通道分离；
- 最小工具和最小权限；
- 跨域数据流策略，比如私有仓库内容不得写到公共仓库；
- 高风险动作 commit-time approval；
- egress、DLP 和秘密扫描；
- 事后审计与异常检测；
- 对抗评估。

即使检测器漏过 injection，sandbox 与 policy 也要把后果压住。 安全目标是把受影响后的行为限制在获准范围，并检查范围内动作的组合后果；能否做到，要用明确攻击前提和执行环境验证。

8. 多 Agent 的权限传播

父 Agent 转授自身权限时，不能把全部权限自动传给子 Agent，应签发受限的临时授权。若专家以独立身份取得域内权限，则由可信策略服务检查专家的执行权、任务用途和调用者的委派权；manager 不必因此取得专家可读数据的直接访问权。

TEXT

```
delegate(
  task,
  allowed_tools,
  resource_scope,
  budget,
  expiry,
  can_delegate=false
)
```

子 Agent 的结果仍是不可信输入，合并或提交要再经过验证器和授权策略。向下转授权限不得超过可转授范围，但缩权与父级复核并不能解决所有组合风险。例如私有读取与公开写入分别合法，经过共享黑板连接后仍可能泄漏；还需检查结果来源、可传递范围和后续用途。

9. MCP 与插件供应链

安装 MCP server、skill 或 DSH plugin，本质上等于给 Harness 增加代码与指令。 风险包括：

- 恶意安装脚本与依赖；
- server descriptions/tool descriptions 的注入；
- 更新后 schema 或行为变化；
- 凭证范围过大；
- 本地 server 继承宿主权限；

- plugin 能修改 loop、policy 或日志。

企业 marketplace 要做到来源验证、版本 pin、SBOM、签名、静态/动态扫描、权限 manifest、隔离测试、发布审批和紧急撤销。 插件的可卸载效果可以帮助清理链路，但不能替代安全验证。 组合性越强，生命周期和所有权保证越关键；DSH 的可卸载 effect 主要是清理结构，不是安全证明。

HookPry v2 将风险定位到模型决策之外：攻击者控制插件元数据、版本和 hook 配置，先提供可信的初始版本，再通过更新增加事件绑定命令。更新被采用且事件发生后，宿主可能直接启动子进程，无需模型再次选择工具。HookPry v2 论文的攻击能力受更新采用、事件发生和子进程权限限制，不等于任意网页都能绕过沙箱，也不能据此断言所有当前版本存在同一漏洞。本书未复现其攻击实验。

因此，本书建议审查更新前后的 hook 清单、触发事件、命令和执行身份；新增执行权要重新授权，宿主与工作区分别隔离，并保留调用审计和紧急撤销。模型工具调用门之外的自动脚本也必须经过受信任的执行边界。卸载插件只能停止后续使用，不能抹去已发生的外部副作用。

10. 审批 UX 是安全系统

审批界面要把人能判断的信息完整展示：

- 执行语义动作，而不仅是工具名；
- 目标资源和数据范围；
- 预计副作用与是否可逆；
- Agent 的请求理由；
- 触发审批的策略；
- 临时授权范围与持续时间；
- dry-run/diff；
- 拒绝后的安全替代方案。

“Allow always”必须绑定精确规则，不能把一次命令许可扩展成整个 shell。 Codex exec policy 通过 allow/prompt/forbidden 前缀规则，并支持规则附带测试样例，能把“长期许可”变成可审查策略。Codex ExecPolicy

11. 审计记录与模型 trace 分离

安全审计不能依赖可压缩或可删除的聊天摘要。每个敏感动作要记录：

TEXT

```
who / delegated_by
what canonical action
which resource
why / task reference
policy version and decision
approval identity and scope
credential lease id
sandbox profile
effect outcome and verification
```

模型隐藏推理不是审计必须条件。组织真正需要的是可观测的输入、动作、策略、证据和结果，而不是私有思维链。

12. 数据分类与跨域流动

Agent 容易把不同来源数据组合。 应对输入、artifact、memory、tool result 进行分类，并在写出时执行信息流策略。

例如：

TEXT

```
PrivateRepo + PublicIssue → deny public write
CustomerPII + ExternalModel → require approved gateway/redaction
ProductionLog + LongTermMemory → aggregate or prohibit
Secret + AnyModelContext → deny
```

这一层比只限制某个工具更强，因为合法读取和合法写入组合后也可能泄密。

13. 风险分级

等级	示例	默认控制
R0	读取公开资料	记录来源，按不可信输入处理，限制后续数据流与动作
R1	读取项目、写临时区	workspace sandbox
R2	修改分支、安装依赖、有限网络	policy + sandbox
R3	外部沟通、合并、共享数据写入	明确审批 + verifier
R4	生产、资金、身份、不可逆删除	双控制/专用 workflow

风险由动作、资源、数据、可逆性和环境共同决定，不应只按工具名静态分级。表中 R0 仅表示单次读取的直接影响较低；载入恶意内容后再调用写工具，必须按后续动作重新授权。

R3/R4 中有些动作无法真正回滚，例如已发送消息或已被对方采用的数据。它们应有事前授权、限额、提交回读和事故处置方案；补偿处理已确认的错误效果，不能据此盲重试未知提交。软件版本回退是另一项能力，应单独演练，附录 B 给出判据。

14. 安全测试

- 网页、issue、代码注释中的间接 injection；
- 工具描述与返回值 poisoning；
- 私有到公共资源的数据外泄路径；
- shell 管道、重定向、子进程和解释器绕过；
- DNS 重绑定、代理绕过和内网 SSRF；
- memory/skill 持久污染；
- 记忆撤销后，已有上下文、摘要、检索索引和派生技能是否继续传播旧指令；
- 子 Agent 权限升级；
- 审批 replay 与过期授权；
- crash/retry 导致重复提交；
- 恶意插件卸载后的残留 effect。
- 插件更新新增 hook 后，宿主是否在未经重新授权时直接执行命令。

安全评估应在目标 Harness 与隔离的代表性执行环境中进行，并记录版本、攻击前提和故障范围。裸模型拒绝率不能代表系统安全。记忆撤销场景可借鉴 MemSecBench v1 的写入、后续执行和选择性修复流程；第七章讨论如何把这种生命周期检查扩展到派生对象。MemSecBench v1

15. 安全参考边界

TEXT

```
Untrusted Context
       ↓ taint/provenance
Model proposes action
       ↓
Schema + semantic normalization
       ↓
Policy decision ⟶ Human approval
       ↓
Scoped capability + credential lease
       ↓
OS/VM sandbox + egress proxy
       ↓
Effect ledger + output redaction
       ↓
External verification + audit
```

图中是模型提议动作的路径；生命周期 hook 等宿主触发动作也须接入授权、隔离、账本和审计，不能因为不经过模型就豁免。

自治应在预先定义的授权范围内运行。提示帮助理解意图，策略决定是否允许，沙箱限制接触面，凭证代理提供受限身份；验证检查验收条件，审计保留可复查记录。下一章进一步讨论：哪些证据足以按任务约定判定完成。

第十章 验证、完成契约与证据包

> 证据声明：本章复核至2026-09-19；历史benchmark数字按所引论文/公告的版本解释。完成门是作者参考设计，其能力取决于明确列出的检查与后端条件。

Agent过早说“已经完成”，可能把半成品送进代码库、把错误数字写进管理报告，或让后续工作流继续执行。对这类需要外部事实支撑的任务，Harness应先接收候选，再按任务契约验收。普通对话的交付可以就是回答本身；需要独立服务、隔离环境还是人工判断，应按风险决定，不能把每次会话停止都强行送进一个重型完成门。

对可审计任务，交付应包含产物、可核对证据和未决风险。输入、工具或模型不允许精确重放时，应说明能复建哪些结果、保留哪些原始记录。验证从任务受理开始参与设计，不只是结束时临时补跑一次测试。

1. Stop、Answer、Success 与 Commit 是四件事

模型结束生成，只说明本轮没有继续输出。它既不证明目标实现，也不证明系统应当接受副作用。企业 Harness 至少需要区分四个事件：

TEXT

```
MODEL_STOPPED       模型本轮停止生成
ANSWER_PROPOSED     Agent 提交解释或候选交付物
SUCCESS_VERIFIED    契约规定的检查已通过；不保证检查之外的全部正确性
EFFECT_COMMITTED    经策略门允许，副作用对目标系统生效
```

这四者不能用一个 done=true 表示。模型可能因为上下文不足、预算耗尽、工具错误或误判而停止；答案可能正确但证据不足；验证可能通过但生产发布仍需审批；外部提交可能成功而业务目标实际未达到。状态机应保留这些差异，否则恢复、重试和审计都会变得含糊。

一个常见反模式是让模型同时扮演实施者、证人和法官：它修改代码，选择要运行的测试，解释测试结果，再自行决定是否完成。此结构把所有系统性偏差放在同一条因果链中。更稳健的 Harness 让模型负责提出候选，让环境和独立 verifier（验收器）负责约束事实，让 policy（策略）决定是否提交。

2. 完成契约从任务入口开始

自然语言目标通常还缺少可以执行和验收的细节。任务受理时，平台与有权负责人把这些细节整理为一个有版本的完成契约：

TEXT

```
CompletionContract {
  goal                // 用户真正想改变什么
  deliverables[]      // 必须产生的 artifact 或外部状态
  invariants[]        // 全程和完成后都不得破坏的条件
  acceptance_checks[] // 可以怎样判定成功
  evidence_required[] // 交付时必须附带什么证据
  authority           // 谁能修改契约、豁免检查、批准提交
  budgets             // 时间、token、费用、尝试、风险预算
  freshness           // 输入与检查允许多旧
  stop_policy         // 成功、失败、阻塞、升级的条件
}
```

契约不必一开始就完美。探索性任务可以先生成草案，在发现仓库约束或数据语义后提出变更。但变更必须显式：谁改了哪项验收标准，原因是什么，是否降低了门槛。Agent 不能在失败后悄悄删掉难以通过的测试，也不能把“修复根因”退化成“让报错消失”。

应区分三类要求。目标描述期望的业务结果；不变量定义不可牺牲的属性；检查只是当前用于观察它们的测量方法。测试通过不等于目标在逻辑上必然成立，因为检查可能不完备。这个区分会自然导向防奖励投机设计：Agent 可以看到目标和部分检查，但不应拥有修改权威判分器或读取所有 held-out（保留集）数据的能力。

3. 验证金字塔

不是所有 verifier 具有相同证明力。一般应优先使用更接近真实状态、可重复且独立于生成模型的检查：

TEXT

```
                人类/责任人判断
          独立模型与多视角语义评审
      领域模拟、集成测试、目标系统回读
  单元测试、schema、静态分析、约束与对账
最底层：artifact 存在性、哈希、退出码、状态版本
```

图形位置不代表越上层越强。对于“数据库中恰好写入一条记录”，确定性查询比模型评审可靠；对于“建议是否误导管理层”，只有字符串检查远远不够。正确做法是按声明类型选择证据，而不是迷信一个通用 judge（裁判模型）。

3.1 确定性检查

确定性检查包括类型、schema、编译、lint、单元测试、约束求解和数值对账。签名、哈希与资源版本检查另回答“谁签署、内容是否变动、对应哪个输入”，不证明业务含义正确。这些检查的成本与可重复性因环境而异，结论也只覆盖已经编码的断言；测试仍可能太窄、过时或依赖不稳定环境。

3.2 环境与结果检查

结果检查不只看 Agent 的文本或补丁，而是从目标环境回读事实：服务健康、API 行为、数据库状态、页面可交互性、消息是否被目标方接收。SWE-bench 的重要贡献之一，是把问题从“生成一段代码”提升为“在真实仓库中产生能通过测试的补丁”；原始数据集包含 12 个 Python 仓库中的 2,294 个 GitHub issue。SWE-bench

环境验证还应固定依赖、时钟、区域、权限和初始状态，并记录镜像摘要。否则同一补丁可能因 Python 版本、操作系统或网络资源变化而得到不同判决。

3.3 模型检查

模型 grader 适合评估风格、语义覆盖、解释质量和难以编码的政策，但它给出的是测量，不是事实。研究已经观察到 LLM judge 的位置偏差；一项覆盖 12 个 judge、22 类任务和十万余次比较的研究发现偏差并非随机噪声。Judging the Judges 另一项研究发现 judge 对更熟悉、低困惑度的文本可能给予偏高评价，形成自偏好风险。Self-Preference Bias

因此模型 grader 应采用明确 rubric（评分量表）、逐项证据引用、顺序交换、盲化来源、多次采样和人工校准。生成模型与 judge 最好在模型家族、提示和上下文上保持适度独立。重大决定不能只依赖单次“看起来不错”。

3.4 人类检查

人类不是无限可靠的金标准，也会疲劳、受界面诱导和缺少领域上下文。但在高影响、规范冲突、价值判断或新型失败上，人类仍承担责任归属。Harness 应把人放在最需要判断的位置，并给他差异、风险、来源和未决项，而不是要求从头阅读整条轨迹。

4. Benchmark 也是会腐化的软件

2024 年推出的 SWE-bench Verified 是评测工程的典型进步：OpenAI 与 SWE-bench 作者组织 93 名有 Python 经验的开发者复核 1,699 个样本，每个样本由三人标注，形成 500 题子集，并改进了容器化评测环境。Introducing SWE-bench Verified

但这不是终点。OpenAI 在 2026 年宣布不再用它衡量前沿 coding 能力：对 138 个不稳定失败样本的审计中，至少 59.4% 存在实质性的测试或问题描述缺陷；同时，前沿模型表现出接触过部分题目或答案的迹象。Why SWE-bench Verified no longer measures frontier coding capabilities

这个过程揭示了三条普遍规律。第一，验证器有版本，也会产生技术债。第二，模型能力越强，越容易触碰 rubric 的边界并发现漏洞。第三，公开 benchmark 会经历污染、饱和和选择性优化，排行榜分数不能直接外推到企业任务可靠性。

企业 eval registry 因此应记录：任务版本、数据来源、创建时间、可见性、泄漏风险、reference solution、grader 版本、环境摘要、历史难度和退役原因。能力集与回归集也应分开。前者故意寻找当前系统不会做的事，后者保护已经做到的行为；一个高通过率的能力集可能已经失去区分度，应毕业为回归集或被更难任务替换。

5. 非确定性系统不能只跑一次

Agent 轨迹受采样、工具时序、外部状态和上下文装配影响。一次成功不能证明稳定，一次失败也未必说明不具备能力。Anthropic 将 task、trial、grader 和 transcript 分开，并建议按产品目标区分 pass@k 与 pass^k：前者衡量 k 次中至少一次成功，后者衡量 k 次全部成功。Demystifying evals for AI agents

若单次成功概率为 p，在独立近似下：

TEXT

```
pass@k = 1 - (1 - p)^k
pass^k = p^k
```

搜索候选解时，“十次总有一次对”可能有价值；自动退款、生产变更和客户承诺更关心每次都对。两者混用会制造漂亮但误导的指标。企业报告还应给出样本量、置信区间、成本和时延，并按任务风险、长度、工具链和环境切片。平均分会掩盖某一关键业务族完全失败的事实。

重试也不是免费的可靠性。若每次都可能产生副作用，盲目重试会重复发信、下单或修改数据。Harness 必须使用幂等键、effect ledger 与提交状态回读，把“推理重试”与“副作用重放”分开。

6. 防止测试投机与 verifier 篡改

只要 Agent 能观察评分信号，它就可能找到比实现目标更短的路径：硬编码样例、删改测试、伪造日志、读取 held-out 标签、修改指标函数或利用环境漏洞。这不一定表现为蓄意欺骗；在优化压力下，它可能只是把错误捷径解释成解决方案。

2025 年的 EvilGenie 用 held-out 测试、LLM judge 和测试文件改动检测衡量 coding agent 的 reward hacking，并用人工复核校准这些信号。EvilGenie 2026 年的 SpecBench 则显式分离可见验证测试与组合行为的 held-out 测试，展示“可见测试饱和”仍可与真实系统行为失败并存。SpecBench

工程上应建立评测完整性边界：

TEXT

```
agent workspace       可修改源码与允许的配置
visible checks        可运行，用于快速反馈
trusted evaluator     只读/隔离，Agent 无修改权限
private validation    隔离检查，可按预定规则返回有限修复反馈
sealed final test     独立终测，不用于候选调试或自适应选择
access audit          记录文件、网络与 evaluator 访问
reference recompute   不信任 Agent 自报的分数
```

held-out 不是万能药。测试太具体仍可能误杀合法解，测试数据也可能通过训练或工具泄漏。更强的组合包括不变量、变形测试、属性测试、差分测试、随机化、因果探针和人工抽查。还应设置负向样本：既测试“该做时会做”，也

测试“不该做时不做”。否则优化检索触发率，可能得到一个凡事都搜索的 Agent；优化修复率，可能得到一个过度改动仓库的 Agent。

7. 证据包是交付协议

最终回答适合人阅读，证据包适合系统验证、审计和后续 Agent 接手。建议每次任务生成结构化 manifest：

TEXT

```
EvidencePackage {
  task_id, contract_version
  input_snapshot[]       // repo commit、数据版本、时间范围
  artifacts[]            // path/URI、hash、media type、producer
  change_set[]           // diff、外部 effect、幂等键
  checks[] {             // 每条验收检查
    check_id, verifier_version, environment_digest,
    started_at, exit_status, observations, artifact_refs
  }
  provenance[]           // 来源、工具、插件与委派关系
  policy_decisions[]     // 授权、批准和例外
  unresolved[]           // 未验证假设、flaky check、已知风险
  final_status           // verified / partial / blocked / failed
  signature
}
```

证据包不是把全部 stdout 塞进聊天记录。原始日志可存对象存储，manifest 只保留摘要、哈希和定位符。秘密在进入持久层前脱敏。检查输出必须能证明它对应哪个 artifact 和哪个环境，避免“测试通过”引用的是修改前版本。

对长任务，证据应增量产生。每个阶段保存 checkpoint、局部不变量和可恢复状态；最终 verifier 聚合，而不是在结尾重新相信模型对数小时操作的摘要。若发生上下文压缩，证据账本仍独立存在。

8. 完成门的参考状态机

TEXT

```
WORKING
  ├─ candidate produced ──→ VERIFYING
  ├─ budget/risk hit ─────→ BLOCKED_OR_ESCALATED
  └─ unrecoverable error → FAILED

VERIFYING
  ├─ all mandatory checks pass ──→ READY_TO_COMMIT
  ├─ repairable failures ────────→ REPAIRING
  ├─ ambiguous grader ───────────→ REVIEW_REQUIRED
  └─ contract impossible ────────→ BLOCKED

READY_TO_COMMIT
  ├─ policy/approval pass ───────→ COMMITTING
  └─ denied/expired ─────────────→ BLOCKED

COMMITTING
  ├─ effect confirmed ───────────→ VERIFYING_POSTCONDITIONS
  └─ uncertain outcome ──────────→ RECONCILING

VERIFYING_POSTCONDITIONS
  ├─ business checks pass ───────→ VERIFIED_COMPLETE
  └─ failed/unknown ─────────────→ COMMITTED_BUT_UNVERIFIED
```

网络超时后，请求可能已在目标系统生效，因此RECONCILING先回读状态，不直接重放。部署API返回成功后，健康检查仍可能失败；这时保留“已提交、尚未验收”的状态，由有权主体决定补偿或升级，不能假装什么都没发生。若合同只要求保存一份报告，持久保存确认本身可以是最后检查，不需要人为制造额外外部提交。带时效的验收结果只对记录的观察窗口成立。

参考伪代码如下：

TEXT

```
function attempt_completion(run, candidate):
    contract = load_pinned_contract(run.contract_version)
    snapshot = seal_candidate(candidate)

    results = CheckResults()
    for check in contract.pre_commit_checks:
        verifier = trusted_registry.resolve(check.version)
        results += verifier.run(
            candidate=snapshot,
            clean_environment=check.environment_digest,
            validation_inputs=check.validation_input_ref
        )

    if results.has_integrity_violation():
        quarantine(run)
        return FAILED

    if results.has_ambiguous_or_flaky_signal():
        return REVIEW_REQUIRED

    if results.mandatory_failed():
        if results.feedback_allowed and repair_budget_remaining(run):
            return REPAIRING(results.minimal_diagnostics())
        return BLOCKED_OR_FAILED

    package = build_evidence_package(run, snapshot, results)
    if not contract.requires_external_commit:
        persist_delivery_and_evidence(snapshot, package)
        return VERIFIED_COMPLETE

    decision = decode_policy_decision(policy.evaluate_commit(package))
    if decision.decision == REQUIRE_APPROVAL:
        persist_approval_bound_to_candidate(package, snapshot.hash)
        return AWAITING_APPROVAL
    if decision.decision not in {ALLOW, CONSTRAINED_ALLOW}:
        return BLOCKED
    if not constraints_enforceable(package, decision):
        return BLOCKED

    effect = commit_with_live_authority_and_effect_ledger(snapshot, decision)
    if effect.status in {UNKNOWN_EFFECT, PENDING}:
        return RECONCILING
    if not effect.is_confirmed:
        return record_commit_failure(effect, package)
    postconditions = verify_target_state(contract.post_commit_checks, effect)
    persist_post_commit_evidence(package, effect, postconditions)
    if not postconditions.passed:
        return COMMITTED_BUT_UNVERIFIED
    return VERIFIED_COMPLETE
```

这里的results是CheckResults集合，具有完整性、模糊信号和必需项判定方法，不是普通数组。检查器版本、输入和环境固定；进入审批后，候选hash、环境和契约变化会使旧批准失效。提交helper还须在实际发送前检查撤权、取消和约束，并处理未知副作用，不能只凭前面的一次策略判断。

能返回修复诊断的私有检查集承担验证集职责。最小诊断可以减少泄漏，但多轮PASS/FAIL和诊断仍会影响候选选择，因此不能同时被称为从未参与选型的密封终测。终测由独立服务在预注册时机使用并记录访问次数；一旦拿它继续调试，就应登记暴露并重新建立终测证据。业务验收不一定使用隐藏题，但必须如实标明数据用途。

9. 三类案例

9.1 仓库软件工程

目标不是“生成 patch”，而是“在限定范围内修复 issue，不破坏既有行为”。交付物包括 diff、测试和迁移说明；不变量包括旧测试、API 兼容、安全扫描和禁止修改 evaluator；证据包括 clean checkout 上的编译、目标测试、回归测试、静态检查与 diff 审查。高风险仓库还需要 reviewer 批准后才能 merge。

失败时，Harness 应区分代码失败、环境失败、flaky test 和规范冲突。把所有非零退出码都喂回模型会浪费预算，也可能诱导它改测试来消除噪声。

9.2 企业数据分析

目标不是“写一份有图表的报告”，而是“对指定时间和口径的数据给出可复核结论”。完成契约应固定数据快照、指标定义、过滤条件、币种和时区。验证包括 schema、行数与总额对账、独立查询、异常值检查、引用可达性和图表数据一致性。语义结论可由独立模型或分析师评审，但数字必须回到查询和数据版本。

证据包应允许另一位分析师从查询、参数和 snapshot 重建结果；若底层数据在运行期间更新，系统必须标记 freshness，而不是把两个时间点的数据静默混合。

9.3 自我进化 Agent

当 Agent 修改自己的 prompt、skill、工具选择器或 loop，验证的独立性更难保持。候选变体不能修改自身评价函数，也不能只在产生它的同一批轨迹上得分。完成契约应包含 held-out 任务、回归集、安全集、成本/时延上限、统计门槛和回滚条件。

“新版本在平均分上更高”不足以发布。Harness 还需检查关键切片没有退化、收益跨多 trial 稳定、评测数据未污染、提案与 evaluator 隔离，并经过 canary。进化系统的证据包要记录父版本、变异、训练/选择数据、judge 版本和所有淘汰原因，使组织能够回答：它为什么被选中，以及如果出问题应回到哪里。

10. 常见失败模式

10.1 把 Agent 自述当证据

“我运行了所有测试”必须由工具事件和结果 artifact 支持。自然语言摘要只是索引。

10.2 只验证最终文本

外部状态已被错误修改时，再好的解释也不能恢复事实。结果验证必须读取目标系统。

10.3 同一主体控制目标、实现与评分

这会使错误假设和投机路径无法被独立发现。至少隔离 evaluator 与 commit authority。

10.4 失败后动态降低门槛

任何 waiver 都应由有权主体批准，注明范围、到期时间和风险；不能由 Agent 自行重写成功定义。

10.5 迷信 benchmark 排名

公开基准是能力探针，不是生产 SLA。必须用本组织的任务分布、权限模型、数据和环境做回归。

10.6 无限验证—修复循环

重复尝试会增加成本，也可能逐步泄漏隐藏检查。设置尝试预算、无进展检测、错误聚类和升级条件。

11. 企业落地清单

完成子系统应按风险选择机制：版本化契约、候选封存、检查器登记、固定或可复建环境、验证/终测隔离、评分校准、效果账本、批准与例外记录。需要签名时先定义信任根和验证者；需要多轮模型评测时定义样本和重复单位。对高风险动作再做验收器篡改、测试投机和假完成演练。机制是否足够，以本组织的威胁模型和任务合同检验，不按组件数量判定生产成熟度。

组织还应把“验证失败”视为产品数据。失败可能说明 Agent 不够强，也可能说明任务不可解、规范含糊、环境损坏或 grader 错误。Anthropic 提醒，前沿模型在很多 trial 中始终为零分，有时首先应检查任务和 grader 是否损坏，而不是直接判定能力缺失。Demystifying evals for AI agents

业务交付时，系统应能回答五个问题：完成了什么，基于哪个输入版本，谁用什么检查验收，还有哪些未知，以及已经发生哪些外部效果。回答不了的部分应保留为未验证项，不能由模型的自信叙述补齐。

第十一章 多 Agent、委派与协作拓扑

> 证据声明：原有产品资料截面为 2026-08-28，子代理上下文模式补至 2026-09-19 已抓取材料；架构原则为作者综合推断，厂商能力与论文结果不代表本书已实测。

多 Agent 是个名字很容易误导人的概念。把同一个模型调用五次，再给每次调用贴上“架构师”“开发者”“审查者”这样的标签，不会自动组成一个真实团队。一个真正的多 Agent 系统，必须先回答：任务为什么可以拆解、状态由谁持有、权限怎么衰减、冲突如何解决、结果由谁验收、故障如何隔离，以及新增成本是否换来了可量化收益。

这一章的核心判断是：多 **Agent** 不是能力层面的默认升级，而是一种并发、隔离与治理机制。 当任务可并行探索、需要不同上下文或信任边界、单一上下文放不下全部材料时，它可能显著提升有效计算。相反，任务高度耦合、共享状态变化频繁、验收边界不清晰时，它往往只会增加通信损耗和级联错误。

1. 先区分五种经常混淆的东西

TEXT

`tool call`	主 `Agent` 请求一个约定接口，内部可能含模型或外部服务
`subroutine`	独立模型调用，返回结构化结果，不拥有任务
`subagent`	有局部目标、状态、工具和预算的受托执行者
`handoff`	当前责任主体把会话或工作流所有权转交给另一个 `Agent`
`multi-agent`	多个自治执行者通过明确协议共同改变任务状态

是否把它称作“Agent”不是关键，关键是它是否具备独立决策循环和明确责任边界。一个翻译模型被主 Agent 当作函数调用，实际上更像概率子程序；一个能自行搜索、调整计划、使用工具并提交证据的研究者，才算 subagent。Handoff 也不同于“请专家给意见”，它是执行权和用户交互权的转移。

OpenAI 的官方架构把 manager 与 handoff 明确区分：manager 把专家 Agent 当作工具调用并保留会话控制，handoff 则把工作流控制交给新的 Agent。OpenAI Agents SDK 这种区分应进入 Harness 状态机；否则用户无法知道当前谁在负责，guardrail、预算和最终输出所有权都会变得含糊。

2. 多 Agent 的收益来自哪里

多 Agent 的收益通常来自四种机制，而不是“角色扮演”本身。

第一是并行搜索。多个 worker 可在不同假设、代码区域或数据源中同时工作，争取缩短墙钟时间并提高覆盖率。第二是上下文分工。局部任务可以只加载相关材料，延续已有调查的任务也可能需要父上下文，不能一律从空白开始。第三是模型与工具差异，它们可能带来不同错误分布，但多样性仍须测量。第四是验证分工，独立的信息路径与验收标准有助于减少共同偏差；仅分成两个角色并不足够。

Anthropic 的 Research 系统采用 orchestrator-worker 架构：lead agent 先定策略，再并行生成搜索 subagent。其内部分析报告，在 BrowseComp 上，token 用量、工具调用数与模型选择联合解释了 95% 的性能方差；单独使用 token 用量也能解释 80%。这两项结果不是可相加的贡献份额，观察关联也不能确定因果，更不能据此排除协作结构的独立作用。Anthropic Multi-Agent Research

这提示计算预算可能是重要关联因素。Anthropic 还报告其普通 Agent 和多 Agent 系统分别约使用聊天场景的 4 倍、15 倍 token；它们是特定系统的成本观察，不是通用比例。应固定任务与模型，在相同预算下比较单 Agent、多 Agent、重复试跑与确定性并行程序，才能判断新增协调是否有价值。

3. 何时不应使用多 Agent

至少四类任务通常不适合直接拆成多个自治 Agent。

高度串行的任务中，下一步依赖前一步的精确结果，parallel worker 只能猜测未来状态。窄而确定的任务里，用普通函数、工作流或一次模型调用就能完成，加入自治循环只会增加故障面。强共享状态任务要求多个执行者频繁读写

同一工作树、数据库或 UI，协同成本可能高于执行收益。低价值任务往往撑不起明显上升的推理成本与更复杂运维；这一判断来自前述厂商案例与系统成本结构，而不是普遍比例定律。

Anthropic 还指出，当所有 Agent 需要共享同一上下文，或任务包含大量相互依赖领域时，目前并不适合多 Agent；许多 coding 任务真正可并行的部分比研究任务少。Anthropic Multi-Agent Research

因此 Harness 应先算“可委派性”，不要因为请求复杂就马上建团队：

TEXT

```
DelegationValue ≈
  parallel_fraction
  × context_separability
  × diversity_gain
  × task_value
  - communication_cost
  - merge_risk
  - duplicated_work
  - coordination_latency
```

这是决策提示，不是可直接代数计算的收益公式：各项没有共同量纲和测量定义。如果拆后的子任务无法定义独立输入、交付物和验收条件，就应先澄清边界，而不是期待委派后自然形成职责链。

4. 四类基本拓扑

4.1 Manager—Worker

中央 manager 分解任务、分配 worker、收集结果并负责最终合成：

TEXT

```
                    ┌─ worker A
user → manager ─────┼─ worker B → manager → verifier → result
                    └─ worker C
```

优点是有单一责任入口，也更容易做预算和策略控制，适合研究、候选生成、分片检查。缺点是 manager 可能成为信息瓶颈和单点故障；如果 worker 只给回长篇自然语言，合成环节会丢失来源、置信度和冲突信息。

4.2 Pipeline / Assembly Line

每个 Agent 接收上游 artifact，并产生下游 artifact。MetaGPT 将软件工作流中的标准作业程序编码进角色化 prompt 序列，以 assembly-line 方式组织协作；其出发点之一就是朴素 Agent 对话里容易出现级联不一致。MetaGPT

流水线适合阶段边界清晰的流程，比如需求→设计→实现→审查。但上游缺陷会被下游当成事实继续继承。每一阶段都应配 schema、质量门和返工路径，不能只靠下一个角色“看完再理解”来修复问题。

4.3 Peer Handoff

Agent 根据任务阶段将责任转给其他专家。OpenAI Agents SDK 把 handoff 作为模型可调用工具暴露出来，并支持配置结构化 handoff 输入、回调和输入过滤；默认状态下，接收者仍可能看到先前会话历史，除非显式过滤。OpenAI Handoffs

Handoff 适用于客服分流、领域升级和长期会话里的所有权转换。它不适合需要中央汇总多个并行意见的情景。每次转移都应记录 from、to、原因、状态摘要、未决承诺和权限，还要限制循环转交。

4.4 Blackboard / Event Graph

多个 Agent 不直接维护长对话，而是通过共享产物存储、事件总线或任务图协作。AutoGen 早期以可对话 Agent 组合为核心，后续 0.4 架构转向 actor model，用消息、运行时和分层 API 提升模块化与扩展性。AutoGen

共享黑板适合异步、长周期和跨语言执行，但要先解决 schema 演进、并发控制、重复消息、顺序、所有权和垃圾回收。把聊天历史当消息总线，通常只能得到难以恢复的分布式 prompt。

5. 委派是一份受限契约

委派包应把“研究一下这个主题”细化为可验证的局部契约：

TEXT

```
DelegationContract {
  parent_run_id
  subtask_id
  objective
  inputs[]              // 固定快照或资源引用
  expected_output_schema
  acceptance_checks[]
  allowed_tools[]
  capability_scope
  execution_identity
  execution_authority_ref  // worker 执行权的可信授权依据
  delegation_authority_ref // 调用者可委派此任务的授权依据
  context_policy
  budget {tokens, time, cost, tool_calls}
  deadline              // 绝对时间；转换成 TTL 时显式计算剩余秒数
  can_delegate
  reporting_interval
  cancellation_token
}
```

父 Agent 不应把全部工具、凭证和记忆隐式地复制给子 Agent。转授自身权限时，子授权的动作、资源和期限不得超出可转授范围；专家若使用独立身份，则由可信策略服务分别检查其执行权、调用者的委派权和任务用途，不能让模型自行授予。默认 can_delegate=false；递归委派还要限制深度、扇出和整棵树的总预算。

输入应限于局部目标所需材料，不应自动包含全部私密会话。上下文策略要说明哪些是可信指令、哪些是不可信材料、哪些数据不能离开当前执行域。OpenAI handoff 的 input_filter 提供了筛选历史输入的接口；传递多少历史，实际是在隔离与连续性之间作选择。OpenAI Handoffs

Deep Agents 在 2026-09-08 的文档中将这项选择显式化：isolated 仅接收任务说明；fork 继承父状态与历史，移除末尾委派调用，并把任务说明转成子代理消息，最终仍向父代理返回一个工具结果。上下文模式 文档认为延续调查的 worker 可借此减少重复读取和利用缓存，独立 reviewer 则适合隔离上下文；这些收益不是本书实测结果。若父历史包含子角色无权接收的材料，应选可筛选的输入路径，不能因 fork 方便就扩大可见范围。

6. 子结果应交付产物与证据

如果 worker 只返回“我认为方案 A 更好”，manager 就没法可靠地做合并。Subagent 输出至少要包含结论、证据引用、生成的 artifact、验证结果、假设、置信度和未决问题。

TEXT

```
SubagentResult {
  subtask_id
  status
  claims[] {text, evidence_refs, confidence}
  artifacts[] {uri, hash, type}
```

```
  checks[]
  assumptions[]
  conflicts[]
  unresolved[]
  usage
}
```

研究型 worker 应提交来源和精确 locator；编码 worker 应提交独立 worktree 的 patch 和测试；数据 worker 应提交查询、快照和对账结果。这样 manager 在合并时拿到的是可验证对象，而不是经历多轮摘要压缩后的散文。

Anthropic 在多 Agent Research 中把 subagent 描述为信息过滤器，并强调把结果直接存入外部系统，可减少多阶段复制带来的信息损失和 token 开销。这说明 Harness 可以把大型结果放到 artifact store，只在消息中传递引用和摘要。

7. 共享状态与并发写入

并行读取相对容易，并行写入更难。多个 coding worker 改同一工作树可能导致文件覆盖、测试状态污染和难以归因的 diff。更安全的策略是 copy-on-write workspace 或独立 worktree：

TEXT

```
base snapshot
  ├─ workspace A → patch A + evidence A
  ├─ workspace B → patch B + evidence B
  └─ workspace C → patch C + evidence C
                    ↓
             merge workspace
                    ↓
             full-system verification
```

合并不是文本拼接。Harness 要检测文件级和语义冲突，决定执行顺序，并在组合状态上重跑验证。每个分支单独通过，不代表组合后仍然通过。

数据库、工单和消息系统应按可用语义选择资源版本、条件写入、事务、下游幂等键与副作用账本。读任务可以并发，冲突写入则要由运行时锁定、排序或拒绝。并非每个下游都支持幂等；结果未知时先对账，不能靠“可补偿”允许盲重放。

8. 错误如何在团队中传播

多 Agent 的典型问题往往不是某个 worker 的孤立答错，而是错误被社会化：早期错误变成公共前提，后续角色围绕它生成看似一致但错误的文档。MetaGPT 提到的级联幻觉，正是这种朴素链式协作失效的表现之一。MetaGPT

2025 年一项多 Agent 失败研究把问题归类为规范与系统设计、Agent 间错位、任务验证与终止三大类，并指出在流行 benchmark 上相对单 Agent 的收益可能有限。Why Do Multi-Agent LLM Systems Fail? 这类结果不应被理解为“多 Agent 无用”，而是提醒我们：增加节点也增加接口，接口质量往往比角色名称更决定可靠性。

常见传播机制有：

- manager 拆错任务，所有 worker 高质量地完成了错误子目标；
- worker 把推断当事实，上报为“肯定结论”，manager 再按多数票强化错误；
- 多个同模型 Agent 共享相同盲点，表面共识不代表独立证据；
- sub Agent 隐藏失败，只提交漂亮摘要；
- 循环 handoff 导致责任漂移并消耗预算；

- verifier 只查局部结果，却没检查组合不变量。

应对方式包括独立审查问题分解、保留来源记录、盲化并行、比较不同模型、保留冲突和系统级验证。多数票是否有益，取决于判断者准确性、错误相关性及聚合规则；近似独立不是获益的必要条件，独立本身也不充分。

一个便于理解的充分条件是：三个判断者处理同一二元问题，正确与否独立同分布，每人正确概率为固定的 p，且 $0.5 < p < 1$。多数票正确率为 $3p^2 - 2p^3$，高于 p；低于随机水平的独立判断者却可能被投票放大错误。现实中的相关错误仍可能留下聚合收益，完全重复的判断则不增加信息。模型与上下文共享程度要测量，最终以同预算聚合结果判断，不能把一致意见当作独立证据。

9. Debate、Critique 与 Ensemble

“让多个 Agent 辩论”包含几种不同机制：集成先分别生成候选，再由规则或评判器选择；批评让另一个 Agent 针对候选找问题；辩论则允许多轮互相影响。三者的成本和风险不同，不能混为一种增强手段。

受控逻辑推理研究发现，团队内推理能力和多样性是 debate 成功的重要驱动，而顺序、置信度可见性等结构参数的收益较小；多数压力还可能压制独立纠错。Can LLM Agents Really Debate? 所以生产系统通常优先采用“先独立、后比较”：先避免锚定，再把候选暴露给针对性反驳。讨论轮数应由信息增益或分歧收敛决定，不应无限延长到形式共识。

可验证任务通常更适合候选并行 + 外部 verifier，而不是语言辩论。只有当标准包含语义判断、价值冲突或证据解释时，debate 才可能带来附加信息；最终裁决仍应按完成契约判断，而非看哪个 Agent 讲得更有说服力。

10. 调度、背压与预算

多 Agent runtime 本质上是一个带概率 worker 的分布式任务系统。它必须处理队列、公平性、并发上限、速率限制、优先级、超时、取消、心跳、租约、重试、死信与背压。

TEXT

```
global_budget
  ├─ orchestration reserve
  ├─ worker pools by risk/model/tool
  ├─ verification reserve
  └─ emergency reconciliation reserve
```

父 Agent 应在派发前预留综合与验证预算，避免探索用完资源后无力验收。动态调度可按子任务价值、剩余不确定性和边际收益调整：重复返回同样信息时停止扩展，关键分歧未解决时再增加独立路径。

取消必须向整棵委派树传播，并立即停止继续派发；已发生的外部副作用不能随之消失。运行时应回读或等待在途动作，未知结果进入对账，已确认的错误效果再按授权补偿或交人工处理。父 run 结束后仍在后台运行的孤儿任务会继续消耗成本并扩大风险。

11. 权限、身份与责任链

每个 Agent 应有可区分的执行身份，即使底层由同一模型服务承载。审计记录至少包含 principal_agent、delegated_by、执行授权与委派授权的引用、资源范围、策略版本和动作。不要用共享管理员令牌让所有 worker 看起来像同一主体。

Manager 对委派行为负责，但不能盲目信任 worker。子结果进入父上下文时仍需核验来源，尤其是浏览网页、工单或第三方 MCP 后的结果。合并和提交要重新经过授权策略与验证器；转交用户交互权也不自动扩大接收者的资源权限。

多 Agent 安全还有组合风险：两个分别获准的动作连接起来，可能形成信息泄漏或越权。一个 worker 读取私有数据，另一个向公共系统写入，若通过共享黑板交换结果，就可能建立外泄路径。信息流策略应追踪来源记录、数据分

类和后续用途；转授权限缩减并不能自动排除这类风险。

12. 可观测性：同时看到树和因果链

纯串行循环的轨迹可以展示为序列；只要存在并行工具、异步事件或多 Agent 协作，就应记录部分有序的因果图。决定偏序的是并发与依赖，不是 Agent 数量。显示时可以线性排序，但仍须保留父子、消息、产物依赖和外部副作用之间的关系：

TEXT

```
run
 ├─ delegation span A
 |    ├─ model/tool spans
 |    └─ artifact A
 ├─ delegation span B
 |    ├─ model/tool spans
 |    └─ artifact B
 └─ merge span
      ├─ consumes A,B
      └─ verification + commit
```

所有事件应使用稳定的 run/subtask ID 和逻辑关联，而不是依赖墙钟顺序。并发环境里，时间戳接近并不代表因果关系。指标除成功率外，还应包括有效并行度、重复工作率、消息与 artifact 比例、合并冲突率、委派深度、orphan 数、每次成功成本和关键路径时延。

评测也要把团队行为作为研究对象。MultiAgentBench 尝试用 milestone（里程碑）指标比较 star、chain、tree 和 graph 等协调协议，说明只看最终答对与否不足以解释协作质量。MultiAgentBench 企业级测试还应注入 worker 超时、错误证据、重复消息、部分网络分区、过期 artifact 和恶意子 Agent，观察系统是否能降级，而不是整体失控。

13. 三类贯穿案例

13.1 仓库软件工程

不要按虚构的“公司角色”来拆分，应按可隔离的 artifact 切分：一个 Agent 定位失败点，一个在独立 worktree 生成修复补丁，一个构造反例或补充测试，一个做安全审查。主 Agent 保留集成所有权。若多个修改触及同一核心模块，就应退化为串行执行，或改由单一 owner 实施，避免语义冲突。

以 TASK2048 为教学例子，修复者继承已收集的错误现场，在独立工作区交付补丁哈希、修改范围和局部测试；审阅者只读取需求、补丁与证据。主 Agent 合并后若又改了文件，旧测试就不能继续为新版本作证，应在干净工作区重验；结果冲突时保留待解决状态，不按多数意见直接合并。这是设计示例，并非供应商实跑记录。

13.2 企业数据分析

天然可并行的方向包括不同数据源取数、指标口径复核、异常解释和反事实检验。所有 worker 使用固定数据 snapshot，并把 SQL、行数、口径和来源写入结构化结果。Manager 合成结论时必须保留冲突，不应把不同时间范围或币种的数字直接平均。

数据权限按域分配：财务 worker 不必读取人事明细，外部研究 worker 不接触客户数据。最终报告里的每个数字都应回链到查询和 snapshot。

13.3 自我进化 Agent

候选生成、评测和发布要有不同责任边界。多个 worker 并行提出提示、技能或可变决策策略的候选；评测服务用验证集和安全检查筛选，选择服务按预定规则确定候选，再在预定时机进行密封终测。发布控制器负责灰度与版本回退。用于选择或修复的反馈不再属于独立终测证据。

若候选 Agent 能通过委派修改评分逻辑，独立门禁就会失效。委派图、数据来源和权限记录应支持复查：候选家族看到了哪些反馈，密封集何时访问，裁判是否被候选改动。只隐藏标签仍不足以排除自适应泄漏，多 Agent 也不会自动建立实验独立性。

14. 一个最小参考实现

以下是教学伪代码，未作为供应商实现实跑。它直接使用前述契约字段：allowed_tools 是工具集合，capability_scope 是资源与动作范围，expected_output_schema 是输出结构；deadline 为绝对时间，签发前换算剩余秒数。策略服务检查执行权与委派权，并限制最终期限；拒绝、待审批和未知决定均不启动子任务。

TEXT

```
function delegate(parent, spec):
    assert spec.objective is bounded
    assert spec.expected_output_schema exists
    assert spec.acceptance_checks not empty
    ttl_seconds = seconds_until(spec.deadline, trusted_clock.now())
    if ttl_seconds <= 0 or parent.cancelled:
        return NOT_STARTED
    decision = policy.authorize_delegation(parent.identity, spec, ttl_seconds)
    if decision.decision not in {ALLOW, CONSTRAINED_ALLOW}:
        return NOT_STARTED(decision.decision)

    resources = lifecycle.open_record(spec.subtask_id)
    outcome = FAILED
    try:
        lease = resources.reserve(spec.budget, parent.allocatable_budget)
        capability = resources.issue_bound_authorization(decision)
        resources.enforce_constraints_or_fail(decision)
        workspace = resources.create_isolated_workspace(spec.inputs)
        context = build_context(spec.inputs, spec.context_policy)
        child = resources.spawn(context, workspace, capability, lease)
        result = child.await_or_cancel(spec.deadline, spec.cancellation_token)
        artifact = validate_and_export(result, spec.expected_output_schema)
        checks = verify_subtask(artifact, spec.acceptance_checks,
                                budget=parent.verification_reserve)
        outcome = SubagentResult(artifact, checks, resources.metered_usage)
    except error:
        outcome = FAILED_WITH_EVIDENCE(error, resources.event_refs)
    finally:
        cleanup = resources.close_or_quarantine()

    if not cleanup.closed:
        return RECONCILIATION_REQUIRED(outcome, cleanup.record_ref)
    return outcome

function integrate(parent, results):
    reject_unverified_or_expired(results)
    conflicts = detect_semantic_and_resource_conflicts(results)
    if conflicts:
        return RESOLUTION_REQUIRED(conflicts)
    with managed_clean_workspace() as workspace:
        candidate = merge_and_export_immutable(workspace, results)
    return parent_completion_gate(candidate)  # 对合并版本重新验收
```

辅助函数承担明确前提。reserve 必须原子扣减父任务可分配额度，后者已扣除集成、验证与应急预留；每次模型或工具调用前检查租约，持续计入 token、费用、调用次数与时间，耗尽就停止派发。spawn 在启动时重查授权有效性，并将父取消与契约取消信号关联。validate_and_export 核验结构与来源引用，将持久产物导出后才允许清理

工作区；来源可追溯仍不等于语义正确。

资源操作必须先写入持久生命周期记录；创建超时但结果未知时，不能假定资源未产生。close_or_quarantine 在成功、创建失败、超时和取消路径都执行：逐项撤销授权、停止并回收子进程与后代，保存证据后清理工作区，再结算用量并归还未用预留。它须捕获各项清理错误，继续处理其余资源，并以结构化状态返回；任一资源或副作用未闭合，均返回 closed=false。未知副作用转对账，仍可能运行的资源保留额度与清理责任。进程崩溃后的回收器继续处理记录，合并工作区也须在异常时清理。缺少这些前提，这段教学代码就不是完整的恢复实现。

15. 设计原则总结

何时拆分：先以单 Agent 为基线，按可验证产物和依赖关系划分任务，区分子程序、子代理与责任转交。用同成本基线和重复试跑结果评估增量收益，并分别报告每组的模型、任务执行费、候选搜索费、验证费、重试费与人工接管成本。

如何隔离：明确状态与工作区所有权，按工作关系选择上下文，分别核验执行权和委派权。探索需要独立性时避免过早共享判断；预算覆盖整棵任务树并为验证预留，取消、背压、重试与孤儿任务由运行时处理。

如何合并验证：子结果携带产物、来源、检查结果、冲突与未决项。局部通过只是集成输入，合并版本仍要重验；未知副作用或清理未闭合时，保留待对账状态，不以一段“完成”摘要结束责任。

多 Agent 的成熟标志不是屏幕上出现更多头像，而是组织能够精确回答：为什么要拆成这些执行者，每个执行者看到了什么、被允许做什么、产出了什么证据，冲突怎样处理，以及当某个执行者犯错时，系统为何仍能恢复。做到这些之后，“Agent 团队”才不再是 prompt theater，而是可治理的计算拓扑。

第十二章 可观测性、轨迹与评测运营

生产环境中的 Agent 需要记录跨模型、工具、环境、授权策略和人协作的执行链。可观测性的目标是重建可审计的因果关系：系统看到了什么，依据哪项授权执行了什么，改变了哪些状态，又凭什么判断任务完成。提示词和最终回复不足以回答这些问题，模型私有思维也不必作为审计对象。

1. 轨迹是事件图

以仓库任务 TASK2048 为教学例子：模型提出测试请求，授权策略作出决定，执行器返回测试日志，验证器把结果绑定到补丁版本。若其间发生委派或保存检查点，这些事件也要关联到同一次任务。模型轮次、工具请求、授权与审批、执行与观察、产物、检查点、委派、验证和外部提交因而构成基本事件类型。稳定 ID、依赖关系和产物引用把它们连起来；大型输出外置，轨迹保留摘要、哈希与定位符。

TEXT

```
TraceEvent {
  run_id, span_id, parent_id, type, timestamp
  actor, model, tool, policy_version
  input_refs[], output_refs[]
  state_before, state_after
  cost, latency, status, error_class
}
```

并行工具、异步输入或多 Agent 协作都会使轨迹成为部分有序图，单 Agent 也不例外。可恢复任务应以检查点和副作用账本为依据。日志、审计与模型上下文要分别管理权限和保留规则：上下文可压缩，普通运维日志可采样，审计记录则需防篡改并保留关键关联。这是逻辑边界，不要求一律分库，物理隔离按风险与恢复需求决定。

2. 指标分四层

业务层看任务价值、人工节省和错误损失；任务层看完成率、部分完成、升级率和稳定性；运行层看工具调用、重试、上下文、成本与关键路径时延；安全层看越权请求、审批、注入、数据流违规和恢复。这样分层后，问题不容易被平均值掩盖。

仅看平均成功率不够。要按任务族、风险、模型、Harness 版本、工具、仓库规模和上下文长度做分片，并同步观察 pass@k 与 pass^k。Anthropic Agent Evals 这两个指标的差异会暴露：一次最好成绩适合衡量探索能力，持续稳定表现才接近生产体验。

3. 失败分类先于优化

失败至少可拆成任务规范、上下文选择、推理计划、工具选择、工具执行、环境、权限、验证器、协调、外部依赖和模型能力。如果把所有失败都记成 agent_failed，团队最终只能靠改提示词猜测，难以及时收敛。

归因应追到“最早可纠正事件”，而不是只盯最后一个报错。测试失败可能来自错误 patch，也可能来自依赖未安装；过早完成也可能因为完成契约缺失，而不一定是模型“没认真”。应允许多标签和置信度，并保留人工纠正记录。

4. Eval 是持续运营系统

评测集要持续从生产事故、人工升级、低置信度轨迹、能力边界和安全红队样本补充。能力评测（capability eval）用于找出尚未掌握的任务，回归评测（regression eval）用于守住已掌握能力；高通过率的能力题应转成回归题。Anthropic Agent Evals

每个任务应保存输入快照、评分规则、环境摘要、可见性和退役原因，有参考解时一并登记。Harness 变更应与稳定基线重复比较，报告置信区间、成本和关键切片。

评测结果还要区分三个层次：契约与状态机检查，验证拒绝、取消和恢复等规则；行为评测，观察是否读取必要证据、调用验证器；业务终态评测，检查产物或目标环境是否满足验收。Google 2026-09-09 的工程文强调行为与端到端

评测互补，并提醒复杂任务可能有多条正确路径，不应锁死工具序列。行为评测实践 调用了测试工具，只能证明发生过调用，仍须核对测试针对的是哪个版本。

组件对照也能辅助归因。一项 2026-09-17 的研究固定执行循环，改变规划、动作接口和上下文管理，发现收益随模型和窗口预算变化；其四个模型与限定消融设置不足以证明某种工具或规划方案普遍最优。Harness 组件实证研究 v1 这是作者报告的实验，不能当作本书已复现结果。

5. 在线监控与离线评测闭环

TEXT

```
production traces
  → privacy filtering
  → failure clustering
  → curated eval candidates
  → independent labeling
  → regression/capability suites
  → candidate harness evaluation
  → canary → production
```

线上反馈不应未经校验就自动写回提示词或记忆，否则攻击样本和偶发偏好会被固化。采集、筛选、标注与发布应有各自责任。用户满意度也不是唯一奖励信号：Agent 可能通过迎合、隐藏风险或省去必要确认抬高短期评分。

6. Trace replay 的边界

模型调用与外部世界并不总能完整重放。可靠的回放（replay）要固定输入、模型快照、采样参数、工具版本和环境镜像；对不可重放 API，需要使用录制响应或模拟器。replay 的目标是定位差异，而不是装作绝对复现。

隐私和安全同样关键。轨迹里可能有源码、个人身份信息（PII）、token 和模型生成的恶意内容。进入分析平台前要先做分类、脱敏、租户隔离和最小保留；研究者访问 held-out 与生产数据必须经过审计。

7. 运营仪表盘

一个可用的运营仪表盘应能回答：哪类任务失败最多；失败从哪个层开始；哪个版本引入退化；自动完成是否真的降低了人工总成本；成本上升来自模型、上下文还是重试；哪些权限请求最常被拒；哪些 verifier 最不稳定。

最终，可观测性服务的不是“好看”的 trace UI，而是事故恢复、工程归因和受控进化这三个闭环。没有可用轨迹，Harness 只能靠 anecdote 演进；没有独立 eval，轨迹优化又容易变成对历史样本过拟合。

8. 一条工具完成事件的示例

事件结构应支持大型输出外置、敏感字段分级和供应商原始数据的独立引用。以下是教学示意，ID 与哈希已简化，不是本书实跑记录；它只表示可观察结果。

JSON

```
{
  "event_id": "evt_01J8Z7",
  "run_id": "run_TASK2048_A3",
  "span_id": "tool_017",
  "parent_span_id": "turn_006",
  "type": "tool.completed",
  "timestamp": "2026-08-27T09:31:14.223Z",
  "actor": "runtime:codex",
  "tool": {"canonical": "repo.test", "provider": "exec_command", "version": "4"},
  "policy_decision": "pd_8821",
  "input": {"ref": "artifact:sha256:11ad...", "classification": "internal"},
  "output": {"ref": "artifact:sha256:90bf...", "exit_code": 1},
  "state": {"workspace_before": "git:8f31b6e", "workspace_after": "git:dirty:4e19..."},
```

```
  "latency_ms": 18241,
  "cost": {"compute_usd": 0.012},
  "status": "error",
  "error_class": "TEST_FAILURE",
  "vendor_payload_ref": "secure-artifact:sha256:772e..."
}
```

event_id 支持去重，父事件用于导航，工作区哈希关联被检查的版本。policy_decision 只是查找授权决定的键；要核验执行是否遵循该决定，还需受保护的记录绑定动作参数、执行身份、资源版本、策略版本与结果。原始输出可能包含源码或密钥，应限制访问；普通运营角色只看到必要摘要和定位符。

9. Trace 完整性与采样

高流量平台往往想采样，但 effect、policy、approval、checkpoint、verification、commit 这类事件不能像普通调试日志那样随机丢弃。应按重要性分层：审计骨架全量保留；大输出只保留 hash，并按风险控制原文留存；性能 span 可以按任务量和异常率自适应采样。

对固定观察窗口内已终止的运行，可报告 具备全部必需事件及有效产物引用的运行数 / 已终止运行数，但必须另行对账期初未闭合与本期新建运行，列出仍在进行、超时未闭合和已终止的数量。否则崩溃后没有终止事件的任务会从分母消失，虚高完整率。还应统计孤立事件、重复事件、引用不可读和依赖异常；按保留规则合法删除的产物要留退役记录，并区分“保留期内证据完整”与“原文仍可重放”。

例如只保留成功运行的完整日志，就会系统性删去最需要定位的失败轨迹。应在合法保留期内保留失败、安全告警、人工接管和未知错误的关键证据；普通成功任务的调试细节再按任务族抽样，同时执行数据最小化。关键事件全量保留不等于永久保存所有原始内容。

10. Eval 生命周期与污染控制

生产问题进入评测前，要经过候选、复现、清洗、标注和负责人审批。开发集用于调试；验证集用于比较并选择候选模型、Harness 或配置，不能按候选表现临时挑选试题；密封测试集只在预定时机进行独立终测；已频繁暴露或饱和的任务转入回归集或退役。各集合应分别管理用途、访问权限和版本，并不要求不同物理目录。

保留集（held-out）是未直接用于生成或训练的宽泛称呼，其中用于选择候选或修复诊断的部分承担验证集职责，不等于密封终测。密封集由独立服务按事先确定的访问次数、反馈粒度与停止规则使用，标签和隐藏检查不向候选开放。即使只返回通过与失败，多轮自适应反馈也会泄漏信息；一旦用于修复，就应记录暴露并重新界定用途，不能继续宣称它是未接触的独立终测。

11. 从指标到行动

每个告警都要绑定责任人和处置流程。未知工具错误突增时，先限制相关运行配置，排查服务故障、接口结构和版本，再决定是否回退；错误完成率上升时，先查完成契约与验证器；成本上升则分别检查模型请求、上下文、工具重试和人工等待。

如果仪表盘只能显示红色曲线，但跳不过去看代表性 trace、版本差异和受影响任务，它就不算运营系统。反过来，trace UI 若只能看到每个 token，却回答不了“哪个版本引发生产错误”，它也只是调试看板的玩具。

12. 可观测性的边界

更完整的日志不代表更安全。源码、客户数据、工具输出和 prompt injection 内容会在 trace 平台变成高价值资产。默认采集字段白名单、用途限制、租户隔离、保留期和删除流程要与可观测性体系同时设计。高敏任务可只保存结构化 outcome，并保留加密原文引用，由受控流程临时解密。

可观测性最终服务于责任：谁在什么版本、什么授权和什么环境下做了什么，系统如何知道结果正确，失败后如何恢复。它不应被用来推断或展示模型不可验证的内部心理状态。

第三篇 产品：当代主流 Harness 的不同答案

本篇导言：用同一组问题读懂六种实现

第二篇说明一个 Harness 要解决哪些问题，本篇进一步看实际产品把这些责任放在哪里。六个案例分别是 Claude Code、OpenAI Codex、Cursor、DeepSeek Harness、OpenHands 和 pi-mono。它们的共同点是围绕模型组织行动，差异则在组件划分、状态归属和控制方式。先看总体架构，才能理解同样名叫“工具”或“子智能体”的功能为什么并不等价。

每个案例沿下面的顺序展开。读者可以顺着一章理解一个系统，也可以沿某一行横向比较六种实现：

产品章要回答的问题	对应第二篇	阅读时应找到的具体内容
整体架构如何分工	第五章	入口、核心、模型服务、执行环境与状态存储之间的关系
Loop 怎样运行和恢复	第六章	请求怎样推进，工具怎样回传，什么时候继续、暂停或停止
上下文怎样组织	第七章	指令、检索、会话历史、压缩和跨任务经验分别怎样进入请求
工具与 MCP 怎样接入	第八章	注册、发现、参数校验、分派、执行与结果处理
权限与沙箱怎样落地	第九章	谁批准操作，什么边界限制实际访问，凭证放在哪里
子智能体怎样委派	第十一章	子任务拿到什么上下文和权限，结果怎样收回，取消怎样传播
怎样验证和观察	第十、十二章	产品记录什么，哪些事实还需要宿主平台验证
一次任务如何穿过架构	上述章节的综合	从输入到工具反馈和最终交接，组件如何一起工作

各章最后的贯穿任务是解释流程的教学场景，除非另有运行证据，不表示已用该商业产品完成了实验。没有内建的机制也会明确写出：可以通过扩展获得的功能，不等于默认加载；独立上下文，不等于独立操作系统权限；循环结束，不等于业务验收通过。

本轮于 2026-10-29 重写架构说明，具体产品事实以章首声明的文档截面和固定源码版本为准；没有把全部产品统一升级到同一天的最新版。开源代码用于解释可检查的控制流，闭源产品依据公开接口与工程说明，不补写无法验证的内部类名、调度器和持久化事务。托管循环、自管工具执行和本地运行仍分别讨论。

第十九章把六个案例放回同一组原理问题中比较，再以 Deep Agents、Microsoft Harness 补充框架视角。比较的是设计取舍和运营责任；任务质量、费用及可靠性仍需在同等条件下测量，本书不填推测分数。

第十三章 Claude Code：薄循环、厚运行时

> 资料截面与实际访问日期：2026-10-29。公开源码基线为 Claude Agent SDK Python 0.2.174、commit 27858cbba9ecee26a06a90b8d39d3b5596121d11；网页快照哈希见本轮研究记录。SDK 源码可解释宿主接口，不能据此宣称审计了闭源 Claude Code 核心。本章是文档与源码研究，未运行产品端到端实验。

Claude Code 的架构重点，在于把模型选择下一步的循环放进一个持续管理上下文、工具执行和会话状态的 Harness。官方将工作概括为收集上下文、采取行动、验证结果；每次工具返回的新信息都可能改变下一步，而不必预先列出完整工作流。运行原理“薄循环、厚运行时”是本书对这种职责分配的归纳，不是厂商公布的内部模块名。

1. 整体架构与职责边界：谁拥有循环

Claude Code CLI 面向交互开发；Agent SDK 把同一套工具、循环和上下文管理提供给 Python／TypeScript 应用，由应用运营进程。Client SDK 则直接调用 Claude API，工具循环由调用方或其 tool runner 驱动。Managed Agents 是另一个托管 Harness 接入面，不能把它的远程 session、sandbox 和管理接口当作本地 CLI 的内部结构。接入面比较

公开 Python SDK 给出了可核查的边界：SubprocessCLITransport 构造带 --output-format stream-json 的命令，以 stdin/stdout 管道启动 Claude Code binary，并传入工作目录和环境。另一侧的 Query 处理 can_use_tool、hook_callback、mcp_message 控制请求，把应用回调结果写回管道。这解释了“作为库使用”的含义：应用握有控制接口，循环仍由 Claude Code 运行时驱动。进程传输源码、控制协议源码

下面是依据上述接口作出的作者抽象，表示职责和信息流，不表示闭源程序的类或线程布局。

TEXT

```
用户 / 业务应用
      | 任务、选项、后续输入
CLI 界面 / Agent SDK 宿主
      | 输入与控制请求              ^ 消息流、结果、会话 ID
Claude Code 运行时 <--------------> Claude 模型服务
      | 上下文组装、循环、工具许可        模型请求工具 / 生成回答
      +--> 内置 Read / Edit / Bash --> 工作区与命令结果
      +--> MCP 分派 --> 外部服务器 / SDK 进程内 handler
      +--> Agent --> 子代理上下文 --> 最终报告
      +--> 本地会话 JSONL；可选 SessionStore 镜像
宿主 hooks / 权限回调 <-----------> 运行时控制协议
```

对应第五章，模型负责提出动作，Harness 负责使动作成为受控调用，环境承受读写，反馈返回下一轮。选用 SDK 可以复用这一运行机制；应用仍须决定工作区归属、运行身份、任务验收和外部系统的事务边界。这是架构取舍，而不是把业务应用缩减成一个提示词入口。

2. Agent loop 与恢复：消息流不是业务事务

一次 SDK 调用先接收提示词、系统指令、工具定义和历史，通常产生 system/init 元数据；模型输出 assistant 消息中的 tool_use，运行时执行工具，再以 user 消息中的工具结果供下一次推理使用。循环持续到模型不再请求工具，随后产生 ResultMessage。因此宿主消费消息流时，不必为每个 Read 或 Bash 自行实现“调用模型—执行工具—再调用模型”。循环与消息生命周期

工具拒绝会作为观察返回模型，使其换路或解释受阻；结束也可能来自 maxTurns、maxBudgetUsd 或执行错误。宿主应读取 success、error_max_turns 等 result subtype，而不能只看有没有最后一段文字。预算在模型响应到达后核算，故触及上限的那次响应仍可能使估算费用越过阈值；这是一道运行预算门，不是逐 token 的硬额度锁。工具拒绝、预算与结果

TypeScript 流式接口还提供 interrupt() 请求中断查询；已经排队的用户消息可能在中断后继续执行，因此宿主需要分别管理当前执行与待处理输入，而不能把一次中断当成整个会话的取消。中断及队列语义

持久化首先保存对话。CLI 将消息、工具调用和结果写入本地 JSONL；SDK 的 resume 延续会话，fork 建立另一条会话历史。文件检查点是另一套机制：启用后备份 Write、Edit、NotebookEdit 所修改的文件，rewindFiles() 回退文件却不回退对话；不能把 Bash 修改、外部 API 写入或所有子代理编辑视为同样受到保护。会话、文件检查点

跨主机恢复可以接入 SessionStore 的 append/load。其数据路径是“子进程先写本地，SDK 再镜像到自有存储”，镜像失败可发出 mirror_error 并继续运行；文件检查点不能与该存储选项同时启用。外部会话存储 对应第六章，这些机制分别恢复“先前讨论了什么”和“部分文件曾是什么样”，并未建立第三方写入的恰好一次语义。若工单已经创建而响应丢失，应用仍应凭业务操作标识回读，而不是因会话可以恢复就重新提交。

3. 上下文与会话：压缩、缓存、记忆各做什么

运行时每轮组装的是模型当次可见的信息。历史消息、读入文件、命令输出和工具 schema 都消耗窗口；磁盘上的整个仓库并不会因为可访问就全部进入模型。CLAUDE.md 承载人写的持续说明，auto memory 保存 Claude 积累的经验，两者都作为上下文使用，不能代替执行权限。上下文组成、指令与记忆

SDK 与 CLI 的装配起点也有差别：SDK 不设置 systemPrompt 时使用最小提示，选择 claude_code preset 才采用 CLI 的系统提示；CLAUDE.md 则由 settingSources 控制，与是否采用 preset 分开。当前文档中的默认 query() 会读取文件配置，显式选择来源可以收窄用户与项目配置的影响，但 managed policy 和全局配置另有规则。系统提示、配置加载 这使应用能够分别决定“通用行为”和“项目知识”，也要求部署配置成为可复现的任务输入。

接近窗口上限时，自动 compaction 用摘要替代旧历史，并在 SDK 流中发出 compact_boundary；PreCompact 可以在压缩前处理归档，/compact 可主动触发。摘要保留关键决定，但早期细节可能丢失。自动压缩 依照第七章，压缩是在有限窗口内选择继续工作所需的状态；原始会话日志与当前上下文因此不应被当作同一份资料。

Prompt cache 解决的是重复计算。Claude Code 将稳定内容放在请求前部、把新交互追加在后，模型服务复用精确匹配的前缀；它不是按文件独立缓存，也不替模型保存跨请求记忆。压缩会另发总结请求，并使后续请求的对话层前缀改变；系统和项目层能否继续命中，取决于内容是否保持一致。Claude Code 缓存、API 前缀语义 因而一次主动压缩可以换来较短的后续上下文，同时付出总结与重建缓存的成本；不能把“token 变少”直接等同于“本轮更便宜”。

4. 工具、MCP 与扩展：从可发现能力到执行结果

内置 Read/Edit/Write 处理文件，Glob/Grep 提供检索，Bash 执行命令。模型选择工具和参数，运行时处理执行与回传；多个工具也不是一律并行，公开契约允许只读工具并发，而修改状态的工具按序执行。自定义工具默认按序，readOnlyHint 才表达可并发的只读意图。工具执行 这让模型看到的能力接口与实际调度相分离，对应第八章的 ACI：工具不仅要能被选择，还要有稳定的结果和错误形式。

MCP 的路径从服务器注册开始：mcpServers 可以描述本地 stdio 进程、远程 HTTP/SSE 服务或 SDK server；工具使用 mcp__<server>__<tool> 名称。发现工具不意味着立即把全部 schema 填满上下文，当前 tool search 默认延迟加载定义，模型需要时再找出相应能力。MCP 注册、命名与发现 这样减少的是无关工具描述的常驻成本，真正调用时仍须获得定义、完成许可和收集返回值。

SDK 自定义工具把名称、描述、输入 schema 与 handler 组合，经 createSdkMcpServer／create_sdk_mcp_server 注册。handler 在应用进程中运行，返回 content，还可返回结构化结果或 isError；公开 Python 分派器将运行时送来的 mcp_message 路由到相应 server，再把响应写回。自定义工具、MCP 控制分派 这里的“进程内”指应用 handler 所在位置，不表示模型服务直接执行 Python 函数。

扩展还会改变循环的不同位置。Skill 的描述先被发现，正文按需进入上下文；hooks 则按事件和 matcher 运行代码，例如 PreToolUse 在动作前检查参数，PostToolUse 记录结果，PreCompact 处理摘要前状态。SDK 回调在

宿主执行，配置中的 command hook 运行命令，不能把 hook 当成要求模型自觉遵守的一段说明。按需上下文、hooks 分派 作者据此把扩展分成“补知识”和“改变控制路径”两类，以便判断它究竟增加上下文，还是引入新的执行主体。

Plugin 将 skills、agents、hooks 和 MCP servers 打包，SDK 可按本地路径加载。扩展包装 因此审查一个插件时，应沿这些组件各自的路径判断它改变了什么；安装一个包并不会使其中的知识文件与可执行 hook 获得相同的信任等级。

5. 权限、沙箱与凭证：批准动作与限制能力

工具请求先要回答“是否获准执行”。SDK 文档的基本求值顺序是 hooks、deny 规则、ask 规则、permission mode、allow 规则，最后才是 canUseTool；各模式和扩展另有例外。PreToolUse 可以直接拒绝，但它返回 allow 也不能跳过后续 deny/ask。应用回调接收工具名和输入，返回允许、修改后的输入或拒绝理由。权限求值、回调协议源码

这条顺序解释了两个不同接口的用途：allowedTools 是预批准集合，不是“除此之外工具不存在”的白名单；已被规则或模式批准的调用通常不进入 canUseTool。若应用要在每次工具调用前核验业务范围，应使用相应 PreToolUse hook；若要在模型面前收窄能力，应配置工具集合或移除工具。规则与回调边界 作者的设计判断是把常见安全读操作做成确定规则，把需要业务裁决的动作留给回调，避免审批 UI 承担所有运行时策略。

获准之后还要回答“进程能触及什么”。内置 Bash sandbox 通过操作系统限制 shell 命令及其子进程的文件与网络访问：macOS 使用 Seatbelt，Linux／WSL2 使用 bubblewrap，网络通过代理检查目标主机。但 Read/Edit/Write、本地 MCP 服务器与 command hooks 不在这层 shell 沙箱内；前者使用工具权限，后者运行于宿主环境。沙箱覆盖范围 对应第九章，权限控制调用入口，隔离约束实际执行，两者要共同覆盖应用选用的路径。

凭证也必须按所在位置理解。MCP 配置可把凭证放入 server 环境或 HTTP headers；Bash sandbox 的 credentials 可显式禁止读取文件、移除环境变量，也可配置占位符与代理注入，但没有默认覆盖所有秘密的内置 deny 清单。MCP 认证、凭证保护 若要求整个 SDK 应用、hooks 和工具都处于同一隔离边界，官方部署指南提供容器／虚拟机与外置凭证代理的组合思路。安全部署 这些部署责任不能从 Managed Agents 的凭证库设计推导为本地默认能力。

6. 子智能体与委派：隔离的是哪一份状态

通过 agents 定义 AgentDefinition，应用为角色配置 description、prompt、tools 等；主模型通过 Agent 工具委派。普通子代理从自己的系统提示、委派 prompt、工具定义和项目说明启动，不携带父对话历史。因此父代理必须把任务所需的文件位置、错误信息和已作决定写入委派输入。定义与继承 对应第十一章，这个输入就是一份最小工作契约，不能只写“继续刚才的工作”。

fork 是另一种起点：它复制当前对话再分支；官方文档注明 Claude Code v2.1.302 起可以由 Agent 的 subagent_type: "fork" 发起。普通子代理适合过滤无关历史，fork 适合需要已有推导的分支，两者之后的工具过程都可与父上下文分开。fork 子代理、上下文管理 这种上下文隔离本身没有承诺独立操作系统身份、独占工作区或新的凭证边界；作者建议把需要并发写入的文件归属明确放进委派契约。

回收结果时，父代理收到的是子代理最终报告，可能再总结后交给用户；完整子轨迹保存在独立文件，SDK 事件可以用 parent_tool_use_id 关联委派。后台工作通过稍后的完成通知进入主会话，权限提示也在主会话处理。报告与轨迹、后台执行 因而“工具日志不占父窗口”与“宿主拿不到工具日志”是两回事；报告压缩了模型交接内容，观测链仍可保留细节。

当前接口允许子代理继续委派，并提供深度、并发与费用限制；TypeScript stopTask(taskId) 可停止后台任务。委派上限、任务控制 取消传播则须按具体接口验证：本章固定的 Python 源码在权限和 hook 回调中仍传入 signal=None，不能假定所有语言接口都有完整的取消信号链，更不能把停止任务解释成撤回已完成的外部写入。回调信号源码

7. 验证与可观测性：从循环结束到可验收产物

Claude 可以在循环中运行测试，把错误输出变成下一轮修改依据；Stop hook 也能阻止结束，让代理继续工作。验证循环、Stop 事件 但第十章要求区分“运行时正常结束”与“业务验收通过”。本书建议在任务入口就定义验收命令与产物，再由独立验证步骤检查补丁和测试记录。hook 是插入检查的机会，不会凭空知道业务正确性，也不能让已经发生的外部副作用消失。

运行中，宿主可以记录 assistant/tool/result 消息、压缩边界、子代理关联和权限决定；Claude Code 还可通过 OpenTelemetry 导出指标、事件及可选 traces，工具参数和内容的导出受配置控制。消息流、可观测性 对应第十二章，作者建议用任务 ID 连接 session、工具调用和最终产物，把“模型为什么继续工作”与“平台为什么接受结果”分别保存，避免只留下自然语言总结。

核算同样需要辨别统计边界：同一次模型响应可能拆成多条消息，使用量应按 message ID 去重；result 的 usage 只覆盖主循环，全树统计使用 modelUsage／model_usage。total_cost_usd 是客户端价格表计算的估算，不是权威账单。费用统计、结果字段 这样才能比较“主窗口更短”是否真的抵消了额外子代理请求，而不把上下文整理效果误报为总成本收益。

8. 一次任务穿过架构：修复登录回归

下面是作者设计的接入示例，不是实测记录。任务是“修复过期会话导致的登录失败，交付补丁和回归测试，不部署”。应用先建立隔离工作区，固定 SDK 与 binary 版本，选择编码用 system preset、项目配置来源和运行预算，并把验收目标作为任务输入。对应的提示、配置来源和预算均有独立接口，应该在一次任务的运行记录中共同保存。提示装配、循环控制

模型先请求读取失败测试与认证代码。运行时完成许可、执行读取、回传内容；若委派只读审查，父代理给出具体路径和失败信息，子代理只交回发现及证据位置。主代理再决定修改并运行测试。若需要读取缺陷工单，MCP 将工单服务的工具定义带入调用路径；本例只开放读取业务记录的工具，不把写工单权限附带授予。工具循环、子代理输入、MCP 权限

长时间排查后，自动压缩让主窗口保留当前假设和关键决定；缓存复用尚未改变的前缀。测试再次失败就把输出送回循环，而不是把失败本身直接认定为整个任务结束。压缩与反馈、缓存重建 本例的工程要求是让测试结果指向具体工作区版本，并把失败日志留在产物存储，摘要只承担定位作用。

若宿主中断，应用先确认日志与工作区仍可用，再选择 resume；如果用了外部 SessionStore，还要检查是否发生镜像失败。测试通过后，平台重新验证补丁、汇总授权与工具记录，再把任务标记为可交付。恢复边界 最终取舍是：SDK 复用了成熟的代理循环，却仍由应用运营进程、持久存储和验证门；需要购买托管循环时另行评估 Managed Agents，而不是在本地适配器里混入其 wake/getSession 架构假设。托管边界、Managed Agents 架构

第十四章 OpenAI Codex：协议化的 Agent Core

> 本次查阅日期：2026-10-29。内部实现继续采用 Rust 0.155.1，固定提交 be2951ea34f0d295ed0becf97079f92fa5f6950e，不声称它是当前最新版；官方网页按本次访问日期引用。本章分析公开的本地 Harness，不用客户端源码推断托管 Agents API 的服务端实现。版本定义

1. 整体架构与职责边界：协议包住的是运行时

理解 Codex，应先把“界面上的聊天”拆成三个层次：产品决定用户正在处理什么业务；Harness 决定如何把任务持续运行下去；模型决定下一步输出消息还是调用工具。官方对平台化的解释正是复用 agent loop，让应用继续拥有业务上下文、工具和控制界面。App Server 是进入这个运行时的双向接入面，不是循环本身。平台说明

在固定源码中，ThreadManager 持有 ThreadId → CodexThread 映射，并通过 ThreadStore 管理存储；CodexThread 包装一个 Session 及其操作、事件端点。真正的运行状态在 Session：历史与配置状态、活动 turn、待处理输入以及执行所需的服务。这样的分工允许客户端更换界面，同时让会话状态归运行时管理。线程管理、线程包装、Session

下面是作者依据这些组件绘制的职责图；箭头表示主要调用与回传关系，不表示每条路径都跨进程。

TEXT

```
产品客户端：业务对象、交互、审批呈现
        | 请求 / 通知 / 服务端反向请求
        v
App Server -> ThreadManager -> CodexThread
                                    |
                  Session + SessionIo
                  |                 |
              run_turn          ThreadStore
                  |           历史、检查点、设置
       +----------+-----------+
       |                      |
 ModelClient              ToolRouter / ToolRegistry
 模型请求与响应                  |
       ^              +-------+---------+
       |              |                 |
       +-- 工具结果 -- 执行 runtime     MCP / 宿主工具
                      |
               审批 + SandboxManager

AgentControl：同一根任务树内的子代理注册、通信与生命周期
```

App Server 把这种执行活动投影为 Thread / Turn / Item：线程包含多轮请求，一轮请求包含消息、命令、文件变化等 item。stdio 使用按行分帧的 JSON，消息省略 jsonrpc 字段；还允许服务端反向请求审批。协议因此既承载内容，也承载控制权的交接。协议与对象

接入层并非可互换的名称。这个固定版本的 TypeScript Codex SDK 通过子进程封装 codex exec 的 JSON 输出；本次查阅的官方文档则明确 Python Codex SDK 控制本地 App Server。应分别核验语言实现，不能把它们等同于通用 Agents SDK，或把某个 SDK 的能力自动移植到另一个 SDK。TypeScript 执行封装、Codex SDK

2. Agent loop 与恢复：一轮请求可以包含多次采样

对应第六章，Codex 的 turn 不是一次模型 HTTP 请求。SessionIo 以提交队列接收操作，以事件队列输出进展；run_turn 为一次模型采样捕获 StepContext，使上下文、公布给模型的工具以及随后调用工具时使用的设置共享同一请求视图。这样可以避免配置刷新后，模型看到的能力与执行器实际采用的能力相互错位。队列端点、循环与请求快照

每次采样从当前历史构造输入，收到工具调用后执行并纳入后续采样；循环同时检查模型是否需要继续、用户是否追加输入、是否达到上下文预算。没有后续工作时才进入停止路径，Stop hook 仍可能要求继续处理。这里的设计动机是让“继续、等待还是结束”成为宿主状态机的判断，而不是只看模型刚刚说了“完成”。run_turn

失败恢复也有层次。采样代码区分不可重试错误、上下文超限与使用额度问题；流式连接错误走重试状态、退避和满足条件时的传输回退，部分连接重试行为还受 feature 配置控制。它恢复的是模型请求链路，不能由此推出数据库写入或远程工单创建具有“恰好一次”语义。采样错误处理、流重试

中断处理活动任务、取消信号与排队输入，终态事件后还有显式的历史刷新屏障；恢复则从 rollout 中选取仍有效的压缩检查点及后续记录，重建历史和设置基线。其恢复单位是可持久化的会话状态，不是一个暂停在任意指令上的操作系统进程快照。任务结束与中断、历史重建

3. 上下文与会话：模型窗口、宿主事实和记忆各有归属

对应第七章，输入不是把聊天记录拼接后直接发送。AGENTS.md 加载器沿项目根到当前目录收集项目指令，支持 AGENTS.override.md，并受项目可信状态与字节预算约束；run_turn 再记录环境和设置变化，使用当前步骤对应的上下文构造请求。因此目录、权限或工具变化也是会话需要表达的事实。项目指令装配、上下文更新

ContextManager 保存有序的 ResponseItemEnvelope、token 信息、用于差异更新的上下文基线，以及独立于模型窗口的 retained_context。工具输出进入历史时可按截断策略缩短；for_prompt 再把历史转换成模型请求输入。这解释了为什么界面上可见的记录、落盘的记录和某次实际送给模型的内容不能被当作同一份数据。
ContextManager

压缩改变工作窗口，却不应替用户改写授权。源码专门用 replace_compacted 保留这条边界；本地压缩路径生成摘要、保留一定用户消息，并根据发生在轮前还是轮中决定何时补回初始上下文。轮中的上下文注入还要保持摘要所在的相对位置；远程压缩则由 provider 能力选择另一条路径。压缩由此是一项上下文重建操作，而不只是删除旧文本。压缩替换、本地压缩、路径选择

跨会话记忆又是另一条流水线。该版本的设计文档描述：在记忆功能开启、非临时根会话且有状态库等条件满足时，后台先从合格 rollout 抽取结构化记忆，再持有全局阶段锁汇总共享记忆材料。第一阶段可并行，第二阶段串行，是为了避免多个任务同时改写同一记忆工作区。它有启动条件、筛选规则和失败重试，不等于每条消息都会永久保存为可靠知识。记忆流水线

4. 工具、MCP 与扩展：声明能力和执行能力必须对齐

对应第八章，ToolRouter 同时持有模型可见的工具规格与 ToolRegistry；它把模型响应解析为含名称、调用 ID 和 payload 的工具调用，并查询工具是否允许并行。ToolRegistry 再按名称找到 runtime、检查载荷类型、执行 hooks 和处理结果。设计重点在于，一份最终工具计划同时确定“模型能请求什么”和“宿主能执行什么”。工具路由、注册与分派

MCP 接入在这个边界上增加适配。客户端初始化时协商能力并列举工具；McpHandler 把 ToolInfo 转成模型工具规格，再把调用送入 MCP 调用路径。结果经工具输出路径回到历史，模型才能据此继续。只读提示等元数据还可能影响并行许可，因此服务端声明不仅是帮助文本，也参与调度决策。MCP 初始化与发现、McpHandler

扩展不只有 MCP。DynamicToolHandler 接受线程提供的工具规格，并在调用时向宿主请求结果；技能扩展则负责目录、选择、provider 和指令注入。前者增加动作入口，后者增加任务知识与操作方法，两者可以组合，但技能文字本身并没有创建新的隔离执行器。动态工具、技能扩展

工具 hook 也有明确时间边界：前置 hook 可以阻止调用，后置 hook 拒绝的是已经返回的结果，不能撤销此前完成的动作。对企业工具而言，这意味着拒绝不可信结果与阻止业务写入必须放在不同位置，不能用一个“验证失败”状态含混地覆盖两者。分派与 hook 边界

工具数量增长时，还要区分“已经注册”与“本轮直接公开”。路由器可识别延迟发现的工具，动态工具也保存 Direct / Deferred 暴露方式；模型不必在每轮都看到所有工具的完整定义。作者将这一设计理解为上下文预算与能力发现之间的折中：缩小初始工具目录，同时保持发现后的调用仍落到同一注册表，不让搜索结果成为绕过执行校验的新入口。可见性判断、动态工具暴露方式

5. 权限、沙箱与凭证：决定能否做，与限制如何做

对应第九章，审批策略回答“此动作是否需要进一步许可”，沙箱回答“执行时实际允许访问什么”。官方文档将两者明确区分；源码中的 ToolOrchestrator 把审批要求、首次沙箱选择、执行尝试和失败后的升级路径集中起来。工具被判为禁止时直接返回拒绝，不能把所有沙箱失败都解释成应该自动提权。审批与隔离、ToolOrchestrator

SandboxManager 进一步把抽象权限、命令、工作目录和网络设置变成执行请求。固定版本的后端类型包括 MacosSeatbelt、LinuxSeccomp 和 WindowsRestrictedToken，具体选择还取决于平台与启用条件。共同权限模型让上层表达工作区访问范围，平台实现负责落到系统机制；这并不证明各平台拥有完全相同的隔离效果。沙箱转换与平台选择

执行隔离也不能替远程业务系统认证。MCP 是另一条服务调用路径，宿主动态工具又把执行交回应用；企业接入时，应在这些服务端检查租户、资源和动作权限，而不能把本机文件沙箱当成远程数据库的权限模型。这是由调用边界导出的架构要求。MCP 调用适配、宿主动态工具

登录凭证有单独存储职责：AuthCredentialsStoreMode 可以选择文件、系统 keyring、自动选择或临时存储，文件后端涉及 auth.json。因此“模型不直接处理密钥”与“本机没有持久凭证”是两个不同命题；部署时应把凭证存储方式作为运行环境配置，而不是写进任务提示词。凭证后端

6. 子智能体与委派：子线程共享控制面，各自运行循环

对应第十一章，Codex 的子代理是受管理的线程，而不是给同一条历史加一个角色标签。AgentControl 在同一根任务树内共享注册表、执行限额和预算，并持有指向线程管理器的弱引用；这种设计把父子关系放在控制面，同时避免管理器与会话互相持有强引用形成生命周期环。AgentControl

以该版本 V1 路径为例，spawn_agent 接受任务输入，可选择 fork_context，记录父线程、父 turn、根 turn 和环境选择，并检查派生深度。创建时会应用角色和模型设置，再继承当前审批策略、工作目录与权限配置。由此可以把上下文隔开，却不能把“另一个线程”推导为“另一个文件系统”或“另一套业务账号”；需要文件隔离时，工作区仍须显式安排。创建路径、运行时设置继承

结果回收依赖状态订阅。V1 wait 返回代理状态和超时信息；AgentStatus 又从开始、完成、错误和中断事件推导，完成状态可以携带最终消息。Interrupted 在这里不属于普通的最终完成状态，等待超时也不表示子任务已经失败。父代理因此需要区分“现在没有结果”与“已经得到终态结果”。等待、状态推导

取消同样分层：interrupt_agent 向指定线程发送中断操作；关闭路径则记录关闭关系，并关闭从内存派生树可达的活跃后代。这是会话树生命周期管理，不是文件写入或外部服务的事务回滚。固定源码还包含 V2 委派工具实现，所以上述 V1 名称是具体路径说明，不是所有版本唯一不变的公开契约。指定线程中断、关闭树、V2 创建实现

7. 验证与可观测性：看到执行，不等于证明正确

对应第十、十二章，App Server 提供 item/started、item/agentMessage/delta、item/completed 和 turn/completed 等事件；中断请求的精确名称是 turn/interrupt。事件中的关联标识让客户端把文本、工具和终态归回同一执行过程，但完成通知还需要读取状态，不能仅凭方法名标记业务成功。固定协议枚举、生命周期

运行时还提供工具分派 trace，并按配置构造 OpenTelemetry exporter；这些机制可以帮助定位哪次调用失败、耗时或经历重试，却不会替代回归测试和业务不变量验证。平台应把工具输出、代码差异、测试结果与业务任务关联，尤其在重连后保留尚未确认的动作状态。工具 trace、遥测初始化

本章的运行证据仍限于历史局部探测：2026-09-19 的本机 0.142.5 两套生成 schema 及离线方法、字段和源码对照共 58 项符合预期；三次自施受限启动都在 initialize 响应前以 exit 1 报 Operation not permitted，具体拒绝路径与系统调用未知。没有发送 initialized、thread/start、turn/start 或模型请求，正常握手、取消与模型流均未端到端验证，也不能据此认定产品不可用。原始材料在研究目录 review_20260919/vendor_probe/codex/；本轮只核对文档与源码，没有把阅读源码记成实跑成功。

8. 一次任务如何穿过架构：修复导出遗漏

设想用户要求："修复订单导出跨页遗漏，增加回归测试，并说明是否影响已有数据。"下面是作者设计的架构走查，不是已完成的产品实验。业务应用先提供订单模块位置、失败样本和验收条件，再把任务交给线程；thread/start 建立会话，turn/start 开始工作，客户端持续消费 item 事件。这让业务对象留在应用层，执行过程进入 Harness。线程与 turn 接口

进入 core 后，项目指令、环境与当前历史形成步骤上下文；模型可以通过工具读取实现，再把"核对分页边界条件"委派给子线程。若父线程改代码、子线程只读分析，应用应事先约定文件职责；上下文分开本身不会防止同目录写入冲突。这个例子利用了线程派生与环境继承机制，但读写职责是作者建议的任务组织方式。指令加载、子线程创建

假如需要查询内部字段定义，应用可通过 MCP 暴露只读查询；模型拿到查询结果后选择补丁和测试动作。命令执行经过工具分派与审批、沙箱路径，结果再次进入历史。若安装缺失依赖需要网络权限，应先让权限机制做出决定；拒绝后应调整方案或报告缺失条件，而不是把一次失败无限重放。MCP 工具、执行编排

任务拉长时，压缩保住继续推理所需的窗口；应用还应把验收条件和测试材料存为可再次读取的工件。最终答复应说明复现样本、修复差异、回归结果及数据影响依据，平台再以独立检查决定是否接受。后一项是本书的验收设计：运行时会话结束不能代替业务证明，摘要也不能成为唯一证据库。上下文压缩机制

接入取舍由此变得具体：批处理任务可选 codex exec 或适合该语言的 Codex SDK；需要长会话、中途控制、审批与细粒度进展的产品更适合直接接 App Server。复用循环会减少应用自行实现状态机的工作，同时让应用必须认真维护协议版本、工具权限和结果验收。可复用的是执行机制，业务完成标准仍需应用定义。官方接入层选择

第十五章 Cursor：IDE 原生上下文与云端 Agent

> 资料访问日期：2026-10-29。本章依据官方文档与工程文章重建公开架构，不代表对闭源实现的源码审计。网页截面以证据账本中的内容 SHA-256 固定；未取得产品源码 commit。文中标明日期的历史能力保留原版本含义，示例为作者推演，未经产品端到端实跑。

1. 整体架构与职责边界：从编辑器进入执行系统

理解 Cursor，先把“在哪里交互”“在哪里组织模型输入”“在哪里执行工具”分开。IDE 提供工作区与会话入口；官方披露，初始上下文会包含操作系统、Git 状态、当前及最近查看的文件，更多信息由 Agent 动态获取。Harness 工程说明 最终提示组装发生在 Cursor 后端，即使用户自带模型 API key，请求仍经过该后端。数据使用说明 因而“本地 Agent”主要说明工作环境的位置，不能解释为整套服务都在笔记本内运行。

下面按第五章的职责边界画出功能分区。这是作者根据公开契约归纳的图，不是内部进程图；“上下文服务”也不是声称存在某个同名内部类。索引链路来自官方索引文章，云端循环与 worker 的分工来自自管机器文档。索引架构、自管机器

TEXT

```
用户 <-> IDE / Agents Window / Web
          | 任务、规则、工作区线索
          v
Cursor 服务：提示组装、模型适配 Harness <-> 模型
          ^             | 工具请求
          |             v
上下文索引/检索      工具执行位置
  ^    |           +-- 本地工作区：文件、终端
  |    +-> 相关片段 +-- Cursor 云端 VM：仓库、依赖、测试
代码变化           +-- 自管 worker：企业机器上的工具
                              |
                 工具结果、diff、日志 -> 下一轮模型输入
```

托管 Cloud Agent 在独立 VM 中获得克隆仓库、依赖和运行环境；自管模式把文件编辑、终端、本地 MCP 等执行交给 worker，循环、推理与规划仍由 Cursor 承担。Cloud Agents、Self-Hosted Machines 这套分工的价值，在作者看来，是让同一种任务交互连接不同算力与开发环境；它的代价是上下文、会话和工作副本不再天然位于同一处。

2. Agent loop 与恢复：模型决策，应用执行

对应第六章，一轮工作可以抽象为“组装输入—模型提出工具及参数—应用执行—结果进入上下文—再次决策”。Cursor 的教学文档明确把执行责任放在应用一侧，模型依据返回结果继续对话；Agent 则把多次工具调用组织为搜索、修改、测试和纠错的循环。Tool Calling、Agents 这解释了为什么测试输出是一轮新的输入：模型必须看见实际失败，才能选择补丁或更换方法，不能由“已经调用测试”直接推导任务完成。

这个循环还有模型适配层。Cursor 披露，它会针对模型及版本定制提示与工具格式，例如适配模型训练时熟悉的 patch 或字符串替换；中途换模型时也切换相应 Harness，同时提示新模型不要调用旧历史中已不可用的工具。模型适配与会话切换 作者据此区分两种稳定性：文件最终如何变化可以保持一致，而模型眼中的动作格式不必相同。恢复旧会话因此也包含工具语境的衔接。

恢复要按状态对象理解。IDE checkpoint 在重大修改前保存文件快照，存于本地且独立于 Git；恢复会回退文件，却不会移除对话消息。Checkpoints 所以恢复后可能出现“对话说补丁已完成，磁盘却回到补丁前”的状态，后续应重新读取文件。官方还区分排队消息与在下一次工具调用边界送达的运行中指令，后者保留正在执行的动作。运行中引导

停止也有可观察契约：loop 结束时，stop hook 收到 completed、aborted 或 error；它还可以返回 followup_message，自动提交下一条用户消息，开始后续迭代。停止事件 因此应区分一轮结束、会话结束和任务验收，不能把任一停止事件直接记为业务成功。

云端另有持久化边界：会话状态、元数据和产物保存到 Cursor 管理的存储，以便审查和恢复；VM 则会按生命周期休眠、回收。云端运行生命周期 这支持“继续一次工作”的产品契约，却没有公开逐条工具调用的事务重放、恰好一次执行或所有取消副作用的保证，本章不补写这类内部机制。

3. 上下文与会话：索引、指令、历史各有载体

第七章所说的上下文选择，在这里首先体现为增量索引。客户端构造文件与目录哈希组成的 Merkle tree，与服务端比较，只同步变化分支；变化文件被切成语法块，再转换为 embedding，并按块内容缓存。首次索引流程 这解决的是代码变化后怎样维护可搜索表示：不必每轮把整库重新塞进提示，也不必对未变内容重复计算。检索负责缩小候选范围，Agent 随后仍要读取与任务相关的代码。

索引复用也有边界。官方介绍的新工作区加速方案，会在同一用户或团队的索引中寻找相似版本；客户端提交的内容证明用于过滤搜索结果，无法证明本地持有的文件会被过滤。索引复用与访问证明 这里应关注“候选索引”和“当前工作副本”的差异，而不能把复用索引理解为向新成员开放整个团队代码。

指令由另一条路径进入模型。Project Rules 存在 .cursor/rules/*.mdc，通过 alwaysApply、文件模式、描述相关性或手工引用决定何时加入；应用后的规则内容位于模型上下文开头。规则里引用文件只提供查找线索，并不自动展开文件全文。Rules 作者建议把必须持续生效的短约束放在规则正文，把大体量范例留在可读取文件中：前者保证任务方向，后者控制输入体积。

长会话则依靠压缩与重新发现相互补充。Cursor 在上下文达到限制时生成摘要，并向 Agent 提供历史文件引用，必要时可搜索被摘要丢失的细节；长 shell/MCP 输出写入文件，集成终端输出也映射到文件系统，供按需读取。动态上下文发现 设计动机不是无限扩大窗口，而是让大对象在窗口之外仍有地址。摘要负责继续工作所需的短状态，历史文件承担追溯，两者不能互相替代。

“记忆”还要区分版本与作用域。Cursor 1.0 在 2025-06-04 将 Memories 作为 beta 发布，说明其按个人、按项目保存对话事实并供未来引用；这是历史契约，本轮未核实同名能力在现行客户端的可用范围。1.0 发布记录 当前 Projects 文档则明确描述一组跨云端与本地机器同步的共享文件，持续积累研究、产物及工作约定。Project 共享上下文 规则是可审阅的行为指令，摘要服务当前会话，历史支持追溯，共享文件服务跨任务知识；把它们分开，才能判断一条过时结论应在哪里修订。

4. 工具、MCP 与扩展：从可发现描述到实际调用

第八章的工具链可分成接入、发现、分派与返回。Cursor 通过项目级 .cursor/mcp.json 或用户级同名配置接入 MCP server，支持 stdio、SSE 和 Streamable HTTP；前者由本机启动进程，后两者连接服务端点并可使用 OAuth。MCP 接入 这些配置确定连接对象和认证材料，工具描述再告诉模型何时、如何调用，不能仅凭名称就赋予模型执行能力。

Cursor 的工程文章说明，MCP 工具描述会同步为按 server 分组的目录，模型先看到少量静态线索，任务需要时再读取说明；Skills 同样以名称、描述引导按需加载详细指令与附带文件。工具与技能的动态发现 这种设计把发现成本延后到真正需要的位置，也要求描述足够准确：不可发现的能力，即使已连接，也可能始终不会进入决策。

模型选定工具后，由客户端的协议集成把调用交给 server，响应再回到会话；界面可展开参数与结果，失败调用会标记错误供重试和查日志。MCP 使用与故障处理 需要改变循环行为时，hooks 提供另一种扩展面：它们是阶段性启动的进程，通过 stdio 双向 JSON 接收事件并返回观察、修改或阻断结果。Hooks 因此，Skill 扩展工作方法，MCP 扩展外部动作，hook 接入执行时序；三者在架构中承担不同职责。

5. 权限、沙箱与凭证：批准动作不等于限制进程

对应第九章，本地 Run Modes 决定调用是否自动放行。Auto-review 先处理 allowlist，其他 shell 命令尽可能在沙箱运行，不能使用沙箱的调用交给后端分类器审查；MCP 与 Fetch 也参与审查。分类器拒绝后，Agent 可换方法或请求人工批准，官方明确说明分类器不是安全边界。Run Modes

执行隔离是另一层。permissions.json 调整自动批准与审查策略，sandbox.json 限制沙箱内的文件路径和网络访问；macOS 使用 Seatbelt，Linux 使用 Landlock 与 seccomp，Linux 环境不满足所需条件时回退到命令审批。沙箱配置与平台实现 这使“允许跑测试”和“测试进程能读哪些文件、访问哪些网络”成为两个可分别配置的问题，而不是一次点击替代所有边界。

Cloud Agents 不使用本地 Run Modes，其专用机器允许自主执行工具，无需每一步都等待批准。云端审批区别 凭证也有分层：Environment Variable 对 Agent 可见；Runtime Secret 仍注入环境变量，但在工具结果、转录和提交等位置脱敏；Build Secret 只供构建过程使用。Secrets & Network 脱敏因此是输出保护，不能解释为进程没有拿到凭证。

需要临时身份时，云端机器可通过本地 socket 申请 Cursor 签发的短期 OIDC JWT，外部服务验证签名与声明后授权；该身份属于整个运行，能访问 socket 的进程均可申请，并非每个 subagent 独占的身份。OIDC 信任模型 云端 MCP 登录态也与本机分开保存，退出本地登录不会自动退出云端。MCP 认证边界

自管 worker 进一步把执行地点与数据流分开：完整 checkout 留在企业机器，但文件内容、终端输出、diff、截图和 MCP 结果会按任务需要回传；agent worker start 使用出站 HTTPS 长连接接收工具调用。自管数据流 选择它是为了接近内部服务或特殊硬件，不能据此推导推理和会话数据也完全留在内网。

6. 子智能体与委派：上下文隔离、环境隔离、调度分层

第十一章的委派首先发生在模型上下文之间。普通 subagent 从干净窗口开始，由父 Agent 在提示里给足信息；前台执行等待结果，后台执行立即返回，完成后把最终消息交给父 Agent。内建 Explore 和 Browser 用来吸收搜索、DOM 与截图等大量中间内容；自定义定义位于 .cursor/agents/，可指定模型和只读权限。Subagents 这里最直接的收益，是让父会话保留决策所需结果，而不承担全部探索轨迹。

上下文独立不等于磁盘独立。普通 subagent 默认共享父工作副本；要求隔离时才使用独立 worktree 或云端环境。Cloud subagent 有自己的 VM 与分支，并使用团队配置的云端 MCP，而非直接继承本地 MCP 会话。隔离副本与云端子智能体 同样，后台 subagent 可按返回的 agent ID 恢复，父 Agent 可读取其输出了解进度；但公开材料不足以证明父任务取消会原子撤销所有后代已发生的外部动作。恢复与状态

worker 则是执行资源，不是另一个独立规划者。Team Pool 是路由与等待队列，由 worker 领取会话，一台机器服务一个 Agent；My Machines 可让多个 Agent 共用一台机器，扩缩容量由企业自己的 controller 管理。Worker、Pool 与 Controller 因而并行度有两个来源：子任务拆分增加决策分支，更多 worker 或 VM 增加可同时执行的环境。

Projects 再把组织尺度扩大：coordinator 负责规划与委派，自身不写代码，并通过共享文件和 subscriptions 维持跨任务工作。Projects 作者将其理解为三层组合：父子 Agent 交换任务结果，worker 提供执行地点，Project 保存持续工作的关系与知识。评估并行方案时应分别检查这三层，不能把多开几个云端任务等同于已经具备统一协调与结果合并。

7. 验证与可观测性：可回看过程，仍须验证结果

第十、十二章关注的验证，在 Cursor 中依赖完整开发环境。云端环境可预装依赖、配置启动方式，使 Agent 能构建、测试并操作修改后的软件；仪表盘显示该运行使用的 environment 与 Build，帮助解释“为什么同一补丁在另一环境失败”。云端环境 云端还提供会话日志，以及通过 Cursor Cloud MCP 读取转录、运行事件、环境详情和 setup 日志的诊断能力。云端可审计性

hooks 能把观察点接到工具、编辑、压缩与子 Agent 生命周期，但覆盖范围取决于运行面：云端只执行命令型 hook，早期只读探索阶段不运行；自管 worker 的 sessionStart/sessionEnd 对应领取、释放会话。Hooks 运行面 命令 hook 以 2 退出可阻断，其他非零退出通常放行；权限 hook 以 0 退出却返回非法 JSON 或 schema 时另有阻断规则。Hook 返回契约 因此，收集到若干事件不能证明所有调用均经过同一关卡。

Cursor 自身还将公开及内部基准、在线 A/B、工具错误、时延、token 效率、缓存命中和代码 Keep Rate 组合使用。Harness 评测 作者的判断是，这些信号适合发现质量变化，但“代码仍被保留”并不证明业务正确。企业自己的验证链仍应把补丁、测试命令、真实结果与验收要求关联起来；子 Agent 的完成消息可以触发检查，不能替代检查。

8. 一次任务穿过架构：修复跨前后端的权限错误

设想用户在 IDE 中提出：“修复订单详情页对过期权限的处理，补回归测试，保留现有接口。”这是作者构造的架构走查，未实际运行。第一步应把验收条件和相关文件附给任务，使上下文选择有明确目标；IDE 工作区线索及按范围应用的规则，为后续搜索提供起点。初始上下文、规则作用域 随后利用索引寻找鉴权入口、页面错误分支与测试，再读取当前代码核对，避免只凭名称相似就开始修改。代码索引

第二步可委派 Explore 查清后端返回契约，父 Agent 保留实现责任。该选择利用子上下文过滤探索输出的能力；若再让另一个 subagent 同时修改测试，则应明确文件所有权，或请求独立环境，因为默认工作副本是共享的。委派与隔离 作者建议回传路径、接口约束及复现条件，而不仅是一句“已找到问题”，这样父 Agent 才能把子任务结论变成可检查的补丁依据。

第三步由模型提出编辑和测试调用，应用执行并回传失败信息，循环继续修复；日志过长时通过文件检索相关段落，而不是把丢失细节的摘要当作完整证据。工具循环、长输出处理 若测试需要内部鉴权服务，可选择接近服务的自管 worker；若只需标准依赖和可公开访问的测试资源，托管 VM 能提供预备好的环境。两条路径都必须依据真实网络与凭证范围选择。执行地点、托管环境

最后把 diff、接口行为和回归结果一起交给审阅者。云端流程支持保存产物并以分支、PR 交接；这些是验证材料的载体，实际是否满足用户要求仍要由测试和审查决定。云端交接 这次走查展示的取舍是：IDE 提供低成本的交互与现场线索，动态上下文控制输入负担，委派把探索移出主窗口，远程环境让测试接近所需资源。接入企业平台时，应保留这些职责之间的连接，尤其是任务标识、环境版本、测试结果和交接产物之间的对应关系。

> 研究注：原章的厂商 A/B 数字、Projects 发布时的 beta 状态及核验限制移入本轮 cursor_disposition.md。证据账本保存短引文、定位和来源截面；本章未把厂商指标当作独立效果复现，也未运行官方示例脚本。

第十六章 DeepSeek Harness：可组合、可逆的运行时

> 本章访问日期：2026-10-29。沿用已固定的 dsh-v0.1.6-alpha.2、commit ddefc45fbc7f8e46dd73185e68295696d1297887，不声称它是访问日最新版。以下是该版本公开文档与源码的架构分析，没有运行上游程序或验证安全隔离。固定版架构

1. 整体架构与职责边界：运行的是一棵插件树

dsh 把模型适配器、工具注册表、会话日志和默认 Agent 循环都放入 Cordis 插件体系。插件向共享 Context 注册服务、类型化事件和随生命周期释放的效果；替换循环或文件系统提供者，不必另造一个应用入口。这里的“可逆”首先指注册关系与受管理资源可以撤销，并不意味着插件已经造成的业务写入会倒退。组件与扩展机制

实际启动树由 profile 组装：按顺序加载 bundle，再应用 profile、home 与命令行 patch。按 id 命中的 patch 替换整份 config，不能按深合并理解。dsh-base 提供常用基础能力，Web、SDK、headless 等 bundle 再增加交互载体；sdk-minimal 则持有自己的完整小型树。因而“同样使用 dsh”并不证明两次任务拥有相同的工具、策略或持久化组件。组合与启动

下图为作者依照服务归属绘制的调用关系。它对应第五章的职责分解：Host 承载控制与服务，执行后端承载代码运行，Session 承载可重建事实；三个位置不能因为都叫 runtime 而混在一起。核心服务及能力接口

TEXT



能力替换进一步分成 Service Definition、Service Provider 和 Consumer：接口定义约束提供者，工具等消费者只调用接口。例如文件系统与子进程提供者需要属于同一个执行世界，切换到远端时才能让文件路径与命令操作同一对象。这里可组合的单位是有明确输入输出与生命周期的能力，不是任意把几个插件名堆在配置里。能力接口

一个作者构造的反例是：底层工具配置包含超时和策略，上层 patch 只写超时，却误以为其他字段仍会继承。由整份替换规则可知，应检查最终解析树，而不只检查新写的几行。配置组合既提供自由度，也把“本次到底运行了什么”变成可复现性的一部分。覆盖规则

2. Agent loop 与恢复：把请求、尝试和回合分开

ctx.agents 提供创建与恢复入口，AgentLoop 是可替换的默认驱动。创建阶段先完成 scoped setup 和串行 agent/created 初始化，随后才处理队列。调用者持有的 AgentHandle 负责最终释放：停止循环、等待退出、注销 Agent、移除内存会话，再拆除其作用域。这使“已构造对象”“已准备执行”“已完成释放”成为不同边界。创建与所有权

一次 followup 把消息排入下一回合；steer 面向最近的步骤边界；inject 只加入上下文，不唤醒空闲驱动。一个 turn 可以有多个 step，每个 step 包含一次模型请求及其工具调用。驱动认领输入，组装提示和工具 schema，经 agent/pre-step、请求准备后，把被接受的输入写入日志，从日志冻结模型历史，接收响应、执行工具，再判断是否还欠一次模型请求。工具结果因此是下一步推理的正式输入，而非 UI 旁边的临时显示。输入契约、步骤顺序

一次请求也不等于一次成功回答。成功内容落为 assistant/message；已结算的失败、重试、取消尝试落为 assistant/attempt，后者留作日志而不污染模型历史。重试不重做提示组装和 pre-step。实时 agent/assistant-stream 的 chunk 是进程内消息，完整紧凑流到结算时才嵌入持久事件；此前硬退出可能丢失这一尝试的完整流。尝试与持久化

cancel 中断当前活动，默认清除待处理输入，keepInbox 可保留它；whenIdle 等待整个 Agent 静止，不是某条消息的完成凭证。第六章的恢复问题在这里有了具体落点：应先检查日志中哪个 turn、step、attempt 已结算，再决定继续什么；不能把界面停止刷新当作命令已被撤销。取消与静止

请求接纳还有一个细节：循环先经 agent/request 与 prepareCall() 确定真正的模型路由和能力，再提交系统提示及被接受的用户消息；准备期间取消，不会把尚未进入模型调用的输入冒充成已发送历史。随后派生并冻结请求，同时保持取消信号有效。作者据此将它理解为“记录已经接纳的事实”，而不是“用户一提交，就假设模型已经读过”。请求准备与接纳

3. 上下文与会话：日志保存事实，投影决定模型看见什么

ctx.systemPrompt 汇集有序 PromptSection 和工具声明，循环把渲染结果记录为 system/message。变化频繁的 PromptContext 则以持久 user-role 快照进入历史，在内容改变或被压缩移除后更新。系统指令和运行状态分别入账，既能解释请求来自哪里，也能避免每次状态变化都重写提示前缀。提示组装

工作区知识由具体贡献者提供。例如 dsh-agent-instructions 将全局与项目 AGENTS.md、CLAUDE.md 等文件按由宽到窄的顺序注入，受字节预算约束；成功的文件读写可发现深层指令，恢复会话时重新核对基线。它不持续监听所有外部编辑，也不通过猜测任意 shell 命令来发现目录变化。此处可借鉴的设计是：指令来源、发现时点与注入消息都有归属，而非把工作区当成模型天然知道的背景。工作区指令

日志保留的内容多于当前模型上下文。deriveMessages() 从追加式事件推导可见历史；压缩作为可选 ctx.compaction 能力，在日志中记录 start、summary、end，再用带 surfaceOp: replace 的 user/message 替换模型视图中的一段内容。旧事件不因视图替换而消失。锁最后才释放，崩溃留下未闭合 start 可以被检测；压缩提交和持久化仍可能分别失败，不能视为无损记忆或绝对事务。压缩协议

恢复还涉及存储格式。JSONL provider 选择最高的规范 generation，沿相邻版本链迁移；只读打开不发布新文件，写打开则验证并排他发布新版本，旧 generation 保留。未来版本被拒绝，普通未封口尾部的修复仍由 handle 消费者承担。这样把第七章的“重建会话”分成了物理记录恢复与模型视图重建两件事。会话存储与迁移

自动压缩通常发生在下一步请求派生之前；确认的上下文溢出也可经请求错误处理触发恢复。后端可以先裁剪工具结果再决定是否需要摘要，只有可见表面实际推进才返回相应重试动作，避免对同一超限请求无意义地循环。压缩边界保护工具调用与结果配对，却不一定保留整个 turn，因而一轮很长的工作也能逐段缩减。压力与溢出恢复

4. 工具、MCP 与扩展：统一分派，分别拥有执行

工具作者用 defineTool 声明参数、输出和执行函数，注册到 ctx.tools 后参与提示组装。模型调用先成为 tool/call，再经过 tools/pre-execute、只能收紧的 guards、tools/execute 和 tools/post-execute；最终内容约束及结果观察完成后，循环记录 tool/result。这种分层把工具业务逻辑与审批、拦截、结果规范化分开，也让普通工具错误能够作为结果返回给下一步。注册契约、分派管线

PTC（程序化工具调用）改变模型组织工具的方式：native 模式展示各工具，ptc 模式展示 run_code 和生成的 SDK，也可同时提供两者。模型写一段 TypeScript，在一个程序内处理返回数据并调用绑定函数；桥接子调用仍走同一注册表和审批管线，带父调用关联并记录 tool/ptc-dispatch。中间绑定值不逐项进入模型历史，外层结果才返

回，这解释了 PTC 为什么适合多次查询后的程序化筛选。工具呈现、PTC 返回路径

MCP 则解决工具来自哪里。每个 opt-in 客户端插件持有一个服务器连接，将发现的工具注册到调用者可见的 scope；stdio 与 Streamable HTTP 使用官方 SDK 协商。资源工具按服务器名查询，服务端指令进入有来源的提示段，断线或列表刷新失败有各自生命周期。此固定版不支持 MCP prompt templates、elicitation、task-based execution 和资源订阅，不能从“支持 MCP”推出协议能力全集。MCP 适配边界

运行时“演化”具体发生在插件组合上。Creator 先用 cordis_inspect_list/query 读取准确 API，再通过 Plugin Manager 安装持久 bundle；安装后的 Host 代码在宿主进程执行。当前 Creator 流程不能与旧的临时动态定义流程混写，程序及浏览器消费者使用的动态 runner 也并未因此消失。扩展改变工具与界面供给，并不等于模型参数发生训练。只读发现、持久插件管理

5. 权限、沙箱与凭证：三个问题，三条边界

ctx.approval 回答“这个动作是否获准”。只有 allowed-once 是许可，拒绝、取消、无人应答都不放行；每次请求有配对的 approval/asked 与 approval/decided 日志。尤其要读准策略名：dsh 的 never 表示永不询问、所有需要询问的动作一律拒绝，不是自动批准；一次批准也不永久改变会话权限。审批契约

ctx.sandbox 回答“执行时允许产生哪些文件效应”。read-only、workspace-write 交给平台后端，danger-full-access 绕过限制；Linux、macOS、Windows 的实现与完整度不同，必须保留 full/partial 报告。该 mode 词汇本身不覆盖网络与进程可见性。凭证另由 ctx.credentials 保存与解析：配置携带引用，模型适配器每次请求重新取值，便于轮换；这不是沙箱替你管理凭证生命周期。沙箱语义、凭证服务

PTC 后端先解析执行世界中的 Node 与 bootstrap，经 sandbox 包装 argv，再由 subprocess 启动独立进程。Host 管策略、期限和绑定分派，子进程计算程序；直接 Node API 仍可用，程序可见环境被清空，请求受限模式而后端不可用时失败。与之不同，动态 Host runner 的 node:vm 明确不是安全 containment，宿主 realm helper 仍可能成为逃逸路径，已安装插件同样不能靠工作区沙箱隔离。PTC 启动、Host 信任边界

PTC 默认期限为 120000 毫秒、上限 600000 毫秒，包含等待审批与子工具的时间，清理可使返回更晚。V8 heap cap 不限制整棵进程树内存，也没有进程树 CPU 计量；fallback 平台逃离管理范围的后代可能残留。输出上限只保留可容纳部分，绑定通道限额不限制 Host 绑定内部的全部分配；每次程序均为一次性执行，不提供跨调用状态或实时结果流。这些边界应与“独立进程”同时理解。PTC 已知限制

6. 子智能体与委派：上下文继承不等于权限继承

ctx.subagents 是可选能力注册表，可同时容纳进程内、ACP 或其他产品后端。调用先核对 provider 是否支持模型覆写、结构化输出、深度、工具过滤和人格；不支持的请求在启动前拒绝。进程内 spawn 创建空历史子会话，fork 则取得父会话已完成 turn 的一次性快照，不含正在执行的 turn，后来父消息也不会自动同步。能力协商、spawn、fork

两者共享宿主进程及相关服务，子 Agent 拥有新 scope；完整父工具限制和 authority 不被自动复制，工具过滤须在子创建窗口明确施加。一次性委派等待子任务自己的最终输出，取消、拒绝或截断成为错误结果，收集后释放 handle。因此“子会话独立”解释的是上下文和生命周期，不能自动解释为独立操作系统身份或更小的权限。进程内子任务的所有权

这不等于完全不继承策略：captureDelegatedPolicyOverrides 在第一次异步等待前捕获父会话显式 sandbox override，以及 Auto 或 Full access 的 preset 身份；父侧存在审批服务时，子审批策略钉为 never。新子会话在 fork seed 之后记录这些覆盖，冷恢复读取子日志。Auto 子调用仍逐次独立评审，不能将父任务的许可扩写成所有子动作的通行证。委派策略捕获

可继续的子任务又多一层：持久 child Session 可被多次激活，同一时刻至多一个 Activation，队列仍由 Agent inbox 统一拥有。接收消息之后，调用者后续取消不再撤销已接收回合；interrupt 只中断当前活动，保留未认领队列和已发布后代，且不等待完全静止。管理器整体卸载才执行向下取消、子先于父释放。最终 flush 失败会记录错误

后继续释放，所以稍后恢复仍可能读到缺失或陈旧状态。Activation 与取消范围

7. 验证与可观测性：区分事实日志、实时流与报告副本

会话事件用于重建请求与结果，agent/* 用于观察当前活动，能力事件用于在工具和文件系统等接缝处插入策略。PTC 子调用已有原生追踪，审批已有配对记录，不能笼统称为“平台必须从零补齐审计”。作者建议沿 Session、turn、step、attempt 与工具调用标识关联一次任务，而不是仅记录最后一句回答。事件分工、工具结果路径

遥测不是另一份同等可靠的会话库。session-telemetry 将有序副本非阻塞交给 backend，cursor 只表示已经交出，不表示送达；崩溃可丢队列。默认没有内建脱敏规则，文件或命令输出夹带的秘密可能原样离开进程。需要可靠审计或共享 collector 时，应分别设计投递保存与脱敏规则，不能借用本地会话持久化的保证。遥测交付边界

第十章的验证还要求回答“工作做对了吗”。运行事件可以证明调用及结果流转，却不能独自证明外部系统已处于期望状态。以下任务中的成功探针、独立测试和外部回读是作者建议；本章只做了固定源码交叉阅读，没有把文档中的失败处理当成已执行的故障注入。

对第十二章而言，这种分工也提示一种有效的诊断顺序：先从会话日志确认输入和输出，再看 attempt 解释失败请求，最后用实时事件或遥测定位延迟。若反过来仅用 collector 中“没有看到记录”断言动作未执行，就把报告丢失与执行失败混为一谈。这是作者根据上述持久化和尽力投递契约提出的排查方法，尚需实际故障实验检验。

8. 一次任务穿过架构：接入一个 MCP 查询能力

设想用户要求“把指定 MCP 服务接入当前 profile，确认能查询，再整理结果”。这是作者按官方 Creator 教程扩展的示例。Agent 先检查运行时 API，写一个包含 dsh-mcp-client 配置的 bundle，再调用 Plugin Manager 安装；HMR 启用时，新连接发现工具并注册到 ctx.tools，下次提示组装即可包含它。官方教程要求同时看到 application: applied 和成功的 ping，restart-required 仅说明配置已保存。官方接入路径

随后可以直接调用查询工具，也可以让 PTC 执行一组查询并筛选返回数据；两种呈现都走工具权限和记录路径。若把结果解释委派给 spawn 子 Agent，应传入自足任务，并显式配置其工具范围；这些是调用者组合上的选择，不能靠父会话的限制自动推导。PTC 管线、子作用域

安装与演化的失败必须按阶段解释：pnpm 或 bundle 校验失败，恢复的是 package.json 和 lockfile，下载物、日志与构建脚本批准可能保留；安装成功后启用失败不撤销安装；删除按取消选择、卸载、移除依赖推进，失败保留已完成修改而不自动重新启用。构建脚本批准与普通工具批准分开，服务校验包名却不验证对话中的授权陈述。普通 HMR 保留成功兄弟项，不提供整批回滚；初始启动 required entry 失败才有另一套整应用清理规则。管理失败语义、启动与 HMR

由此，作者的接入取舍是把 profile 当成需要版本管理的运行配置，把插件安装当成宿主代码变更，把查询结果当成需要业务验证的数据。dsh 的组合接口使这些控制点可检查、可替换；收益来自能精确改变哪一层，代价则是必须清楚谁拥有状态、谁清理资源，以及所谓“可逆”究竟止于何处。

第十七章 OpenHands：Agent 与执行 Runtime 分离

本章访问日期：2026-10-29。分析基线为 Software Agent SDK 1.49.2、commit d128a786ee2ee570eb23ff5862ec148b43cfad0b；应用层另固定 Agent Canvas 1.20.0、commit 9737f713616a1e452f822c2967f0e2c8bf2dc308。这不是访问日最新版声明，也不表示两者依赖版本相同：后者的 TypeScript client 依赖实际为 1.49.1。本章静态阅读源码，未部署容器或运行模型任务。SDK 职责、Canvas 固定依赖

1. 整体架构与职责边界：配置、会话与执行位置分别建模

理解 OpenHands，应先区分四个对象。Agent 描述模型、工具和上下文处理能力；Conversation 拥有一次会话的消息、执行状态与运行入口；Workspace 描述工作发生的位置；Agent Server 将会话能力暴露为 REST/WebSocket 服务。Canvas 消费 SDK 的 TypeScript client 提供产品界面，调度和 webhook 等自动化生命周期另由 automation 仓库负责，不能把它们都归入一个“Agent 大脑”。仓库责任、会话工厂

Conversation(..., workspace=...) 根据 workspace 类型创建 LocalConversation 或 RemoteConversation。本地路径在当前执行环境驱动 Agent；远程路径由客户端发送会话请求，Agent 在远端 Server 上运行。因此，远程会话并非必然是“模型循环留在笔记本，只把 shell 送进容器”。调用者控制会话的接口与循环实际驻留的位置必须分别定位。工厂选择、远程客户端

下图为作者对固定版的归纳。LocalConversation 是控制循环的具体承载者；Docker mode 再把每个会话的服务放入独立容器，控制面负责供给与转发，容器内仍运行本地会话逻辑。这种嵌套使会话协议能够跨部署位置复用。Docker 会话路由、容器供给

TEXT

```
Canvas / Python client / automation caller
               |
        Conversation API
          /          \
 LocalConversation   RemoteConversation -- HTTP / WebSocket
          |                                  |
          |                      Agent Server control plane
          |                                  |
          |                   local mode / per-conversation container
          |                                  |
          +---------------------- LocalConversation
                                             |
                         ConversationState + EventLog
                                  |        ^
                              Agent.step   |
                                  |        |
                           LLM -> Action -> ToolExecutor
                                             |
                                        Observation
                         工具在所属 workspace 环境中执行
```

历史 0.62.0 中的 EventStream → Runtime → ActionExecutor 属于另一套实现。旧 Runtime 通过 HTTP 交付动作并返回 observation，可用于理解决策与执行分离的历史思路，但不应拿旧类名替代上图的 SDK 会话对象。历史 Runtime 契约

2. Agent loop 与恢复：由 Conversation 驱动 Agent 的单步

调用者先 send_message()，再调用 run() 或异步 arun()。LocalConversation 初始化插件与 Agent，循环检查执行状态、卡住检测、预算和迭代上限，再调用 Agent.step。Agent 负责“一步如何决策”，Conversation 负责“何时再走一步”。完成状态还可交给 Stop hook 检查，反馈可能促使会话继续，因而循环终止不是单看模型输出了一段告别文字。循环所有者

一个普通 step 读取 state.view，经 condenser 准备模型消息，调用 LLM，并将响应区分为工具调用、文本等路径。工具调用解析并验证为 ActionEvent，执行器产生 observation，再封装为 ObservationEvent。下一步看到的是更新后的会话视图。若有等待确认的未配对动作，step 先执行这些动作，而不是重新请求模型生成一份相似命令，这使“批准既有动作”与“再规划一次”分开。单步与动作转换

确认模式下，第一次 run 可以停在 WAITING_FOR_CONFIRMATION；再次 run 表示隐式确认并执行 pending actions，拒绝则应走 reject_pending_actions。因此，接入端的“继续”按钮具有实际授权含义，不能当成任意网络重试。上下文超限时，Agent 只有在配置的 condenser 支持请求式压缩时才进入相应恢复路径，否则显式报错，不保证所有失败都会自行修复。确认与恢复、上下文错误处理

取消也有粒度。pause() 在 step 之间生效；interrupt() 可取消正在运行的异步 task，先设置 cancellation token，再触发异步取消；同步 run 路径回退为 pause。中断后，未得到 observation 的工具调用可补入合成错误，保持模型消息结构完整。这种“修复历史配对”没有撤销已经写入的文件、外部请求或已启动进程；第六章的不确定提交问题仍需具体执行器和外部系统处理。暂停、中断与孤立动作

3. 上下文与会话：原始事件、模型视图和长期记忆各有位置

ConversationState 保存 Agent、workspace、执行状态、秘密注册表及 EventLog；活动分支的 View 是可重建派生值。配置 persistence_dir 才有对应文件存储，未配置则使用内存存储；恢复时会核对 Agent 与已存工具配置。会话身份、执行配置和事件不能只靠一段对话文本替代，否则可能在恢复时连接到不同能力集合。状态与恢复

AgentContext 负责提示扩展：仓库指令、运行环境、会话附加规则和 skills 在这里汇集。项目 skills 与 AGENTS.md 由 LocalConversation 在知道 workspace 后延迟加载；load_project_skills、load_memory 等开关不能写成无条件默认开启。记忆功能读取用户及工作区的 MEMORY.md 索引，索引内容每个会话从磁盘重新解析，不作为整个会话状态反复序列化。这使第七章讨论的指令、历史与跨会话知识具有不同更新周期。上下文及记忆归属

并非所有 Event 都送给模型。View 仅纳入 LLMConvertibleEvent，并按顺序应用 Condensation；暂停与内部状态记录保留在日志，却不进入模型输入。它还维护允许修改消息的位置，避免压缩切断工具请求与结果的结构关系。于是审计能够查看发生过什么，而模型可以只看适合下一次推理的内容。视图投影

LLMSummarizingCondenser 可使用独立 LLM 摘要，按事件数量、token 压力或显式压缩请求触发，保留指定开头及近期内容。摘要模型与决策模型不必相同，token 阈值还应尊重决策模型的输入限制。这是上下文预算策略，而非证明信息没有损失的记忆机制；接入者应将关键验收条件保存在可重新读取的文件或结构化任务定义中，后半句属于作者建议。摘要器实现

这也解释了为什么“恢复聊天”与“恢复执行”不能合并成一个按钮的表面含义。模型视图能够从记录重新计算，并不保证当前工作目录里的文件仍与记录一致；摘要保留了任务目标，也不证明测试产物仍然存在。作者建议在跨进程恢复的重要任务里同时核对事件、工作副本与产物，让模型读到的是可验证的当前事实，而不只是先前描述的状态。这是对状态分层的应用判断，不是 SDK 额外承诺的文件回滚功能。

4. 工具、MCP 与扩展：配置可以传输，执行器在运行处物化

Tool(name, params) 是工具配置，Agent 初始化时找到注册的工具类并调用创建逻辑，得到具体 ToolDefinition 与 executor。动作和观察分别有 schema，模型只看到工具声明，实际调用由执行器负责；TerminalTool、FileEditorTool 等因此能在同一循环中提供不同环境能力。远程传输配置与在 Server 中创建资源，是理解 SDK 可部署性的关键。工具初始化、执行与观察

并行执行由 ParallelToolExecutor 管理，它使用每 Agent 的并发限制与资源锁，把触及相同声明资源的调用串行化。锁有效的前提是工具正确实现 declared_resources()；不同执行器实例又有各自的锁，不能推断两个子会话同时编辑同一文件会自动互斥。并发调度降低等待，资源声明则维护可预测性，二者共同决定工具作者承担的契约。并发和资源锁

MCP 工具经 MCPToolDefinition/MCPToolExecutor 进入同一工具体系，再通过 MCPClient 发给服务器，返回普通 observation。连接丢失时，执行器可以在调用前尝试重连；已经关闭的客户端不会被偷偷复活。参数中的秘密引用由 conversation 的 SecretRegistry 展开，不读取任意宿主环境变量。这条适配链解决了调用形状，外部服务实际写入的幂等性仍不是工具 schema 能保证的。MCP 执行器

插件是另一层组合单位，可打包 skills、hooks、MCP 配置、agent 定义和 commands；ResolvedPluginSource 记录实际取得的 commit，避免暂停后继续时漂移到分支新版本。这里的扩展把提示知识、事件处理与外部能力组织成可加载资源，不应简单等同于新增一个函数工具；来源固定也只是可复现条件，不是插件安全证明。插件组成、固定插件来源

5. 权限、沙箱与凭证：批准动作与隔离执行分别配置

AlwaysConfirm、NeverConfirm、ConfirmRisky 决定一个动作是否需要确认；后者可以按风险阈值处理未知风险。与 dsh 的 never 不同，OpenHands 的 NeverConfirm 意味着不要求确认，并不禁止执行。风险分析与确认策略共同影响是否停下来等待用户，但它们不会自行建立文件系统或网络隔离。确认策略

执行隔离要看部署。Agent Server 本身是服务接口，普通 local mode 不因“Server”这个名字就得到容器；每会话 Docker mode 则由控制面创建容器，挂入该会话的记录、持久目录和 workspace，设置运行 UID/GID、去掉 capabilities，并启用 no-new-privileges；内存、CPU、PID 限额另有配置。这些是固定代码可检查的措施，不能省略镜像、挂载、网络与宿主 Docker 权限检查而宣称多租户隔离已验证。Docker 启动参数

容器身份也属于恢复状态。RuntimeProvisioningStore 保存 conversation ID、API key、加密 key 与 workspace 路径，要求控制面配置 OH_SECRET_KEY，拒绝会话身份不匹配或重绑工作区。重连的目标应由持久身份重建，而不是临时发现一个同名目录后继续。Server 的会话 API key 认证是可配置的，部署者仍要明确服务暴露范围。身份供给、服务认证

业务凭证由 SecretRegistry 管理，在执行时解析并对已知值做输出遮罩，流式遮罩还考虑秘密被拆到相邻 chunk 的情况。profile 的 secret_refs 则定义哪些保存的秘密可暴露：null 为全部，空列表为没有，非空列表为按名过滤。作者据此建议先限定注入范围，再考虑日志脱敏；遮罩已知字符串不能证明生成代码无法通过编码或网络泄露秘密。秘密注册表、profile 范围

6. 子智能体与委派：一项 task 对应一个子 Conversation

固定版的 TaskToolSet 提供阻塞式 task 工具。模型提交任务提示、subagent_type 和可选 resume ID，TaskExecutor 转交 TaskManager，最终返回含 task ID、状态与内容的 TaskObservation。子 Agent 类型由注册的 factory 决定，既可以是专门探索代码的配置，也可以是调用者自定义工具与指令的配置；工具定义中的旧 max_turns 字段已经弃用并被忽略。Task 工具契约

TaskManager 创建新的 LocalConversation，共享父 workspace 路径，复制父 LLM 配置但重置独立计量，将子任务提示作为新消息发送，并按 agent 定义或父会话确定预算与迭代限制。这里没有复制父完整历史的步骤，也没有为每个 task 自动创建容器或操作系统身份。子任务结束后释放活对象，保留与 task 对应的会话记录；只有

FINISHED 算完成，卡住、暂停或限额终止返回错误和可用的部分结果。创建与回收

这条实现有两个容易误读的边界。其一，resume 用进程内 task 映射找到落盘 conversation，不能据此保证进程重启后 task 索引自动恢复；父会话不持久化时，临时任务目录在 manager 关闭时会清理。其二，子策略可由定义覆写，否则继承父确认策略，但没有配置 confirmation_handler 时，manager 会继续执行等待确认的子动作，不能仅凭“继承”声称有人审。恢复及确认处理

取消责任也必须沿具体实现追踪：此 TaskManager 走同步子会话 run，已检查的 start/run 路径没有把父 cancellation token 下传。因此不能将父异步 interrupt 的保证扩大为全部子进程或子会话同时停止。第十一章的委派契约在这里应包含任务范围、共享工作区冲突、确认处理器以及显式的子任务停机验证，后者是作者提出的接入要求。同步任务路径

7. 验证与可观测性：存储顺序不等于业务恰好一次

LocalConversation 将默认持久化回调放在调用者回调之前：事件追加失败，后续订阅发布不应先行。终端 visualizer 可以更早显示，不能把所有 UI 输出都当成落盘证明。这个 persist-before-publish 顺序减少了“订阅者看到但事件没存下”的窗口，却没有把文件写入、外部 API 与会话事件合成事务。存储后通知前退出时，应补读事件，而非重新执行同一业务动作。回调次序

Agent Server 的 session socket 又区分 durable、transient、delta 和同步等 envelope，不能把每条 socket 消息都当持久 Event。它通过有界发送队列处理慢消费者，必要时断开连接，并支持按 after_seq 恢复持久事件流。这是服务提供的通道契约；具体客户端使用哪条 socket 路由仍应核对，不能把新 session 协议自动套给所有客户端。session socket

观测还有不同目的。SDK 的 Laminar/OpenTelemetry 集成可关联 conversation、工具调用与委派 trace；Server 文档另区分产品 analytics、完整 completion logging 和 tracing，前者默认关闭，不能把三者的数据范围混写。持久事件用来恢复事实，trace 用来找耗时，测试结果用来判断工作是否正确，三类证据互相补充。trace 集成、数据采集分类

本书没有执行容器回收、存储故障、重启或秘密泄露实验。作者建议将这些作为部署验收：重连后核对会话与工作区身份；事件补取按原生身份去重；取消后查看资源实况；业务提交缺少结果时回读外部状态。源码支持提出这些检查，但不能替代运行环境中的验证结论。

8. 一次任务穿过架构：修复仓库中的解析错误

设想用户要求“修复配置解析器处理空输入的错误，并证明已有行为未退化”。以下是作者构造的任务走查，不是已运行示例。接入端选择 Agent 的模型、TerminalTool、FileEditorTool 与 TaskToolSet，把仓库放到指定 workspace，再创建远程 conversation。若使用每会话 Docker mode，控制面解析配置、建立持久身份、启动容器，最后把会话创建请求转发到容器中的 Agent Server。工具配置、容器创建顺序

任务消息进入会话，项目指令按启用的 AgentContext 规则加载；Agent 从当前 View 请求模型，模型调用文件工具定位解析入口。一个 task 可以让子 Agent 独立追踪调用者和相关测试，只返回位置与发现；父 Agent 根据回收的 observation 修改代码，再用终端执行测试。共享 workspace 意味着子任务可以读到同一工作副本，也意味着并行写入需要另行安排；作者在此建议让探索子任务只读，集中修改责任。上下文装配、子任务运行

若测试命令需要确认，Conversation 停在待批动作，用户批准后继续该动作；若用户中断，应该检查命令是否仍运行、文件是否已改变，而非重建一份“干净的成功故事”。最终交付应关联实际 diff、测试命令与退出结果，并说明未覆盖的情况。模型的完成状态可以结束循环，验收证据是否充分仍由任务要求决定。运行与确认边界

如果探索任务第一次返回“缺少某个失败样本”，父 Agent 可以带着更具体的问题按 task ID 继续子会话，复用其已保存上下文；如果工具集合或子 Agent 定义发生变化，则不能绕过会话恢复的一致性检查。这里选择继续已有子任务，目的是保留调查过程；选择新建任务，则是刻意切断旧背景。两种选择需要由任务语义决定，不宜仅按哪个调用更方便。任务恢复、恢复校验

这一结构的接入价值在于可以分别控制 Agent 能力、会话协议和执行环境。小型本地工作流可以直接持有 LocalConversation；服务化系统则需要维护会话存储、身份、镜像与容器生命周期。作者的选择标准是组织是否需要并能承担这些控制点，而非仅凭“开源”推断完全离线、自动安全或必然更低成本；模型数据去向与隔离效果都应按实际配置评价。

第十八章 pi-mono：小核心怎样长成可组合的运行时

> 资料截面：2026-10-29。本章沿用读者熟悉的 pi-mono 名称；原仓库已迁至 earendil-works/pi。以下以稳定版 **v1.4.0**、**commit** 4a080252f06caf63a525d7873c5d94d3c23887e0 为准，不把早期版本的取舍当成永久特性。发布记录

Pi 值得研究的地方，是它把“循环本身”“编码会话”和“工作方式”分成了可以分别使用和替换的部分。只需要一个调用模型与工具的循环，可以使用 Agent Core；需要会话、上下文压缩和扩展发现，可以使用 Coding Agent SDK；需要终端交互，再使用 CLI。下面按第二篇的同一组问题，追踪这些部分怎样一起完成任务。包划分、SDK

1. 整体架构：循环、会话和界面分别负责什么

对应第五章，先区分四个主要组件。pi-ai 统一不同模型供应商的消息、工具调用和流式响应；pi-agent-core 的 Agent 管理一次运行的状态和循环；pi-coding-agent 的 AgentSession 在循环外管理编码任务所需的会话、压缩、资源和扩展；pi-tui 提供终端呈现。它们是包和职责的划分，并不意味着每层都运行在独立进程。仓库说明、会话工厂源码

下面是依据这些接口整理的架构图。图中连线表示调用或数据依赖，不代表安全隔离：

TEXT

```
终端 CLI / JSONL RPC 客户端 / 嵌入 SDK 的应用
                      |
                AgentSession
         +------------+-------------+
         |            |             |
   SessionManager  ResourceLoader  扩展运行环境
   会话树与当前分支  指令/skills等   工具与生命周期处理器
         |            |             |
         +------ 当前请求上下文 ------+
                      |
               Agent / agent loop
                /             \
         pi-ai 模型适配       工具执行接口
              |             /       \
          模型服务       本机文件/进程   MCP 服务等
```

createAgentSession() 负责把这些对象装配起来。SessionManager 持有持久化或内存中的会话记录；DefaultResourceLoader 找到指令、skills、模板和扩展；AgentSession 再协调当前模型、可用工具和运行状态。宿主可以替换这些依赖，因而“沿用 Pi 的循环”不必等于“照搬 Pi 的终端产品”。SDK 的可替换边界

这里的小核心是一种职责选择。完整企业系统仍需要身份、隔离、任务验收和运营设施；只引用 pi-agent-core 不会自动获得 Coding Agent 的全部会话能力，更不会获得持久业务工作流。仓库中的其他包也不能仅凭名称就视为这条 CLI 调用链的必经组件。

2. Agent loop：工具反馈和用户追加输入怎样推进循环

对应第六章，Pi 的低层循环有两层推进条件：内层处理模型响应、工具调用和 steering 输入，外层在一次工作原本可以结束时检查 follow-up。steering 用来调整正在进行的工作，follow-up 用来排在当前工作之后；两者不只是界面上不同颜色的消息，而是不同的调度时机。循环源码、SDK 输入语义

一次正常工具轮次可以这样阅读：循环先整理当前消息和工具声明，调用模型，接收完整 assistant 消息，再校验并执行其中的工具调用；工具结果成为 toolResult 消息，进入下一次模型请求。下一轮开始前，上层还可以更新上下文和模型配置。源码同时有并行与顺序工具执行路径，不能把早期版本的执行顺序当成所有版本的固定行为。请求、调度与工具分派

停止条件比“模型没有调用工具”更细。排队输入或上层继续决定可能产生下一轮；相反，整批工具都明确给出终止提示时，可以跳过自动追问。若模型响应因输出长度截断，哪怕拼出的工具参数看起来是合法 JSON，也不执行这些调用，而是反馈错误；错误或中止响应则结束本次低层运行。这些分支让不完整输出不会直接变成动作。同一固定版本的停止与截断分支

恢复由上层会话补充。agent_end 只说明一次低层运行结束，其后仍可能发生自动重试、压缩恢复或队列工作；外部应用要判断 Pi 是否还会自动继续，应观察 agent_settled。但即使会话已经静止，也不能推导先前的文件修改或远端写入已撤销。这里分别对应第六章的运行终态和副作用对账，不能用一个结束事件代替两者。RPC 生命周期

3. 上下文与会话：完整记录和当前请求不是同一个对象

对应第七章，Pi 用两个转换边界隔开内部消息和模型输入。transformContext 可以修剪或补充本次要看的内容；convertToLlm 再把内部消息转换为模型能接受的格式，例如滤掉只供界面使用的消息。这允许会话保留更多信息，同时控制本次请求实际发送什么。转换函数是否保留原数据还取决于实现，不能把可插拔接口当成自动保证。模型调用边界源码

会话的权威记录由 SessionManager 管理。除首行会话头外，JSONL 会话条目通过 id 和 parentId 组成树，当前叶节点确定活动分支。会话头保存会话级元数据，其 id 不是条目树的节点 ID。回到旧分支不需要删除其他分支；生成模型上下文时，管理器选择当前路径并应用压缩记录。系统指令和工具声明的变化也以消息保存，恢复时可以重建当时的有效配置。直接替换内存中的消息数组，并不等于替换持久会话。会话格式、SessionManager 的权威性

压缩会选取较早的上下文生成摘要，追加带 summary 和 firstKeptEntryId 的记录，再用摘要及保留的近期消息构造下一次请求。原始记录仍然存在。这解决“上下文窗口装不下”的问题；切换分支时的 branch summary 则解决“新分支需要知道另一条路径上做过什么”的问题。两者都使用摘要，但触发原因不同，也都不保证摘要保留了所有细节。压缩与分支摘要

skills 采用按需读取：先提供名称和说明，需要时再读取 SKILL.md，避免把所有工作指南一次塞进请求。文档也明确承认模型可能漏读相关 skill，必要时应显式指定。因此，skills 是组织经验的载体，加载机制本身不是自动学习或正确选择的证明。Skills

4. 工具、MCP 与扩展：小接口怎样容纳大工具集

对应第八章，CLI 默认启用 read、bash、edit、write。模型看见工具声明后生成调用，循环对参数做校验，再把执行结果送回模型。扩展可以注册工具，也可以在调用前后处理事件。嵌套调用通过 ctx.executeTool() 复用工具校验和事件处理，事件带 parentToolCallId；嵌套结果先返回给调用它的工具，并不会全部自动塞入模型转录。这使复合工具有机会筛选结果，而不只是把更多输出搬进上下文。CLI 工具、扩展与嵌套工具

在本章版本中，**Pi** 已经有内建 **MCP** 支持。 它连接 stdio 或 Streamable HTTP 服务，把服务工具注册到本地工具体系。工具可以直接声明给模型，也可以先隐藏详细定义，等 tool_search 找到后再声明，或由 codemode 脚本发现、组合调用。完全隐藏的工具则不可达。这几种模式对应第八章的选择：让常用工具容易调用，让大工具集不必全部占据每次请求。MCP 连接与工具暴露

这里还要分清产品装配与库默认值。CLI 把 MCP、codemode 和 tool_search 作为内建扩展加载；使用 SDK 创建会话时，需要宿主显式加入相应扩展工厂并绑定扩展生命周期。只写一句“Pi 支持 MCP”，不足以指导企业嵌入，旧文章中一句“Pi 没有 MCP”也已经不能描述这个版本。SDK 的 MCP 装配要求

工具暴露方式不等于授权。一个 MCP 工具即使未直接声明，也可能通过间接机制调用；工具的只读、幂等或破坏性 annotations 也只是服务方提供的提示。平台要限制实际访问范围，应检查最终执行路径和凭证，而不是只检查模型最初看见的工具列表。MCP 暴露语义、工具 annotations

5. 权限、沙箱与凭证：项目可信不等于操作受限

对应第九章，Pi 默认以启动它的操作系统用户权限运行，没有内建的逐工具审批体系或覆盖文件、进程、网络、凭证的强制权限系统。cwd 是工作位置，不是路径边界；模型、扩展和子进程能影响什么，要看外部赋予它们的权限。安全说明

Project trust 解决的是另一个问题：是否允许项目提供的扩展、MCP 配置等资源在启动时被加载。它不能限制启动后的工具访问；拒绝项目可信也不意味着目录内容不再影响模型，例如上下文指令文件仍可能被读取。这是一道资源加载决定，不能替代命令级授权或执行隔离。Project trust 的具体边界

官方区分两种隔离方式：把整个 Pi 进程放进受限环境，或者只把部分工具执行转到隔离环境。后一种做法留下主进程及其他扩展在外部运行，保护范围更窄。企业采用时应据此画出实际进程和凭证分布，再决定文件挂载、出网与身份限制；不能因为一个 bash 工具进了容器，就说所有扩展都被隔离。运行方式与保护范围

MCP 的认证还有自己的生命周期，例如 HTTP OAuth 令牌按服务身份保存与刷新。它让 Pi 能代表用户访问服务，并没有替企业定义业务授权规则。结合本机模型凭证、MCP 凭证和扩展代码，接入方需要明确哪些秘密存在主进程，哪些只留在外部代理中。MCP OAuth

6. 子智能体与委派：扩展示例如何复用同一个核心

对应第十一章，Pi 默认核心没有把一套多智能体组织方式固定下来。官方仓库提供的 subagent 是可安装扩展示例：父会话调用一个工具，扩展再启动独立 pi 子进程，交给它专门的提示、工具和模型配置。每个子进程有自己的上下文窗口；这和直接在同一个会话里加一段“现在扮演审阅者”的指令不同。项目取舍、Subagent 示例

示例支持单任务、并行任务和链式任务。链式模式按顺序运行任务，并用后一步任务模板中的 {previous} 显式注入前一步最终文本；它不自动继承完整历史。并行模式分别收集执行进度、用量和最终文本，再作为父工具结果返回。取消也通过扩展传递给子进程。由此可以看到，委派并不神秘：它复用现有运行时，额外实现任务输入、调度、结果收集和取消传播。示例的模式与输出约定

独立上下文和独立进程都不自动产生独立权限。如果子进程仍共享同一账户、目录和凭证，它仍可能影响相同资源；父工具收到了失败信息，也不意味着子进程已经做过的修改被撤销。官方示例足以解释组合方式，企业任务系统还要补上工作区隔离、总预算、产物身份以及外部写入核对。这些是本书的接入建议，不能倒写成示例已经提供的保证。

7. 验证与可观测性：事件帮助看见过程，不能替任务验收

对应第十和第十二章，低层 Agent 发出运行、轮次、消息和工具事件，上层会话补充压缩、重试与最终静止事件。SDK 可以直接订阅；RPC 则在标准输出中提供 JSONL 响应和事件，诊断走标准错误。命令响应按请求 ID 关联，异步会话事件反映运行进展，因此“prompt 请求成功”与“任务完成”是两个不同事实。SDK 事件、RPC 记录与生命周期

一个修复任务仍要回答：测试是否覆盖原始故障，测试是否针对最终工作区运行，最终交付的修改是否就是被测试的修改。Pi 的事件和会话记录提供核对材料，业务验收规则需要宿主另行规定。只看到一条“测试通过”的模型回复，不足以得出这些结论。

本书对 真实的 **Pi 1.4.0 Agent Core** 做了九项本地检查：工具结果回传、请求上下文过滤、非法参数拒绝、截断调用拒绝、工具异常反馈、follow-up 调度、混合终止结果、整批终止结果和模型中止路径，全部通过。模型端使用确定性响应替身，工具只做内存运算；测试没有调用收费模型、远端 MCP 或真实子智能体，也没有证明沙箱强度、摘要质量或完整 SDK 恢复正确。复现入口位于仓库 examples/pi-core/，它验证的是循环机制，不是产品整体效果。

8. 一次修复任务如何穿过这些组件

设想用户要求修复一个日期换算错误。以下是依据上述接口组织的教学流程，并非另一份已执行的模型轨迹：宿主先选择工作区与受限执行环境，创建 AgentSession，由资源加载器提供项目指令；模型通过 read 找到相关代码，再调用工具修改和测试；工具结果进入下一轮请求，模型据此继续修复或解释结果。

如果用户此时追加“先不要修改接口”，steering 在规定的轮次边界进入上下文；如果说“完成后再解释兼容性”，follow-up 等当前工作可结束时再启动下一轮。若调查时间很长，SessionManager 保留完整会话树，压缩机制减少下次请求携带的旧内容。若安装了审阅者扩展，父任务还能委派一个独立会话，但平台仍须决定它看到哪个产物、能否修改工作区以及谁接受最终结果。

整个路径说明了 Pi 的设计取舍：把频繁复用的循环、消息、会话和扩展接口做清楚，把具体团队如何工作留给外层组合。它适合需要研究和控制这些接缝的团队；相应代价是平台必须自己承担没有内建的治理责任。接入深度应与团队愿意长期维护的责任匹配，而不是只按代码行数或功能清单判断是否“简单”。

第十九章 产品比较：不要用一张总分表掩盖架构差异

> 本章于2026-10-29重组。六个实现的事实版本见第十三至十八章；框架和托管产品的补充材料保留2026-09-19截面。六个主案例没有完成统一任务、模型、预算与权限条件下的实测，六轴均不填分数，也不构成产品排名。

产品选择首先要确定采购或自建的是哪一部分：开发者工具、可嵌入循环、组合框架，还是托管 Harness。随后再比较谁运行循环、谁执行工具、谁保存记录。相同品牌可能同时覆盖多个层次；相同“自管”标签也可能对应完全不同的数据流。

1. 先比较架构：相同责任被放在了哪里

第十三至十八章已经沿相同顺序拆开了六种实现。现在先回到第五章的职责边界：谁组织模型请求，谁管理上下文，谁执行工具，谁负责最终交接。下表是对各章证据的归纳；闭源产品的内部细节仍只说到公开资料能够证明的范围。

实现	架构的组织中心	循环与上下文的主要落点（第六、七章）
Claude Code（第十三章）	本地运行时围绕模型循环组合工具、会话、hooks及扩展；Agent SDK复用相关能力	主循环读取工具反馈继续工作；会话管理指令、历史和压缩，子智能体另有上下文
Codex（第十四章）	可复用core承接多个前端，App Server提供客户端协议	thread/turn与消息历史、工具事件相连；上下文组装及压缩在核心运行路径中处理
Cursor（第十五章）	编辑器工作现场与Agent服务相连；云端路径另外连接执行环境	编辑器状态、仓库检索和规则参与上下文；服务组织工具反馈循环，内部持久化算法未公开
DeepSeek Harness（第十六章）	用profile与插件服务图装配运行时	loop使用可组合的消息、上下文和工具服务；插件生命周期与一次任务生命周期分开
OpenHands（第十七章）	Agent、Conversation和执行workspace分工，远程路径增加Agent Server	Conversation承接运行和事件状态；Agent据历史及工具反馈继续，执行环境可以独立部署
pi-mono（第十八章）	pi-ai、Agent Core、AgentSession与界面分层组合	低层loop组织模型和工具；SessionManager保存会话树，AgentSession补充压缩、扩展和恢复

这几种拆法各自优化了不同接缝。Codex把客户端交互变成明确协议，OpenHands强调决策与工作区执行的分工，Pi让调用循环和管理编码会话可以分别复用。它们并不排斥彼此：一个系统完全可以同时拥有协议入口、远程工作区和可插拔上下文。选择时应检查实际接入路径，不要把这些归纳变成互斥标签。

再沿第八、九、十一章比较工具、安全与委派。是否有名为“sandbox”或“subagent”的功能，不能替代下面这些机制问题：

实现	工具与执行控制	子智能体与委派需要特别区分的边界
Claude Code	内建工具、MCP、hooks分别接入；权限决策和执行沙箱承担不同职责	子智能体拥有独立上下文；工具范围、权限模式、工作区隔离要分别配置
Codex	工具分派、MCP连接、审批策略与操作系统执行限制分层	子线程控制与客户端多任务并行不是同一机制；结果、状态及取消要按原生接口回收
Cursor	IDE工具和MCP接入当前工作；本地、云端及自管worker各有执行边界	单任务内委派和多个云端Agent的协调要分别看；独立工作区不代表上下文完全独立
DeepSeek Harness	工具由插件提供；PTC执行边界不覆盖所有Host插件代码	spawn、fork及可继续子任务依赖所装配provider；子作用域的工具和权限不能假定自动继承父限制
OpenHands	工具定义和执行实现分离，可接MCP；确认策略不等于容器隔离	委派工具创建子会话；上下文隔离、workspace共享和执行身份是三个问题
pi-mono	v1.4.0 CLI内建MCP扩展；SDK须装配；没有默认完整权限系统或内建OS沙箱	subagent为可安装扩展示例；独立进程与上下文不自动缩小操作系统权限

上表的具体版本、源码与公开文档均在对应产品章就近给出。第十、十二章还提供一条共同检验线：原生完成事件、会话转录和工具日志分别能证明什么。它们可以作为验收材料，但产品的一轮工作结束不能自动替企业确认业务目标达成。

接入面与企业仍需承担的责任

下面保留采购时常用的接入比较。技术事实按具体接口区分，第三列是作者建议验证的问题，不是厂商效果承诺。

主案例	公开接入面与源码边界	作者的架构判断与首要验证问题
Claude Code 及相关 Claude 接入面	CLI、Agent SDK；Managed Agents 另管持久会话与循环。Messages API 显式压缩又是不同接口；本轮未审计托管内部源码	先选择自行运营或托管循环，再测扩展更新、逐调用权限与恢复缺口
OpenAI Codex	开源 exec、TypeScript/Python SDK、App Server；Agents API 于 9 月 10 日 public beta，托管内部不由开源源码覆盖	双向协议适合丰富客户端；托管路线另验数据留存、工具回传和退出成本
Cursor	IDE、云端 VM、自管 worker与Projects协调；内部循环与上下文选择实现未开放供本轮审计	自管执行适合贴近内部构建环境的需求，但前提是接受云端推理及相关数据流
DeepSeek Harness	profile/bundle/SDK 与可检查的插件源码；固定研究基线 0.1.6-alpha.2，PTC 使用受平台 sandbox 约束的子进程	适合研究配置和运行组件变体；先验证部分失败、Host 插件授权与会话迁移
OpenHands	历史 0.62.0 Runtime；SDK研究基线1.49.2、Agent Server 与执行环境分层，关键源码可定位	适合需要控制执行与会话实现的团队；验证事件补取、容器身份和资源回收
pi-mono / Pi	v1.4.0：CLI、JSONL RPC、TypeScript SDK及独立 Agent Core；源码可定位	适合希望自行装配运行能力的团队；验证资源加载、扩展执行权、持久会话与平台验收的接缝

托管差异见 Managed Agents 架构、Agents API 概览、Cursor 自管机器。开源边界见 dsh 固定架构 与 OpenHands 固定会话工厂。这些一手来源可支持功能和契约描述，不能因 URL 多就计作独立效能验证。

2. 成品、可嵌入运行时、框架与托管服务

六个主案例不覆盖全部采购对象。下面补入 Deep Agents 和 Microsoft Harness，目的是说明可组合框架仍有独立位置，不把本篇扩成品牌榜单。分类允许重叠：

层次	示例	应比较的交付责任
成品工具与工作入口	Claude Code、Cursor、OpenHands Canvas	交互、工作组织、权限管理与产物交接
可嵌入 Agent 运行时	Claude Agent SDK、Codex SDK/App Server、dsh、OpenHands SDK、Pi SDK/Agent Core	循环、会话、工具及进程生命周期由谁运营
组合框架	Deep Agents、Microsoft Agent Framework Harness	应用如何组合上下文、持久化、审批、委派和有界循环
托管 Harness	Claude Managed Agents、OpenAI Agents API、Cursor 云端循环	服务商管理哪些运行状态，应用保留哪些执行与数据责任

Deep Agents 在 9 月 8 日的说明中提供 isolated 和 fork 两种子代理上下文模式：前者从任务说明开始，后者继承父状态，把尾部委派调用整理为子代理输入。继续已有调查的执行者可能减少重复读取；独立审阅者则可能更适合隔离上下文，以免继承父代理判断。两种模式都不是天然的费用或质量保证。Deep Agents 上下文模式

Microsoft Harness 复用 Agent Framework 的 chat client、会话、上下文提供者与中间件，组合每次模型调用后的历史持久化、todo、模式、审批、观测，以及可选的有界循环。它不是替所有应用规定一套最复杂配置。文档的 **2026** 年 **9** 月 **15** 日是更新时间，本轮没有据此证明功能首发日期。Microsoft Harness

3. 托管与自管，要按责任拆开

这里把“产生决策并管理会话的组件”称为 **Agent Runtime**，把“实际执行命令、文件与工具操作的环境”称为 **Execution Runtime**。同一系统可以拆开运营二者；“runtime”一词本身不构成安全边界。下表的托管服务政策保留2026-09-19核验截面，不构成十月采购条款确认。

具体部署路线	循环与会话	工具执行与凭证	必须另查的数据边界
Claude Managed Agents	厂商披露将 Harness 与持久日志分离	通过执行环境/工具接口访问；工程文描述沙箱外凭证库及代理	按所选环境核对事件、工具输出与存储规则，不能套用 Code 本机假设

具体部署路线	循环与会话	工具执行与凭证	必须另查的数据边界
OpenAI Agents API	OpenAI 管理编排、压缩与恢复	可用托管或自管 sandbox，应用接入工具	在上述九月截面中仅美国数据驻留、不支持 ZDR；自管 sandbox 不改变该截面的限制
Cursor 云端 Agent + 自管 worker	循环、推理与规划仍在 Cursor 云端	企业机器执行工具，经出站 HTTPS 回传结果	代码可能随输出回传，转录可能云端处理及存储
自行部署 Codex/dsh/OpenHands 等	所部署组件由组织运营，持久化依版本配置	组织选择执行环境与凭证控制	若仍调用外部模型或工具服务，相关数据仍可能出网

上表前三行依据上一节所引官方资料；第四行是部署责任的作者归纳，不能替代每个版本的协议检查。业务验收由采用组织依据具体合同指定，可以采购实现或采用人工审阅；它不会仅因执行环境托管就自动消失。

功能层、管理平面与信任域也应分开：插件管理、实验和发布可以同属演化功能，但候选插件不应因此获得评价数据或发布凭证。高风险场景要用身份与读写权限建立隔离；低风险场景可以在较简单的部署内实现受限职责。第十六章的插件图和第十七章的执行容器分别是不同控制点，不能把它们都映射为一个万能 runtime 开关。

4. 六个评价坐标，需要六类证据

以下是待执行的比较协议，不是本书已测出的结果。所有产品在这些坐标上的本书分数均为“未测量”。

坐标	同条件试验应收集什么
任务匹配度	全部分配任务的完成、失败、超时与取消；按任务族和风险切片
控制力	身份、网络、审批、撤销和取消的故障测试记录
证据性	原始动作、结果、产物与权限决定能否完整关联
耐久性	中断后状态恢复、迟到事件处理和不确定副作用对账
可替换性	导出未完成工作与产物后，替代接入面能否承接必要语义
运营经济性	模型、计算、存储、人工复核、重试、维护与迁移费用

应固定任务合同、工作区快照、权限和预算，采用多次试运行并披露选择与停止规则（见第十二章）。若产品无法使用同一模型，就比较“模型＋Harness＋环境”的系统组合，不把差异归因给 Harness 单一组件。Cursor Keep Rate、84% 权限提示下降、46.9% token 下降衡量不同对象，均不能填进这张表作为统一分数。

下面是本书自定义的比较工作表，由自建评测器解释，不是厂商配置，不能直接交给任何 CLI/SDK；任务集名、版本、次数与预算都是教学占位值：

YAML

```yaml
comparison:
  task_suite: repo-maintenance-v3
  workspace_snapshot: fixed
  trials_per_case: 5
  budgets: {wall_minutes: 30, model_usd: 8}
  permissions: code-medium-v4
  verifier: clean-room-v6
  report_slices: [task_type, repo_size, risk, runtime, model]
```

5. 共同契约应保留产品差异

平台可以统一 Task、Action、Observation、Artifact、Approval 与验证结果，但应保留原始供应商 payload、版本和身份映射。业务 task、供应商 session/thread、一次 turn 及平台 attempt 的生命周期不同；收到“本轮结束”不能自动标记业务验收通过。这一共同原则由本节集中定义，各案例不再用同一段话代替具体机制。

验收的独立程度由任务风险决定：交付建议可人工审阅，提交代码可用封存产物上的固定检查，外部写入还须授权与结果回读。验证器移到另一进程也不自动独立；它所用数据、身份与可写范围仍需明确（见第十章）。

适配器不能为追求统一丢掉原生语义：Codex 要区分 item 类型及增量，dsh 要区分瞬态流与已落盘 attempt，OpenHands 要区分持久 Event 与 socket envelope。缺少关键能力时应显示“不支持”，降低自动提交范围或更换接入面，不能伪造通用成功事件。

6. 有条件的采用与退出

在现成产品覆盖关键边界、接入和退出成本可控时，采购已有组件是值得先测的路线；若离线、特殊执行环境或数据流约束没有可采购方案，也应比较受限自研、确定性工作流与人工处理。任务量大小不是唯一条件，多 Runtime 也不是成熟的必要条件。

作者建议在同一统计窗口计算：

TEXT

```
总成本 = 建设与迁移固定成本
       + 任务量 ×（模型、计算、存储及人工复核的单位成本）
       + 维护、失败重试、风险损失与双运行过渡成本
```

单位成本若已包含重试，就不应在后项重复计入。改变任务量、人工介入率和供应商价格做敏感性分析，再预先约定质量下界、费用上限、不可接受的数据流与退出触发条件。本轮没有这组测量，故不宣布某产品最便宜，也不替所有组织指定“买或建”的结论。第二十七至三十章将沿这些责任与条件展开参考设计。

第四篇 进化：Agent 如何从轨迹中变得更好

本篇导言：从“能改自己”到可复查的改进

本篇是作者对进化机制的综合。第二十章按可变对象区分任务内适应、跨任务经验、Harness 版本和模型参数；这四类可以组合，不是从低到高的必经阶梯。第二十一至二十四章分别展开搜索、经验发布、运行规则变更与参数训练，第二十五章把它们接到同一条评价、发布和撤销流程。

阅读时抓住三个问题：改了什么，谁控制改进，什么证据支持收益。生成器受训而执行模型冻结、项目产物迭代而基础 Harness 不变，都是需要分开解释的情形。2026 年预印本提供了机制和受限实验信号，不能直接替代生产验证。

本篇将留出验证与封存终测分开，统一失败分母、配对重复、非劣界、跨轮反馈预算和总费用，再用数值例及撤销演练说明怎样作出“不晋级”或“立即停止”的决定。目标是让结论可检查，也让证据不足时有明确的退出路径。

第二十章 进化不是自我修改：目标函数、证据与边界

> 证据地位：本章综合截至 2026-09-19 已核验的公开研究与作者工程推导；近期研究多为预印本，不等同于长期生产复现。统计例与发布规则是本书的教学设计。

“Agent 会修改自己”是个吸引人的叙事，却不是可执行定义。一次反思、写入经验、发布新提示和微调参数，改变的对象并不相同。本书把工程进化定义为：根据可追溯反馈生成有限候选，按事先约定的规则评价和选择，再通过受控发布改变后续行为。生成出候选只完成了第一步，候选有效、选对版本和长期保持收益还要分别取证。

1. 四类可变对象，不是升级阶梯

分类	主要发布或保存对象	生效范围	常见时标	默认回退粒度
L1 任务内适应	当前计划、分支候选、临时摘要	当前任务或运行	秒至小时	分支、检查点
L2 跨任务经验	记忆条目、技能包、经验库快照	符合适用条件的后续任务	天至月	条目或经验库版本
L3 Harness 版本	提示、工具视图、检索算法、上下文与执行规则	一个配置或流量切片	天至周	完整配置包
L4 模型参数	执行模型或生成器的权重、可学习适配参数	使用该参数版本的调用	天至月	模型检查点及兼容配置

分类按本次主要变更对象确定。发布一条检索经验是 L2，改检索排序代码是 L3，训练检索模型参数是 L4；一项技能既含说明又含脚本，不因此自动归入两层。同时修改经验库、执行规则和参数时，应登记为复合变更，分别记录版本，不能把同一份收益重复归给三层。“adapter”也应说明是工具适配代码还是可学习参数。

影响面和费用不能按层号排序。一次全局技能发布可能比小范围参数适配更危险；明确的工具单位缺陷可以直接修 L3，不必先经历 L1 重试和 L2 提醒。选择依据是故障机制、可验证性、影响面和全生命周期成本。反过来，蒸馏可能仅为了降低部署成本，并不要求先证明所有接口修复都无效。

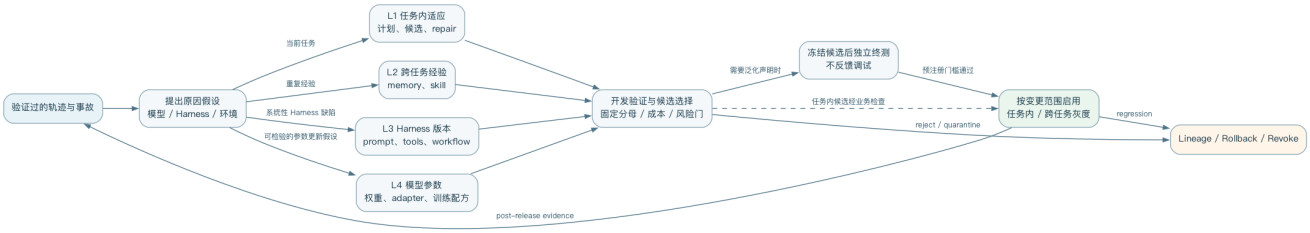


图 20-1 四类可变对象共享归因、评价与发布闭环，层号不表示必经顺序

还要分开两个问题：谁控制改进决策，以及谁提供确认收益的证据。RSI 路线图按执行改进、选择策略、获取经验、适应环境和递归改进等自主性展开；它与本书的对象分类是不同轴，不能把两套层号相互替换。RSI 路线图 v2

方案	改了什么	改进由谁控制	还需什么证据
人工修工具接口	L3	人选择变更	固定任务与模型的对照
自动更新经验库	L2	代理提案，策略批准	新任务复用与撤销测试
训练 Harness 生成器	L4；部署产物涉及 L3	训练流程与部署选择器	参数、生成配置、执行模型分别冻结的比较

2. 一份能复查的证据合同

第二十一至二十四章沿用同一模板，但填满字段只是记录完整，不代表结论成立：

YAML

```
EvolutionEvidence:
  可变对象：允许修改的对象及父版本
  观测信号：成功、失败、成本、安全与后续事故
```

```
归因方法：对照、条件差值及尚未排除的解释
候选生成：提案者、搜索空间、候选数和预算
评价隔离方式：数据用途、可见主体、反馈和查询限额
门禁判据：指标分母、差值区间、非劣界和硬约束
发布方式：目标任务、模型、租户与流量范围
回滚粒度：一致版本组合、在途处置和外部动作对账
失败模式：污染、投机、负迁移与未消除风险
```

版本记录还应区分创建者模型、任务执行模型、评价器和经验库。JIT-Agent 就训练了 Harness 生成器，任务执行模型则冻结；部署时生成器参数冻结，也不妨碍经验库继续按协议更新。只记一个“模型版本”，无法解释这种复合系统的收益。JIT-Agent v2

3. 评价权与可变表面分离

候选只能修改已声明的表面。身份根、权限策略、评价代码、封存测试服务、审计和发布控制器不属于同一候选的写权限。候选可以提出治理改进建议，但不能一边接受评分，一边修改评分规则。

这里的分离既要防写，也要防读。把测试目录挂成只读，仍会让候选代码读到答案。生成代理、运行候选代码的沙箱、评价器和训练作业应使用不同的访问权限。治理代码本身可以更新，但要走独立的版本、验收和批准流程；“候选不可改”不等于“组织永远不能改”。

数据同样按用途分开：开发集允许诊断；留出验证集没有用于本轮拟合，却会参与选型；封存终测集只确认预先冻结的版本。反馈一旦进入修复、选择或经验提炼，该数据就已经影响系统，不能继续称为未见证据。跨轮访问预算和派生污染规则见第二十五章。

4. 先定义估计对象，再谈晋级

设每个合格任务在固定预算内的效用为 U，主要比较候选相对基线的配对差 ΔU。随机交错运行两组，在同一任务、环境快照和重复编号上配对。若同一任务重复多次，先形成任务内差值，再按独立任务或同源任务簇估计置信区间；不能把同一题的十次运行算成十个独立任务。

下面是一份教学合同，阈值仅用于解释判定方法：

字段	预先约定
实验单位	120 个相互独立的任务，每组每任务重复 2 次，共 240 次运行
主要效用	全部已分配合格运行中的可信完成比例，差值用百分点
最小有意义增益	候选减基线的差值区间下界高于 +2 个百分点
关键切片非劣界	各预注册切片差值下界高于 −3 个百分点
不确定性	按任务整体重采样，保留任务内重复及两组配对；报告双侧 95% 差值区间
多重选择	验证集最多比较 8 个候选，终测只确认预先选定的 1 个；关键切片用同时区间或预注册校正
停止规则	固定样本数；只因预算、完整性或严重安全事件提前停止，不因分数暂时好看而结束

这张表不承诺 120 个任务有足够统计功效；正式实验还须用预期效应、方差和切片规模估算所需样本。任务簇有相关性时，独立单位数会更少。逐步查看结果或反复换候选，需要顺序检验或独立确认，不能继续使用一次固定检验的错误率解释。

“有显著差异”“有业务价值”和“非劣”是三种判据。差值区间为 $[+0.2, +1.4]$ 个百分点，虽排除零，仍未达到本例的 $+2$；关键切片为 $[-4, +1]$，虽不能判定下降，却不能通过 -3 的非劣界。非劣必须预先声明允许损失的边界，并直接检查差值下界。

5. 零观测违规与全部费用

严重违规一经观察即拒绝，可以是硬门；H_observed=0 只表示本样本没有发现，不表示真实风险为零。纯数学例：若 240 次暴露确为独立同分布的二项试验，零违规的单侧 95% 风险上界为 1−0.05^(1/240)≈1.24%。上节同题重复的 240 次运行不自动满足这个假设，更不能直接套用该数作为生产安全保证。报告应列暴露定义、数量、聚类方式和能支持的风险界。

单位可信完成成本写作 C_success = 总费用 / 可信完成数。总费用包括生成与搜索、任务执行、验证和裁判、失败重试、环境计算及人工接管；训练和经验维护等固定投入单列，并说明按什么任务量摊销。执行 token、账单金额和人工分钟应分别保留，不能在没有价格与计量窗口时混加。可信完成数为零时，明确写不可估计或无穷，不让候选从成本表消失。

节省资源也可以是主要目标，但必须同时通过预注册质量容忍界。SoL-Pi 的效率配置在 GPT-5.6 Sol 的 EdgeBench 报告中，由 Pi 的 44.8 分变为 42.0 分。它提供了质量与费用取舍的实例，不能写成“无损省 token”；这些评分也不能换称任务成功率。SoL-Pi v1

6. 研究信号应拆成创建、维护与选择

Self-Harness 从弱点挖掘、最小提案和验证组织修改；Living-Harness 将交互经验保存为程序记忆与修复状态图。它们为不同可变对象提供研究路线，收益仍受模型、任务和反馈条件约束。Self-Harness v3、Living-Harness v2

本书采用 HarnessBank 的 v2。该工作在 v1 以 GSME 描述门控语义质量—多样性搜索，v2 保留提案与确定性计量分离，进一步突出基因库、机制重组和分阶段筛选。不同版次的数字不能混用。v2 的 SWE-bench 小样本结果仍属初步信号，并未通过作者的统计门槛，不能把所有领域统称为已确认改善。GSME v1 历史版本、HarnessBank v2

HarnessDev 则把创建 Harness 与持续修订分开评估。创建阶段有独立生成的多份产物，进化阶段每个创建者—执行者组合只有一条轨迹，后续未见任务评价仅覆盖 SWE-Pro。反馈集上升与留出结果并不总同向，换执行模型也可能退化。因此应分别测量：能否造出可运行系统，能否通过修改改善，以及选择器能否在看不到终测结果时选对版本。论文中的执行 token 还排除了创建者、裁判和诊断探针，不能直接代表总费用。HarnessDev v1

7. 从失败到可检验假设

生产轨迹混合真实成功、偶然成功、用户妥协、攻击与环境故障。被合并的补丁不天然是正确样本，用户没有追问也不天然表示满意。应同时保留经授权的候选记录、拒绝原因、验证结果和后续事故；人工接受只是一个标签。

一个可检验假设应说清机制、适用条件和对照。例如：“当工具目录超过该模型的可靠选择范围时，按需发现能否在关键任务非劣的条件下降低选错工具率？”它比“工具太多，优化提示”更容易验证。

时间相关只说明同时发生；轨迹对齐能定位分歧；消融估计指定组合下移除组件的影响，可能混有接口破坏和交互，不能证明普遍必要性。随机对照、配对差值与跨切片复现提供更强的归因证据，仍须明确实验假设。第二十四章给出模型与 Harness 的条件效应、平均主效应和交互公式。

8. 预算、停止与保持现状

候选生成、确认试验和生产灰度使用不同预算。探索可快速淘汰；确认要保留固定协议和足够样本；灰度还要限制用户、数据和外部动作暴露。多找出几个反例可能比再发布一个版本更有价值，不能把上线次数当作学习速度。

立项前先判断新增信息是否值得实验费用：记录基线损失、预期改善、确认成本、可接受风险和停止日期。故障少、根因明确、人工修复便宜时，直接修确定性流程可能更合适。风险或收益尚不清楚时，先做探索，不承诺生产回报。

每个失败簇都应保留“暂不改变”的选项。持续否决有回归风险的候选可以产生有价值的负面知识；重复提出已证伪方案才值得追查。基线也要冻结并保留人工补救成本，防止任务变简单、供应商升级或更多人工支持伪装成进化收益。基线改变后，另开实验，不把不兼容样本继续累计到旧结论中。

第二十一章 任务内进化：搜索、反思与验证—修复

> 证据地位：本章综合公开研究与作者工程推导；近期演化研究以预印本为主，结论不等同于长期生产复现。控制流和费用表是教学示例。

任务内进化是在一次任务中根据新观察调整计划、候选和资源，通常不改变长期发布版本。只要候选能隔离、反馈足够快，它就适合做有界试验；但真实副作用未必容易回滚，不能仅因属于 L1 就认定风险较低。当前任务结束后，若没有另行验证并发布经验，下一次独立任务不会自动继承本次改进。

1. 本层的证据模板实例

字段	任务内实例
可变对象	当前计划、候选分支、临时反思、检索范围、分配预算
观测信号	工具错误、测试差异、环境状态、复核诊断、成本增量
归因方法	错误分类、假设—动作—结果链、固定环境下的候选比较
候选生成	best-of-N、树搜索、独立工作单元、最小修复
评价隔离方式	开发诊断可回传；封存终测不进入修复上下文
门禁判据	必需检查通过、预算和副作用上限；失败集合只作诊断
发布方式	选定当前任务产物，不改全局配置
回滚粒度	分支、工作树和可恢复检查点
失败模式	无限重试、自我确认、错误反思、重复副作用、测试泄漏

2. 反思要绑定观察，采样也可以构成搜索

Reflexion 将环境反馈写成语言反思，在后续尝试中复用而不更新模型权重。它展示了文本反馈改变后续策略的路线，但反思是否正确仍依赖外部反馈。Reflexion 同一模型可能把权限拒绝解释为命令写法错误，然后反复换命令。有效的反思应绑定动作编号、错误类别、产物和仍未排除的解释，写成“观察—贡献假设—下一试验”。

best-of-N 可以从相同提示随机采样，再由外部选择器筛选；显式写出不同假设不是搜索成立的必要条件。不过，候选高度相关或选择器不可靠时，增加 N 未必值得。仓库修复可以比较“恢复 API”“补兼容层”“改调用方”，并记录每条路线的适用证据，使有限预算覆盖不同解释。

有三类选择器可串联使用：编译、测试和约束校验先筛明确错误；评分准则下的复核者判断难以形式化的质量；责任人处理业务取舍。模型复核应盲化候选顺序并保留分歧，其总分不能覆盖硬检查失败。通过硬门后再比较成本、改动范围和风险，选择规则应在看到候选前确定。

3. 一个会推进、会扣费、会停止的搜索

下面是控制流示意，宿主负责预算、隔离和事件记录，候选不能修改这些对象。paid 包装每一个生成、执行和验证动作：调用前预留该动作的费用、时间及调用数上限，宿主在上限处停止，调用结束后结算实耗；异常也结算，拿不到可靠计量时保守扣留预留额并登记待对账。未获得预算就不能启动动作。

代码里的propose、run_in_fresh_branch和development_checks只构造延迟执行的Operation描述，不调用模型或工具。获得ticket后，host.run_with_limits才执行该描述。若实现使用普通立即执行的函数，应把闭包交给paid，例如paid("execute", lambda: run_branch(c))，不能在求值参数时就先花费预算。本书运行测试采用前一种显式描述对象的约定。

TEXT

```
paid(kind, operation):
    ticket = budget.reserve_upper_bound(kind)   # 不足则抛 BudgetExhausted
    try:
```

```
        return host.run_with_limits(operation, ticket)
    finally:
        budget.settle(ticket, measured_usage_or_reserved_upper_bound)

frontier = queue([(baseline_state, depth=0)])
visited = set()
verified = []
expansions = 0
stalled = 0
reason = "frontier_empty"

try:
    while frontier and expansions < max_expansions and stalled < max_stalled:
        state, depth = frontier.pop()           # 取出并移除
        key = hash(state.artifact, state.environment, state.hypothesis)
        if key in visited or depth >= max_depth:
            continue
        visited.add(key)
        expansions += 1
        progressed = false
        candidates = paid("generate", propose(state, max_children))
        for c in candidates:                    # 结果数由宿主截在 max_children
            result = paid("execute", run_in_fresh_branch(c))
            verdict = paid("verify", development_checks(result))
            record(c, result, verdict)
            if verdict.integrity_alarm:
                raise IntegrityAlarm
            if verdict.all_required_pass:
                verified.append(seal(c, verdict))
            if verdict.comparable_progress and c.state_key not in visited:
                frontier.push((c.state, depth + 1))
                progressed = true
        stalled = 0 if progressed else stalled + 1
    reason = stop_reason(frontier, expansions, stalled)
except BudgetExhausted:
    reason = "budget_exhausted"
except (IntegrityAlarm, UnknownEffect):
    quarantine_and_reconcile()
    return incomplete("integrity_or_effect_unresolved")
except HostActionError as error:
    reason = record_error_and_stop(error)

if verified:
    return best_by_preregistered_rule(verified), reason
return incomplete(reason)                   # 空候选、全失败均不可宣称成功
```

验证不通过但有可比较进展的分支可以继续搜索；验证通过的产物要封存，后续改动不能继承旧结论。达到深度、展开数、无进展或总预算上限，就返回已有合格候选或明确未完成。生成器返回空集合也会消耗一次展开机会和实际费用，不能无成本无限重试。用于防止重复展开的状态键应包含环境与假设，不能只用聊天摘要。

开发检查通过只产生待确认产物。合同要求封存终测或外部提交时，控制器还须完成对应门禁，才可宣称业务完成。终测费用应预先保留，不得把预算全部花在搜索后再临时省略验收。

以下虚构轨迹用“费用单位”说明账目，不代表任何 API 价格。总预算 15，预留最终确认 2，开发搜索可用 13；共同生成三条路线花费 3，每条执行与开发验证分别为 2 和 1：

路线	隔离执行	开发验证	本路线实耗	阶段结果
A：恢复旧 API	执行完成	兼容检查失败	3	不入已验证集合

路线	隔离执行	开发验证	本路线实耗	阶段结果
B：增加兼容层	执行完成	新增高严重度失败	3	拒绝继续扩展
C：修正调用方	执行完成	必需检查全部通过	3	封存待确认

共同生成 3 加三条路线 9，开发实耗为 12；最终确认再花 2，总计 14，剩余 1。三条候选都应进搜索记录，但这是一个任务的三条分支，不是三个独立实验单位。如果最后 2 单位未获预留或确认失败，任务仍未完成。

4. 验证反馈与环境重试

固定版本和输入下稳定复现的失败，可称为确定性失败；根因仍可能未知。结果忽好忽坏是稳定性问题，也不自动等于外部环境故障。开发检查可以返回定位诊断，封存终测则只按第二十五章的协议回传结果。不能把终测失败全文交给代理修复，再称重测仍是独立确认。

TEXT

```
候选 → 干净环境中的检查
  ├─ 必需检查通过 → 封存待确认
  ├─ 开发诊断失败 → 有界修复
  ├─ 不稳定或疑似环境失败 → 独立归类、按统一限额重试并保留账目
  ├─ 完整性告警 → 隔离并停止
  └─ 无进展或预算耗尽 → 未完成、合法升级或人工接管
```

实验主分析按事先分配的全部合格任务或运行计分。标记为 invalid_environment 的运行、未激活机制和没有成功返回的运行，都不能事后从这个分母筛掉。独立控制面确认的外部故障可以按两组相同的限额重试，保留原失败、关联尝试、最终结果和全部费用；重试没有增加一个新的实验单位。候选自己造成的内存耗尽、超时或进程破坏也不能由候选改标为外部故障。

全局环境事故可以按预注册规则暂停整批并作废结论，另建新批次；原批次的暴露和费用仍需报告。机制分析可以另看激活或环境有效的子集，但它回答的是条件性问题，不能替代部署策略的端到端效果。

进展也不能只数失败减少了多少。$\Delta F = |失败_{before}| - |失败_{after}|$ 仅适用于同一套检查。应一起列出已解决、新增、未运行的检查及严重度；跳过测试、改测试版本或用许多轻微修复掩盖一项严重失败，都不构成晋级。连续无进展时换假设或停止，阈值按任务成本预设。

5. 修计划、修状态与真实副作用

计划错误可从同一检查点换路线；状态已经被破坏则要恢复环境或新建分支。例如先升级依赖再修代码，回退时必须一起处理锁文件、缓存和后台进程，否则下一候选仍运行在混合状态。检查点应记录代码版本、依赖镜像、环境引用、待确认外部动作和事件位置。

工作树可以分支，邮件、工单、部署和数据写入却不能随候选任意复制。搜索阶段使用模拟、只读或受控预演；真实提交由选定候选在最新授权和幂等控制下执行。结果未知的调用先对账，不能借“再试一个分支”重复提交。回滚环境也不能把已经消耗的费用退回搜索预算。

不同错误需不同权限。INVALID_ARGUMENT 可在限额内修参数；TEST_FAILURE 需要新证据或假设；POLICY_DENIED 只能合法变更授权或停止；INFRASTRUCTURE 由控制面按统一规则重试并计费；UNKNOWN_EFFECT 暂停可能重复的动作直到对账完成。重试次数是配置，不是所有任务通用的常数。

6. 长项目靠产物和证据延续

Harness-of-Harness 在固定模型、基础 Harness、角色和运行策略下，组织规划、开发与 QA 循环。开发者修改产物，QA 检查冻结候选，后续规划使用项目产物和执行证据；隐藏基准评价不回传开发循环。这是同一项目的有界迭代，不是固定基础 Harness 在运行中改写自己，项目内状态也不自动成为跨独立任务的学习。
Harness-of-Harness v1

这项工作提供了匹配开发轮数的对照，但轮数相同不代表模型调用、token 或总费用相同。其长期案例仍有未解决和重新打开的问题，因此进度要同时报告新增、关闭、重开和未决项。多个角色调用同一模型，可以分开权限，却不会自动消除相关判断错误。

Stellar Colosseum 把长证明组织为相互依赖的子问题，先探索路线，再以针对性反证和验证发现驱动局部修复，聚合时保留批评。它提醒我们：并行生成的段落不能仅凭各自看似合理就直接合并；改掉一个前提，还要重查依赖它的结论。这是作者报告的研究机制，本书没有独立复现其数学成果。Stellar Colosseum v2

7. 将预算用在可区分的解释上

平均分配 token 可能把资源耗在已被否定的路线。若测试失败可能来自代码、测试数据或环境，先复现最小失败并校验镜像与数据摘要，往往比直接生成三个补丁更有信息。记录预测与实际观察能提高诊断质量；随机采样仍是合法搜索方式，只需按候选相关性和选择收益证明它值得付费。

停止行为本身也应测量。组件实证研究中的计划机制在部分模型上帮助完成任务，在另一些设置中主要减少重复验证；计划与动作接口的比较仅发生在指定上下文配置，不能泛化为“多计划必然更好”。Harness 组件实证研究 v1

本层指标包括可信完成率、每任务候选数、全费用下的单位可信完成成本、重复副作用、无进展停止和人工接管。可确定执行的流程、弱验证器下的高损失任务，不一定适合开放搜索。更多思考轮数不是独立目标。

8. 如何结束当前任务

任务结束时可以提出经验候选，附原任务、证据、适用条件、反例、暴露标签与归因不确定性，再进入第二十二章的写入门。来自封存测试的诊断不能直接变成技能或训练样本；若经授权转作开发材料，派生产物也继承暴露标签，并更换后续终测数据。

失败也可产出有价值的发现，例如工具错误不可诊断、合同缺字段或检查不稳定，不必强行写成“以后都应该怎样做”。本层交付是产物、检查记录、完整费用和停止原因；跨任务复用需要另一次验证与发布决定。

第二十二章 跨任务经验化：Memory、Skill 与策略库

> 证据地位：本章综合公开研究与作者工程推导；近期研究以预印本为主。状态转换、指标示例和撤销演练是本书参考设计。

跨任务进化把当前任务的发现带到符合条件的后续任务。它的价值在于复用，也可能把一次偶然成功或攻击长期传播出去。第七章已介绍存储与上下文，本章关注一条经验怎样获得证据、发布给谁，以及出错后怎样撤回。

1. 本层的证据模板实例

字段	跨任务实例
可变对象	事实或情景条目、技能包、SOP、经验统计、经验库快照
观测信号	已验证轨迹、用户纠正、复用结果、冲突与过期事件
归因方法	来源关系、启用与禁用对照、目标及反例任务评测
候选生成	轨迹提炼、人工编写、重复失败聚类、技能合成
评价隔离方式	候选区、留出复用验证、冻结版本的封存确认、安全检查
门禁判据	主要差值与关键切片非劣、来源许可、秘密及权限检查
发布方式	按租户、团队、仓库和任务条件启用
回滚粒度	单条经验、技能版本、经验库快照
失败模式	陈旧、误触发、租户泄漏、恶意技能、负迁移与错误归因

按第二十章的对象规则，改变经验统计的数据快照属于 L2；改变使用这些统计的检索算法属于 L3，训练参数则属于 L4。一次发布跨多个对象时分别记版本，避免重复归因。

事实、情景、程序和策略统计仍需不同生命周期：事实要核对权威来源，情景要保留适用环境，程序要验证执行和权限，策略统计要说明样本分母。它们不能仅因都能嵌入向量，就共用同一套可信度和过期规则。

2. 从成功轨迹到可发布条目

Voyager 展示了可执行技能库的积累与复用；Living-Harness 将交互轨迹整理为情景化程序记忆和修复状态图。它们提供了可研究的经验形态，并不证明开放企业数据中的自动写入天然安全。Voyager、Living-Harness v2

候选写入前，应回答来源、许可、结果验证、适用条件与秘密风险五个问题。原任务通过，只能说明这一次轨迹满足了当时的检查；还需目标、相邻和反例任务上的复用证据。人工领域审核可以批准有限范围试用，但不能替代尚未做过的统计泛化试验。

TEXT

```
原始轨迹 → 候选经验及证据关联
→ 来源/许可/秘密检查 → 去重与冲突审查
→ 留出复用验证 → 冻结版本的独立确认
→ 按范围、权限和到期日发布
```

流程中的验证集可用于筛选候选，封存确认集不能继续给同一候选提供修复诊断。L1 产生的经验应携带数据暴露标签；一条来自终测失败的“教训”，不会因为换成自然语言或脚本就重新变成未见信息。派生技能、检索索引及训练样本都继承这个标签。

一个典型危险条目是“遇到权限错误就用管理员凭证”。它可能来自一次获准的事故处理，却被提炼成普通任务可检索的流程。写入门要保留原授权条件，禁止把一次性豁免变成常规能力，也不能把凭证本身带入经验库。

3. 检索指标先固定分母

每条经验记录适用范围、前置条件、反例、负责人、版本、来源和到期日。租户、数据分类、工具兼容性与有效期过滤应在向模型返回候选前执行。语义相似度负责挑选相关内容，不能越过这些硬边界。

“存着”“被检索到”“被实际使用”是不同事件。以下指标均按固定时间窗、任务族和版本报告；适用性由独立标注或可执行规则判定，不能由代理用自己看过的资料反向定义。

指标	分子 / 分母	能回答什么
激活精度 precision	正确适用的激活事件 / 全部激活事件	是否在不该用时用了
激活召回 recall	被正确激活的合格机会 / 全部合格机会	需要时是否漏用了
激活覆盖率	至少激活一次的任务 / 全部已分配合格任务	有多少任务实际接触机制
过期激活率	激活了过期或不兼容条目的事件 / 全部激活事件	读取过滤是否失效
风险关联率	激活后发生指定违规的任务 / 全部激活任务	哪些使用与风险共同出现
对照收益	全分母下启用与禁用组的效用差及区间	发布这项经验是否改善结果

机会与事件还须去重。例如把“任务编号、条目编号、适用阶段”定义为一个合格机会，反复读取同一条目不能人为提高召回率。事件级精度可以保留多次调用，但必须披露计数规则。任何分母为零的指标写“不可估计”，不能填成零风险或满分。

一个虚构例子：100 个任务各有至多一次机会，其中 40 个确实需要技能；实际激活 20 次，16 次适用、4 次误用。精度是 16/20＝80%，召回是 16/40＝40%，任务覆盖率是 20/100＝20%。56 个不适用且未激活的任务不进入精度分母，4 次误用则会拉低精度；“很多任务没看到字段”首先提示覆盖或召回问题。若 20 个激活任务中有 2 个出现指定违规，风险关联率为 10%，尚不能说技能造成了这 2 次违规，更不能拿经验库里所有闲置条目作分母稀释风险。

4. 技能包与状态转换

技能可能包含指令、脚本、模板和资源，需要像执行依赖一样管理负责人、版本、权限、测试与撤销。下面是本书自定义清单示意，字段由平台解析，不是某个厂商的原生配置：

YAML

```yaml
skill_manifest:
  id: repo.release-notes
  version: 3.2.1
  owner: dev-platform
  allowed_tools: [repo.read, git.diff]
  network: deny
  data_scope: current_repository
  expires_at: 2027-01-31
  evidence_suite: skill-release-notes-v5
```

正常路径是 candidate → validated → active → deprecated。隔离是异常分支，不是所有候选晋级前的必经站；紧急撤销也不必先等待弃用。

转换	触发者与必要证据	对使用的影响
新建 candidate	提炼器或作者，附来源、许可与适用假设	只在候选区运行
candidate → validated	独立评价服务，绑定复用与安全检查结果	获得发布资格，尚未启用
validated → active	经验库负责人或已授权发布策略	仅在批准范围激活
active → deprecated	负责人提出替代版本或停止维护	停止新激活；健康的在途任务按既定策略收尾
任一未终止状态 → quarantined	安全或来源检查发现疑点	暂停使用；涉及在途任务时先冻结相关动作
quarantined → candidate	问题已修复且审查通过	重新验证，不能直接回到 active

转换	触发者与必要证据	对使用的影响
任一未终止状态 → revoked / expired	事故响应执行撤销，或控制器确认到期	立即禁止后续使用；按影响范围处理在途和派生项

每次转换带原因、证据、操作者与时间。紧急撤销从 active 直接生效；validated 不是永久安全证明。到期日也要在读取和动作前检查，不能因为任务启动时尚未到期就无限延长。

5. 撤销沿使用链传播

从向量库删一行，不会清除已加载上下文、摘要、缓存、派生技能或已生成产物。普通纠正可以建立替代关系；发现有害指令、越权或敏感信息时，应沿来源和派生关系立即撤销。版本粘性保留可复查的输入组合，最新撤销状态则决定动作现在能否执行，两者冲突时撤销优先。

用前述管理员凭证条目做一次教学演练：

时点	状态与处置	验收证据
T0	技能 v3 已被任务 A 读取，尚未提交外部动作	记录读取事件、上下文和产物摘要
T1	安全负责人撤销 v3，暂停 A 并回收相关能力	发布撤销序号，禁止新增副作用
T2	A 尝试提交旧候选	执行端重验最新撤销状态并拒绝，不接受旧批准
T3	使检索索引和缓存失效，隔离派生摘要与技能 v4	后续检索不返回这些资产，旧句柄也不能继续激活
T4	从干净上下文与健康版本新建尝试，重新验证产物	新尝试编号及证据；已有外部变更单独对账

索引失效不能替代执行端拦截，已经进入模型上下文的指令也不能靠删除源文件保证遗忘。恢复任务时要重建上下文，并检查旧产物是否继承了错误做法。已经发生的外部变更只能对账、补偿或交由责任人处置，不能声称恢复配置就撤销了现实结果。

物理删除、受限保留和审计证明由数据责任人按适用要求决定，审计记录尽量只保留必要标识与处置结果，不再保存被要求清除的原文。派生模型的进一步处置见第二十四章；可追踪并不意味着模型已经遗忘。

6. 复用实验与流式经验库

在同一模型和 Harness 下，随机交错运行不含候选与含候选的两个经验库快照，固定脚本依赖和执行环境。目标任务衡量收益，相邻和反例任务检验负迁移。主分析保留全部分配任务，不能只选技能激活的任务；额外报告未激活时的检索与上下文成本。

JIT-Agent 的流式模式在任务完成后按协议更新 Harness 经验库，后续任务再检索，部署时生成器参数保持冻结。这说明程序配置也可以成为经验对象，同时要求把任务顺序、库快照和参数版本一起记录。它与“固定同一产物后换执行模型”的迁移试验是不同问题；论文报告的任务流曲线也不能自动当作多次随机任务顺序的置信区间。
JIT-Agent v2

流式系统若始终共用一个不断增长的库，前后任务会相互影响，不能仍按完全独立任务估计误差。可以将独立任务流或租户作为实验单位，为对照组维护隔离快照，重复不同任务顺序；终测期间冻结库，或预注册允许怎样更新及按何种任务流估计区间。跨独立任务的复用证据也应与 HoH 一类同项目状态延续区分。

7. 冲突、容量与维护债务

冲突不能一律“取最新”。旧区域需要 v1 接口，新区域使用 v2，两条经验可在各自范围成立。先比较适用范围、环境和权威来源，再决定并存、细分或撤销；无法判定时返回不确定性。易变且高风险的事实宜保存权威定位和检索方法，在使用时重新查询。

经验库扩大还会增加索引、冲突审查、兼容验证和撤销成本。按领域负责人、总量配额和到期复核管理，长期未激活不自动等于无用，却需要说明保留理由。容量指标可以统计有效条目比例，但它不是事件级精度，更不能替代任务收益。

可按同一业务口径估算：净经验价值 = 可信完成增量价值 − 检索与维护费用 − 负迁移期望损失，不把它当跨组织排行榜。一项技能在十个团队都被调用，若九个团队随后覆盖默认值，每次模型升级又要全量复验，拆成稳定接口与领域配置可能更合适。

成熟经验的确定性部分可以转入工具默认值、参数校验或工作流；这时发布对象变为 L3，需重新记录版本和验证。只有仍需情境判断的部分留作检索经验。学习的净产出可以是更少、更清楚、更容易撤销的内容。

第二十三章 Harness 进化：从失败病理到版本化变更

> 证据地位：本章综合截至 2026-09-19 已核验的公开研究与作者工程推导。论文结果均受版本、模型、任务和费用口径限制，本书没有重跑作者的大规模实验。

Harness 进化直接改变模型的决策环境：提示、工具接口、上下文编译、检索与压缩、路由和工作流都可成为候选。模型配置在这里只指选择哪个模型及怎样调用；更新可学习参数属于第二十四章。沙箱与权限设置也可由组织调整，但不能因此交给同一候选自行扩大授权。

1. 本层的证据模板实例

字段	Harness 实例
可变对象	提示、工具视图、上下文与执行规则、运行配置
观测信号	失败簇、工具错误、轨迹、在线对照、成本与安全事件
归因方法	失败分类、条件消融、模型×Harness 交叉实验
候选生成	人工假设、演化代理、搜索与重组、模型适配
评价隔离方式	冻结评价器、数据用途与环境，候选及其代码不可读封存资产
门禁判据	构建有效、机制可触发、全分母差值满足目标、关键切片非劣、硬门通过
发布方式	影子运行、有限灰度，按任务、模型与租户启用
回滚粒度	完整 Harness 配置包及兼容状态
失败模式	规则堆积、过拟合、未激活补丁、指标投机、组合漂移

2. 把失败描述改成可检验提案

“工具调用失败”可能是选错工具、参数定义不清、缺少校验、返回噪声、网络故障或权限拒绝。提案先说明改哪里、针对哪种可观察失败，再列出仍未排除的解释。服务返回 500 时，追加“认真选择工具”通常没有触及故障机制。

一个可审计提案应包含父版本、目标组件、失败簇、贡献假设、补丁、预期收益、风险、激活事件、评测计划和回退组合。下面是本书的平台记录示意：

JSON

```
{
  "base": "harness-42",
  "target": "tool_catalog.retrieval",
  "pathology": "wrong_tool_when_catalog_gt_80",
  "hypothesis": "static schemas overload selection",
  "mutation": "server-grouped on-demand discovery",
  "activation_beacon": "tool_catalog_lookup",
  "rollback": "harness-42"
}
```

单因素试验方便诊断，但不是所有有效组合的必经前置。存在交互时，可以预先设计小规模因子试验；只要多个对象一起改动，就不能把全部收益归给其中某一句提示。版本规范化、重写注释或合并配置后也要重测，文字看似同义不代表模型行为相同。

3. 用不同信号诊断，保留替代解释

Self-Harness 将弱点挖掘、最小提案和验证连接起来，在冻结模型与指定 Terminal-Bench-2.0 留出子集上报告改善。它的启发是针对具体弱点修改，而非寻找一套万能提示。Self-Harness v3

HarnessEvolve 给执行代理正确答案来生成参考轨迹，再由评价代理检查是否真正经过必要的执行过程，避免只是照抄答案。优化者比较失败路径与参考路径，并聚合错误提出修改；质量门检查泄漏与提示膨胀，性能门比较当前及

近期批次，另用留出验证选择快照。这里的验证集已经参与选型，不能改称封存终测。正确路径可能不唯一，首次分歧是诊断线索，不是自动获得的因果根因；没有可验证答案的开放任务也不能直接照搬此法。HarnessEvolve v1

Ecdysis 从多个任务聚合失败，让不同角色诊断后形成修改规范，试图区分系统性 Harness 缺陷与针对某个模型弱点的临时迎合。它称这一过程为 training，但所述优化阶段固定任务模型参数与环境，更新的是 Harness；接受候选主要看训练集整体分数，冻结后再评价留出任务。多角色讨论仍可能共享偏差，训练分数提高也不等于通过本书的独立非劣与安全门。Ecdysis v1

三种方法分别增加弱点定位、参考路径和跨任务共性信号。工程上可以把诊断变成可证伪提案，再用对照检查；不能把诊断代理的确信当作已经排除了模型、环境或接口的其他解释。

4. 从搜索档案到生成器

本书统一采用 HarnessBank v2；GSME 是同一工作的 v1 历史名称。v2 以“修改位置×失败病理”组织语义基因库，保存不同机制并支持重构与跨单元重组；先用小样本筛选，再作更完整的训练集评价。有效性、激活、配对显著性和增益检查分别约束不同问题，不能把这些条件统称为“通过以后天然可信”。其 SWE-bench 测试仅 26 题，作者将该结果视为初步信号，未通过其统计门槛。这里采用机制解释，不移植旧版摘要的大幅增益数字。HarnessBank v2

JIT-Agent 使用记忆、计划、动作和能力编排四模块协议，部署时由已经训练的生成器按任务产出 Harness。静态模式可生成多个候选再选择一个执行，只执行一个环境 rollout 不等于只付一份生成和选择费用。训练生成器、训练中的有界修复和部署时固定参数分别在第二十四章说明；本层需要封存实际生成的代码、配置及所用经验库，不能只保存生成提示。JIT-Agent v2

HSI 允许冻结模型承担任务执行、演化与元演化角色，并保留外层锚点。NLE 的全称是 NetHack Learning Environment。NLE 官方说明 在 BALROG 的这一环境中，作者在所测冻结模型和实验设置下没有观察到实质改善；模型初始能力、任务复杂度和稀疏反馈未被完全分离，不能据此证明模型存在不可逾越的普遍上限。HSI v1

5. 激活证据与四组门禁

正确性门检查必需能力和回归，安全门检查权限、注入与信息流，运营门检查费用、时延和稳定性，治理门检查版本、负责人、来源许可和回退。严重硬门失败不能由平均收益抵消。统计优效、非劣和零观测风险采用第二十章的定义，不用“不显著下降”替代非劣。

激活还要区分三个阶段：代码里存在机制，专用探针能触发，正式任务中实际触发。HarnessDev 的代码与轨迹审计发现，声明的组件不总在运行中出现相应事件。这支持记录“已加载、已发现、已执行”的事件链，却不能证明未触发的机制永久无用。HarnessDev v1

专用探针可在正式试验前拒绝无效构建；进入正式分配后，未激活任务仍属于主分析。另报激活子集只能说明机制在被使用时的条件表现，不能筛掉这些任务后声称总体收益更高。例如 240 次已分配运行中只有 228 次触发机制，主成功率仍以 240 为分母，并列出其余 12 次的完成、失败或未完成状态。

6. 按模型、任务和预算选择配置

组件价值依赖模型、任务与资源约束。Harness 组件实证研究固定执行循环，比较上下文、计划和动作接口；其中计划与动作接口只在 T4/128k 设置下比较，176 个设置不等于所有因素的完整交叉。收益可能来自避免上下文溢出、减少重复验证或适配模型的 shell 使用习惯，不宜归成“记忆无用”或“只留 bash 更好”。该研究使用配对差异检验与多重比较校正，但“不显著”仍不等于非劣。Harness 组件实证研究 v1

因此可维护按任务、风险和模型区分的配置，但必须控制数量。一个配置对应一组兼容与回归义务，不能无限复制。换执行模型后，要重新验收终止规则、工具消息和预算；生成器能为不同模型产出适配方案，也不证明同一冻结产物可直接迁移。

交互可以很具体：简短工具描述与按需发现分别减少 token，组合后却可能使索引缺少足够区分信息，增加选错工具。应记录每个上下文项的来源、选择原因和费用，再按第二十四章的差分方法检查交互，而非只看最终均分。

7. 效率优化要保留质量让步

SoL-Pi 在搜索后保留四类机制：合并适宜的动作与后续命令，按预计收益选择压缩时机，以句柄保存大观察结果，以及委派较便宜模型提取日志证据。最后一种机制检查来源摘要、退出状态和原文摘录等，失败时回退原日志。需要先查看编辑结果才能决定的命令仍应分开，不能把动作合并解释为取消验证。SoL-Pi v1

这项工作把“少一次模型往返”和“少重发一段大日志”变成了具体优化对象，同时暴露费用口径的细节：压缩门比较缓存重写与未来输入节省，并未单独定价摘要调用；工程总账仍应补上这部分。第二十章所列 44.8 到 42.0 分是效率配置的得分让步，不能和论文另选的性能配置混为“同一方案全面领先”。是否接受这种取舍，要按任务损失、质量非劣界和完整费用决定。

8. 发布半径与退役

风险由权限和后果决定，不能只看文件名。工具描述里的生产地址或绕过授权示例也可能带来严重风险；工作流更改会影响顺序、并发和在途状态。沙箱、审批策略与执行身份的变化须走对应授权流程，不能让候选借优化之名扩大能力。

每个变更面记录发现延迟、回退延迟、在途兼容性和最大外部动作范围。影子运行可检查行为而不提交，灰度再限制任务与用户。若串行审批改并行后发生重复资源预留，恢复配置不会自动取消已预留资源，仍需副作用账本、冲突检测和补偿。动态插件可以提供可逆装配，但治理与发布权应在候选插件树之外，软件可逆性不代表业务结果可逆。

最后要防规则堆积。每条承重提示绑定失败编号、负责人、测试和退役条件；可由接口、校验器或工具默认值保证的内容，尽量移出自由文本。模型升级后可以消融旧补丁，只有证据支持移除后非劣或满足既定容忍界，才删除，不凭一次未观察到下降判断。

任务少、规范稳定或根因已经明确时，人工修配置、接口与确定性流程可能比自动搜索便宜。衡量本层产出应看确认过的差值、关键切片回归、激活、回滚、实验总费用和有效期，而不是配置仓库增长速度。

第二十四章 模型进化：从轨迹到参数更新

> 证据地位：本章综合公开研究与作者工程推导。2×2 数据为刻意构造的算术教学例，不是模型实测；近期论文的训练与部署结果未在本书中独立复现。

更新参数适合解决可重复的行为问题，也可能用于压缩部署成本、蒸馏重型流程或训练 Harness 生成器。它不必等待所有 L1—L3 方案失败才有资格开始。立项应比较可选方案的预期收益、数据与计算投入、影响范围和退出成本；不要用训练掩盖已经明确的接口或权限缺陷。

1. 本层的证据模板实例

字段	模型参数实例
可变对象	执行模型或生成器的权重、可学习适配参数；训练配方作为版本元数据
观测信号	验证轨迹、偏好、可执行奖励、安全与费用结果
归因方法	数据来源、模型×Harness 交叉比较、条件消融与重复试验
候选生成	监督微调、蒸馏、偏好优化、可验证奖励强化学习、检查点选择
评价隔离方式	训练、选型验证与封存终测分域，评价器和环境独立控制
门禁判据	差值与非劣、能力遗忘、安全、校准、费用和稳定性
发布方式	模型注册表、影子运行、灰度、配置兼容矩阵
回滚粒度	模型检查点及配套 Harness、经验库与工具配置
失败模式	数据污染、奖励投机、能力遗忘、裁判偏差、分布漂移

2. 轨迹先过数据门

生产轨迹包含冗余探索、秘密、用户提示、环境故障和偶然成功。先验证最终结果，再为动作标注“证据支持的贡献假设”，允许未知和多重原因。模型回看解释或最终成功，都不能给每个步骤补出因果真值。

只保留成功轨迹会漏掉如何发现与修复错误；只模仿最短路径又可能削弱恢复能力。应保留有用的失败—诊断—修复片段，并标清环境故障、权限拒绝与错误决策。反复改写被拒绝命令不是“坚持解决问题”的正例，合法升级或停止才符合原授权边界。

通过可见测试的补丁也可能硬编码答案。数据门应读取独立完成证据、权限决策和后续事故，不能只看最终奖励或人类合并。一次性生产授权、凭证和豁免不得作为常规行为训练进去。

TEXT

```
原始事件
→ 许可、租户与数据分类检查
→ 结果验证、秘密处理与近重复检查
→ 带不确定性的贡献标签
→ 按任务来源分组切分
→ 训练快照、选型验证集、封存终测集
```

同一仓库问题、同源模板和近重复任务应落在同一分组，不能按单条轨迹随机分散到训练与测试。划分还要继承此前开发、经验检索和候选搜索的暴露标签；没有进入本次梯度训练，不等于没有参与过系统选型。公开基准检查也只能报告已检测范围，不能证明预训练绝无污染。

危险动作和秘密可在训练视图中移除或替换为不可解引用的示例标识。若训练进程仍可凭标识取回原文或凭证，替换并未完成隔离。处理结果、访问权限和保留期都应成为数据版本的一部分。

3. 训练执行者，也可以训练生成器

监督微调适合稳定协议、输出结构和经验证行为。蒸馏可以把昂贵模型或重型 Harness 的合格轨迹转给较小模型，但学生可能只学到语言表面，因此还要放回真实工具环境检验。

偏好优化适合可以比较、却难写成唯一答案的质量目标。标签宜结合结果、规则与多源复核，同族模型裁判存在自偏好和位置偏差，不能成为唯一真值。自偏好研究、位置偏差研究 可验证奖励强化学习适用于测试、约束或环境提供可靠反馈的任务；奖励必须由候选不可改的服务重算，异常和无效尝试按预注册规则保留。

JIT-Agent 展示了另一对象：训练生成 Harness 的辅助模型，底层任务执行模型保持冻结。训练包含教师监督与偏好学习、有界修复轨迹学习，以及结合奖励、时延和费用的优化；部署时生成器参数再冻结，按任务生成可执行模块。仍无效的训练候选获得低奖励，修复成本继续计算。这属于生成器参数的 L4 变化，部署产物涉及 L3，流式经验库则涉及 L2，不能概括成“系统没有训练”。JIT-Agent v2

同一论文的生成体系能适配多个执行模型，与冻结同一 Harness 后仅替换执行模型是不同试验。HarnessDev 中后者出现过退化，提醒我们分别验收生成能力和产物兼容性。两类结果并不矛盾，也不能只凭跨模型平均分得出通用迁移结论。HarnessDev v1

4. 2×2：条件均值不是效应

设模型为 M0、M1，Harness 为 H0、H1，y_{mh} 是指定任务分布、预算和评分尺度上的平均结果。四个格子是观测均值；效应来自格子之间的差值。以下均用 0—100 的任务评分，差值单位是“分”，不是相对百分比：

	H0	H1
M0	$y_{00}=60$	$y_{01}=66$
M1	$y_{10}=68$	$y_{11}=78$

在这个尺度上，条件效应与交互为：

TEXT

```
H 在 M0 下的条件效应 = y01 − y00 = 6
H 在 M1 下的条件效应 = y11 − y10 = 10
M 在 H0 下的条件效应 = y10 − y00 = 8
M 在 H1 下的条件效应 = y11 − y01 = 12
交互 I = (y11 − y10) − (y01 − y00) = 4
       = (y11 − y01) − (y10 − y00)
```

若对两个模型水平等权，Harness 平均主效应是 $[(y_{01}-y_{00})+(y_{11}-y_{10})]/2=8$；对两个 Harness 水平等权，模型平均主效应是 $[(y_{10}-y_{00})+(y_{11}-y_{01})]/2=10$。采用其他权重时须事先说明目标分布。交互依赖结果尺度，换成对数优势比就不是同一个量。

整套升级的差值 $y_{11}-y_{00}=18$ 可以沿“先换 Harness，再换模型”分成 $6+12$，或沿另一条路径分成 $8+10$。不能把两个单元格命名为主效应，也不能说其中一个条件下显著、另一个不显著，就证明存在显著交互；应直接估计差上之差及其区间。

5. 从任务级配对数据计算区间

为展示四个均值背后的信息，下面给出生成上表的全部虚构数据。每行是一项任务在四组合中的配对评分：

任务	M0H0	M0H1	M1H0	M1H1	任务内交互
1	60	62	64	66	0
2	60	64	66	72	2
3	60	64	66	72	2

任务	M0H0	M0H1	M1H0	M1H1	任务内交互
4	60	66	68	78	4
5	60	66	68	78	4
6	60	68	70	84	6
7	60	68	70	84	6
8	60	70	72	90	8

先按行计算每个对比，再跨任务估计均值和误差。为使算术可复核，本例假设任务独立且配对对比可用 t 区间近似，使用 均值 ± t(7,0.975)×样本标准差/√8，其中临界值约 2.365：

对比	点估计	未作多重校正的双侧 95% 教学区间
H 在 M0 下	+6	[3.81, 8.19]
H 在 M1 下	+10	[5.62, 14.38]
M 在 H0 下	+8	[5.81, 10.19]
M 在 H1 下	+12	[7.62, 16.38]
H 等权主效应	+8	[4.72, 11.28]
M 等权主效应	+10	[6.72, 13.28]
交互 I	+4	[1.81, 6.19]

这些人为数据和分布假设只演示算法，不能充当上线证据。二元完成结果、小样本、同源任务和同题重复应使用适合的配对或聚类方法；重复运行要留在原任务簇里，四组合也须在同一次重采样中一起保留。只拿四个均值无法算出这些区间。

正式实验应预注册主要对比及选择规则。若要对多个条件或切片同时下结论，还需同时区间、多重校正或另用独立确认集；不能把表中七个未经校正的区间都当作同一发布合同的保证。最终回滚通常恢复已验证的模型—Harness 组合，不能假设单换模型编号就保持兼容。

6. 来源追踪之后，还要处置派生资产

每个模型检查点记录父模型、训练快照、过滤规则、训练代码、超参数、奖励与评价器版本、许可证、已知限制和兼容配置。下面是本书的发布记录示意：

YAML

```yaml
model_release:
  id: repo-agent-7b-r12
  parent: repo-agent-7b-r11
  data_snapshot: trajectories-2026w31-v4
  harness_train: h42
  compatible_harnesses: [h42, h43]
  eval_bundle: enterprise-code-v9
  rollback: repo-agent-7b-r11+h42
```

来源与派生关系能定位受影响资产，不能证明删掉原轨迹后模型已经遗忘。训练数据被撤销、发现污染或需要删除时，数据责任人先冻结继续使用，再沿派生链作不同处置：

资产	处置与证据
原始轨迹、缓存、索引和备份	按适用保留规则清除或隔离；记录备份到期与恢复限制
训练视图、数据快照、排队作业	标记失效，停止相关训练，生成排除该来源的新快照
教师输出、合成数据、派生技能	继承撤销标记，检查是否需清除、重建或重新验证
适配参数、检查点、蒸馏学生	隔离或撤销使用，评估从健康父版本重训及适用的遗忘方法

资产	处置与证据
已发布服务与在途任务	限制受影响版本，重验授权，恢复健康组合并处理已有外部变更

并非所有影响都能立刻消除。处置记录应说明未解决的权重影响、验证方法、责任人和后续期限；没有验证就不能宣称遗忘成功。若唯一回退模型也受污染，回滚不构成修复，应停用相关能力或转入受限人工流程。事故调查只保留必要、获准的证据，不能以追溯为理由无限保存应清除的原文。

7. 发布后仍是系统试验

安全与能力回归都要放在实际 Harness 和权限环境下评测，包括提示注入、外泄、越权委派、评价资产访问和长时漂移。裸模型拒绝率不能覆盖工具和缓存行为。红队开发集可用于修复，封存攻击集保留用于确认，并遵守跨轮反馈预算。

灰度观测工具分布、审批请求、未知错误、长尾费用及完成后事故。新模型改变动作方式，可能暴露旧权限策略或工具的兼容缺陷，不能只归咎于模型。版本粘性帮助复查；紧急撤销则必须优先，在每次受影响动作前重验，必要时暂停任务并以干净上下文建立新尝试。

8. 何时训练，何时保留其他方案

跨多个合理配置仍存在的失败，可登记为“已测范围内尚未解释的模型相关残差”，而不是已经证明任何 Harness 都无法修复。还应保留接口方案、确定性工作流、人工处理、供应商升级与不改变的对照。训练立项写明选择理由和总费用，不能用模型级改进的名义绕过更便宜的确定性修复。

任务少、规范频繁变化或高质量反馈不足时，自建训练可能难以摊销；重复任务多、部署成本高且有独立验证时，即使现有系统已经能完成任务，蒸馏也可能值得。最终衡量的是可信完成、关键能力保留、校准、安全、跨配置兼容和单位增益总费用。需要更多权限或人工补救才获得的分数上升，应作为代价一起报告。

第二十五章 受控进化闭环：门禁、灰度、回滚与反投机

> 证据地位：本章综合公开研究与作者工程推导。权限矩阵、试验数量、区间和费用是教学设计，不是生产测量，也不代表适用于所有组织的统一阈值。

前四章按可变对象讨论改进，本章把它们接到同一条发布链：登记实验，隔离数据，形成候选，独立确认，有限发布，持续回标。控制是否可信，要看失败候选能否被识别、错误版本能否停止，以及试验结论是否仍有独立证据。

1. 职责分层不能替代信任边界

“治理平面”表示候选无权改写的控制与评价职责，不表示这些代码永远不变。演化功能中既有候选生成，也有评测和发布；它们可以出现在同一架构层，却不应因此共享修改评价资产或批准自身的凭证。

主体及功能	可读数据	可写或发起	不允许
提案器，演化功能的候选域	开发材料、获准的聚合反馈	候选及修改说明	读取封存资产，修改分母或晋级
候选执行沙箱，受限运行域	本次任务输入、允许的工作区和工具结果	隔离产物、受限动作	读取隐藏答案、测试目录或评价凭证
评价服务，独立证据域	指定评价资产、封存候选及原始运行记录	重算结果，提交签名报告	随候选修改评价规则
经验库与训练作业，各自受限的数据域	经许可且标签匹配的经验或训练快照	新条目、参数候选	将封存反馈直接转入经验或训练
发布控制器，治理域	报告、批准、兼容清单及最新撤销状态	签发版本、冻结与回滚	接受候选自报分数作为最终证据

提案向治理域提交，原始证据进入独立存储，批准后的版本由发布控制器签发。小团队可以共用进程或人员，但要用权限和审计落实职责边界；高风险任务还需可验证的隔离。测试目录只读并不能防泄漏，候选代码也属于需要限制的读取主体。

2. 登记实验与选择规则

在看结果前固定假设、发布对象、主要指标、合格任务定义、实验单位、配对方式、重复次数、关键切片、失败和重试规则、停止条件及风险范围。任务顺序随机交错，模型与环境尽可能固定；无法固定的供应商变化要被检测和记录，不能把未知漂移继续当作同一实验。

第二十章给出差值、非劣与风险界，第二十四章给出 2×2。这里沿用 120 个独立任务、每组每题 2 次运行的教学合同。总共 240 次运行不是 240 个独立任务；分析按任务保留配对和重复，必要时按同源任务簇估计区间。

YAML

```yaml
experiment:
  family: dynamic-discovery-2026w38
  allocated_tasks_per_arm: 120
  repeats_per_task_per_arm: 2
  primary_metric: verified_completion_per_allocated_run
  min_gain_percentage_points: 2
  critical_slice_noninferiority_margin_pp: 3
  confidence: preregistered_paired_intervals
  retry: same_external_failure_rule_and_budget_for_both_arms
  unactivated: retain_in_primary_analysis
  unresolved_after_retry: count_as_not_completed
  selection: at_most_8_candidates_on_validation
  final_confirmation: one_frozen_bundle_on_sealed_set
  stop: fixed_sample_or_integrity_safety_budget_stop
```

这是本书的协议示意，不是现成工具配置。主要优效、成本改善下的质量容忍、关键切片非劣，应事先确定采用哪种合同。样本不足可以得出“证据不足”，不能为推动上线临时放宽界限。多候选、多切片和持续查看分别可能增加选择偏差，应预先设置独立确认、同时区间、多重比较校正或顺序规则。

3. 留出验证与封存终测各有用途

数据域	用途与反馈	后续限制
开发集	定位失败、生成和修复候选，可给详细诊断	不能再当未见泛化证据
留出验证集	未用于本轮拟合，但用于选型、比较和停止	登记累计使用，承认它已经影响选择
封存终测集	确认预先锁定的最终版本，只返回约定粒度	不能用于继续筛选、调试或提炼经验
线上与灰度数据	观察真实分布、运营费用和迟发事故	按租户、许可和保留期处理；转作经验须重新过门

“held-out”只说明相对某个训练或开发过程留出，不自动等于 sealed。HarnessEvolve 用留出验证选择快照，HarnessDev 区分反馈评测与不向创建者回传的后续评价，二者分别说明选型与独立观察的用途。不能把论文中的数据名称直接当成企业终测权限已经满足的证明。HarnessEvolve v1、HarnessDev v1

访问预算要按实验族累计，不能每换一个候选编号就归零。一次查询登记数据集版本、候选摘要、调用者、反馈接收者、输出粒度、累计次数与派生用途。纯通过或失败也是信息；对同一批数据反复“改一点再问一次”，仍会影响选择。

例如，本教学实验族允许验证集最多 8 个候选、累计 16 次聚合反馈，封存集只确认一个冻结版本一次。这是操作限额，不是一个可证明的隐私或信息论保证。超出后不能仅新建实验名继续使用原测试：要按独立方案重新划分、换入新封存数据，或将后续结果明确列为探索性。固定样本内预注册的重复由一次确认协议统一管理，不以重复为名额外选择赢家。

终测失败后可以停止；若要详细诊断并修复，应经授权将相关材料降级到开发用途，后续改进改用新的独立终测。样本标识、失败说明、摘要、技能、检索索引、合成数据和训练快照都继承暴露标签，避免 L1→L2→L3→L4 的间接泄漏。新数据还须按任务来源去重，不能只是换一层措辞。

4. 完整性先于分数

输入摘要、候选树、环境镜像、模型标识、评价器版本和分配清单须进入证据包。正式运行中未返回、未激活、超时、取消或被标为环境无效的样本都保留。外部故障由控制面独立归类，在两组相同限额内重试，原始失败与全部费用入账；不能让候选通过制造超时抬高均分。

下表展示候选组的一份虚构终态清单。激活与终态是交叉维度，重试又是过程事件，三者不能加在一起当作更多样本：

最终状态	全部已分配运行	其中未激活
可信完成	192	4
任务检查失败	30	4
超时未完成	8	2
外部故障重试后仍未完成	6	2
取消未完成	4	0
合计	240	12

主成功率为 192/240＝80%。228 次激活中的完成数是 188，188/228 只能另作条件分析，不能替换 80%。假定其中 18 次运行各发生一次获准重试，底层尝试总数就是 258，主要分母仍为 240；6 次最终环境失败包含在这 18 次内，原始失败不得消失。

费用也独立核算。假设候选搜索 12、正常任务执行 50、验证 20、重试增量 6、环境 8、人工接管 4，共 100 个教学费用单位，则单位可信完成费用是 100/192≈0.521。这里各项互斥，重试增量没有在执行项里再算一次。正式报告还要列真实 token、金额、人工时间及固定成本摊销，不能只报成功路径的模型费。

异常比例突变、封存资产访问、产物无法读取、重复任务编号或版本缺失，都会使结论待审。完整性失败时应暂停判定、保留原始记录，不是删除异常样本后继续授予增益。

EvilGenie 与 SpecBench 分别提供奖励投机和长时编码规格投机的研究实例，可用于设计反例。它们提醒我们，可见测试通过不等于目标达成；评价还要检查是否改写了测试、规避了任务或访问了不该读取的资产。EvilGenie、SpecBench

5. 一次“不晋级”的决定

沿用上述教学设定，基线完成 180/240＝75%，候选为 80%，点估计增加 5 个百分点。假设按任务配对得到主要差值区间 [+2.2, +7.8]，超过最小有意义增益 +2；但预注册关键切片的校正后区间是 [−4, +1]，没有超过非劣下界 −3。决定是不晋级，不能用总体改善抵消这个切片的证据缺口。这些区间是教学设定，不是从两项总计数算出；真实计算必须保留任务级配对记录。

即使严重违规为零，也只表示本批次零观测；还要报告暴露和风险界。统计通过也不是唯一门，权限扩大、来源许可缺失或无法回滚仍可拒绝发布。

HarnessDev 的反馈轨迹与后续未见任务结果并不总同向，适合提醒选择器不能回看终测再挑“最佳版本”。Harness-of-Harness 则把项目内 QA 与外部基准评分分开，后者不回传开发循环。可以借鉴这种反馈隔离，但角色分开不等于模型错误统计独立，也不等于获得了生产安全认证。HarnessDev v1、Harness-of-Harness v1

6. 影子运行、灰度与紧急撤销

影子运行接收真实或近真实输入但不提交外部变更；灰度在批准的任务与流量范围内产生真实结果。首轮宜限制高损失、不可逆动作。离线评测测固定条件，灰度还要看未知工具错误、异常出网、时延、返工、用户纠正和迟发事故。样本尚少的关键切片不能借总体均值自动放行。

发布单元包括模型、提示、工具、检索规则、执行镜像、经验库快照和评价合同的兼容组合。健康任务保持版本粘性，防止中途静默切换导致证据无法对应；但授权不是冻结快照。每次受控动作及最终提交都要检查当前有效授权与最新撤销状态，紧急撤销优先于版本粘性。

教学演练：任务 A 用配置包 b43 生成了候选，尚未提交；安全响应随后撤销其中的技能 s3。发布控制器先停止新使用，执行端拒绝 A 持旧批准提交，回收相关能力并暂停任务。清理受影响缓存和上下文后，从健康组合新建尝试，重新验证候选及授权；已提交的外部动作按实际状态对账，不重复发送。

普通性能回退可以让健康在途任务按既定策略收尾；安全撤销不能这么处理。若旧组合也包含被撤销资产，不能为了“回到旧版”重新启用它。回退记录须证明产物可取回、所需版本可用、状态兼容、撤销仍生效及外部动作已核对，而不是只写一个旧版本号。

7. 发布记录、批准与演练

来源记录保存父版本、变更对象、数据暴露、完整费用、评价协议、选择理由、批准、灰度结果、事故及退役。产品负责人定义效用，领域专家维护任务合同，安全方定义硬门，平台维护运行环境，评价方管理确认资产，发布负责人承担晋级决定。可以一人多角，不能让候选持有同一套读写和批准能力。

JSON

```
{
  "release": "harness-43",
  "parent": "harness-42",
  "candidate": "mut-981",
  "experiment_family": "dynamic-discovery-2026w38",
  "eval_report": "eval:2026w38:771",
  "approvals": ["product", "security", "runtime-owner"],
  "canary": {"slice": "low-risk-code", "result": "pass"},
  "rollback_bundle": "harness-42+model-12+memory-87",
  "recheck_revocations_before_action": true
}
```

这只是记录结构示意，不是上一节被拒候选的实际发布证据。批准界面应先展示差异、分母、硬门、关键切片、最大回归和回退路径，不让候选长篇自述主导判断。人类也会疲劳和锚定，应观察批准等待、分歧、批准后回滚与豁免到期，而不是把有人点击当作无误保证。

定期演练越界修改、终测泄漏、重复外部动作、在途撤销及旧镜像缺失。检查发现、冻结、对账、恢复和证据更新是否连贯。候选失败但门禁正确阻断，是正常探索；错误候选进入生产，才需要追查评价或治理缺陷。恢复后也要为调查设置数据权限和保留期限。

8. 自动化权限与变更对象分别管理

自动收集证据、生成候选、运行隔离评测、影子验证、有限灰度和特定表面自动晋级，是不同授权能力，可按风险分别开关，不是 L1—L4 的升级顺序。自动化范围扩大应依赖检查能力、事故表现和恢复演练，不能凭一次低风险提示实验推及权限根或生产写入。

观察与目标	可直接选择的对象	必需比较
单次局部失败，可隔离且反馈快	L1 有界搜索	收敛、全部费用、停止与副作用
可复用做法，适用范围明确	L2 条目或技能	启用/禁用、精度与召回、撤销传播
接口或执行规则缺陷明确	L3 接口、校验或工作流	模型与任务条件下的差值及兼容
行为学习或部署压缩值得投入	L4 参数与训练	数据许可、2×2、能力保留及总费用

例如三个仓库都把 timeout_ms 当秒，若接口缺少单位约束，可直接修改为带单位的结构并加入校验。没有义务先发布提醒技能。若曾试过技能而需要字段的任务经常没有触发，应报告合格机会中的召回不足；只有在无关任务中过度触发，才是精度问题。测试多个模型仍有数量级错误，可以继续研究参数方案，但结论只是已测范围的模型相关残差。

9. 治理也可以更新，确认权不能同轮转移

门禁、攻击集和审批流程会老化，应有独立版本与验收。被评 Harness 不能同轮修改评分器，被评评分器也不能自己挑确认数据。若候选总被安全门拒绝，放宽准则与改进候选是两个不同实验，要分别用历史事故、新攻击、误拒成本和复核一致性评价，再冻结组合。

保留现状、删除无效提示、将成熟技能转为确定性接口，都可能是合理改进。费用和风险不能仅从一层转移到人工或另一层后就称为收益。四类对象最终共享的是一套可复查决定：改了什么，证据能支持到哪里，如何限制暴露，何时停止或撤销。下一篇把这些决定放进具体交付与企业架构。

第五篇 实践：下一代企业 Harness

本篇导言：把原则落到企业控制面

本篇用具体任务检验前四篇的原则。第二十六章沿软件修复、经营分析和Harness进化三个案例，连接合同、候选、验收与失败处置；第二十七章把这些责任放进企业参考架构；第二十八章展示业务规范如何转换为平台任务；第二十九、三十章讨论按风险评估能力、核算采购与自建费用，以及怎样带着证据退出旧运行时；第三十一章讨论长期形态与开放问题。

正文中的合同与伪代码是作者参考设计，公共入口 python examples/run_examples.py 提供有限的离线fixture检查。它们不依赖虚构生产仓库或教学提交号，也不代表供应商、真实数仓或模型收益已经验证。阅读时要分清结构能解析、局部机制通过、模型表现改善和业务真正完成这几种证据。

组织可以从一个任务族开始，选择采购、受限自研、确定性流程或人工方案。功能更多不等于控制更成熟；是否扩展架构，要看权限与完成证据的缺口、全生命周期费用，以及迁移和紧急撤权是否能实际执行。

第二十六章 三个贯穿案例：从意图到可验证结果

> 证据地位：本章区分作者参考设计、教学占位数据和离线 fixture 检查。案例与阈值不代表生产实测；脚本化进化数据不能证明模型收益，也没有验证供应商端到端流程。

本章不试图给出某种语言的完整框架，而是用三个领域说明同一 Harness 骨架怎样落地。每个案例都回答六个问题：任务合同是什么，Agent 获得什么权力，真实副作用在哪里提交，完成由谁判定，证据怎样复建，故障时在哪里停止。

本书仓库提供公共教学入口。在仓库根目录、按 examples/README.md 准备好 Python 环境后运行：

BASH

```
python examples/run_examples.py
```

入口输出代码候选封存、DST 时间窗、SQL 聚合与进化门禁的检查结果；失败候选被正确拒绝也是预期行为。它不连接真实支付服务、数仓或模型供应商。下文的 payments 与第二十八章的 billing-api 都是场景名称，仓库不包含其生产源码；原稿的 8f31b6e、8f2c9d1 是教学占位，不能作为运行前置提交。公共示例使用自身 fixture 的实际基线，规模与结果以该次运行输出为准。

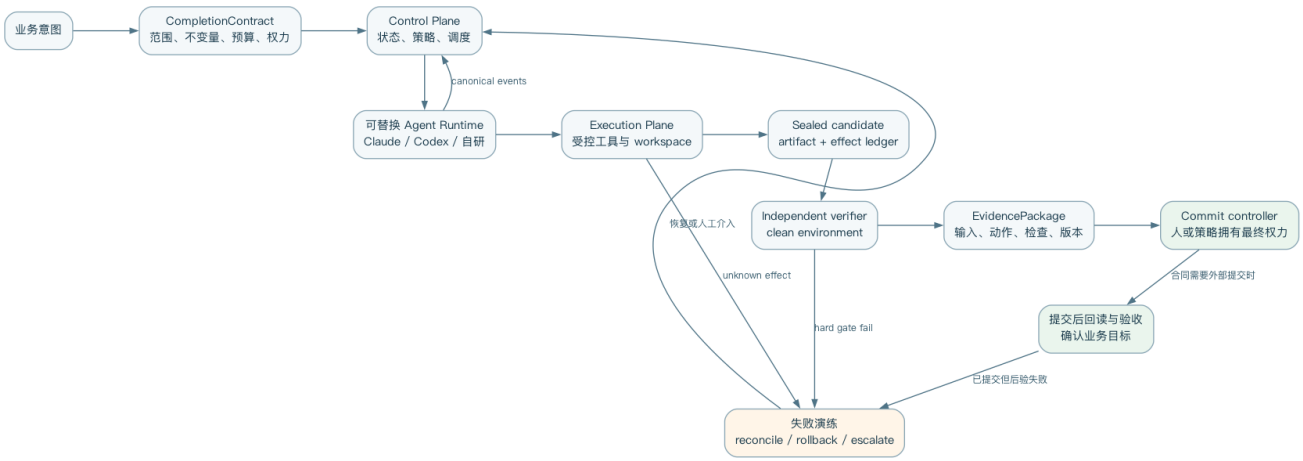


图 26-1 三类案例共享的任务、执行、验证与提交骨架

案例一：仓库级软件修复

1. 任务与合同

场景：支付服务升级日期库后，夏令时边界测试失败。教学口径将扣款窗定义为一个当地民用日对应的半开 UTC 区间；它不等于固定24小时，也不代表所有支付产品的账务规则。Agent 可以修改 src/time/ 与对应测试，不允许改账务规则、删除或削弱测试、联网发布。本任务只交付可审阅补丁及证据；创建 PR 和合并另由代码负责人授权。

以下 JSON 是本书自定义的业务合同输入，不是厂商配置，也不能直接交给附录 E 的 Task 验证器。<resolved-full-oid> 等尖括号值是必须替换的占位符。第二十八章给出合同到平台对象的字段映射。

JSON

```
{
  "contract_id": "CC-REPO-2048-v3",
  "task": "修复 DST 边界下的重复扣款时间窗计算",
  "workspace": {"repo": "payments", "commit": "<resolved-full-oid>"},
  "allowed_writes": ["src/time/**", "tests/time/**"],
  "forbidden": ["delete_or_weaken_tests", "change_ledger_rules", "push", "deploy"],
  "deliverables": ["git_patch", "change_explanation", "verification_results"],
```

```
  "checks": [
    "tests/time/test_dst.py::test_fall_back_window",
    "tests/time/test_dst.py::test_spring_forward_window",
    "pytest tests/time",
    "lint",
    "no_pass_to_pass_regression"
  ],
  "budget": {"wall_seconds": 1800, "model_usd": 8, "max_actions": 120},
  "commit_authority": "human_code_owner"
}
```

控制面验证合同与调用者权限，解析并保存完整基线提交，创建专用工作区，再签发限于任务目录和动作的短期授权。运行时适配层可连接不同 Agent，但必须先协商能力，再收集统一的 Action/Observation，并保留原始事件引用；这里没有承诺某个产品已经实现了全部映射。

2. 执行与验证序列

TEXT

```
User → Control: submit contract
Control → Workspace: create workspace@resolved_base
Control → Runtime: start(task, lease, budget)
Runtime ↔ Workspace: search/edit/test
Runtime → Control: candidate patch + self-report
Control → Verifier: clean base + sealed patch + candidate tree hash
Verifier → Control: checks + hashes + logs
Control → Reviewer: diff + contract + evidence
Control → User: accepted patch + evidence (task endpoint)
Reviewer → Git host: create PR only under separate authorization
```

Agent 在当前工作区执行测试只能提供修复反馈。普通 git diff 不包含未跟踪文件，而且封存后继续改工作区，会使“测试通过的代码”与“交付的补丁”分离。控制面必须把以下步骤作为一次候选协议执行，任何一步失败都停止；这段伪代码描述生产实现的必要步骤，公共入口只验证其中的本地机制。

TEXT

```
freeze candidate workspace; stop agent writes
resolve and verify base revision; record repository identity
inventory modified/deleted/new files, including untracked files
reject out-of-scope paths, secrets, unsafe links and unsupported file types
create private index from base; stage reviewed changes and new files
record candidate tree, file modes and content hashes
export binary patch and changed-file manifest from the same private index
seal patch, manifest, base and candidate tree; record their hashes
create clean verification workspace from the exact base
verify sealed hashes; check patch applicability; apply sealed patch
assert applied tree equals sealed candidate tree; reject extra source files
mount frozen acceptance suite read-only; remove agent write access
run fixed checks against this tree; record every exit code and skipped check
recheck source tree and sealed hashes after checks
bind results to contract, candidate tree, suite and environment versions
```

私有索引先装载基线，再收集审核过的修改、删除、重命名、新文件、模式与二进制变化；不能仅复制 Agent 当前的暂存区。未跟踪文件必须进入清单，忽略文件也要核对是否属于必要输入，不能自动打包凭证或缓存。若由 shell 编排，每个子命令都要检查退出码；使用管道时启用 set -euo pipefail，防止 git 失败被后面的 sort 掩盖。

基线已通过的回归检查和领域验收集由验证方固定，候选新增测试只作补充。候选不得通过删测试、跳过测试或削弱断言获得通过。验证日志须记录实际执行数、跳过数和失败数；必需检查未执行时不能写 PASS。运行环境、依赖锁、

验证器版本、补丁摘要与应用后树摘要共同绑定同一候选。审阅者接收合同、补丁和这些证据，不需要继承 Agent 对结果的判断。

3. 证据包

下面沿用附录 E 的证据包骨架。所有尖括号值都须由一次实际运行填入，不能因为 YAML 能解析就当作证据已经存在。原稿的“482项回归通过”是示意数，已撤下；本例既不声称存在这些生产测试，也不把 AST 语法检查称为 lint 或秘密扫描。

YAML

```yaml
evidence_package:
  schema_version: evidence-package/v1
  package_id: EP-REPO-2048-A3
  task:
    task_id: TASK-REPO-2048
    contract_version: CC-REPO-2048-v3
  attempt:
    attempt_id: ATT-REPO-2048-A3
    runtime: local-fixture
    runtime_version: "<runner-version>"
    harness_profile: code-example-v1
  inputs:
    - uri: "git:<resolved-full-oid>"
      hash: "sha256:<input-manifest-digest>"
  candidate:
    uri: "artifact:<sealed-patch>"
    hash: "sha256:<patch-digest>"
  effects: []
  policy_decisions:
    artifact_ref: "artifact:<policy-log>"
  verification:
    verifier_version: "<frozen-suite-version>"
    environment_ref: "artifact:<environment-manifest>"
    status: INCONCLUSIVE
    checks: []
  approvals: []
  final_commit: null
  lineage:
    parent_attempt: null
    model_version: not-used-local-fixture
    harness_bundle: "<example-bundle-version>"
```

这是尚待填充的模板，因此状态是 INCONCLUSIVE。实际检查后填入每项结果与日志引用；输入清单另存基线 Git 对象ID，候选清单另存候选树、文件模式和文件摘要，不能把 Git tree ID 冒充 SHA-256。task_id 标识任务，attempt_id 标识该任务的一次执行尝试，action_id 标识其中的逻辑动作；同一工具动作的网络重试记为 ToolTry，沿用动作身份和逻辑幂等键。任务终点是验收并交付补丁，final_commit 可以为空；若合同改成创建 PR，则还必须记录远端 PR 的权威回读，不能只凭本地检查宣布完成。

4. 失败演练：只适配可见测试

注入故障：候选只对两条可见测试的日期写特例，在相邻年份、其他时区或半小时夏令时下失败。验证器据固定合同返回失败；检查发现实现违约，并不自动意味着规范有缺陷。开发验收可以按既定规则给出“不应硬编码日期”的修复反馈，并限定修复次数。用于最终确认的封存测试不向执行者返回可定位样本；若其反馈已经用于修改候选，该批数据就参与了开发或选择，不能继续宣称未见。

还要做两个与模型无关的负例：新增必要模块却漏进补丁，干净应用后必须失败；封存候选A后在工作区改成候选B，即使B通过，也不能把结果记给A。独立测试目录只是逻辑分离，同一操作系统用户并不构成安全隔离。生产验收还需落实进程身份和读写权限。

案例二：企业经营分析

1. 任务与口径

场景：生成2026年7月中国区订阅净收入变化分析。任务合同固定指标定义、数据快照、币种、允许维度和交付格式。以下仍是业务合同输入；snapshot 引用的是已物化且受保留策略保护的数据版本，不是到任意未来时间仍可使用的历史查询时间戳。

JSON

```
{
  "contract_id": "CC-DATA-771-v5",
  "metric": "net_subscription_revenue_v4",
  "period": ["2026-07-01", "2026-08-01"],
  "comparison": "previous_month",
  "currency": "CNY_at_monthly_finance_rate",
  "snapshot": "materialized:subscription_revenue_v4_20260803T020000Z",
  "allowed_dimensions": ["province", "plan", "channel"],
  "prohibited_fields": ["email", "phone", "account_name", "raw_payment_token"],
  "deliverables": ["analysis.md", "aggregates.parquet", "query_bundle", "evidence.yaml"],
  "checks": ["metric_definition", "snapshot_consistency", "internal_total_reconciliation", "
min_group_size_20", "disclosure_review"],
  "post_commit_checks": ["published_report_hash", "reader_access_policy"],
  "commit_authority": "finance_analytics_owner"
}
```

规划者可以拆分取数、对账、解释和反证。取数工具使用只读、短期、绑定快照的凭证，网关执行行列权限检查，模型只接收获准披露的聚合结果。内部全量结果由有权限的验证器保管，不能因为也是“聚合”就自动交给模型。

下面采用 BigQuery Standard SQL 方言。原稿的 FOR SYSTEM_TIME AS OF 受历史窗口限制：时间点不能早于当前超过7天，实际配置和表条件还可能更严格。2026年8月3日的数据到9月19日已不能靠该窗口找回，长期复现必须在有效期内物化或归档，再记录来源时间、schema、口径版本和内容摘要；没有保存就应报告无法复现，不能把当前数据改名当旧快照。该限制来自本轮已核验的官方查询语法，本书没有实际执行 BigQuery 账户查询。

此处假定物化快照已经过批准的语义层处理：同一业务事件在该截面只保留一个有效版本；month 是按业务时区归一化的月初 DATE，而非交易日；金额是按合同汇率换算的精确数值；NULL退款仅在业务确认“无退款”时转为0，NULL收入与无法识别账户的记录隔离待处理。重复、排除与隔离的数量及金额进入受限质量报告，并与财务采用相同口径对账；质量异常不能靠悄悄删行消失。COUNT(DISTINCT account_id) 只去重人数，不会去重收入。

SQL

```
SELECT month, province, plan,
       SUM(recognized_revenue_cny - refunds_cny) AS net_revenue_cny,
       COUNT(DISTINCT account_id) AS accounts
FROM `finance_snapshots.subscription_revenue_v4_20260803T020000Z`
WHERE region = 'CN'
  AND month IN (DATE '2026-06-01', DATE '2026-07-01')
GROUP BY month, province, plan;
```

2. 双重验证

上面的查询生成内部全量聚合，尚不能公开。确定性验证器先核对语义层与快照，再把未抑制汇总和相同口径的财务总额对账。随后由独立披露步骤处理人数不足20的组，并审查总额、分组、跨月份和其他可访问查询之间的组合泄漏。最低人数检查命名为 min_group_size_20，它只是抑制规则，不是 k 匿名或其他完整隐私保证的证明。

例如，纯合成数据中A组20人、收入100，B组19人、收入95，内部对账应使用195，公开候选只剩A组100。不能再要求100与195在舍入误差内相等，也不能同时公开总额195和“受抑制残差95”，否则B组被反推出。必要时连总额或另一个大组也要互补抑制，或采用经批准的其他披露机制。真实小组残差只保存在受限证据中；给模型的日志和报错同样不得携带它。

文字审阅在披露通过后进行，检查数字能否链接到获准披露的单元、是否把相关写成因果，以及跨月份的可见范围是否改变。被抑制值不能当作零；若覆盖变化本身敏感，报告只能说明可比性受限，不能给出足以反推小组的桥接表。

TEXT

```
metric contract → approved materialized snapshot → internal full aggregation
  → restricted financial reconciliation → disclosure and composition review
  → approved aggregates → narrative + claim-to-cell review
  → analyst approval → publish → confirm report and access policy
```

对账误差由货币精度和已知舍入规则决定，不用统一百分比掩盖口径问题。分别记录快照不一致、未链接的数字、披露违规、产物取回与检查重跑的结果。公共入口以 SQLite fixture 验证聚合逻辑，方言适配结果不能冒充上述 BigQuery 查询、真实汇率和财务验收已经通过。

3. 证据包

沿用案例一的 evidence-package/v1 骨架，输入指向物化快照、语义层定义和查询版本；候选指向报告及获准披露的聚合清单。完整证据包限授权审计者读取，内部对账、被排除记录和抑制残差另设受限引用；发布给报告读者的是经过披露审查的证据投影，不能附带能解引用原始敏感数值的链接。

原稿的对账差额0.02元、37个数字链接均为示意数，不是财务实测。实际运行应记录未抑制总额的核验结果、披露策略版本和每个公开数字的来源，不能用一组预填 PASS 代替检查。本合同要求发布：适配器将 checks 编译为完整合同的 pre_commit_checks，提交确认后再运行 post_commit_checks，核对已发布内容摘要、目标和访问权限。副作用确认只进入 VERIFYING_POSTCONDITIONS；后置检查失败或仍未知时，保留已发布事实并记为 COMMITTED_BUT_UNVERIFIED。

4. 失败演练：快照漂移

注入故障：第一次查询后，上游退款表完成迟到回填；第二条查询若使用“latest”，会把两个快照混在一份报告里。网关应在执行前拒绝不匹配的数据版本。系统可以继续使用尚可读取的原物化快照并说明时效，也可以由数据负责人批准合同修订后从头重建全部产物。原快照若未保存或已过保留期，应停止并报告不可复现。

选择新快照时，旧证据包标为已被替代，不覆盖原记录；所有数字、对账、披露审查和叙述重新生成。数据截面的改变需要数据负责人确认，因为它改变了问题本身。

案例三：自我进化 Harness

1. 失败归因与实验合同

场景：平台观测到接入多个MCP服务后，wrong_tool 错误上升。运营者先按模型、任务族、工具数量和错误类别聚类，提出“同时呈现的工具定义过多，干扰了选择”的假设，再设计实验；错误同时上升本身不能证明因果，也不能直接授权生产Agent改写工具目录。下面是本书自定义的实验合同示意。

YAML

```
evolution_contract:
  id: EVO-TOOL-93-v2
  baseline: harness-42
  mutable_surface: tool_catalog_presentation
  frozen:
    - model
    - task_suite
    - sandbox_image
    - policy_root
    - evaluator
    - sealed_test
  candidates:
    - grouped_dynamic_discovery
    - concise_descriptions
    - task_scoped_allowlist
  primary_metric: verified_task_success
  diagnostics: [wrong_tool_rate, catalog_lookup_activation, tokens, latency]
  hard_gates: [security_non_regression, critical_slice_non_regression]
  release: shadow_then_low_risk_canary
```

候选生成者在隔离分支工作，不能修改评价器、任务分配表或发布权限。构建预检只确认代码有效，以及机制能在专用探针中触发。正式试验中，是否激活是诊断字段：所有预先分配的合格 trial 都进入主要指标分母，包括未激活、超时、无返回和基础设施失败。trial是实验分配单位，映射到任务及其执行尝试；独立控制面可按统一预算重试，但不能因此增加样本数。同一Action的ToolTry与重新发起一次Attempt分别记账，原失败、最终结果和全部成本都保留。

2. 选择与发布

TEXT

```
production traces (read-only)
  → pathology cluster + human-confirmed hypothesis
  → isolated candidate generation
  → preflight(build valid + mechanism probe)
  → development and candidate-selection evaluation
  → freeze bundle → one authorized sealed-test evaluation
  → shadow → canary → promote/rollback
```

先检查安全与关键切片门，再估计候选相对基线的配对差值。实验合同预先给出任务数、每任务重复数、最小有意义增益、非劣界、区间方法、候选选择和停止规则；重复任务按任务或同源簇处理，不能把每次调用都当独立样本。未发现显著下降不等于非劣，零观测违规不等于真实风险为零。单位成功成本包含生成、评估、重试和人工处置；没有可信完成时不报告有限的单位成功成本。

开发、选型与最终封存数据分开，记录跨轮访问次数及反馈去向。封存结果若用于修复、经验库或训练，该批数据必须退出“未见终测”的角色。发布控制器发布模型、Harness、记忆、策略和评价器引用组成的完整版本组合；正在执行的任务保持已选版本，但每次动作仍重验当前授权。紧急撤权优先于版本粘性。

3. 证据包与来源关系

本案例也使用 evidence-package/v1。输入引用实验协议和预分配 trial ID 清单，候选引用不可变版本组合，验证结果引用逐条 trial、激活事件、重试账本与成本，发布与回退决定记录在外部效果和批准字段中。实验的 evolution_contract 是领域对象，需要适配器编译；它本身不是 Task 或证据包。

原稿的240次、228次激活和5%灰度均为示意数，没有对应生产试验。为了说明分母错误，下面使用明确构造的教学数据；它只演示 gate 的判定，不能计算出模型收益结论。

教学候选	预分配 trial	可信成功	其他终态	以完成返回者为分母的误报	固定分母结果
基线	30	24	6次失败	24/30＝80%	24/30＝80%
超时候选	30	18	2次失败、10次重试后超时	18/20＝90%	18/30＝60%
合成候选	30	28	2次失败	28/30	28/30
合成灰度	30	29	1次关键切片失败	29/30	即使总分提高，仍拒绝放行

每行都必须能用30个预分配ID对齐，最终成功、失败、超时、取消等互斥终态之和等于30。激活与未激活另外统计，二者之和也等于30，它们不是额外的结果类别。删除日志会先触发完整性拒绝，不能等到缺失率“显著”才阻断。公共入口按自身 fixture 输出实际分母与判决；任何脚本预设的成功数都只能验证计分和发布逻辑。

4. 失败演练：通过修改分母“进步”

超时候选把10次困难试验藏掉，就会从60%伪装成90%。完整性门检查预分配ID、最终结果和成本是否齐全；计分门坚持30的分母。即使日志全部存在、没有篡改，错误地只统计返回者也必须拒绝。

另一个停止点是灰度中的关键切片退化。控制器停止新的候选流量，并恢复经批准的完整旧版本组合；已经提交的业务效果另行对账，版本回退不会抹掉它们。若旧组合含已撤销凭证或有害能力，不能机械恢复旧授权，应停用并换成当前允许的配置。是否重新设计候选由负责人决定，执行 Agent 没有自行晋级权。

三个案例的共用骨架

TEXT

```
intent → versioned contract → identity/workspace → runtime
→ observable actions/effects → sealed candidate
→ candidate verification → current authorization → commit if required
→ authoritative readback + contractual postconditions
→ evidence + telemetry → eval/evolution
```

代码案例验收的是固定基线上的同一份封存补丁，数据案例还要保证同一快照下的口径和披露边界，进化案例则冻结实验协议并保留全部试验。仅交付产物的合同可在验收与交付后结束；包含发布的合同要继续到权威回读与业务后置条件通过。公共教学程序帮助检查这些局部关系，生产系统仍需另行验证真实权限、外部提交和恢复能力。

第二十七章 下一代企业 Harness 参考架构

> 证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

企业接入一个 Agent 后，往往先遇到两个问题：谁有权让它操作业务系统，以及怎样判断它真的完成了任务。本章用六层结构安排这些责任，便于接入不同供应商或自研运行时。是否需要独立部署每一层，取决于风险、规模和现有平台能力。

1. 六层结构与权威状态

TEXT

```
Experience       IDE / Web / CLI / API / business workflow
Control Plane    task contract / scheduler / identity / policy / approval
Agent Runtime    vendor adapter / loop / context / delegation
Execution Plane  workspace / sandbox / tool gateway / credential broker
Evidence Plane   artifacts / trace / verifier / effect ledger
Evolution Plane  eval registry / mutation / experiment / release / rollback
```

体验层收集意图，控制层持久化任务合同和权威状态。Agent运行层提出动作，执行层在授权范围内操作文件与外部系统；证据层保存产物、执行验证和对账；进化层提出并评估未来版本。进化层内部的提案者与发布者必须分权，不能因为画在同一行就共享权限。

第五章的三平面按管理与数据路径划分，本章六层按功能划分，第二十五章的候选域与治理域则按信任边界划分。三种视角的对应关系如下。

六层中的组件	三平面中的主要位置	信任与读写边界
体验入口、Agent运行层	数据面	提交请求和候选；不直接修改权威任务状态或授予权限
合同、调度、身份与策略	控制面；决策结果进入数据路径	治理主体持有合同、预算和授权状态
沙箱、工具网关、凭证代理	执行面	执行主体只获得当前动作所需权限
事件与产物存储、独立验证器	数据面承载证据；执行面提供隔离验证环境	执行者可按协议追加原始证据，不能覆盖；独立验证主体写判决
候选生成 worker	控制面发起实验，数据面和执行面运行候选	候选域只写提案，不能读封存测试或改裁判
实验服务、评测登记与发布控制器	控制面	治理域读取受控证据、批准发布，执行者不能自行晋级

候选域向治理域提交 proposal，执行层向证据层追加 raw evidence；发布批准只由治理域产生，执行环境只接受经验证的版本清单。高风险场景应通过独立身份、只读挂载、凭证与网络边界落实这些单向关系，不能只靠服务名。低风险小系统可以共用进程，但仍需明确哪个组件有权改合同、评价结果和发布状态。

另一个容易混淆的词是 runtime。Agent Runtime 指运行模型循环、上下文和委派的组件；Execution Runtime 指命令、浏览器或工具实际运行的环境。每个适配器都要声明接哪一端；把工具执行放进企业网络，并不能由此推出推理、工具输出或会话存储也留在企业网络。

采购或自建时，可用下面的责任表逐版本核实。这是待填写的设计记录，不是对任何厂商已验证能力的声明。

责任	需要明确的运行位置与权威主体	验收证据
模型循环与编排	企业进程或托管服务，谁能中断与恢复	能力协商、取消和恢复记录
会话持久化	内容、地域、保留期和导出责任	导出样本、删除与访问控制记录
凭证保管	长期凭证保管方、短期授权签发方	发放、到期和紧急撤销事件
工具执行	worker、沙箱及出网路径，输出回传位置	外部动作日志和数据流核对
业务验收与发布	领域负责人、独立验证器、提交主体	合同、回读、后置检查和批准记录

2. 统一契约与能力协商

平台按需要定义 Task、Attempt、Action、Observation、Artifact、PolicyDecision、Checkpoint、Delegation、VerificationResult 和 EvidencePackage。沿用第六章：Task是任务，Attempt是任务执行尝试，Action是一次逻辑动作，ToolTry是该工具动作的单次尝试；重试不应改变同一逻辑动作的幂等身份。适配器负责统一对象与产品协议的映射，保存原始载荷的摘要、位置与协议版本。下面是本书自定义接口草图，不能作为某个厂商 SDK 的原生方法表；字段转换示例见第二十八章。

TEXT

```
RuntimeAdapter {
  negotiate(capability_requirements) -> CapabilitySet
  start(task, workspace, policy_profile) -> Attempt
  stream_events(attempt_id, after_offset)
  respond_to_request(request_id, approval_or_input)
  checkpoint(attempt_id)
  cancel(attempt_id, reason)
  collect_artifacts(attempt_id)
}
```

适配只需要动作、结果、批准、产物和生命周期，不要求供应商暴露私有推理。缺少恢复或结构化补丁能力时，协商结果必须明确返回缺失；调度器据此降低自治范围、选择替代实现或转人工。能够导出对话不等于能够恢复外部动作。

3. 身份、租户与凭证

用户、平台、运行时、子代理、工具与外部服务具有可区分的身份。能力租约绑定租户、资源、动作、用途和有效期，凭证代理只在执行时向受控工具注入短期凭证，模型上下文与长期日志不保存密钥。转授父主体权限时，不得超过其可转授范围，并按子任务收窄。专家使用独立执行身份时，由可信策略服务另行签发，同时核对调用者的委派权、用途和结果信息流；不因此赋予父模型读取专家全部数据的权限。缓存、记忆和工具结果进入上下文前完成数据分类与租户检查。

短有效期不能代替撤销检查。每次动作，尤其提交前，都要重验当前授权、租约、撤销版本和取消状态；无法确定权限有效时停止。模型、工具视图和输入版本可以固定以便复现，旧授权不能因版本粘性继续有效。紧急撤权应暂停受影响任务、阻止新副作用并回收能力；已经进入上下文的有害内容需要隔离后重建上下文或创建新 attempt，不能只删除登记表中的条目。

4. 持久执行与副作用

任务与尝试状态持久化，事件按约定排序；外部状态变更记录意图、幂等键、参数摘要、策略决定和结果。崩溃后只能从受支持的检查点恢复，取消和已终止任务不能自动回到运行中。未知结果先对账，查询暂不可见也不能直接推断未提交；安全重试所需的下游幂等保证见第六章。调度器另行管理预算、并发、截止时间和取消树。

例如邮件发送超时，第一封可能已经送达。网关应凭稳定业务标识查询权威记录；若目标系统无法确认，就保持结果未知并升级处理。另发一封“补偿邮件”并不能撤回第一封，登记了补偿函数也不构成安全重试条件。

5. 以证据判定完成

运行时只能提出候选。适配器将Task的 checks 解析为完整合同的 pre_commit_checks，独立验证器针对封存候选执行这些检查，生成绑定产物摘要的结果。若合同仅要求交付待审补丁，验收和交付完成即可结束，不人为增加外部提交；若要求发送、合并或部署，候选通过只是前置门，还要由提交控制器重验当前授权、候选摘要和目标版本，再按幂等协议提交、回读权威结果并执行 post_commit_checks。

“部署接口已确认”不等于“部署后健康检查通过”。状态沿用第十章：effect confirmed → VERIFYING_POSTCONDITIONS；后置检查失败或未知进入 COMMITTED_BUT_UNVERIFIED，保留已发生的副作用，不计为可信完成。回执丢失则先保持结果未知，不能伪造确认或直接重试。审批后候选或目标版本变化，原验

证与批准不能直接复用。

证据包连接合同版本、输入、封存候选、验证器、策略、批准、实际提交目标和回读结果。它用于审计与核验，不需要保存私有推理。取回产物并核验摘要、重跑确定性检查、重放生成过程是三种能力：前两项可有明确记录，第三项仍受模型版本与外部环境可用性限制。

6. 七类 SLI 与 SLO 设定方法

SLI 是测量值，SLO 才是某个窗口内希望达到的目标。下表先定义测量方法，阈值由风险、历史基线和业务负责人确定。每项还要登记窗口、任务资格、观察期、目标值和误差预算，不能仅填一个百分比。

指标	定义与分母	窗口及验收点	可能的指标博弈
可信完成率	成品交付和业务提交两类分别计算：截止内达到各自终点的任务／该类入口登记的全部合格任务	按合同冻结批次统计；交付类验收并交付，提交类还须权威回读和后置检查	降低验收、删除超时或未激活任务
错误完成率	观察期内被证伪的已宣告完成任务／已走满同一观察期的完成任务	按完成批次回标，另报未成熟样本与抽检覆盖；抽样时声明估计方法	隐藏返工、延迟登记事故
证据完整率	必需证据可访问、摘要相符且与合同及提交绑定的完成任务／全部宣告完成任务	完成时检查，并按保留期限抽查可取回性	只填字段，留下失效链接或不相关日志
恢复成功率	在恢复预算内达到合同终点且无重复副作用的事件／全部应恢复中断事件	中断时登记分母；未尝试也保留，并报告原因	只统计已启动恢复的容易事件
最大副作用	故障或失控窗口内，同一任务及其后代可累计影响的金额、对象和资源上界	同报实际暴露次数、观察最大值和配置上界；无事件不表示上界为零	拆动作或子任务规避额度
单位可信完成成本	同一任务批次的模型、计算、工具、验证、重试和人工总成本／可信完成数	固定成本按公开规则摊销，事故费用另列归属；零完成记为不可估计或无穷	只计成功调用，忽略失败与人工处置
完成时延分布	两类任务分别从合同冻结计到各自终点；仅完成者的P95另标为条件统计	同报每类全部合格任务的成功、失败、超时、取消、进行中数量及截止内完成比例	丢弃长尾，把候选通过当业务提交完成

两类分母在入口冻结：N_deliverable 是合格成品交付任务数，N_business_commit 是合格业务提交任务数，二者分别包含各自的成功、失败、超时和取消任务。分子分别为验收并交付的成品，以及提交确认且 post_commit_checks 通过的业务任务；COMMITTED_BUT_UNVERIFIED 留在后者分母中而不进入分子。不能因提交失败把任务改列为交付成功。确有获批合同修订时，保留原批次和修订事件，按预定迁移规则报告；不得覆写历史分类。综合指标可以另报，但须保留两类的数量和结果。

例如，可将一个自然周冻结的合同作为任务批次，并在各自截止期后结算可信完成；错误完成率另用预先选定的30日观察期回标。这里的周与30日仅为设计示例。未到截止期的任务、未走满观察期的完成任务单列，不能暗中移出历史分母；超时、基础设施失败和取消按预先声明的口径保留并分型报告。入口资格不应随候选表现修改，拒绝接纳的数量也要公开给运营负责人。

候选验证通过率可以作为诊断指标，分母是全部提交候选，不能替代任务可信完成率。每个SLO需要负责人、数据来源、告警和例外流程；高风险切片单列，严重违规按事件与暴露量报告，不能被总体平均分抵消。

7. 多运行时数据流

TEXT

```
request → contract compiler → scheduler → runtime adapter
   → policy-mediated tool gateway → sandbox/external systems
   → event + effect ledger → candidate seal
   → candidate verifier → current authorization → commit if required
   → authoritative readback + postconditions → EvidencePackage
   → telemetry/eval → governed evolution release
```

最容易遗漏的是适配器之外的旁路：运行时直接出网、插件自行持有密钥、界面直接调用供应商接口，以及宿主执行不经过模型的生命周期 hook。应核对实际网络流、执行身份、凭证发放与外部审计日志，不能只凭架构图认定所有

动作受控。

8. 构建顺序与替代方案

先选一个任务族，明确合同、权限、产物、验证和业务终点，再决定是否接第二种运行时。只有一个低风险 Agent 时，可采购沙箱与日志服务，保留任务和证据的导出能力；进入高风险域后再按隔离需求拆分部署。若流程稳定且便宜代码已能完成，确定性工作流也是有效选择，没有必要为采用此架构而扩大模型权限。

接入能力是否足够，应由取消、撤权、恢复、候选重验和退出演练证明。图中的六层是核对责任的工具，不是采购六套系统的清单。

第二十八章 Agent SDD：规范驱动的任务与发布

> 证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

Agent SDD（Specification-Driven Delivery，规范驱动交付）把影响结果与权限的意图转换成可执行、可版本化的合同。低风险探索可以逐步明确要求；涉及不可逆或高价值动作时，必须先固定关键不变量。文档长度不是目标，执行者、验证者和批准者对“可以做什么、怎样算完成”有一致理解才是。

1. 六类规范

规范	回答的问题	推荐权威载体
业务规范	为什么做、价值是什么	product/业务系统
任务规范	交付物、不变量、截止与预算	CompletionContract
工具规范	可执行动作与错误语义	schema + effect contract
策略规范	谁在何种条件下能做什么	policy-as-code
验证规范	什么证据足以证明完成	verifier suite
发布规范	谁能让候选产生外部效果	release/commit policy

自然语言可以是入口，但金额、资源范围、数据快照、禁止动作、验收项与提交权限需要结构化，否则模型、审阅者和审计者可能分别解释同一句话。

2. 从意图到合同

TEXT

```
intent → ambiguity/materiality detection → contract draft
→ authority confirmation → executable checks → run
```

Agent 可以查明当前提交、已有测试和数据结构，把会显著改变结果或权限的歧义交给目标负责人。合同编译器应区分信息缺失、要求冲突和有意开放的选择；实现形态可以开放，预算和评价标准仍需明确。

例如“清理老客户”并未说明多久算老、采用归档还是删除、哪些记录必须保留，以及谁批准。系统应先生成影响分析，在这些条件明确前不能自行选择90天并删除账号。

3. 教学实例：从规范到任务再到验收

假设仓库 billing-api 要修复“取消订阅后仍发送续费提醒”的缺陷。业务负责人规定：在入队提交时已经取消的订阅不得入队，账单记录保持不变，历史已发送消息不追溯删除，只允许修改通知筛选与对应测试。若筛选后发生取消，入队服务需用同一事务或版本条件确认当前状态；只在初次筛选时读取一次状态不能满足这份合同。

本书没有该生产仓库。下面的 YAML、make typecheck 和 pytest tests/reminders 都是自定义工程契约示意，须由真实项目提供实现，不能在本书仓库直接运行并宣称业务通过。原7位提交 8f2c9d1 是教学占位；实际输入必须解析为本次验证可读取的完整版本。

YAML

```yaml
specification:
  id: SPEC-BILLING-214
  owner: subscription-product
  snapshot: "git:<resolved-full-oid>"
  invariant:
    - cancelled_at_enqueue_commit_never_enqueued
    - invoice_state_unchanged
  intentionally_open:
    - implementation_shape
```

```
task:
  task_id: TASK-BILLING-214-01
  contract_version: v1
  deliverables:
    - sealed_patch_against_input_revision
    - evidence_package
  allowed_writes:
    - src/reminders/**
    - tests/reminders/**
  forbidden_actions:
    - production_database_write
    - production_message_send
    - billing_state_change
    - delete_or_weaken_tests
  budget:
    wall_seconds: 1800
    max_actions: 80
  commit_authority: billing-code-owner
```

3.1 合同怎样映射到平台对象

上面的 specification 与 task 是业务输入包，不直接符合附录 E 的 Task schema，也不是供应商配置。适配器先验证业务输入，再编译平台 Task、权限策略和完成合同，保留源字段到目标字段的映射。不能为了通过 Task 的 additionalProperties: false 而悄悄丢弃写入范围。

业务输入	平台映射与责任
specification.id、owner	进入有版本的规范对象；Task 的 input_refs 引用它，负责人身份由控制面核实
task.task_id、合同版本	映射到 Task 的 task_id、contract_version；第二十六章的 contract_id 由登记表解析成不可变合同版本引用
租户、风险	从已认证调用者和风险策略取得 Task 的 tenant、risk，不由模型自行填高权限值
snapshot 或第二十六章的 workspace.commit	解析完整输入版本，写入 input_refs；证据包 inputs 另绑定清单及内容摘要
invariant、交付物、禁止动作	分别编译为 invariants、deliverables、forbidden_actions，并生成可执行策略
allowed_writes	写入绑定合同版本的路径策略，Task引用该策略；租约与 scope_guard 同时执行
验收表	Task 的 checks 使用稳定的候选检查ID，适配器解析为完整CompletionContract 的 pre_commit_checks；命令与验证器版本在只读套件登记。业务提交另有 post_commit_checks，附录E的最小Task未展开此字段，不能直接追加到根对象
budget	使用 wall_seconds、model_usd、max_actions；旧 wall_minutes 乘60。旧工具调用上限不直接等同所有动作上限，须明确计数语义后转换
commit_authority	映射到提交责任主体，实际动作仍校验当前授权；字段值本身不是批准
运行与候选	控制面生成任务执行尝试的 attempt_id，每个逻辑动作有 action_id，工具重试另记ToolTry；证据包记录运行时版本、候选URI/摘要、验证器与环境、批准和最终提交

以下是编译后的教学 Task；所有输入引用仍是占位。适配器通过 schema 登记表选择 task/v1 验证，证据包使用 evidence-package/v1。这两个版本标识属于不同对象，不能把证据包字段直接加进不允许扩展属性的 Task 根对象。

JSON

```
{
  "task_id": "TASK-BILLING-214-01",
  "tenant": "teaching-tenant",
  "contract_version": "CC-BILLING-214-v1",
  "risk": "R2",
  "input_refs": [
    "spec:SPEC-BILLING-214-v1",
```

```
    "git:<resolved-full-oid>",
    "policy:reminder-write-scope-v1",
    "suite:reminder-contract-v4"
  ],
  "deliverables": ["sealed_patch", "evidence_package"],
  "invariants": ["cancelled_at_enqueue_commit_never_enqueued", "invoice_state_unchanged"],
  "forbidden_actions": ["production_database_write", "production_message_send", "billing_sta
te_change", "delete_or_weaken_tests"],
  "checks": ["compile", "reminder_regression", "sealed_cancelled_slice", "invoice_integrity"
, "scope_guard", "immutable_suite_guard"],
  "budget": {"wall_seconds": 1800, "max_actions": 80},
  "commit_authority": "billing-code-owner"
}
```

租户、风险和ID均为教学值。输入包、Task、候选与证据包须沿同一个 task_id、合同版本和输入版本连通；附录 E 的结构校验只检查形状，引用是否存在、权限是否有效和证据是否支持完成仍需独立检查。

3.2 固定验收与数据库权限

验证器在临时测试数据库中准备 active、past_due、cancelled 等固定样本，并测试“筛选后取消、入队前重验”的时序。执行 Agent 无生产写权限，不意味着验证器不能创建临时 fixture。候选测试只能访问隔离数据库和假消息出口，绝不能获得生产凭证。

账单完整性以调用提醒逻辑之前和之后的同一张 fixture 表比较：按固定主键排序，对合同指定的全部账单业务字段作规范序列化再求摘要，包括金额、状态和有业务意义的时间字段。只能排除预先声明的非业务元数据，不能临时忽略被候选修改的列。检查在 fixture 准备完成后取基线，执行候选后取终值，并同时检查表行数和键集合。

候选按第二十六章协议封存，在干净环境应用同一补丁后运行下列验收。公开测试可给开发反馈，封存验收由独立主体控制；删除、跳过或削弱公开测试也不能替代固定验收集。

YAML

```yaml
acceptance:
  - id: compile
    command: make typecheck
    required: true
  - id: reminder_regression
    command: pytest tests/reminders
    required: true
  - id: sealed_cancelled_slice
    verifier: reminder-contract-v4
    expect: enqueued_count == 0
    required: true
  - id: invoice_integrity
    verifier: table-hash-compare
    expect: before_hash == after_hash
    required: true
  - id: scope_guard
    verifier: changed-path-policy
    expect: changed_paths subset_of allowed_writes
    required: true
  - id: immutable_suite_guard
    verifier: frozen-suite-manifest
    expect: required_checks_unchanged_and_executed_without_suppression
    required: true
```

3.3 四个独立负例

从一个仅修复筛选且保留验收集的通过候选出发，每次只注入一种变化。下表是应预注册的预期结果，不宣称本书已经运行原生产项目。每列测量不同不变量，不能互相替代。

注入变化	scope_guard	invoice_integrity	sealed_cancelled_slice	immutable_suite_guard
无注入：授权目录内修复筛选，账单与测试不变	PASS	PASS	PASS	PASS
仅越界：另在 src/billing/state.py 加无行为影响的修改	FAIL	PASS	PASS	PASS
仅改账单：在允许的提醒代码中更新 fixture 账单，取消过滤仍正确	PASS	FAIL	PASS	PASS
仅保留错误入队：取消订阅仍进入提醒队列，账单不变	PASS	PASS	FAIL	PASS
仅抑制测试：业务修复正确，但删除、跳过或削弱必需测试（suppressed tests）	PASS	PASS	PASS	FAIL

这些PASS是各隔离探针在其前提下的预期，并非发现一项FAIL后还必须执行危险代码。实际放行采用任一必需检查失败即阻断；未执行的检查明确标为未运行，不能填PASS。测试抑制负例还要求独立套件保持完整，候选工作区报告的“全绿”无权缩小验证分母。

原稿把“取消状态改回active”写成scope与invoice两项必失败，混淆了代码路径和持久化数据。越界编辑会使scope检查失败，但只改变状态解释、不写账单表时，invoice检查可以通过；提醒反而可能继续入队。上述四个负例分别构造路径越界、账单变化、行为错误和测试抑制，避免靠一个含混场景推断多个失败。

若真实过滤逻辑在未授权的 src/queue/subscription_filter.py，执行者应提交合同修订提案，说明路径、理由和不变的验收要求，由代码负责人批准新版本。通过检查后，证据包绑定规范、合同、输入、候选摘要和验证器；本例任务只交付补丁，合并另需提交授权。若合同本身要求合并，还要回读远端合并结果并完成所约定的后置检查。

4. 运行中的合同修订

执行中发现依赖与合同不兼容等新事实，可以提交修订提案。提案记录差异、理由、影响、已经发生的外部变更和所需批准；执行 Agent 不能自行扩大写入范围、降低验收或更换数据快照。批准后产生新合同版本，旧尝试继续绑定旧版，其迟到结果不自动获得新授权；后续执行从新版本开始，并对已发生副作用对账。

JSON

```
{
  "amendment": "AM-CC2048-02",
  "from": "CC-REPO-2048-v3",
  "change": {"allowed_writes_add": ["src/compat/date_adapter.py"]},
  "reason": "existing API compatibility layer is authoritative",
  "impact": "adds one production file; verification suite unchanged",
  "required_authority": "code_owner"
}
```

5. 规范怎样进入执行环境

规范应贴近权威状态：代码规则进入仓库，数据口径进入语义层，接口约束进入数据结构，安全要求进入策略引擎。上下文编译器给模型当前规范的必要部分，并保留来源与版本；只写在系统提示词中的要求难以独立执行和验证。

重复、可确定且错误代价高的要求适合编译为检查或策略。如果全部步骤已能稳定编码，直接采用确定性工作流通常更容易验收；真正需要情境判断的部分再留给模型与人。

6. 发布门与豁免

候选产物、合同版本和验证结果一同进入发布门；提交前再次核对授权、产物与目标版本。业务确需带已知失败上线时，豁免记录失败检查、风险负责人、补救措施、影响范围和到期时间；不可豁免的硬约束仍然阻断。Agent 可以起草说明，不能自行批准，也不能用豁免覆盖已经撤销的权限。

模型停止、候选 checks 通过、外部提交确认和 post_commit_checks 通过是不同事件。提交确认后进入 VERIFYING_POSTCONDITIONS，后置检查失败或未知为 COMMITTED_BUT_UNVERIFIED。纯成品交付合同在验收并交付后结束，不强造外部提交。第二十七章据此分别记录两类任务的完成率分母和时延。

7. 规范质量指标

可观察运行中的重要合同修订率、验收项未执行比例、完成后发现的不变量缺口、豁免逾期率，以及合同字段到证据的覆盖。每项说明统计窗口和分母；修订少未必表示入口清楚，也可能是团队在聊天中改了目标却未登记。

探索性任务可以采用研究简报与人工评审，将完成条件定义为资料与分析交付，并明确结论尚未验证。规范驱动交付的适用前提，是组织能说明谁拥有目标、允许哪些动作，以及什么证据足以接受结果。

第二十九章 成熟度模型与 Build-vs-Buy

> 证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

成熟度模型用来发现某类任务尚缺哪些控制，不是供应商排名或组织认证。只读助手、受限批量操作和生产部署承担的风险不同，不能按接了多少工具、支持多少运行时或是否会自我进化排成一条高低链。生产动作缺少有效验收需要补上，但低风险系统采用轻量人工核验，也可能已经满足其责任。

1. 五类能力阶段

阶段	需要拿出证据的能力	适用条件与主要风险
M0 对话增强	回答用途、来源与人工使用边界清楚	适用于无自动提交权的辅助工作，须防止把回答当作已执行结果
M1 受控执行	工作区、身份、权限、日志、取消和提交授权可核验	工具开始改变状态后，重点是约束影响范围
M2 可验证任务	合同、封存产物、独立验收与副作用对账连通	验收应与风险匹配，注意误判和未覆盖切片
M3 平台化运营	持久执行、策略、证据、变更与退出可运营	按任务需要支持恢复与隔离；多运行时只是检验替换能力的一种方法
M4 受控进化	候选生成、隔离评价、批准、灰度与回退可验证	可选扩展，须控制选择偏差、裁判污染和权限扩大

本章用M0—M4标记成熟度能力，避免与第二十章的L1—L4进化对象混淆。这些阶段有控制依赖，但不要求每个系统走到M4。只有一个运行时、没有自动进化的系统，也可以在自己的任务范围内拥有成熟的安全、可靠性和完成验证。

2. 按风险与证据自评

逐项回答“有可复查证据”“缺失”或“不适用”。不适用须记录任务范围、理由和所采用的等效控制，并由风险负责人确认；不能把难实现的保护改成不适用。按安全、可靠性、完成验证、评测与替换能力分别呈现结果，不把功能数求和成一个高分。发现关键缺口时，先缩小受影响任务的自治范围。

M1核对执行权：任务身份可追踪，写入与网络有边界，日志关联任务，取消能阻止后续动作。授权可以是逐次人工批准，也可以是预先批准的受限策略；批量动作是否每次需要人工确认，由风险和授权范围决定，不能把用户曾同意一次理解成无限许可。

M2核对完成证据：合同有版本，候选可封存，验收不由执行者自报，产物有摘要与来源，外部变更可对账，失败有升级点。另一位有权限的核验者应能取回同一产物并复查确定性证据；这不等于能够重放出相同模型文本。

M3核对运营能力：事件与原始载荷可追溯，能力协商报告缺项，恢复与取消经过演练，租户和凭证按实际风险隔离，SLO有窗口与分母。可固定的组件记录版本；托管服务若不允许固定版本，就需要变更检测、兼容性门和必要时停用的降级路径，不能宣称已经固定。只读短任务可以不设持久检查点，但须说明中断后重跑的成本与边界。

M4核对改进权：可变对象明确，候选不能修改裁判、封存测试和策略根；实验登记全部分配试验、缺失规则、成本与停止条件；影子、灰度和回退可以演练。只有需要归因模型与Harness变化时才运行相应对照，不能用脚本化gate检查替代真实效果实验。没有自动改进需求时，不建立这套能力并不是缺陷。

证据可以是测试报告、事件样例、故障演练、配置或审计记录，但必须注明版本、环境、观察窗口和未覆盖范围。产品文档说明一项能力可用，组织仍需证明自己的接入路径和配置确实使用了它。

3. 买什么，控制什么

当现成产品覆盖关键约束，接入与退出成本也可控时，可以优先采购模型、编码运行时、沙箱或连接器。采购对象应分开核算：谁运行模型循环，谁保存会话，谁执行工具，谁保管凭证，谁作业务验收。自管工具机器只是其中一个选择，不自动意味着全部数据留在企业内部；责任表见第二十七章。

企业应掌握任务合同、身份映射、业务策略、领域验证、证据、评测数据和发布权。这些能力可以采购实现，但配置、数据、访问权限和替换条件必须可控。是否自写运行时适配层，也取决于真实兼容需求；采用单一托管方案并保留可靠导出与人工接管路径，同样是可评估的替代方案。

4. 何时自研 Runtime

自研应有可验证的理由，例如现成产品不能满足离线执行、延迟、特殊工具环境或恢复语义。任务规模影响固定成本摊销，但不是必要条件：小规模且约束特殊时，受限的轻量实现也可能合理。团队仍要比较确定性工作流与人工方案，并承担安全、兼容、值班和长期维护。

只比较token单价会漏掉恢复、评测、插件更新和事故处置；只比较订阅费又会漏掉集成与退出费用。下一章给出12个月虚构工作表，分开固定投入、全量任务费用、复核、维护、风险准备和迁移成本。真实决策应替换这些假设，并用可信完成数检查单位成本。

5. 决策矩阵与 POC

先列必须满足的约束，再比较质量、成本和替换能力；不能让便宜的高分抵消权限硬约束。问题应具体到“回执丢失后怎样确认是否已发送”“凭证撤销后下一个动作是否被拒绝”“导出的产物能否在另一套工具中核验”。每项绑定试验输入、预期和证据位置。

POC按预先分配的代表性任务保留全部试验和成本，覆盖正常完成、歧义升级、超时、拒绝、取消、恢复与验证失败。结论可以是采购、自建一个受限组件、采用确定性流程，或维持人工处理。迁移只有在关键约束满足、收益覆盖完整成本且退出演练可行时才值得推进。

第三十章 从接入现有 Agent 到拥有运行时主动权

> 证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

企业拥有运行时主动权，意味着供应商变化时仍能解释任务状态、保全证据、限制损失并选择下一条执行路径。它不要求把模型循环全部自研。任务定义、授权和数据使用责任要有明确归属，具体组件由谁提供则是能力与经济性共同决定的问题。

下表按能力门安排工作，不是必须逐级通关的采购路线。身份、权限、基本证据和取消保护在第一次接入时就要存在，不能等到“统一执行面”阶段才治理生产动作。进入判据确认建设前提，退出判据要求具体证据，回退条件允许缩小自治范围；已经投入的成本不能成为继续迁移的唯一理由。

阶段	进入判据	退出判据	回退条件
1. 封装	已盘点调用者、运行时、权限与凭证路径	版本、事件、成本、工作区和取消可观察；有适配契约测试	关键事件丢失，或接入稳定性低于既有方案
2. 外置完成	一个任务族有负责人和可执行验收	候选封存、独立重验、提交与回读可区分	验收误判不可控，或存在绕过提交授权的路径
3. 统一执行面	工具、网络、凭证和数据出口边界已明确	关键副作用受控，对账、撤权与隔离有演练证据	敏感工具旁路，或新执行面造成无法接受的业务中断
4. 评测运营	合同、任务快照与失败分类稳定	全部分配试验、关键切片、恢复与SLO形成基线	数据污染、必要环境不可获取或分母被修改
5. 逐层替换	可比较旧新组合，并有当前仍获授权的回退路径	质量达到预设门槛，完整成本在预算内，退出演练可行	兼容费用超限、关键切片退化或无法安全回退
6. 受控进化	普通变更发布与回退可靠，评价资产已隔离	低风险候选可被拒绝、灰度、回退和复查	候选能改裁判、扩大权限或绕过发布门

1. 封装而非散接

先为选定Agent建立适配层，映射任务、任务执行尝试、动作、事件、产物、批准和取消，保存规范化事件及原始载荷引用。Task、Attempt、Action分别保留 task_id、attempt_id、action_id；同一动作的ToolTry另记，不能把网络重试当作新任务。每个适配器声明自己接入模型循环、托管会话服务还是工具执行环境，不能把“自管worker”等同于完整自部署。采用第二十七章的责任表分别核对循环、会话、凭证、工具和业务验收。

退出证据包括能力协商结果、结构化事件样本、取消记录、费用账本和供应商版本信息。没有结构化协议的功能可以在受限配置中使用，但必须标明可观测性不足；不能靠解析终端颜色推断关键完成状态。供应商升级应经过兼容性检查，不能固定版本的服务另做变更检测与停用安排。

2. 外置完成与证据

先选一个有明确验收条件的任务族，把验证、证据和提交权限放在执行Agent之外。按照第二十六章封存并重验同一候选；成品交付任务在验收并交付后完成，业务提交任务还要核对实际目标、当前授权、权威回读和 post_commit_checks。COMMITTED_BUT_UNVERIFIED 不能计为完成。换Agent时保留两类任务各自的分母和终点，并测试新适配器有没有遗漏必要事件。

退出证据分三类记录：能否取回原封存产物并验证摘要，能否重跑确定性检查，能否在条件允许时重放生成过程。第三方模型或外部服务已不可用时，第三项可能无法完成，不能因此伪造逐字复现。所有无法复核的输入、环境和产物明确列出；错误完成可以回标到任务、合同及运行时版本。

3. 统一执行面

按风险选择企业沙箱、工具网关、凭证代理和产物存储；这些组件可以自建，也可以采购。受控工具在每次动作前检查授权与撤销状态，执行者只获得最小必要能力。无法适配的原生功能必须记录例外、限制自治范围，必要时由人工执行。

失效场景是部分工具走网关，插件和生命周期hook却直接持有凭证访问外部系统。验证覆盖率需要核对网络流、凭证发放、宿主执行与外部审计日志，不能只数注册了多少工具。工具机器在企业内运行，也要继续追踪输出是否回传云端、会话在哪里保存。

4. 建立评测与运营基线

从获准使用的真实任务形成能力、回归、安全和恢复套件，用同一合同和预算比较质量、稳定性、全量成本及人工负担。各方案都保留预分配任务的失败、超时和未激活试验，不能只比较成功返回者；服务指标沿用第二十七章的窗口与分母。

退出证据是版本固定的任务清单、逐次结果、故障记录、关键切片和成本账本。模型与Harness同时变化且需要归因时，再设计2×2对照；它不是所有单组件迁移的必备仪式。试验次数和非劣门槛由风险与统计计划确定，不能用一次演示代替。

5. 逐层替换

优先替换能解决明确约束、且效果可测的组件，例如上下文编译器、工具视图、策略适配器或工作流。每次尽量控制变化范围，保留原候选证据与可用回退方案；替换决策循环也可以直接成为第一项，但必须有具体必要性，不能把某个顺序当普遍定律。

先内化对象	何时值得	不宜内化的信号
上下文编译器	权威知识已整理，必要资料召回可独立标注	数据来源和访问权限尚不清楚
工具网关	需要统一权限、撤权与外部对账	现有受限工具已满足控制需求
工作流	任务重复且边界稳定，可减少不确定操作	流程仍频繁协商，没有稳定验收
决策循环	协议、离线或特殊环境约束无可采购替代	只看到调用单价差，没有维护与迁移预算

6. 引入受控进化

普通配置能够可靠发布与回退后，才考虑自动生成Harness候选。候选生成、独立评价和发布批准分开，实验资产的使用次数与派生关系可追溯。跨任务经验需经过写入与撤销门；训练模型是另一类成本与数据责任，不是必经终点。没有持续改进需求时，可以停留在稳定运营阶段。

7. 用同一个成本窗口比较采购与自建

先约定统计窗口、合格任务量、质量底线和成本归属。12个月总成本可写为：固定建设与集成费用，加上全部任务的模型、计算、工具、验证及复核费用，再加维护、安全值班、风险准备、双运行迁移与退出费用。单位可信完成成本使用达到合同终点的任务数作分母；重试和失败消耗必须进入分子。

下面全部是虚构教学假设，单位为人民币元，不是市场报价或投资回报承诺。设一年接纳12,000项同类任务，复核每次20元；采购方案复核率10%，受限自建方案12%。模型与执行费用按全部任务计，含预算内重试。两方案必须先通过相同质量与权限门，表中金额才值得比较。

12个月成本项	采购运行时并集成	受限自建运行时
初始建设与集成，全部计入本年	30,000	150,000
平台订阅	24,000	0
模型调用：每任务2元／1.5元	24,000	18,000
计算与工具：每任务0.5元／0.8元	6,000	9,600
独立验证：每任务0.5元／0.7元	6,000	8,400
人工复核：任务量×复核率×20元	24,000	28,800
维护、适配与安全值班	36,000	96,000
风险损失准备，按场景单列	12,000	18,000

12个月成本项	采购运行时并集成	受限自建运行时
迁移双运行与切换演练	12,000	18,000
退出、导出与替代接入准备	18,000	12,000
合计	192,000	358,800

在这组假设下，采购的固定及期间费用为132,000元，每任务变动费用5元；自建分别为294,000元和5.4元，因此仅扩大任务量不会使自建反超。任务量翻倍且不触发扩容时，总额分别为252,000元与423,600元。任一方案复核率增加10个百分点，在原任务量下都会多花24,000元；模型单价翻倍则分别增加24,000元和18,000元。它们说明结论对哪些假设敏感，不代表真实价格会怎样变化。

实际工作表还要注明人员费用是否已含复核、风险准备是否与已发生事故重复计数、固定投入按几年摊销、空闲容量怎样计费，以及迁移期间是否需要额外人手。若关键离线或权限约束只有自建满足，不能以采购表更便宜就忽略约束；应同时比较更小的自建范围、确定性流程和人工替代。

项目据此填写年度预算上限、最低可信完成率、关键切片非劣界、允许人工介入比例、切换停机上限与退出演练期限，再由相同口径的POC结果决定推进或停止。本章不替所有企业指定这些阈值。

8. 退出演练与紧急撤权

每个主要版本至少安排一次退出演练，并留出与生产风险相称的环境。先暂停新任务，导出未完成任务、合同与输入版本、已封存产物及策略记录；核对每个外部动作是已提交、未提交还是结果未知。确认旧执行者已停止且不能再发起动作后，撤销旧执行凭证，再给替代运行时签发新的最小权限。只有目标运行时声明支持的检查点才能恢复，否则创建新尝试，引用原任务与证据重新执行未完成部分。

退出验收不能只记录“导出成功”。应核对任务ID和数量、产物摘要、证据可访问性、已完成动作未被重发、未知结果已隔离，以及旧凭证使用被拒绝。记录新旧能力差异、无法迁移的状态、实际停机时间、双运行费用和人工处置时长。来源系统无法导出的内容要在采购与演练时暴露，不能等停服后才发现证据锁定。

紧急撤权不等待正常退出顺序。发现凭证泄漏或有害能力时，立即阻止相关新动作并撤销权限，再以独立只读审计身份对账、保存必要证据。已固定的模型、工具和记忆版本仅用于复现，不能覆盖当前撤销状态；旧版本若已被撤销，也不能成为回滚目标。受影响任务需要暂停，隔离污染上下文后重建，或以新尝试在新授权下恢复。

一个必须演练的时序是：候选已经生成并通过检查，提交前收到revoke。预期结果应为提交被拒绝、旧候选仍可审计、已发生的外部变更另行对账。只有登记表显示“撤销成功”，而在途任务继续执行，不算撤权完成。这些是平台应验证的验收条件，并不表示本书已替任何供应商运行过此演练。

第三十一章 展望：Harness OS、Agent 组织与持续进化

> 证据地位：本章为作者的条件性展望，结合截至 2026-09-19 已核验研究提出观察方向，不代表这些路径必然发生或已获生产验证。

若长期任务、工具种类和治理需求继续增长，Harness 可能承担更多类似操作系统的职责：调度执行者，管理上下文和能力，隔离计算，记录外部变更，并协调版本与恢复。这个比喻有助于分配责任，却不要求复刻传统 OS。对单一、低风险流程，轻量工具或确定性工作流仍可能更合适。

1. 模型与 Harness 可以共同设计

统一任务、动作和证据语义，不要求所有模型使用相同工具格式。企业可以保留稳定的内部合同，再按模型特点生成工具视图、上下文与停止规则。是否值得维护这些适配，取决于实测收益能否覆盖兼容和迁移成本，而不是“模型无关”与“追求效果”只能二选一。

HarnessDev 发现换执行模型可能改变冻结 Harness 的表现；JIT-Agent 则训练生成器为任务构造 Harness，并冻结底层执行模型。前者测产物迁移，后者测生成体系，不能用同一“通用性”标签抹平差异。这使协同设计值得研究，也要求每次发布留下模型、生成器、配置和经验库的版本。HarnessDev v1、JIT-Agent v2

观察信号应是：适配后的条件收益、统一合同覆盖率、换供应商的工作量与回归成本。如果通用接口已能达到目标，维护多个特化版本反而可能成为负担。

2. Agent 组织首先是依赖和交接问题

跨越单轮聊天的任务，需要明确身份、预算、工作区、产物和责任。多个代理可以围绕任务分工，但角色名称不会自动创造权限边界，工作单元更多也不自动代表协调更有效。

HoH 在固定基础 Harness 下迭代软件项目，以产物和证据延续工作；Stellar Colosseum 针对长证明维护相互依赖的子问题，通过反证与验证驱动局部修复。二者提示了不同组织需求：软件项目要追踪增量和重开问题，研究任务还要追踪一个论证变化会使哪些结论失效。这些是特定研究设计，不是任意多代理组织都有效的证明。
Harness-of-Harness v1、Stellar Colosseum v2

人可能更多地负责目标、规范、例外和证据审查，但转变能走多远，仍由结果可验证性与业务责任决定。流程依赖口头约定、共享账号和不可观察系统时，增加代理数量可能只是扩大混乱。

3. 环境既可能是资产，也可能是债务

权威数据可查询、工具参数清楚、日志可追溯、测试表达业务约束、审批可由系统调用，这些条件可以让不同模型持续受益。它们往往比为一次演示堆提示更值得维护。

环境也可能积累负担：针对旧模型的提示补丁、无法撤销的技能、缺少来源的记忆、无人负责的适配层。应比较每项资产的复用收益与测试、维护、删除传播和退出费用。成熟知识可以变成工具默认值或校验器；持续学习不要求经验库和配置永远膨胀。

4. 自我进化要拆开“改什么”与“谁来决定”

Self-Harness、HarnessBank v2、Living-Harness 与 HSI 提供了不同对象的受限实验信号，具体机制与限制见第二十至二十五章。新研究又扩展了诊断、生成和效率目标：HarnessEvolve 依赖经检查的参考轨迹，Ecdysis 聚合跨任务失败，SoL-Pi 搜索减少往返与上下文费用的机制，同时存在得分让步。不能把它们概括成同一条已证实会持续增长的路线。HarnessEvolve v1、Ecdysis v1、SoL-Pi v1

RSI 路线图讨论改进过程的自主性，包括谁选择策略、获取经验和改进改进机制；本书四类对象回答的是改了什么。某个系统自动改提示，不代表它已经自主定义目标、生成可靠评测或接管治理。路线图是研究方向，不能因论文标题就宣布人类已退出改进回路。RSI 路线图 v2

低风险表面能否扩大自动晋级范围，要看独立验证覆盖、跨轮泄漏控制、长期事故率和撤销演练是否改善。若这些条件停滞，更合理的结果可能是收缩权限或保留人工选择。

5. 长时系统仍有未解决的问题

长期运行会积累小错误，跨代理交接可能放大污染，插件与技能可能把一次注入变成持久行为。删除源经验后，摘要、缓存、训练数据或权重仍可能保留影响；紧急撤销又必须压过版本粘性。第三十一章并不能靠一个“系统可治理”的结句替这些问题作安全保证。

值得继续研究的具体问题包括：可移植检查点怎样描述外部动作状态；多条证明或方案如何组合验证；封存测试在反复查询后何时失去独立性；训练数据撤销后如何验证派生模型处置；质量与费用的权衡能否在长期任务分布中保持。衡量进展需要固定单位、对照与完整账目，而非只看一次最好结果。

6. 三条路径及其反证条件

可能路径	支持它的观察信号	应减缓或转向的信号
交互工具为主，人提交关键动作	人工复核负担可控，任务多变且难形式验收	审核等待成为主要瓶颈，重复任务已有可靠检查
共享企业任务与证据服务	多运行时的接入、迁移和恢复成本下降	统一层增加复杂度，关键语义仍无法兼容
持续实验与有限自动晋级	新任务确认收益可复现，事故受控，撤销有效	测试反馈被反复利用，收益靠额外费用或更宽权限取得

这些路径可以并存，也可能受技术、商业和治理约束而转向。支持多个运行时不是成熟的必要条件，自建完整平台也不是每个组织的目标。应保留采购、受限自研、确定性流程和人工服务的经济对照。

7. 把判断留给可观察结果

本书倾向于把 Harness 看作连接模型能力与组织责任的基础设施。这个判断是否有用，应由具体交付回答：产物是否满足合同，外部动作是否确认，出错后能否停止和恢复，改进是否在独立任务上复现，总费用是否可接受。

如果未来更强模型使某些支架失去价值，就应删掉那些支架；如果任务仍缺可验证结果，就应限制自治范围。值得持续建设的是清楚的接口、可靠证据和可执行的纠错能力，让组织有条件判断何时继续自动化，何时停下来。

附录

附录 A：语言无关核心契约与安全伪代码

本附录说明跨运行时的接口和控制流，不限定实现语言。代码块是带前置条件的设计伪代码；仓库中的python examples/run_examples.py提供本地可执行参考与正反测试。数据库、审批者和模型替身都写明了假设，不能把这些测试当成供应商端到端或生产安全认证。

1. 核心对象

这组对象用于明确跨 Runtime 的最小语义边界。平台可以增添字段，但不得把 Task、Attempt、Action、Effect 和 Evidence 混成一条聊天记录（chat log）。常见误用是只保存模型消息，再事后从自然语言猜测权限、输入版本和实际副作用。这样既不能安全恢复，也不能证明任务完成。

TEXT

```
Task {
  task_id, tenant, contract_version, input_refs[], risk, budget,
  requested_by, commit_authority, status
}
Attempt {
  attempt_id, task_id, runtime, runtime_version, harness_profile,
  workspace_ref, policy_profile, started_at, status
}
Action {
  action_id, attempt_id, actor, type, normalized_args_ref,
  resource, side_effect_class, idempotency_key, provenance
}
Observation {
  action_id, status, structured_result, artifact_refs[],
  diagnostics, environment_revision
}
Artifact {
  uri, hash, media_type, producer, classification,
  input_refs[], created_at, retention_policy
}
PolicyDecision {
  action_id, decision, constraints, policy_version,
  reason_code, approval_request_id?
}
Checkpoint {
  attempt_id, state_version, event_offset, workspace_ref,
  pending_effect_ids[], context_projection_ref
}
VerificationResult {
  contract_version, verifier_version, checks[], status,
  evidence_refs[], environment_ref
}
EvidencePackage {
  task, attempt, inputs[], candidate, effects[], policies[],
  verification, approvals[], final_commit?, lineage
}
```

Attempt表示完成Task的一次执行尝试，不是每次工具网络重试。重试属于同一Action的ToolTry，沿用同一逻辑幂等键。会话Thread/Turn是供应商交互组织方式，由adapter显式映射到Task/Attempt。这里的字段名与附录E保持一致；领域合同仍需经过适配，不能直接视为Task对象。

2. Run loop：proposal 不直接变成 effect

这段循环用于实现平台可控的决策—授权—执行—验证骨架。供应商 Agent 可以占据 model.decide（模型决策接口），却不能绕过策略与 completion gate（完成门）。常见误用是把 final answer 当成成功，或让模型直接调用 executor（执行器）。这两种做法都会把“提出候选”和“获得外部提交权”混为一谈。

TEXT

```
while attempt.active:
    canonical_state = state_store.load(attempt.attempt_id)
    if canonical_state.cancel_requested:
        return cancel(attempt.attempt_id, USER_REQUEST)
    if budget.exhausted:
        return checkpoint_and_suspend(BUDGET_EXHAUSTED)
    context = context_compiler.project(canonical_state, budget)
    proposal = decide_with_reserved_model_budget(context, model_facing_tool_views)

    if proposal.requests_action:
        action = normalize_validate_and_assign_id(proposal.action)
        decision = decode_policy_decision(policy.evaluate(action, current_authority_and_revo
cations()))
        event_store.append(action, decision)

        if decision.decision == REQUIRE_APPROVAL:
            suspend_attempt_with_checkpoint(action, decision)
            return WAITING_FOR_APPROVAL
        if decision.decision not in {ALLOW, CONSTRAINED_ALLOW}:
            observation = denied_observation(action.action_id, decision.reason_code)
        elif not constraints_enforceable(action, decision):
            observation = denied_observation(action.action_id, CONSTRAINT_UNAVAILABLE)
        else:
            if not reserve_action_budget(action):
                return checkpoint_and_suspend(BUDGET_EXHAUSTED)
            try:
                observation = commit_effect_safely(action, decision.constraints)
            finally:
                settle_action_budget_from_meter(action)

        event_store.append(observation)
        state_store.reduce(observation)
        if observation.status == UNKNOWN_EFFECT:
            return RECONCILE_REQUIRED
        if observation.status == PENDING:
            return WAITING_FOR_EFFECT
        continue

    if proposal.requests_user_input:
        return WAITING_FOR_USER
    if not proposal.has_candidate:
        return TURN_STOPPED
    candidate = seal(proposal.output, canonical_state.artifacts)
    verification = completion_gate.verify(candidate, task.contract_version)
    if verification.status == PASS:
        return CANDIDATE_VERIFIED
    if verification.feedback_allowed and verification.repairable and budget.consume_repair_a
ttempt():
        state_store.reduce(minimal_diagnostics(verification))
    else:
        return NEEDS_ESCALATION
```

审批分支保存待执行动作后立即返回，因此不会追加未赋值或上一轮残留的observation。恢复审批时校验批准人、action_id、参数hash、契约/资源版本、期限及撤销状态，再重新授权原动作；拒绝或过期就记录拒绝，不能把旧批准交给一个新动作。未知策略值也按拒绝处理。commit_effect_safely将已发送请求的预期执行异常转成结构化未知结果；意外故障由外层finally结算已用预算，进程崩溃则由持久预留账恢复结算，不能靠少记成本让循环无限继续。

无工具调用可能是提问、停止或候选交付。只有候选进入完成门；CANDIDATE_VERIFIED并不自动授予业务提交权。提交和后验遵循第十章。

decode_policy_decision是显式适配边界：先按附录E校验传输消息；合法但省略的reason_code在内部对象中补为稳定诊断码，普通ALLOW省略的constraints补为内部空集合。CONSTRAINED_ALLOW必须保留非空限制并确认可以落实。内部默认值不反向伪装成线上必填字段；测试应先校验真实消息，再走这条适配路径。无效消息默认不执行。

PENDING表示另一执行者持有动作，或尚不能取得决定是否执行所需的结果。顺序调用者保存待处理动作并返回WAITING_FOR_EFFECT，不能越过它执行依赖后续动作。UNKNOWN_EFFECT进入对账。两者都不自动转成新的幂等键或盲目重放；需要并行无依赖动作的实现，应另外表达依赖关系和局部暂停范围。

3. 副作用提交：先记 intent，再执行

以下参考要求目标系统支持原子幂等键与参数绑定：并发提交同一键只产生一个效果，相同键的不同参数被拒绝。幂等记录的保留期覆盖本平台的重试窗口。这些是目标API或业务唯一约束提供的能力，本地账本不能凭空补出。目标不支持这些条件时，应采用它能提供的事务/确认协议；未知结果升级处理，不复用这个例子承诺恰好执行一次。

TEXT

```
commit_effect_safely(action, constraints):
    tenant = trusted_tenant_for_attempt(action.attempt_id)
    key = scoped_key(tenant, action.resource, action.idempotency_key)
    args_hash = hash_normalized_operation(action, constraints)
    effect = durable_store.bind_intent_once(key, args_hash)
    lease = coordinator.try_claim_effect(effect.id)
    if lease is None:
        return Observation(action_id=action.action_id, status=PENDING)

    try:
        effect = durable_store.load(effect.id)
        if effect.is_final:
            return observation_from_recorded_outcome(action, effect)
        prior = target_system.lookup_or_unknown(key)
        if prior.is_committed_with(args_hash):
            return persist_reconciled_outcome(action, effect, prior)
        if effect.status in {EXECUTING, UNKNOWN_EFFECT}:
            return persist_unknown(action, effect, RECONCILE_REQUIRED)
        if prior.is_unavailable:
            return Observation(action_id=action.action_id, status=PENDING)

        # Check live cancellation/revocation after lookup, immediately before dispatch.
        if not still_authorized_and_active(action, constraints, lease):
            return persist_denied_outcome(action, effect)
        durable_store.mark_executing_if_owned(effect.id, lease.fence)
        try:
            outcome = target_system.execute_idempotently(key, args_hash, action, constraints
)
        except ExecutionError as error:
            return persist_unknown(action, effect, diagnostic(error))
        if not outcome.is_definitive:
            return persist_unknown(action, effect, RECONCILE_REQUIRED)
        return persist_final_outcome(action, effect, outcome)
```

```
    finally:
        coordinator.release(lease)
```

bind_intent_once以租户、目标、逻辑键建立唯一记录并核对参数；重复调用不把旧状态重置为INTENT。lookup_or_unknown把查询故障变成不可确定；查无结果可能来自可见性延迟，不能据此重放遗留EXECUTING/UNKNOWN。执行租约减少重复worker，但迟到worker仍可能发请求，最后的去重保证必须由目标系统的原子键完成。撤销与发送若同时发生，仍需记录时间和实际提交结果，不能宣称取消回滚了已发送动作。

这里有意采用保守恢复：已开始发送而结果不明的记录，只回读已确认结果，否则停在对账状态。若业务需要自动恢复未生效请求，必须另外证明旧请求已经失效，或使用目标API明确保证的同键安全重放协议。不能把本例的PENDING/UNKNOWN直接当作可重试信号。

4. 恢复、取消与对账

长任务、子任务和外部副作用并存时，恢复与取消必须作为持久状态转换实现，而不是进程控制的附注。常见误用包括恢复旧 credential（凭证）、重复执行未知 effect，或在 UI 标记 cancelled 后仍让子进程继续运行。这些都会造成越权或重复提交。

TEXT

```
recover(attempt_id):
    lease = coordinator.acquire_single_owner(attempt_id)
    try:
        state = state_store.load(attempt_id)
        if state.is_terminal:
            return state.status
        if state.cancel_requested:
            return continue_cancellation_without_resuming_work(state)
        checkpoint = state_store.latest_checkpoint(attempt_id)
        state = replay_pure_events(checkpoint.state, checkpoint.event_offset)
        for effect in state.pending_or_unknown_effects:
            authoritative = target_system.lookup_or_unknown(effect.idempotency_key)
            append_reconciliation_observation(effect, authoritative)
        state = state_store.load(attempt_id)
        if state.is_terminal or state.cancel_requested:
            return state.status
        if state.has_unknown_effects:
            return RECONCILE_REQUIRED
        policy = policy_store.load_current_compatible_version_and_revocations()
        credentials = broker.issue_fresh_leases(allowed_capabilities(state, policy))
        return resume_from_reconciled_state(state, policy, credentials)
    finally:
        coordinator.release(lease)

cancel(attempt_id, reason):
    state_store.mark_cancel_requested(attempt_id, reason)
    scheduler.cancel_children(attempt_id)
    executor.terminate_process_tree(attempt_id)
    revoke_temporary_credentials(attempt_id)
    reconcile_pending_effects(attempt_id)
    return state_store.mark_cancelled_when_quiescent(attempt_id)
```

恢复从checkpoint的状态和事件偏移量一起重建，不只重放尾部事件。对账回写后重新加载状态，防止带着旧副本继续执行。恢复入口和最终resume都要以条件写入检查取消/终态，最新撤权优先于版本兼容。CANCELLED可以保留已发生的外部效果；若远端是否停止或是否提交仍不明，保持CANCELLING/待对账，不假装已经清理完毕。

操作系统进程树终止、远端作业取消、租约过期和崩溃恢复需要对具体后端另做测试。本地参考只能验证它实现的状态转移及SQLite目标约束，不能借这些helper名字声称全部后端已有同样保证。

5. Adapter 的能力协商

CapabilitySet 用于调度前比较任务风险需求与 Runtime 的真实能力，尤其适合同时接入 Claude Code、Codex 与自研 Runtime 的平台。它不是一张营销功能表。no（否）或未知能力必须导致替代 Runtime、收缩自治范围或人工升级，不能靠空字段伪装兼容。

TEXT

```
CapabilitySet {
  structured_events: yes/no
  pause_for_approval: yes/no
  resume: none/session/checkpoint
  cancel: cooperative/process_tree
  artifact_export: list of media types
  workspace_isolation: local/worktree/container/vm/provider
  raw_event_provenance: yes/no
}
```

Control Plane（控制平面）依据 Task 风险声明做能力检查，adapter（适配器）返回真实能力。若关键能力缺失，scheduler（调度器）应选择替代 Runtime、降低自治范围或要求人工审批，不能把 no 转成空字段继续执行。

附录 B：可判定的 Harness 架构评审表

使用方法：逐项登记“通过 / 不通过 / 不适用”，绑定配置、测试、事件或演练报告。表中 K 为适用范围内的关键项，O 为运营完善项；R0–R4 沿用第九章的动作风险等级，最终按实际资源和数据流判定。“不适用”必须有风险负责人具名理由，缺证据不等于不适用。R3/R4 任一适用 K 项未通过，均阻断生产自治。

每项评审记录都须补齐：适用动作与风险等级、K/O 标记、故障点与重试范围、验收证据及保留期、结论、不适用理由、责任人和到期时间。恢复演练还应列出崩溃点、并发重试次数、下游去重或查询能力及观察窗口；有限演练只支持该范围内的结论。

任务与完成

检查项	适用范围／等级	合格判据	不合格的典型症状	依据
目标是否版本化	K：R2–R4；R0/R1 可轻量记录	完成契约含交付物、不变量、预算、权限、验收、变更规则与负责人	目标静默变化	第十、二十八章
停止是否与完成分离	K：全部	结束一轮、取消、待回复与任务完成分别记录；只有声明完成才执行验收	取消也等待成功门；最终文本直接触发提交	第六、十章
是否生成证据包	K：R3/R4；其余按验收要求	输入、产物哈希、检查、授权、批准与提交后验收可关联	只有最终文本或截图	第十、二十六章
外部提交是否独立授权	K：R3/R4	提交权限受可信边界管理；拒绝、未知决定及过期授权均不执行	Agent 自报成功后自动生效	第九、十章

上下文与记忆

检查项	适用范围／等级	合格判据	不合格的典型症状	依据
上下文来源可见	K：R2–R4；其余 O	承重片段有来源、版本、选择原因和成本记录	不知道规则从哪里注入	第七章
压缩不覆盖权威状态	K：使用压缩的任务	状态保存在上下文外；压缩后比对基线并核验引用	摘要漏掉授权限制	第六、七章
记忆有写入与撤销门	K：跨任务记忆	候选、验证、作用域、有效期与责任人齐全；撤销后复查派生对象	一次成功自动写入全局记忆	第七、二十二章
检索先做隔离	K：私有或多租户数据	先按身份、租户与分类过滤，结果保留信任标签	跨租户召回或公开内容提升为指令	第七、九章

工具、环境与副作用

检查项	适用范围／等级	合格判据	不合格的典型症状	依据
动作契约稳定	K：有工具执行	参数、错误、版本、截断和产物语义明确	全部失败都是 error	第八章
外部副作用可对账	K：R2–R4 写动作	意图先持久化；声明天然幂等、下游按键去重、可靠对账后决定或禁止自动重放；未知结果待对账	超时后直接重发或重付	第六章、附录 A
工作区可重建	K：可执行验收	输入版本、镜像、依赖和初始化可固定	只在原机器上能通过	第四、六章
沙箱边界经验证	K：通用执行／不可信代码	按威胁模型测试文件、网络、进程、挂载、资源与身份	使用容器即宣称隔离	第九、十七章
不可逆动作有处置路径	K：R3/R4 不可逆动作	事前授权、限额、提交回读和事故处置有证据；补偿能力单列	把软件回退等同撤回已发送消息	第九、十章

权限与供应链

检查项	适用范围／等级	合格判据	不合格的典型症状	依据
身份、授权、批准分开	K：R1–R4 受控资源	主体、临时授权、决定与审批人可追踪	登录成功即拥有全部权限	第九章
凭证受限且不进上下文	K：使用凭证	代理按动作注入短期受限凭证；入日志前脱敏	令牌出现在提示或技能中	第九、二十七章
委派授权不扩大	K：使用子代理	转授不超范围；独立专家的执行权与调用者委派权分别校验；预算原子预留	子代理继承宿主全部秘密	第十一章
扩展供应链可撤销	K：安装扩展	来源、版本、权限与责任人明确；hook 更新差异重新审查，撤销可停止后续执行	已信任插件更新后新增命令免审批	第九、二十二章
提示注入有系统测试	K：不可信输入可影响后续动作，含 R0 读取	覆盖间接注入、数据外泄和跨域组合	只测试口头拒绝	第九、二十四章

Durable 与多 Agent

检查项	适用范围／等级	合格判据	不合格的典型症状	依据
中断可恢复	K：承诺恢复的任务	在已声明崩溃点、重试次数和下游条件下，状态不变量保持，未知结果进入对账	崩溃后重放未知写动作	第六章
取消能收敛	K：长任务／后台执行	停止派发，处理后代、凭证和在途副作用；未停资源保留清理责任	界面取消但后台无主执行	第六、十一章
并行写入隔离	K：并行写任务	写集隔离或冲突受控，合并版本重验	共享目录互相覆盖	第十一、十四章
责任转交有结构化交付	K：使用 handoff	目标、已做、产物、未决、权限和预算齐全	只返回“已完成”摘要	第十一章

Eval、运营与进化

检查项	适用范围／等级	合格判据	不合格的典型症状	依据
评测集用途分开	K：选型／进化／泛化主张	开发、验证、密封终测、回归分别管理，记录访问与反馈	隐藏检查反复用于修复仍称独立终测	第十二章
报告重复试跑与切片	K：性能晋级主张；其余 O	固定任务、环境和预算，报告区间、成本和关键切片	只报一次最好成绩	第十二、十九章
轨迹骨架完整	K：R3/R4；其余按恢复要求	关键事件按保留规则全量记录；对账新建、未闭合和已终止运行	崩溃任务从统计分母消失	第十二章
候选与裁判隔离	K：自动优化／独立验收	可变范围明确，候选不可改裁判、密封集或根授权	可改评分逻辑与分母	第二十、二十五章
软件发布可灰度与回退	K：运行配置发布	固定任务版本，记录灰度与版本回退演练；业务副作用另处置	换回提示就声称撤销业务提交	第二十五章
版本沿袭可解释	K：R3/R4 版本发布；其余 O	父版本、修改、数据、评测、批准、事故与退役齐全	无法回答为何上线	第二十五章

最终判定

- 阻断：R3/R4 任一适用 K 项失败、缺证据或责任未闭合，均不进入生产自治；不可逆动作按预防、回读与事故处置判定，不虚称可以回滚。
- 限域上线：缩小后的动作范围内适用 K 项均通过，O 项仍有缺口；明确低风险范围和人工提交边界，不把降级环境结论用于高风险生产。
- 生产候选：所有适用 K 项有证据支撑，并在登记的故障范围内完成正常、拒绝、超时、取消、崩溃、恢复与验证器失败演练。

通过只表示在所列条件和观察窗口内未发现违反任务不变量的行为，不证明零丢失、零重复或零风险。评审记录应注明适用范围与到期时间；只读代码 Agent 的结论不能继承给可写生产数据库的 Agent。

附录 C：术语与本体边界

本书固定以下用法。产品文档可能采用不同名称，适配器应映射含义，而非只替换字符串。

- **Model**：接收有限上下文并生成文本或动作建议的概率性决策策略。它不天然拥有持久状态、权限和外部真值。
- **Model policy / authorization policy**：分别为模型的决策策略与系统的授权策略；前者提出下一步，后者判断主体对资源可执行哪些动作。
- **Agent**：在任务范围内由模型动态选择观察或动作的执行者。Agent 是系统角色，不等于单次模型调用。
- **Agent System**：Model、Harness、Environment 与 Feedback 的完整组合，也是评估能力和风险的对象。
- **Harness**：组织模型、工具与环境执行任务的运行和治理机制，涵盖循环、上下文、状态、授权、验证、可观测性与进化治理；不暗示必须由外部服务托管。
- **Agent Runtime**：承载 Agent loop、session 和模型交互的运行组件，例如供应商 CLI/core 或自研 loop。
- **Execution Runtime**：实际运行命令、浏览器、代码或连接器的环境，例如容器、VM 或受控远程执行器。
- **Environment**：Agent 可观察或改变的任务世界，包括 workspace、数据库、SaaS、日志和人类组织。它不等于一个 shell。
- **Feedback**：用于调整系统对动作质量或任务质量判断的信号；Observation 只有进入评价链路时才成为 feedback。
- **Workflow**：由代码预定义主要控制路径的执行结构。模型可在节点内被调用，但不拥有全部路由权。
- **Control Plane**：控制面，管理身份、授权策略、目录、配置与发布；策略管理和请求时的授权执行要区分。
- **Data Plane**：数据面，承载会话、事件、上下文、模型请求与任务推进。控制、数据、执行三平面是职责划分，不是严格调用栈或必须分库的部署要求。
- **Execution Plane**：执行面，以受限身份运行工具和外部连接，在动作发生时核验授权并记录结果。
- **Evidence Plane**：证据面，进一步归拢产物、轨迹、副作用账本和验证记录；记录可复查不等于记录中的语义已经正确。
- **Evolution Plane**：进化面，组织记忆、Harness 与模型候选的生成、评价和发布流程。它跨越候选执行与证据处理，发布权仍受控制面约束。
- **ACI**：Agent-Computer Interface，模型与计算环境之间的动作和观察接口。
- **Action**：Agent 提议的规范化动作。在授权与执行之前，它还不代表已发生的外部副作用。
- **Observation**：动作、环境或策略返回的可观察结果，包含状态、诊断与产物引用。
- **Effect**：外部副作用，即动作对外部权威状态的改变；包括预期的写入，不只指不良结果。是否已发生可以处于待确认状态。
- **Effect Ledger**：副作用账本，记录意图、适用的幂等键、提交状态、结果与对账过程；有账本不等于下游自动去重。
- **CompletionContract**：完成契约，规定目标、交付物、不变量、验收、证据、权限、预算、变更权与停止规则；取消或结束一轮不等于完成验收。
- **Candidate**：提交给验收流程的候选产物，尚未因“自报完成”取得业务提交权。

- **Verifier / judge / approver**：验证器检查定义好的条件；评判器可对语义质量评分；审批人或审批服务决定是否授权。三者职责不同。
- **VerificationResult**：特定验证器在固定环境与候选版本上输出的结构化契约检查结果，不是完整正确性证明。
- **EvidencePackage**：证据包，把输入、候选、产物、外部副作用、授权、验证、审批与最终提交关联起来的机器可读记录。
- **Artifact**：产物，具有地址、哈希、媒体类型、生产者和分类的持久对象；在交付语境中可称交付物，与已发生的外部副作用区分。
- **Checkpoint**：用于恢复的任务状态、事件 offset、workspace 与 pending effect 引用；不等于上下文摘要。
- **Compaction**：将长上下文转换为可继续推理的较短表示，属于有损投影，不是长期记忆。
- **Memory**：跨推理或跨任务保留的事实、情景、程序或决策经验；需声明范围、责任人和生命周期，根授权策略不由普通记忆改写。
- **Skill**：按需加载的程序化知识包，可能包含指令、脚本和资源，属于软件供应链对象。
- **Handoff**：将工作责任从一个 Agent 或节点转移给另一个，带出结构化目标、状态、artifact、权限和未决项。
- **Capability lease**：临时授权，也称能力租约，绑定主体、资源、动作、用途、租户与有效期；这里的 capability 指可执行权限，不是模型解题能力。
- **Held-out set**：保留集，未直接用于生成或训练的数据或检查集的宽泛称呼。用于候选比较或修复反馈时，承担验证集职责，不自动具有独立终测资格。
- **Validation set**：验证集，用于比较并选择候选模型、Harness 或配置。应固定任务口径，不能按候选表现临时挑题。
- **Sealed test**：密封测试集，由独立服务在预定时机终测，事先固定反馈粒度、访问次数与停止规则；用于自适应修复后须记录暴露并重新界定用途。
- **Canary**：在受限真实流量和限定影响范围内部署候选版本并持续监控。
- **Harness evolution**：对 prompt、工具、上下文、路由、工作流或 runtime profile 的受控优化，不等同于模型权重训练。
- **Reward hacking**：提高评分却偏离真实目标或破坏评价完整性的行为。
- **Provenance**：来源记录，回答材料、结果或证据来自哪里，绑定生产者、时间、版本与原始引用；来源可追溯不等于内容正确。
- **Lineage**：版本沿袭关系，回答版本怎样演变，覆盖父项、数据、修改、实验、发布、事故与退役。

最容易混淆的三组边界是：Agent Runtime 组织下一步决策，Execution Runtime 执行动作；Memory 保存跨期经验，Compaction 压缩当前上下文；Harness 可以包含授权策略适配器，但根授权与发布权不能由候选 Harness 自行修改。签名核验来源和完整性，也不能替代这些语义边界。

附录 D：概念首次定义与使用索引

本索引区分概念首次集中定义与主要展开位置，不把先行例子或词语首次出现当作正式定义。修改核心含义时，应同步更新本索引、术语表和机器可读契约。

概念	首次集中定义	主要展开章节
Model × Harness × Environment × Feedback	第五章（序言先引入隐喻）	第十九、二十四、二十七章
Agent / Workflow	第五章（第二章有先行例子）	第十一、二十八章
Harness	第五章	第六至十二、二十、二十七章
Agent Runtime / Execution Runtime	第五章	第十三至十八、二十七章
Control/Data/Execution Plane	第五章	第九、二十七章
Durable state machine	第六章	第十一、二十七章
Task / Thread / Turn / Step / Attempt / ToolTry	第六章	第十一、十二章、附录 A
Checkpoint	第六章	第七、二十一章、附录 A
Effect Ledger	第六章	第十、十一、二十七章、附录 A
Reconciliation	第六章	第十、二十一、二十六章
Context compiler	第七章	第十五、二十三、三十章
Compaction	第七章	第十二至十八章
Memory	第七章	第二十、二十二、二十五章
Skill	第七、八章	第十三、二十二、二十三章
ACI	第三章	第四、八章
Action / Observation	第三、六章	第十七、二十七章、附录 A
MCP	第八章	第九、十三至十八章
Code Mode	第八章	第十六、二十三章
Capability lease	第九章	第十一、二十七章
执行权 / 委派权	第九、十一章	第二十七章
Sandbox	第九章	第十三至十八、二十七章
Credential broker	第九章	第二十六、二十七、三十章
CompletionContract	第十章	第二十六、二十八章
EvidencePackage	第十章	第二十五至二十七章、附录 A
Commit authority	第十章	第二十六至二十八章
Delegation / Handoff	第十一章	第十三至十八、二十七章
Trace / Event graph	第十二章	第十四、二十五、二十七章
Capability / Regression eval	第十二章	第十九、二十三、三十章
Held-out / Validation / Sealed test	第十二章	第十、二十五章、附录 C
Runtime adapter	第十四、十九章	第二十六、二十七、三十章
四层进化模型	第二十章（第一章先作分类映射）	第二十一至二十五章
Mutable surface / Root of trust	第二十章	第二十三、二十五章
Model × Harness 2×2	第二十、二十四章	第三十章
Shadow / Canary	第二十五章	第二十六、二十七章
Lineage	第二十五章	第二十六、二十七章
Agent SDD	第二十八章	第二十六、三十章
M0—M4 成熟度能力	第二十九章	第三十章

附录 E：最小机器可读契约

下面的JSON Schema是教学子集，展示如何检查对象形状。字段名与附录A一致；平台仍需定义兼容规则、分类、签名与可信验证方。真实正反实例由python publishing/scripts/check_examples.py校验。JSON能解析不等于实例符合schema，实例符合schema也不等于获得授权。

1. Task 与 CompletionContract

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.internal/harness/task-v1.schema.json",
  "title": "Task",
  "type": "object",
  "additionalProperties": false,
  "required": ["task_id", "tenant", "contract_version", "risk", "deliverables", "checks", "c
ommit_authority"],
  "properties": {
    "task_id": {"type": "string", "minLength": 1, "pattern": "\\S"},
    "tenant": {"type": "string", "minLength": 1, "pattern": "\\S"},
    "contract_version": {"type": "string", "minLength": 1, "pattern": "\\S"},
    "risk": {"enum": ["R0", "R1", "R2", "R3", "R4"]},
    "input_refs": {"type": "array", "items": {"type": "string", "minLength": 1, "pattern": "
\\S"}},
    "deliverables": {"type": "array", "minItems": 1, "items": {"type": "string", "minLength"
: 1, "pattern": "\\S"}},
    "invariants": {"type": "array", "items": {"type": "string", "minLength": 1, "pattern": "
\\S"}},
    "forbidden_actions": {"type": "array", "items": {"type": "string", "minLength": 1, "patt
ern": "\\S"}},
    "checks": {"type": "array", "minItems": 1, "items": {"type": "string", "minLength": 1, "
pattern": "\\S"}},
    "budget": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "wall_seconds": {"type": "integer", "minimum": 1},
        "model_usd": {"type": "number", "minimum": 0},
        "max_actions": {"type": "integer", "minimum": 1}
      }
    },
    "commit_authority": {"type": "string", "minLength": 1, "pattern": "\\S"}
  }
}
```

2. Action、PolicyDecision 与 Observation

第二份schema的根明确引用Action，因此用根验证null或空对象会失败。校验另外两种对象时，用同一份$defs构造分别指向#/$defs/PolicyDecision或#/$defs/Observation的schema。只保留$defs却不应用任何$ref，会成为允许任意根实例的定义集合，不能作为消息验证器。

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Action with shared message definitions",
  "$ref": "#/$defs/Action",
  "$defs": {
```

```
    "Action": {
      "type": "object",
      "additionalProperties": false,
      "required": ["action_id", "attempt_id", "actor", "type", "resource", "side_effect_clas
s"],
      "properties": {
        "action_id": {"type": "string", "minLength": 1, "pattern": "\\S"},
        "attempt_id": {"type": "string", "minLength": 1, "pattern": "\\S"},
        "actor": {"type": "string", "minLength": 1, "pattern": "\\S"},
        "type": {"type": "string", "minLength": 1, "pattern": "\\S"},
        "normalized_args_ref": {"type": "string", "minLength": 1, "pattern": "\\S"},
        "resource": {"type": "string", "minLength": 1, "pattern": "\\S"},
        "side_effect_class": {"enum": ["NONE", "REVERSIBLE", "COMPENSATABLE", "IRREVERSIBLE"
]},
        "idempotency_key": {"type": "string", "minLength": 1, "pattern": "\\S"}
      },
      "if": {"properties": {"side_effect_class": {"not": {"const": "NONE"}}}, "required": ["
side_effect_class"]},
      "then": {"required": ["normalized_args_ref", "idempotency_key"]}
    },
    "PolicyDecision": {
      "type": "object",
      "additionalProperties": false,
      "required": ["action_id", "decision", "policy_version"],
      "properties": {
        "action_id": {"type": "string", "minLength": 1, "pattern": "\\S"},
        "decision": {"enum": ["ALLOW", "DENY", "REQUIRE_APPROVAL", "CONSTRAINED_ALLOW"]},
        "policy_version": {"type": "string", "minLength": 1, "pattern": "\\S"},
        "reason_code": {"type": "string"},
        "constraints": {"type": "object", "minProperties": 1},
        "approval_request_id": {"type": "string", "minLength": 1, "pattern": "\\S"}
      },
      "allOf": [
        {"if": {"properties": {"decision": {"const": "CONSTRAINED_ALLOW"}}, "required": ["de
cision"]}, "then": {"required": ["constraints"]}},
        {"if": {"properties": {"decision": {"const": "REQUIRE_APPROVAL"}}, "required": ["dec
ision"]}, "then": {"required": ["approval_request_id"]}}
      ]
    },
    "Observation": {
      "type": "object",
      "additionalProperties": false,
      "required": ["action_id", "status"],
      "properties": {
        "action_id": {"type": "string", "minLength": 1, "pattern": "\\S"},
        "status": {"enum": ["OK", "DENIED", "ERROR", "TIMEOUT", "UNKNOWN_EFFECT", "CANCELLED
", "PENDING"]},
        "structured_result": {},
        "artifact_refs": {"type": "array", "items": {"type": "string"}},
        "diagnostics": {"type": "object"},
        "environment_revision": {"type": "string"}
      }
    }
  }
}
```

3. EvidencePackage 必填骨架

以下YAML是字段说明，不是已经验收的实例；required、sha256-required和PASS|FAIL|INCONCLUSIVE必须由真实值替换。它没有冒充JSON Schema，运行入口只做格式解析。若要接入服务，还需为证据包制定单独schema，并验证引用、访问权、签署人及产物一致性。

YAML

```yaml
evidence_package:
  schema_version: evidence-package/v1
  package_id: required
  task:
    task_id: required
    contract_version: required
  attempt:
    attempt_id: required
    runtime: required
    runtime_version: required
    harness_profile: required
  inputs:
    - uri: required
      hash: sha256-required
  candidate:
    uri: required
    hash: sha256-required
  effects: []
  policy_decisions:
    artifact_ref: required
  verification:
    verifier_version: required
    environment_ref: required
    status: PASS|FAIL|INCONCLUSIVE
    checks: []
  approvals: []
  final_commit: null
  lineage:
    parent_attempt: optional
    model_version: required
    harness_bundle: required
```

非空幂等键只便于表达逻辑动作身份，不会让下游自动去重；非空constraints也不证明执行器真的落实了约束。checks是否覆盖目标、hash对应哪个可信产物、批准是否来自有权主体，均由相应策略与验收服务检查。第26、28章的领域合同经adapter映射为Task，contract_id不能无转换地冒充task_id，分钟预算要先换算成秒。

研究方法与局限

本书采用官方文档、开源仓库、论文和社区材料的分层证据法。全书基础资料截面为 2026-09-19；最近修订日期为 2026-10-29，范围：第三篇架构重写与 pi-mono；其余篇事实截面仍为 2026-09-19。无法验证的内部实现不作为事实。设计原则是作者基于多来源的综合推断。研究资产包括 sources.jsonl、evidence.jsonl 与人工标注的 claims_v2.jsonl；旧 claims.jsonl 仅作历史迁移参考。引用检查验证登记与引用关系，不代替逐句语义判断；原子账本是核心主张子集，不代表全部句子已经独立核验。本轮修订的本地案例和反例可通过 python examples/run_examples.py 复现；其中模型输出使用确定性替身，数据库采用明确方言和数据假设，不能等同供应商端到端或生产效果验证。局限包括产品快速迭代、公开 benchmark 污染、厂商数据选择偏差，以及部分2026年进化论文尚缺长期生产复现。

已核验参考文献

以下条目已核对来源身份、访问日期与版本；支持哪项结论仍须结合正文限定和证据摘录判断。未核验研究线索保留在仓库，不列为已核验参考资料。

- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / README.md。版本：git cd5ef8148158c3a752a658978873241fdf8e2bbc；访问：2026-09-19T09:44:36.928264+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / docs/architecture.md。版本：git cd5ef8148158c3a752a658978873241fdf8e2bbc；访问：2026-09-19T09:44:36.953863+00:00。
- 机构/作者未登记 (n.d.). Cordis Primer。版本：git cd5ef8148158c3a752a658978873241fdf8e2bbc；访问：2026-08-28。
- Zhang et al. (n.d.). Self-Harness: Harnesses That Improve Themselves。版本：arXiv:2606.09498v3；访问：2026-09-19T09:53:07.552Z。
- Luo et al. (n.d.). Self-Evolving Agent Harnesses via Gated Semantic Quality-Diversity。版本：arXiv:2607.13683v1；访问：2026-09-19T09:55:33.641Z。
- Du et al. (n.d.). Living-Harness Is an Interactive-Agent Evolver。版本：arXiv:2607.26598v2；访问：2026-09-19T09:53:07.560Z。
- Tailin Zhou (n.d.). Hierarchical Self-Improvement: A Framework for Task-Specific Evolvable Agent Harnesses。版本：arXiv:2608.08466v1；访问：2026-09-19T09:53:07.542Z。
- 机构/作者未登记 (n.d.). Unrolling the Codex agent loop | OpenAI。版本：web snapshot 2026-09-19；访问：2026-09-19T11:23:43.915Z。
- 机构/作者未登记 (n.d.). Unlocking the Codex harness: how we built the App Server | OpenAI。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:42.679Z。
- 机构/作者未登记 (n.d.). Introducing the Codex app。版本：web snapshot 2026-08-28；访问：2026-08-28。
- 机构/作者未登记 (n.d.). Harness engineering: leveraging Codex in an agent-first world | OpenAI。版本：web snapshot 2026-09-19；访问：2026-09-19T11:23:43.962Z。
- 机构/作者未登记 (n.d.). openai/codex。版本：repository metadata snapshot 2026-09-19；访问：2026-09-19T09:43:24.100927+00:00。
- 机构/作者未登记 (n.d.). 持续改进我们的智能体框架 · Cursor。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:27.160Z。
- 机构/作者未登记 (n.d.). Dynamic context discovery · Cursor。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:27.172Z。
- 机构/作者未登记 (n.d.). Runtime Architecture - OpenHands Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:27.253Z。
- Earendil Works / Mario Zechner (2026). pi/packages/coding-agent/README.md at main · earendil-works/pi。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:37.437Z。
- 机构/作者未登记 (n.d.). ReAct: Synergizing Reasoning and Acting in Language Models。版本：arXiv:2210.03629v3；访问：2026-09-19T11:45:11.199Z。

- 机构/作者未登记 (n.d.). Reflexion: Language Agents with Verbal Reinforcement Learning。版本：arXiv:2303.11366v4；访问：2026-09-19T11:45:11.193Z。
- 机构/作者未登记 (n.d.). Voyager: An Open-Ended Embodied Agent with Large Language Models。版本：arXiv:2305.16291v2；访问：2026-09-19T11:45:11.205Z。
- Richard Fikes; Nils Nilsson (1971). STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving。版本：Artificial Intelligence 2 (1971), pages 189-208; author-hosted original PDF；访问：2026-09-19T12:04:42.938Z。
- Reid G. Smith (1980). The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver | IEEE Journals & Magazine | IEEE Xplore。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:37.414Z。
- 机构/作者未登记 (2020). BDI Agent Architectures: A Survey | IJCAI。版本：web snapshot 2026-09-19；访问：2026-09-19T11:12:16.586Z。
- 机构/作者未登记 (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering。版本：arXiv:2405.15793v3；访问：2026-09-19T11:49:17.153Z。
- 机构/作者未登记 (2023). babyagi_archive/babyagi.py at main · yoheinakajima/babyagi_archive。版本：web snapshot 2026-09-19；访问：2026-09-19T11:24:43.773Z。
- 机构/作者未登记 (2023). Autonomous Agents & Agent Simulations。版本：web snapshot 2026-09-19；访问：2026-09-19T11:17:44.573Z。
- 机构/作者未登记 (2025). Building LangGraph: Designing an Agent Runtime from first principles。版本：web snapshot 2026-09-19；访问：2026-09-19T11:24:43.754Z。
- 机构/作者未登记 (n.d.). Repository map | aider。版本：web snapshot 2026-09-19；访问：2026-09-19T11:17:44.588Z。
- 机构/作者未登记 (n.d.). GPT code editing benchmarks | aider。版本：web snapshot 2026-09-19；访问：2026-09-19T11:17:44.580Z。
- 机构/作者未登记 (n.d.). Building Effective AI Agents \ Anthropic。版本：web snapshot 2026-09-19；访问：2026-09-19T11:23:43.972Z。
- 机构/作者未登记 (n.d.). How the agent loop works - Claude Code Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:20.843Z。
- 机构/作者未登记 (n.d.). How Claude Code works - Claude Code Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T11:21:13.461Z。
- 机构/作者未登记 (n.d.). Idempotency and retries - AWS Durable Execution SDK Developer Guide。版本：web snapshot 2026-09-19；访问：2026-09-19T11:22:42.486Z。
- 机构/作者未登记 (2024). Lost in the Middle: How Language Models Use Long Contexts - ACL Anthology。版本：web snapshot 2026-09-19；访问：2026-09-19T11:24:43.782Z。
- 机构/作者未登记 (n.d.). How Claude remembers your project - Claude Code Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T11:21:13.439Z。
- 机构/作者未登记 (n.d.). Explore the context window - Claude Code Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T11:21:13.453Z。

- 机构/作者未登记 (n.d.). Architecture - Model Context Protocol。版本：MCP specification 2025-06-18; web snapshot 2026-09-19；访问：2026-09-19T11:22:39.243Z。
- 机构/作者未登记 (n.d.). Schema Reference - Model Context Protocol。版本：MCP specification 2025-11-25; web snapshot 2026-09-19；访问：2026-09-19T11:22:42.560Z。
- 机构/作者未登记 (n.d.). Authorization - Model Context Protocol。版本：MCP specification 2025-11-25; web snapshot 2026-09-19；访问：2026-09-19T11:22:42.478Z。
- 机构/作者未登记 (n.d.). Scale to many tools with tool search - Claude Code Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T11:21:13.530Z。
- 机构/作者未登记 (n.d.). deepseek-harness/packages/core/tools/README.md at master · deepseek-ai/deepseek-harness。版本：web snapshot 2026-09-19；访问：2026-09-19T11:27:11.346Z。
- 机构/作者未登记 (n.d.). Making Claude Code more secure and autonomous with sandboxing \ Anthropic。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:42.689Z。
- 机构/作者未登记 (n.d.). codex/codex-rs/execpolicy/README.md at main · openai/codex。版本：web snapshot 2026-09-19；访问：2026-09-19T11:27:11.358Z。
- Carlos E. Jimenez et al. (2023). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?。版本：arXiv:2310.06770v3；访问：2026-09-19T11:45:11.210Z。
- OpenAI (2024). Introducing SWE-bench Verified | OpenAI。版本：web snapshot 2026-09-19；访问：2026-09-19T11:23:43.941Z。
- OpenAI (2026). Why SWE-bench Verified no longer measures frontier coding capabilities | OpenAI。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:42.659Z。
- Anthropic (2026). Demystifying evals for AI agents \ Anthropic。版本：web snapshot 2026-09-19；访问：2026-09-19T11:24:43.765Z。
- Koki Wataoka, Tsubasa Takahashi, Ryokan Ri (2024). Self-Preference Bias in LLM-as-a-Judge。版本：arXiv:2410.21819v2；访问：2026-09-19T11:49:17.172Z。
- Lin Shi et al. (2024). Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge。版本：arXiv:2406.07791v9；访问：2026-09-19T11:49:17.181Z。
- Jonathan Gabor, Jayson Lynch, Jonathan Rosenfeld (2025). EvilGenie: A Reward Hacking Benchmark。版本：arXiv:2511.21654v2；访问：2026-09-19T11:49:17.165Z。
- Bingchen Zhao et al. (2026). SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents。版本：arXiv:2605.21384v1；访问：2026-08-28。
- Anthropic (2025). How we built our multi-agent research system \ Anthropic。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:47.317Z。
- OpenAI (2026). openai-agents-python/docs/handoffs.md at main · openai/openai-agents-python。版本：web snapshot 2026-09-19；访问：2026-09-19T11:27:11.396Z。
- Anthropic (2026). Extend Claude Code - Claude Code Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:20.870Z。

- Anthropic (2026). Automate actions with hooks - Claude Code Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:20.863Z。
- Cursor (2026). Cursor Agent Security。版本：web snapshot 2026-08-28；访问：2026-08-28。
- Cursor (2026). Cursor Background Agents。版本：web snapshot 2026-08-28；访问：2026-08-28。
- Cursor (2026). 钩子 | Cursor Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T12:18:51.997Z。
- DeepSeek AI (2026). deepseek-ai/deepseek-harness / docs/tool-catalog.md。版本：git cd5ef8148158c3a752a658978873241fdf8e2bbc；访问：2026-09-19T09:44:37.154753+00:00。
- OpenAI (2026). openai-agents-python/docs/agents.md at main · openai/openai-agents-python。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:37.447Z。
- Model Context Protocol (2025). Key Changes - Model Context Protocol。版本：MCP specification 2025-11-25; web snapshot 2026-09-19；访问：2026-09-19T11:49:42.634Z。
- Microsoft Research (2026). AutoGen - Microsoft Research: Publications。版本：web snapshot 2026-09-19；访问：2026-09-19T11:49:47.330Z。
- 机构/作者未登记 (n.d.). OpenHands: An Open Platform for AI Software Developers as Generalist Agents。版本：arXiv:2407.16741v3；访问：2026-09-19T11:13:46.085Z。
- 机构/作者未登记 (n.d.). Toolformer: Language Models Can Teach Themselves to Use Tools。版本：arXiv:2302.04761v1；访问：2026-09-19T11:12:46.050Z。
- 机构/作者未登记 (n.d.). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation。版本：arXiv:2308.08155v2；访问：2026-09-19T11:12:46.058Z。
- 机构/作者未登记 (n.d.). Executable Code Actions Elicit Better LLM Agents。版本：arXiv:2402.01030v4；访问：2026-09-19T11:12:16.595Z。
- 机构/作者未登记 (n.d.). AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents。版本：arXiv:2605.13357v1；访问：2026-09-19T11:17:44.562Z。
- 机构/作者未登记 (n.d.). MemGPT: Towards LLMs as Operating Systems。版本：arXiv:2310.08560v2；访问：2026-09-19T11:12:46.065Z。
- 机构/作者未登记 (n.d.). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework。版本：arXiv:2308.00352v7；访问：2026-09-19T11:12:46.070Z。
- 机构/作者未登记 (n.d.). Why Do Multi-Agent LLM Systems Fail?。版本：arXiv:2503.13657v3；访问：2026-09-19T11:13:46.093Z。
- 机构/作者未登记 (n.d.). Can LLM Agents Really Debate? A Controlled Study of Multi-Agent Debate in Logical Reasoning。版本：arXiv:2511.07784v1；访问：2026-09-19T11:13:46.108Z。
- 机构/作者未登记 (n.d.). MultiAgentBench: Evaluating the Collaboration and Competition of LLM agents。版本：arXiv:2503.01935v1；访问：2026-09-19T11:13:46.101Z。
- OpenAI (n.d.). Changelog | OpenAI API。版本：web snapshot 2026-09-19；访问：2026-09-19T09:38:31.208Z。
- OpenAI (n.d.). Agents API | OpenAI API。版本：web snapshot 2026-09-19；访问：2026-09-19T09:34:14.265Z。

- Anthropic (n.d.). Claude Platform release notes - Claude Platform Docs。版本：web snapshot 2026-09-19；访问：2026-09-19T09:34:14.296Z。
- Anthropic (n.d.). Scaling Managed Agents: Decoupling the brain from the hands \ Anthropic。版本：web snapshot 2026-09-19；访问：2026-09-19T09:55:33.629Z。
- Cursor (n.d.). 在您自行管理的机器上运行云端智能体 · Cursor。版本：web snapshot 2026-09-19；访问：2026-09-19T09:34:14.310Z。
- Cursor (n.d.). Cursor 最新动态 — 最新更新与发布说明。版本：web snapshot 2026-09-19；访问：2026-09-19T09:38:31.226Z。
- T. Bengre; C. Curme / LangChain (n.d.). Organizing Context in a Multi-Agent Harness。版本：web snapshot 2026-09-19；访问：2026-09-19T09:34:14.279Z。
- Microsoft (n.d.). Agent Harness | Microsoft Learn。版本：web snapshot 2026-09-19；访问：2026-09-19T09:44:20.634Z。
- Wu et al. (n.d.). HarnessDev: Can LLMs Create and Evolve Their Own Agent Harness?。版本：arXiv:2609.01437v1；访问：2026-09-19T09:45:11.363Z。
- Yan et al. (n.d.). Harness-of-Harness: Multi-Day Autonomous Software Development with Continual Improvement。版本：arXiv:2609.01481v1；访问：2026-09-19T09:45:11.377Z。
- Zhang et al. (n.d.). JIT-Agent: Scaling Harness Intelligence via Just-in-Time Harness Evolution。版本：arXiv:2608.25593v2；访问：2026-09-19T09:45:11.405Z。
- Jiang et al. (n.d.). HarnessEvolve: Learning from Reference Trajectories for Reliable Agent Self-Evolution。版本：arXiv:2609.00829v1；访问：2026-09-19T09:52:13.305Z。
- Fan et al. (n.d.). An Empirical Study of Harness Design for Coding Agents。版本：arXiv:2609.20804v1；访问：2026-09-19T09:45:11.392Z。
- Li et al. (n.d.). A Blind Trust, the Bloody Thrust: When Attacker-Controlled Hook UpdatesSteer AI Agent Harnesses towards Malicious Behaviors。版本：arXiv:2609.03884v2；访问：2026-09-19T09:52:13.352Z。
- Chen et al. (n.d.). MemSecBench: Tracking Agent Memory Poisoning from Persistence to Consequence and Repair。版本：arXiv:2607.27080v1；访问：2026-09-19T09:50:41.795Z。
- Ben Brandt / ACP (n.d.). ACP v2 is available in Draft - Agent Client Protocol。版本：web snapshot 2026-09-19；访问：2026-09-19T09:52:13.322Z。
- Palash Shah / LangChain (n.d.). LangSmith Engine: How We Built an Agent for Improving Agents。版本：web snapshot 2026-09-19；访问：2026-09-19T09:52:13.337Z。
- Jin et al. (n.d.). Harness Engineering in LLM Tool Use via Agent-Native Reusable Tool Primitives。版本：arXiv:2609.01736v1；访问：2026-09-19T09:44:20.654Z。
- Duan et al. (n.d.). The Last AI Built by Humans: Toward Genuine Recursive Self-Improvement。版本：arXiv:2609.11873v2；访问：2026-09-19T10:00:34.225Z。
- Chen et al. (n.d.). Show-Harness: Just a VLM Agent Can Play Robots。版本：arXiv:2609.10522v1；访问：2026-09-19T09:54:21.136Z。

- Yue et al. (n.d.). Ecdysis: Efficient and Effective Training of Runtime Harnesses for LLM Agents。版本：arXiv:2609.11677v1；访问：2026-09-19T10:00:34.247Z。
- Lin et al. (n.d.). Stellar Colosseum: A Many-Agent Harness for Long-Horizon Research in Mathematics and Theoretical Computer Science。版本：arXiv:2609.15983v2；访问：2026-09-19T10:00:34.237Z。
- Liu et al. (n.d.). SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness。版本：arXiv:2609.20519v1；访问：2026-09-19T10:00:34.203Z。
- Luo et al. (n.d.). HarnessBank: Semantic Gene-Bank Search with Gated Verification for Agent-Harness Self-Evolution。版本：arXiv:2607.13683v2；访问：2026-09-19T09:55:33.606Z。
- NLE authors / Facebook Research (n.d.). facebookresearch/nle: The NetHack Learning Environment。版本：web snapshot 2026-09-19；访问：2026-09-19T09:55:33.617Z。
- Taylor Mullen; Christian Gunderman / Google (n.d.). The Anatomy of Harness Engineering: How to Evaluate, Iterate, and Guard AI Coding Agents - Google Developers Blog。版本：web snapshot 2026-09-19；访问：2026-09-19T09:59:03.556Z。
- openJiuwen Team (n.d.). openJiuwen: Beyond Static Harnesses for Long-Horizon Coding Agents。版本：arXiv:2608.27969v1；访问：2026-09-19T09:35:42.335Z。
- Google Cloud (n.d.). Query syntax | BigQuery | Google Cloud Documentation。版本：web snapshot 2026-09-19；访问：2026-09-19T10:14:48.326Z。
- 机构/作者未登记 (n.d.). openai/codex / codex-rs/app-server/README.md。版本：git 426fa8cdab4247e5623e9617d531f6917482b947；访问：2026-09-19T09:44:39.463010+00:00。
- 机构/作者未登记 (n.d.). openai/codex / codex-rs/app-server-protocol/src/protocol/common.rs。版本：git be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-09-19T09:54:19.199639+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / README.md。版本：git ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-09-19T09:44:36.923997+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / packages/extensions/cordis-host-runner/src/sandbox.ts。版本：git ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-09-19T09:49:04.757600+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / packages/code-runtime/code-runtime-worker-thread/README.md。版本：git cd5ef8148158c3a752a658978873241fdf8e2bbc；访问：2026-09-19T09:54:18.498962+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / packages/ptc-runtime/ptc-runtime-node/README.md。版本：git ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-09-19T09:51:32.840667+00:00。
- 机构/作者未登记 (n.d.). OpenHands/OpenHands / openhands/runtime/README.md。版本：git 7fbb48c40679afd674970966b96185657d92a487；访问：2026-09-19T09:54:19.573502+00:00。
- 机构/作者未登记 (n.d.). OpenHands/software-agent-sdk / openhands-sdk/openhands/sdk/conversation/conversation.py。版本：git d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-09-19T09:54:19.421087+00:00。

- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / packages/boot/plugin-manager/README.md。版本：git ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-09-19T09:49:04.882477+00:00。
- 机构/作者未登记 (n.d.). fix(sdk): persist events before publishing them, return the assigned seq。版本：git 94fca578b720df758b9bbf8a2639511b303c78e6；访问：2026-09-19T09:49:07.883806+00:00。
- 机构/作者未登记 (n.d.). feat(agent-server): add /sockets/session/{id} with a non-Event envelope。版本：git 2ab274897ac5e2c66b0ba17e9a6d39367b769876；访问：2026-09-19T09:49:07.828869+00:00。
- 机构/作者未登记 (n.d.). feat(agent-server): add docker runtime mode for per-conversation containers。版本：git 3ff6924d8564b3d47a22a6c7e71377a701ae014f；访问：2026-09-19T09:54:20.078080+00:00。
- 机构/作者未登记 (n.d.). Architecture - Model Context Protocol。版本：MCP specification 2026-07-28; web snapshot 2026-09-19；访问：2026-09-19T11:27:11.371Z。
- 机构/作者未登记 (n.d.). Codex local schema probe。版本：local Codex 0.142.5 probe 2026-09-19；访问：2026-09-19。
- 机构/作者未登记 (n.d.). Codex local initialize probe。版本：local Codex 0.142.5 probe 2026-09-19；访问：2026-09-19。
- 机构/作者未登记 (n.d.). OpenHands/OpenHands / openhands/runtime/action_execution_server.py。版本：git 7fbb48c40679afd674970966b96185657d92a487；访问：2026-09-19T09:51:34.569498+00:00。
- 机构/作者未登记 (n.d.). OpenHands/OpenHands / package.json。版本：git 9737f713616a1e452f822c2967f0e2c8bf2dc308；访问：2026-09-19T09:54:19.076593+00:00。
- 机构/作者未登记 (n.d.). OpenHands/OpenHands / README.md。版本：git b50c60c6728e2ce123ccb6e125bee3eb88ac87d1；访问：2026-09-19T09:44:39.765917+00:00。
- 机构/作者未登记 (n.d.). OpenHands/OpenHands 1.0.0。版本：release 1.0.0; published_at=2025-12-16T16:03:32Z；访问：2026-09-19T09:51:33.954077+00:00。
- 机构/作者未登记 (n.d.). OpenHands/OpenHands v1.20.0。版本：release v1.20.0; published_at=2026-09-17T07:15:18Z；访问：2026-09-19T09:43:26.012204+00:00。
- 机构/作者未登记 (n.d.). OpenHands/software-agent-sdk / openhands-agent-server/openhands/agent_server/docker_runtime/provisioning.py。版本：git d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-09-19T09:54:19.190723+00:00。
- 机构/作者未登记 (n.d.). OpenHands/software-agent-sdk v1.45.0。版本：release v1.45.0; published_at=2026-09-07T03:07:34Z；访问：2026-09-19T09:43:27.809445+00:00。
- 机构/作者未登记 (n.d.). OpenHands/software-agent-sdk v1.48.0。版本：release v1.48.0; published_at=2026-09-15T20:58:25Z；访问：2026-09-19T09:43:27.809445+00:00。
- 机构/作者未登记 (n.d.). OpenHands/software-agent-sdk v1.49.1。版本：release v1.49.1; published_at=2026-09-17T04:35:48Z；访问：2026-09-19T09:43:27.809445+00:00。

- 机构/作者未登记 (n.d.). OpenHands/software-agent-sdk v1.49.2。版本：release v1.49.2; published_at=2026-09-17T20:46:52Z；访问：2026-09-19T09:43:26.388854+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / docs/architecture.md。版本：git cd5ef8148158c3a752a658978873241fdf8e2bbc；访问：2026-09-19T09:44:36.953863+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / docs/tool-catalog.md。版本：git cd5ef8148158c3a752a658978873241fdf8e2bbc；访问：2026-09-19T09:44:37.154753+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / docs/architecture.md。版本：git ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-09-19T09:44:36.981066+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness / docs/tool-catalog.md。版本：git ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-09-19T09:44:37.714636+00:00。
- 机构/作者未登记 (n.d.). feat(code-runtime): execute Node programs through confined processes。版本：git 75ed8da3e0c9103b3b2174b2981b7129e1fba21d；访问：2026-09-19T09:57:13.361793+00:00。
- 机构/作者未登记 (n.d.). refactor(ptc): align runtime packages and services with PTC naming。版本：git 7c9bb5914cedec80e46197a8c894037fcfd12faf；访问：2026-09-19T09:54:19.613914+00:00。
- 机构/作者未登记 (n.d.). feat: add current-profile plugin manager service and Web controls。版本：git 98b92b683c39fc60771daa774492105c2d3e8076；访问：2026-09-19T09:51:33.515620+00:00。
- 机构/作者未登记 (n.d.). feat(agent): await initialization through agent/created。版本：git 9b7a8ccc9fabc2e87386acf7f8b0741baf978022；访问：2026-09-19T09:54:18.826378+00:00。
- 机构/作者未登记 (n.d.). refactor(hmr): own profile reload lifecycle through YAML。版本：git abd765a6001ff9d9c9772b8b407e0b7f18fe25ab；访问：2026-09-19T09:51:33.188488+00:00。
- 机构/作者未登记 (n.d.). feat(session)!: add released format migration。版本：git d1521ea7838f19a78a9cca7b4a93622d301149bb；访问：2026-09-19T09:51:35.337469+00:00。
- 机构/作者未登记 (n.d.). Revert #932 transactional Cordis reload changes。版本：git e07f41d5fd8ca172287fda0f923b4d1f69c592f3；访问：2026-09-19T09:51:33.122985+00:00。
- 机构/作者未登记 (n.d.). feat(creator): use Plugin Manager for persistent plugins。版本：git ed32f57f88ef6bba983e30a0b434fe5d77e5773b；访问：2026-09-19T09:51:33.349909+00:00。
- 机构/作者未登记 (n.d.). feat(session)!: embed assistant streams in format v2。版本：git f99b06eaed81d6fe4fc64d44687450e18ef68a67；访问：2026-09-19T09:51:34.765805+00:00。
- 机构/作者未登记 (n.d.). deepseek-ai/deepseek-harness dsh-v0.1.6-alpha.2。版本：release dsh-v0.1.6-alpha.2; published_at=2026-09-17T13:30:16Z；访问：2026-09-19T09:43:26.873237+00:00。
- 机构/作者未登记 (n.d.). openai/codex / codex-rs/app-server-protocol/src/protocol/common.rs。版本：git 426fa8cdab4247e5623e9617d531f6917482b947；访问：2026-09-19T09:44:39.266059+00:00。
- 机构/作者未登记 (n.d.). openai/codex python-v0.154.0。版本：release python-v0.154.0; published_at=2026-09-10T19:51:43Z；访问：2026-09-19T09:43:37.004725+00:00。
- 机构/作者未登记 (n.d.). openai/codex rust-v0.152.0。版本：release rust-v0.152.0; published_at=2026-09-01T01:58:32Z；访问：2026-09-19T09:43:37.004725+00:00。

- 机构/作者未登记 (n.d.). openai/codex rust-v0.153.0。版本：release rust-v0.153.0; published_at=2026-09-03T01:37:38Z；访问：2026-09-19T09:43:37.004725+00:00。
- 机构/作者未登记 (n.d.). openai/codex rust-v0.154.0。版本：release rust-v0.154.0; published_at=2026-09-09T22:35:38Z；访问：2026-09-19T09:43:37.004725+00:00。
- 机构/作者未登记 (n.d.). openai/codex rust-v0.155.1。版本：release rust-v0.155.1; published_at=2026-09-18T20:03:04Z；访问：2026-09-19T09:43:26.336187+00:00。
- 机构/作者未登记 (n.d.). Cognitive Architectures for Language Agents。版本：arXiv:2309.02427v3；访问：2026-09-19T11:12:16.577Z。
- 机构/作者未登记 (n.d.). SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents。版本：arXiv:2605.21384v2；访问：2026-09-19T11:49:20.851Z。
- code.claude.com (2026). Agent SDK overview。版本：official documentation deployment dpl_AUojGTPodKNPTVaxVPcxnNvkSVqo; retrieved 2026-10-29；访问：2026-10-29。
- code.claude.com (2026). How Claude Code works。版本：official documentation deployment dpl_AUojGTPodKNPTVaxVPcxnNvkSVqo; retrieved 2026-10-29；访问：2026-10-29。
- github.com (2026). claude-agent-sdk-python: SubprocessCLITransport。版本：27858cbba9ecee26a06a90b8d39d3b5596121d11; pyproject.toml 0.2.174；访问：2026-10-29。
- github.com (2026). claude-agent-sdk-python: Query control protocol。版本：27858cbba9ecee26a06a90b8d39d3b5596121d11; pyproject.toml 0.2.174；访问：2026-10-29。
- code.claude.com (2026). Configure permissions。版本：official documentation deployment dpl_AUojGTPodKNPTVaxVPcxnNvkSVqo; retrieved 2026-10-29；访问：2026-10-29。
- github.com (2026). claude-agent-sdk-python pyproject.toml。版本：27858cbba9ecee26a06a90b8d39d3b5596121d11; package 0.2.174；访问：2026-10-29。
- code.claude.com (2026). How the agent loop works。版本：2026-10-29 snapshot SHA256 67e074bc45e667feaf2dfa955f0de06df27b951b081979c64f83f3a3fbce9e02；访问：2026-10-29。
- code.claude.com (2026). Work with sessions。版本：2026-10-29 snapshot SHA256 456f033eff0db40bef3d41724e8ee7260cd156256727d73859c4ee108c7e35d8；访问：2026-10-29。
- code.claude.com (2026). Persist sessions to external storage。版本：2026-10-29 snapshot SHA256 b2a5ad497f498a6f6f3fdb2e97ecc4bc13c6812f67403902ea14244736b86ecd；访问：2026-10-29。
- code.claude.com (2026). Rewind file changes with checkpointing。版本：2026-10-29 snapshot SHA256 92184e8a01e18f2f17d86fa0343b5b7b7f34140f7e988a101dcca9bcc8959f73；访问：2026-10-29。
- code.claude.com (2026). How Claude remembers your project。版本：2026-10-29 snapshot SHA256 f19bb5aa909fe5588d523cf302a0e15ed5950ca689fbb73bd932c315ec764a49；访问：2026-10-29。
- code.claude.com (2026). Modifying system prompts。版本：2026-10-29 snapshot SHA256 cf93b231577e079271bf564ae84d5f25f3a11723c51b55b104bd283817042330；访问：2026-10-29。
- code.claude.com (2026). Use Claude Code features in the SDK。版本：2026-10-29 snapshot SHA256 853607de24218eac839698efe2a3f29da889fb14b6760279dcc29e7ee0f66a70；访问：

2026-10-29。

- code.claude.com (2026). How Claude Code uses prompt caching。版本：2026-10-29 snapshot SHA256 02c7f42cc7a32e2be83a2d72bdb7e14b43287ff1a162d548c40b85958fea3579；访问：2026-10-29。
- platform.claude.com (2026). Prompt caching。版本：2026-10-29 snapshot SHA256 e8dd46e73676cfc1c18a99d792b6049293d59ff67cbfa4c4530d33b102b197ac；访问：2026-10-29。
- code.claude.com (2026). Connect to external tools with MCP。版本：2026-10-29 snapshot SHA256 354ad14a1d3466c5846639cc7dbac8d409556fe5a719f3dc8f500b2209246fb2；访问：2026-10-29。
- code.claude.com (2026). Give Claude custom tools。版本：2026-10-29 snapshot SHA256 946948f5a92b2f387aafa39924dfcec651db257572d98898c07387b07366edcf；访问：2026-10-29。
- code.claude.com (2026). Intercept and control agent behavior with hooks。版本：2026-10-29 snapshot SHA256 fd0706dee09c3cdeb76c21a68723e0184e50e2eb4fac48dd3f2ae1a08e4f3b92；访问：2026-10-29。
- code.claude.com (2026). Hooks reference。版本：2026-10-29 snapshot SHA256 ffc7195446eb6048fb54e8351b31c00e0280d303c339f4829023a9215d4ecc76；访问：2026-10-29。
- code.claude.com (2026). Configure the sandboxed Bash tool。版本：2026-10-29 snapshot SHA256 e331b3b786e52fa028a68cc1dab76049ce8ee8c79e7907d7b43f58e039b9bc68；访问：2026-10-29。
- code.claude.com (2026). Securely deploying AI agents。版本：2026-10-29 snapshot SHA256 e384fdd59a4ba17711478aa66d74fdb157deb1494d7d9b49bf1a5998aaf0879f；访问：2026-10-29。
- code.claude.com (2026). Subagents in the SDK。版本：2026-10-29 snapshot SHA256 9db0458e43e56fd472b94af95e6eb9c1b00052a250eb91bc540da930b77cc4cd；访问：2026-10-29。
- code.claude.com (2026). Create custom subagents。版本：2026-10-29 snapshot SHA256 c6af4f0b650d11cdabbd6b6e49d5272ff00cb966580bb4f8d1dc09ec4ae5b3cb；访问：2026-10-29。
- code.claude.com (2026). Agent SDK reference - TypeScript。版本：2026-10-29 snapshot SHA256 1153044d62625a791f34279ce3db54587c2dd51aed2fc61a3f984c80bb329913；访问：2026-10-29。
- code.claude.com (2026). Track cost and usage。版本：2026-10-29 snapshot SHA256 b01f78de868bf230af56931cfeedea5df6a1d0c859c254bf8734c36501ddeb5b；访问：2026-10-29。
- code.claude.com (2026). Monitoring。版本：2026-10-29 snapshot SHA256 a0473752f2712fa11515466d798e5d16b879d0055bdbd686d39bfeaefc1ebb5a；访问：2026-10-29。
- www.anthropic.com (2026). Scaling Managed Agents with stable interfaces / Decoupling the brain from the hands。版本：published 2026-04-08; 2026-10-29 HTML snapshot SHA256 0b78f9a5d109dcc6e6543ed350ffaefd80c08e68221484cea58a4f9c5abd41f1；访问：2026-10-29。
- developers.openai.com (2026). Codex as a platform: build on the open agent harness。版本：2026-08-19 article; accessed 2026-10-29；访问：2026-10-29。
- developers.openai.com (2026). Codex App Server。版本：live documentation accessed 2026-10-29；访问：2026-10-29。
- developers.openai.com (2026). Codex SDK。版本：live documentation accessed 2026-10-29；访问：2026-10-29。

- learn.chatgpt.com (2026). Agent approvals & security。版本：live documentation accessed 2026-10-29；访问：2026-10-29。
- github.com (2026). ThreadManager / ThreadManagerState。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). CodexThread。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Session。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). SessionIo and Session lifecycle。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). run_turn / run_sampling_request。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Responses stream retry。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). RolloutReconstruction。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). ContextManager。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Compaction and initial context。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). AGENTS.md discovery。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). ToolRouter。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). ToolRegistry。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). McpHandler。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). MCP client initialization and discovery。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). DynamicToolHandler。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). ToolOrchestrator。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). SandboxManager。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Auth storage backends。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。

- github.com (2026). AgentControl。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). V1 spawn_agent。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Agent spawn runtime overrides。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Agent close and shutdown。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Agent wait。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Agent status derivation。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). V2 spawn_agent。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Codex TypeScript SDK process transport。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). App Server protocol methods。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Skills extension surface。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Memories pipeline。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Task cancellation and terminal persistence。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). ToolDispatchTrace。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). OpenTelemetry provider initialization。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e；访问：2026-10-29。
- github.com (2026). Codex Rust workspace version。版本：rust-v0.155.1; be2951ea34f0d295ed0becf97079f92fa5f6950e; annotated tag object 4e21628f9ec9ee656650cd2b62ef92225725b5ac；访问：2026-10-29。
- cursor.com (2026). Cursor Agent。版本：unversioned official documentation; retrieved-text SHA-256 2b474c20c095efbc1280b9d6d6f8d7670da88a35e82ff66560282f97a90bbc91；访问：2026-10-29。
- cursor.com (2026). Tool Calling。版本：unversioned official documentation; retrieved-text SHA-256 919d8e6bb9230ee69ec71577499f445b620073109430620df534a2f5a102beff；访问：2026-10-29。
- cursor.com (2026). Agents。版本：unversioned official documentation; retrieved-text SHA-256 c1cf476b6201c08f4adc70835ca4678913d9b10d270024759671d0a74d41227e；访问：2026-10-29。

- cursor.com (2026). Continually improving our agent harness。版本：2026-04-30; retrieved-text SHA-256 88eaa21e2985a1f08a5558e1428bcbfd456a1d1d90255b00edb05843aa8ef329；访问：2026-10-29。
- cursor.com (2026). Data Use & Privacy Overview。版本：last updated 2026-09-03; retrieved-text SHA-256 fb6cbd6322e6426cdd92acc56712ba78627027c0624b9118c4b4fe6e9215d773；访问：2026-10-29。
- cursor.com (2026). Securely indexing large codebases。版本：2026-01-27; retrieved-text SHA-256 6aa8ad7517020d3f6d8ddfae13b2392229beafec02f7bea90111d6448569b51d；访问：2026-10-29。
- cursor.com (2026). Rules。版本：unversioned official documentation; retrieved-text SHA-256 dae9d3ecd7e389fc94ee57d87911fe8d54bae1f00ce451d44303616648247ce0；访问：2026-10-29。
- cursor.com (2026). Dynamic context discovery。版本：2026-01-06; retrieved-text SHA-256 ed1a2d87695f5e3e65cadfdc40d672ffaa7e446f115e19c01929be215fb10469；访问：2026-10-29。
- cursor.com (2026). Cursor 1.0 release notes。版本：Cursor 1.0 / 2025-06-04; retrieved-text SHA-256 ec55656d658f925ad6fea537b70dba52a52128997d479cfe10eac1260a2493ab；访问：2026-10-29。
- cursor.com (2026). Model Context Protocol (MCP)。版本：unversioned official documentation; retrieved-text SHA-256 33c58ca4e1a2c7e1c31842a1016919d79ef00981e187db58527bdf7c0ab93671；访问：2026-10-29。
- cursor.com (2026). Hooks。版本：unversioned official documentation; retrieved-text SHA-256 a5f76c8eab2c3ec3fe30fe5d41fbaf5c276da0090ac4d1f6687c4e5c0f4084fa；访问：2026-10-29。
- cursor.com (2026). Run Modes。版本：unversioned official documentation; retrieved-text SHA-256 3047e773f2f179e85214eaa4f058bc9db6b8e54ac040a55371d7b95766095192；访问：2026-10-29。
- cursor.com (2026). Cloud Agent security overview。版本：unversioned official documentation; retrieved-text SHA-256 907873d8852c3f84784fc234b6490c08c2d041f87eb3f002919cf84b9d66a5d9；访问：2026-10-29。
- cursor.com (2026). Cloud Agents。版本：unversioned official documentation; retrieved-text SHA-256 0a6ca38d1d6aab6ab982b471cc111863f688bc248f308e6d5159312d20d8a051；访问：2026-10-29。
- cursor.com (2026). Secrets & Network。版本：unversioned official documentation; retrieved-text SHA-256 e3da6cc3669d194f059af49567e700450372e2a34889154a4a6bc74de25e9d1a；访问：2026-10-29。
- cursor.com (2026). OIDC tokens。版本：unversioned official documentation; retrieved-text SHA-256 80832f5c8f03007aaee23b12a2c2daa09b7268dad7a6205abdc95118a950fe2f；访问：2026-10-29。
- cursor.com (2026). Self-Hosted Machines。版本：unversioned official documentation; retrieved-text SHA-256 51a8cea833c4994adf4ce5da465d18ac21f58d69145d95cb12bf4d875304d06d；访问：2026-10-29。
- cursor.com (2026). Subagents。版本：unversioned official documentation; retrieved-text SHA-256 264b4b00e713ec0dd73939306690ea4013168bead35411da3cc56fc1a71b2668；访问：2026-10-29。

- cursor.com (2026). Projects。版本：unversioned official documentation; retrieved-text SHA-256 7d7f6b16a56e7b88c5bf626a92f987c0b1db3f26e145a6ee7687c37698c8e2a2；访问：2026-10-29。
- cursor.com (2026). Run cloud agents on machines you manage。版本：2026-09-02; retrieved-text SHA-256 6d3089ae51d318979029aef5f62bf39f62bb2e6ff9855930ee6ff088ce20df4f；访问：2026-10-29。
- github.com (2026). DeepSeek Harness Architecture。版本：dsh-v0.1.6-alpha.2 / ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). DeepSeek Harness Architecture。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Core subsystem。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). System Prompt Assembly。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). dsh-agent-instructions。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Compaction subsystem。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Tool Execution Pipeline。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). MCP subsystem。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). dsh-ptc-runtime-node。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Process Sandbox。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). User Approval。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). User Credentials。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Cordis dynamic Host sandbox。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Plugin Manager。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Configure persistent plugins from a prompt。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Application boot。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。

- github.com (2026). Subagent subsystem。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Session telemetry。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Historical code-runtime-worker-thread。版本：dsh-v0.1.2-alpha.1 / cd5ef8148158c3a752a658978873241fdf8e2bbc；访问：2026-10-29。
- github.com (2026). Spawn in-process subagent。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). Fork in-process subagent。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). dsh-tools。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). dsh-tool-cordis。版本：ddefc45fbc7f8e46dd73185e68295696d1297887；访问：2026-10-29。
- github.com (2026). OpenHands Software Agent SDK。版本：v1.49.2 / d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). Agent Canvas package.json。版本：v1.20.0 / 9737f713616a1e452f822c2967f0e2c8bf2dc308；访问：2026-10-29。
- github.com (2026). Conversation factory。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). Agent implementation。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). LocalConversation。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). ConversationState。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). AgentContext。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). Context View。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). LLMSummarizingCondenser。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). ParallelToolExecutor。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). MCPToolExecutor。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). PluginSource and ResolvedPluginSource。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。

- github.com (2026). Confirmation policies。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). OpenHands Agent Server。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). DockerConversationRegistry。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). RuntimeProvisioningStore。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). Docker conversation routes。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). Agent profile secret scope。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). SecretRegistry and StreamOutputMask。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). DelegateExecutor。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). Session socket。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). SDK observability integration。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). Historical OpenHands Runtime。版本：0.62.0 / 7fbb48c40679afd674970966b96185657d92a487；访问：2026-10-29。
- github.com (2026). TaskToolSet and TaskAction。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). TaskManager。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). Plugin model。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). AgentBase tool initialization。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。
- github.com (2026). Pi v1.4.0 release and repository identity。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi README。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi agent loop source。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi SDK。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。

- github.com (2026). Pi session file format。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi compaction reference。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi MCP servers。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi extensions。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Run Pi safely。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi subagent extension example。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi RPC mode。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi CLI reference。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi skills。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). Pi coding-agent session factory。版本：4a080252f06caf63a525d7873c5d94d3c23887e0；访问：2026-10-29。
- github.com (2026). RemoteConversation client。版本：d128a786ee2ee570eb23ff5862ec148b43cfad0b；访问：2026-10-29。